

DJ PARTY SYSTEM

ΜΕΛΗ ΟΜΑΔΑΣ:

ΠΑΝΙΤΣΑΣ ΦΩΤΗΣ	A.M. 1100969	3 ^ο ΕΤΟΣ
ΜΕΝΤΖΑΣ ΓΡΗΓΟΡΙΟΣ	A.M. 1108335	3 ^ο ΕΤΟΣ
ΤΟΥΝΤΑΣ ΔΗΜΗΤΡΙΟΣ	A.M. 1093504	5 ^ο ΕΤΟΣ
ΚΑΠΠΟΣ ΚΩΝΣΤΑΝΤΙΝΟΣ	A.M. 1088444	5 ^ο ΕΤΟΣ
ΔΕΡΒΕΝΗΣ ΙΑΣΩΝ ΑΘΑΝΑΣΙΟΣ	A.M. 1097437	5 ^ο ΕΤΟΣ

Link για github:

<https://github.com/dervenisja/Technologia-logismikou>

ΠΕΡΙΕΧΟΜΕΝΑ

- 1) PROJECT DESCRIPTION**
- 2) USE CASES**
- 3) DOMAIN MODEL**
- 4) ROBUSTNESS DIAGRAMS**
- 5) SEQUENCE DIAGRAMS**
- 6) CLASS DIAGRAM**
- 7) ΚΩΔΙΚΑΣ**
- 8) TEST CASES**
- 9) SCREENSHOTS ΕΚΤΕΛΕΣΗΣ**

1) PROJECT DESCRIPTION

Το σύστημα που αναπτύσσεται αφορά μια εφαρμογή διαχείρισης μουσικής και ατμόσφαιρας σε περιβάλλον πάρτι, με στόχο τη δημιουργία μιας διαδραστικής και εξατομικευμένης εμπειρίας ψυχαγωγίας. Οι χρήστες μπορούν να δημιουργούν και να συμμετέχουν σε πάρτι, να αλληλεπιδρούν με τη μουσική και να συμβάλλουν δυναμικά στη διαμόρφωση της συνολικής εμπειρίας.

Κάθε πάρτι δημιουργείται και ελέγχεται από έναν χρήστη με ρόλο PartyHost, ο οποίος οργανώνει το event, διαχειρίζεται τη λίστα αναπαραγωγής (playlist) και ελέγχει βασικές ρυθμίσεις όπως ο ήχος και ο φωτισμός. Οι υπόλοιποι χρήστες (PartyGuests) συμμετέχουν στο πάρτι και μπορούν να υποβάλλουν αιτήματα τραγουδιών (song requests), για παράδειγμα μέσω σάρωσης ενός μοναδικού QR code που αντιστοιχεί στο συγκεκριμένο πάρτι. Τα αιτήματα αυτά λαμβάνονται υπόψη κατά τη διαμόρφωση της μουσικής ροής.

Μέσω ενός μηχανισμού προτάσεων, προτείνονται τραγούδια που ταιριάζουν στο ύφος του πάρτι και στις επιλογές των συμμετεχόντων, βελτιώνοντας τη συνολική εμπειρία.

Επιπλέον, το σύστημα διαθέτει δυνατότητες αυτόματης μίξης μουσικής (auto-mixing), επιτρέποντας ομαλές μεταβάσεις μεταξύ διαδοχικών τραγουδιών. Οι μεταβάσεις πραγματοποιούνται με τρόπο που διατηρεί τη συνοχή και τη

ροή της μουσικής εμπειρίας και μπορούν να περιλαμβάνουν ηχητικά εφέ, προσομοιώνοντας τη λειτουργία ενός DJ.

Παρέχεται επίσης η δυνατότητα στον PartyHost να καταγράφει το περιβάλλον του χώρου μέσω της κάμερας της συσκευής του, ώστε το σύστημα να προσαρμόζει αυτόματα τις ρυθμίσεις ήχου. Με βάση τα χαρακτηριστικά του χώρου, βελτιστοποιούνται παράμετροι όπως η ένταση και η ισορροπία του ήχου, με στόχο τη βέλτιστη ακουστική εμπειρία.

Το σύστημα μπορεί να αλληλεπιδρά με εξωτερικές συσκευές, όπως συστήματα φωτισμού, επιτρέποντας την προσαρμογή της ατμόσφαιρας με βάση τη μουσική. Παρέχονται επίσης ρυθμίσεις ήχου και περιβάλλοντος για τον καλύτερο έλεγχο της εμπειρίας.

Η χρήση του συστήματος υποστηρίζει συνδρομητικό μοντέλο, στο οποίο μόνο ο χρήστης που δημιουργεί και διαχειρίζεται το πάρτι (PartyHost) απαιτείται να διαθέτει ενεργή συνδρομή για την πλήρη αξιοποίηση των δυνατοτήτων του συστήματος, ενώ οι υπόλοιποι συμμετέχοντες μπορούν να λαμβάνουν μέρος χωρίς να απαιτείται συνδρομή.

Η ομάδα συμφωνεί ομόφωνα ότι η προσπάθεια όλων των φοιτητών ήταν ισοδύναμη άρα $E_i = 0.20$

Όλοι είχαμε συμμετοχή σε όλα τα διαγράμματα και στον κώδικα.

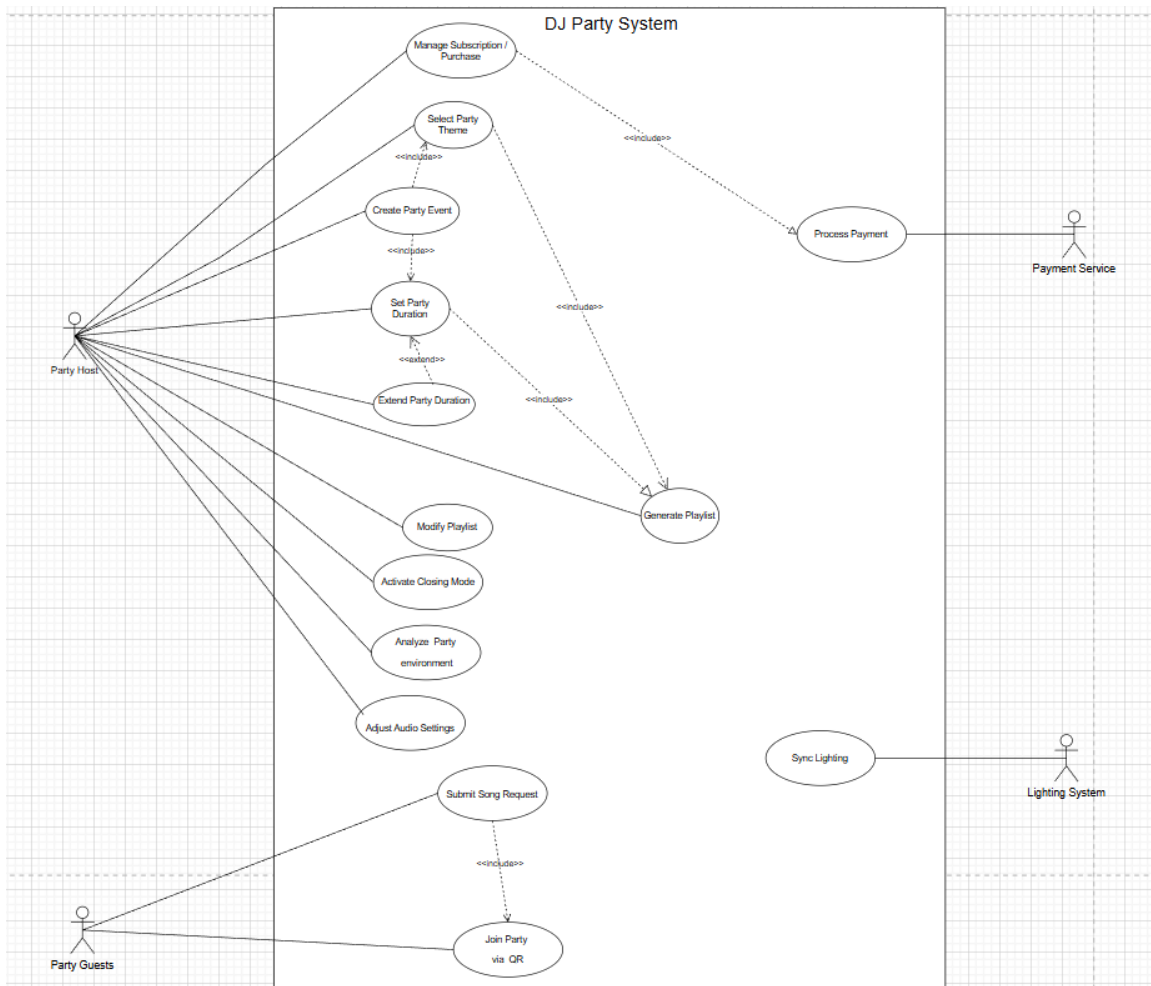
2) USE CASES

Το Use Case Diagram αποτυπώνει τις βασικές λειτουργίες του συστήματος από την οπτική των χρηστών (actors) και τις αλληλεπιδράσεις τους με το σύστημα.

Οι βασικοί actors είναι ο Party Host, οι Party Guests, το Lighting System και το Payment Service. Ο Party Host αποτελεί τον κύριο χρήστη του συστήματος και είναι υπεύθυνος για τη δημιουργία και διαχείριση του party, ενώ οι Party Guests αλληλεπιδρούν με το σύστημα μέσω υποβολής song requests. Τα εξωτερικά συστήματα (Lighting System και Payment Service) συμμετέχουν σε επιμέρους λειτουργίες, όπως ο συγχρονισμός φωτισμού και η επεξεργασία πληρωμών.

Τα use cases περιγράφουν τις βασικές δυνατότητες του συστήματος, όπως η δημιουργία party, η δημιουργία και τροποποίηση playlist, η διαχείριση διάρκειας, καθώς και η υποβολή αιτημάτων τραγουδιών από τους χρήστες.

Η χρήση των σχέσεων <<include>> και <<extend>> εφαρμόζεται επιλεκτικά, ώστε να εκφράζει απαραίτητες ή προαιρετικές λειτουργίες, διατηρώντας το διάγραμμα απλό και κατανοητό χωρίς περιττή πολυπλοκότητα.



USE CASE 1: CREATE PARTY EVENT

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Εισαγωγή δεδομένων από τον Host

- **Ενέργεια ηθοποιού:** Ο PartyHost συμπληρώνει τη φόρμα δημιουργίας party (όνομα, περιγραφή, κλπ.)

- **UI Component:** CreatePartyFormUI → συλλογή δεδομένων

Βήμα 2: Υποβολή αιτήματος

- **Ενέργεια ηθοποιού:** Ο Host πατάει "Submit" / "Create Party"
- **UI → InputHandler:** Τα δεδομένα μεταφέρονται στο InputHandler

Βήμα 3: Έλεγχος δικαιωμάτων (authorization)

- **InputHandler → AuthorizationController:** check role
- **Αποτέλεσμα 1:** authorized → συνεχίζει
- **Αποτέλεσμα 2:** not authorized → εμφάνιση μηνύματος, τερματισμός

Βήμα 4: Validation δεδομένων

- **InputHandler → ValidationController:** valid input
- **Αν αποτυχία:** ValidationMessageUI → εμφάνιση σφάλματος → επιστροφή στο Βήμα 1

Βήμα 5: Δημιουργία αντικειμένου Party

- **ValidationController → PartyFactory:** create()
- **PartyFactory:** Δημιουργεί νέο Party object

Βήμα 6: Σύνδεση με τον Host

- **PartyFactory → PartyHost:** link to host (το party συνδέεται με τον δημιουργό)

Βήμα 7: Αποθήκευση

- **PartyFactory → PersistenceController:** save
- **PersistenceController:** Αποθηκεύει το νέο party στη βάση

Βήμα 8: Επιβεβαίωση

- **PersistenceController → ConfirmationUI:** show confirmation
- **ConfirmationUI → PartyHost:** Εμφάνιση μηνύματος επιτυχίας

Preconditions

1. Ο χρήστης είναι authenticated στο σύστημα
2. Ο χρήστης έχει ρόλο PartyHost (authorization)
3. Το σύστημα είναι διαθέσιμο

Postconditions

1. Νέο Party object έχει δημιουργηθεί
2. Το party έχει μοναδικό ID
3. Το party είναι συνδεδεμένο με τον PartyHost
4. Το party είναι σε κατάσταση "draft" ή "pending" (πριν την ολοκλήρωση)
5. Τα δεδομένα έχουν αποθηκευτεί στο persistence layer

USE CASE 2: SELECT PARTY THEME

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Πρόσβαση στη φόρμα επιλογής theme

- **Προϋπόθεση:** Υπάρχει party event (από Use Case 1)
- **Ενέργεια ηθοποιού:** PartyHost ανοίγει ThemeSelectionUI

Βήμα 2: Επιλογή theme

- **Ενέργεια ηθοποιού:** Ο Host επιλέγει ένα theme (π.χ., "Rock", "Pop", "Jazz")
- **UI → InputHandler:** Το theme ID/όνομα μεταφέρεται

Βήμα 3: Validation του theme

- **InputHandler → ThemeValidationController:** valid
- **Αν theme δεν υπάρχει:** invalid → ErrorUI → "Invalid theme" → επιστροφή

Βήμα 4: Προεπισκόπηση theme (optional, αλλά υπάρχει στο diagram)

- **ThemeValidationController → ThemePreviewUI:** showpreview
- **ThemePreviewUI → PartyHost:** Εμφάνιση προεπισκόπησης (π.χ., sample playlist)

Βήμα 5: Ενημέρωση του party

- **ThemeValidationController** → **PartyController**: settheme
- **PartyController** → **Party**: Ενημερώνει το theme του party object

Βήμα 6: Αποθήκευση

- **PartyController** → **PersistenceController**: save
- **PersistenceController**: Αποθηκεύει την αλλαγή

Βήμα 7: Επιβεβαίωση

- **PersistenceController** → **ConfirmationUI**: show
- **ConfirmationUI** → **PartyHost**: "Theme selected successfully"

Preconditions

1. Υπάρχει party event (έχει δημιουργηθεί)
2. Το party είναι σε κατάσταση που επιτρέπει αλλαγή theme (π.χ., όχι active/closing)
3. Ο χρήστης είναι PartyHost και έχει πρόσβαση στο συγκεκριμένο party

Postconditions

Το theme έχει οριστεί στο party object

1. Η αλλαγή έχει αποθηκευτεί
2. Το party μπορεί να χρησιμοποιήσει το theme για playlist generation

USE CASE 3: SET PARTY DURATION

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Πρόσβαση στη φόρμα εισαγωγής διάρκειας

- **Προϋπόθεση:** Υπάρχει party event
- **Ενέργεια ηθοποιού:** PartyHost ανοίγει DurationInputUI

Βήμα 2: Εισαγωγή διάρκειας

- **Ενέργεια ηθοποιού:** Ο Host εισάγει διάρκεια (π.χ., 120 λεπτά, 2 ώρες)
- **UI → InputHandler:** submit data

Βήμα 3: Validation διάρκειας (βασικός έλεγχος)

- **InputHandler → DurationValidationController:** check duration
- **Έλεγχοι:**
 - Μη αρνητική τιμή
 - Μη μηδενική (αν όχι allowed)
 - Αριθμητική τιμή
- **Αν αποτυχία:** invalid duration → ErrorUI → επιστροφή

Βήμα 4: Επιχειρηματικοί κανόνες

- **DurationValidationController → BusinessRulesController:** valid

- **BusinessRulesController:** Ελέγχει αν η διάρκεια είναι εντός ορίων (π.χ., min 30 min, max 480 min)

Βήμα 5: Ενημέρωση του party

- **BusinessRulesController** → **PartyController:** setDuration
- **PartyController** → **Party:** Ενημερώνει τη διάρκεια

Βήμα 6: Αποθήκευση

- **PartyController** → **PersistenceController:** save
- **PersistenceController:** store → αποθήκευση

Βήμα 7: Επιβεβαίωση

- **PersistenceController** → **ConfirmationUI:** show
- **ConfirmationUI** → **PartyHost:** "Duration set successfully"

Preconditions

1. Υπάρχει party event
2. Το party δεν είναι ακόμα active (δεν έχει ξεκινήσει)
3. Ο χρήστης είναι PartyHost

Postconditions

Η διάρκεια έχει οριστεί στο party object

1. Η διάρκεια είναι αποθηκευμένη
2. Το party είναι έτοιμο για playlist generation

USE CASE 4: GENERATE PLAYLIST

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Ο Host επιλέγει "Generate Playlist"

- **Προϋπόθεση:**
 - Party event exists
 - Theme has been selected
 - Duration has been set
- **Ενέργεια ηθοποιού:** Πατάει "Generate Playlist"

Βήμα 2: Το σύστημα λαμβάνει το theme

- **PartyController** → **ThemeService**: `getTheme(party.theme)`
- **ThemeService**: Ανακτά τη λίστα τραγουδιών που αντιστοιχούν στο theme

Βήμα 3: Το σύστημα λαμβάνει τη διάρκεια

- **PartyController** → **Party**: `getDuration()`

Βήμα 4: Δημιουργία playlist

- **PartyController** → **PlaylistGenerator**: `generate(theme, duration)`
- **PlaylistGenerator**:
 - Επιλέγει τραγούδια από το theme
 - Υπολογίζει συνολική διάρκεια

- Προσθέτει/αφαιρεί τραγούδια ώστε να ταιριάζει στη διάρκεια

Βήμα 5: Αποθήκευση playlist

- **PlaylistGenerator** → **PersistenceController**: savePlaylist()
- **PersistenceController**: Αποθηκεύει τη νέα playlist

Βήμα 6: Παρουσίαση στον Host

- **PersistenceController** → **PlaylistEditorUI**: display(playlist)
- **PlaylistEditorUI** → **PartyHost**: Εμφάνιση playlist

Βήμα 7: Επιβεβαίωση

- **PlaylistEditorUI** → **ConfirmationUI**: "Playlist generated successfully"

Alternative Flows

A1: Δεν έχει επιλεγεί theme

- **Σημείο ελέγχου**: Πριν το Βήμα 2
- **Ενέργεια συστήματος**: ErrorUI → "Please select a theme first"
- **Επόμενο βήμα**: Redirect to ThemeSelectionUI

A2: Δεν έχει οριστεί διάρκεια

- **Σημείο ελέγχου**: Πριν το Βήμα 3

- **Ενέργεια συστήματος:** ErrorUI → "Please set duration first"
- **Επόμενο βήμα:** Redirect to DurationInputUI

A3: Αποτυχία δημιουργίας playlist

- **Σημείο ελέγχου:** Κατά το Βήμα 4
- **Πιθανές αιτίες:**
 - Theme service unavailable
 - No songs found for theme
 - Duration too short/long for available songs
- **Ενέργεια συστήματος:** ErrorUI → "Failed to generate playlist. Please try again or adjust settings."

Preconditions

1. Έχει δημιουργηθεί party event
2. Έχει επιλεγεί party theme
3. Έχει οριστεί διάρκεια party
4. Το ThemeService είναι διαθέσιμο
5. Υπάρχουν τραγούδια για το συγκεκριμένο theme

Postconditions

1. Έχει δημιουργηθεί playlist object
2. Η playlist περιέχει τραγούδια που ταιριάζουν στο theme

3. Η συνολική διάρκεια της playlist προσεγγίζει τη διάρκεια του party
4. Η playlist είναι αποθηκευμένη
5. Η playlist είναι διαθέσιμη για αναπαραγωγή και επεξεργασία

USE CASE 5: MODIFY PLAYLIST

Βήμα 1: Πρόσβαση στη διεπαφή επεξεργασίας

- **Προϋπόθεση:** Υπάρχει playlist (από Generate Playlist)
- **Ενέργεια ηθοποιού:** PartyHost ανοίγει PlaylistEditorUI

Βήμα 2: Επιλογή ενέργειας (προσθήκη ή αφαίρεση)

- **Ενέργεια ηθοποιού:** Ο Host επιλέγει να προσθέσει ή να αφαιρέσει ένα τραγούδι
- **UI → EditRequestHandler:** action (add/remove)

Βήμα 3: Αναζήτηση τραγουδιού (για προσθήκη)

- **EditRequestHandler** → **SongService**: fetch song
- **SongService**: Αναζητά το τραγούδι
- **Αν δεν βρεθεί**: invalid → **ErrorUI** → επιστροφή

Βήμα 4: Υποβολή αιτήματος

- **EditRequestHandler** →
ValidationController: request → submit
- **ValidationController**: Ελέγχει αν η ενέργεια επιτρέπεται
(π.χ., όχι σε closing mode)

Βήμα 5: Επεξεργασία playlist

- **ValidationController (valid)** →
PlaylistController: add/remove song
- **PlaylistController** → **Playlist**: Ενημερώνει τη playlist

Βήμα 6: Αποθήκευση

- **PlaylistController** → **PersistenceController**: store → save
- **PersistenceController**: Αποθηκεύει τις αλλαγές

Βήμα 7: Επιβεβαίωση

- **PersistenceController** →
ConfirmationUI: showconfirmation
- **ConfirmationUI** → **PartyHost**: "Playlist updated
successfully"

Preconditions

Υπάρχει playlist (έχει δημιουργηθεί)

1. Το party δεν βρίσκεται σε closing mode (ή ανάλογα rules)
2. Ο χρήστης είναι PartyHost

Postconditions

Η playlist έχει ενημερωθεί (προσθήκη ή αφαίρεση τραγουδιού)

1. Οι αλλαγές έχουν αποθηκευτεί
2. Η playlist είναι έτοιμη για αναπαραγωγή

USE CASE 6: EXTEND PARTY DURATION

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Πρόσβαση στη φόρμα επέκτασης

- **Προϋπόθεση:** Έχει οριστεί διάρκεια (από Set Party Duration)
- **Ενέργεια ηθοποιού:** PartyHost ανοίγει DurationExtensionUI

Βήμα 2: Εισαγωγή επιπλέον διάρκειας

- **Ενέργεια ηθοποιού:** Ο Host εισάγει τα λεπτά/ώρες για επέκταση
- **UI → InputHandler:** submit

Βήμα 3: Validation

- **InputHandler → ValidationController:** validate
- **Έλεγχοι:** Μη αρνητική τιμή, αριθμητική, όχι υπερβολικά μεγάλη
- **Αν αποτυχία:** invalid duration → ErrorUI → επιστροφή

Βήμα 4: Εφαρμογή επιχειρηματικών κανόνων

- **ValidationController (valid) → RulesController:** apply
- **RulesController:** Ελέγχει αν η επέκταση επιτρέπεται (π.χ., max total duration)

Βήμα 5: Ενημέρωση διάρκειας

- **RulesController → PartyController:** extendDuration
- **PartyController → Party:** Αυξάνει τη διάρκεια

Βήμα 6: Αποθήκευση

- **PartyController → PersistenceController:** store → save
- **PersistenceController:** Αποθηκεύει τη νέα διάρκεια

Βήμα 7: Επιβεβαίωση

- **PersistenceController → ConfirmationUI:** showconfirmation

- **ConfirmationUI → PartyHost:** "Duration extended successfully"
-

Preconditions

1. Έχει οριστεί διάρκεια
2. Το party είναι ενεργό
3. Ο χρήστης είναι PartyHost
4. Η επέκταση δεν υπερβαίνει το μέγιστο όριο

Postconditions

1. Η διάρκεια έχει αυξηθεί κατά την επιπλέον τιμή
2. Οι αλλαγές έχουν αποθηκευτεί

USE CASE 7: ACTIVATE CLOSING MODE

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Πρόσβαση στο Control Panel

- **Προϋπόθεση:** Το party είναι ενεργό
- **Ενέργεια ηθοποιού:** PartyHost ανοίγει ControlPanelUI

Βήμα 2: Επιλογή closing mode

- **Ενέργεια ηθοποιού:** Ο Host επιλέγει "Activate Closing Mode"
- **UI → ActionHandler:** submit

Βήμα 3: Επεξεργασία ενέργειας

- **ActionHandler → ModeController:** process action
- **ModeController:** Προετοιμάζει την αλλαγή mode

Βήμα 4: Εφαρμογή mode

- **ModeController → PartyController:** apply mode / setClosingMode
- **PartyController → Party:** Θέτει το party σε closing mode

Βήμα 5: Αποθήκευση

- **PartyController → PersistenceController:** store → save
- **PersistenceController:** Αποθηκεύει την κατάσταση

Βήμα 6: Επιβεβαίωση

- **PersistenceController → ConfirmationUI:** show
- **ConfirmationUI → PartyHost:** "Closing mode activated"

Preconditions

1. Το party είναι ενεργό
2. Ο χρήστης είναι PartyHost
3. Το party δεν είναι ήδη σε closing mode

Postconditions

1. Το σύστημα λειτουργεί σε closing mode
2. Νέα requests δεν γίνονται δεκτά (ή υπάρχει ειδική συμπεριφορά)
3. Η κατάσταση έχει αποθηκευτεί

USE CASE 8: SCAN QR CODE

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Άνοιγμα scanner

- **Προϋπόθεση:** Υπάρχει ενεργό party
- **Ενέργεια ηθοποιού:** PartyGuest ανοίγει QRScannerUI → open scanner

Βήμα 2: Σάρωση QR code

- **Ενέργεια ηθοποιού:** Ο Guest σκανάρει το QR code
- **UI → ScanController:** scan

Βήμα 3: Αποκωδικοποίηση

- **ScanController → QRDecoder:** decode
- **QRDecoder:** Αποκωδικοποιεί το QR content

Βήμα 4: Validation

- **QRDecoder → ValidationController:** validate

- **ValidationController:** Ελέγχει αν το QR είναι έγκυρο και αντιστοιχεί σε ενεργό party
- **Αν αποτυχία:** invalid QR → ErrorUI → τερματισμός

Βήμα 5: Έλεγχος πρόσβασης

- **ValidationController (valid) → AccessController:** verify
- **AccessController → Party:** Επαληθεύει ότι το party είναι ενεργό

Βήμα 6: Χορήγηση πρόσβασης

- **AccessController → PartyGuest:** grant access
- **AccessController → AccessGrantedUI:** show success

Βήμα 7: Επιβεβαίωση στον Guest

- **AccessGrantedUI → PartyGuest:** Εμφάνιση μηνύματος επιτυχίας

Preconditions

1. Υπάρχει ενεργό party
2. Το QR code έχει εκδοθεί από το σύστημα
3. Ο Guest έχει συσκευή με κάμερα

Postconditions

1. Ο χρήστης έχει πρόσβαση στο σύστημα
2. Ο Guest μπορεί να υποβάλλει song requests

USE CASE 9: SUBMIT SONG REQUEST

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Άνοιγμα Song Request UI

- **Προϋπόθεση:** Υπάρχει ενεργό party, Guest έχει πρόσβαση (από Scan QR Code)
- **Ενέργεια ηθοποιού:** PartyGuest ανοίγει SongRequestUI

Βήμα 2: Αναζήτηση τραγουδιού

- **Ενέργεια ηθοποιού:** Ο Guest πληκτρολογεί τίτλο/καλλιτέχνη
- **UI → SearchController:** Search

Βήμα 3: Εμφάνιση αποτελεσμάτων

- **SearchController → Song:** Αναζήτηση στη βάση
- **SearchController → UI:** results → εμφάνιση λίστας

Βήμα 4: Επιλογή και υποβολή

- **Ενέργεια ηθοποιού:** Ο Guest επιλέγει τραγούδι και πατάει submit
- **UI → RequestHandler:** submit

Βήμα 5: Validation τραγουδιού

- **RequestHandler → SongValidationController:** validate

- **SongValidationController:** Ελέγχει αν το τραγούδι είναι διαθέσιμο
- **Αν όχι:** no access / invalid → ErrorUI → εναλλακτική ροή A2

Βήμα 6: Επεξεργασία από RequestController

- **SongValidationController (valid) → RequestController:** valid
- **RequestController:** Προετοιμάζει το αίτημα

Βήμα 7: Προσθήκη στην ουρά

- **RequestController → QueueManager:** add to queue / update
- **QueueManager → Playlist:** Ενημερώνει την ουρά

Βήμα 8: Δημιουργία SongRequest object

- **QueueManager → SongRequest:** create
- **SongRequest:** Δημιουργείται το αντικείμενο αιτήματος

Βήμα 9: Αποθήκευση

- **SongRequest → PersistenceController:** save
- **PersistenceController:** Αποθηκεύει το αίτημα

Βήμα 10: Επιβεβαίωση

- **PersistenceController → ConfirmationUI:** confirmation → show
- **ConfirmationUI → PartyGuest:** "OK" / "Request submitted successfully"

Preconditions

1. Υπάρχει ενεργό party
2. Ο χρήστης έχει πρόσβαση στο σύστημα μέσω QR code
3. Το party δεν βρίσκεται σε closing mode

Postconditions

1. Το song request έχει καταγραφεί
2. Το τραγούδι είναι διαθέσιμο για ένταξη στην playlist
3. Το αίτημα συνδέεται με τον Guest και το party

Alternative Flows

A1: Μη έγκυρο QR code

- **Σημείο ελέγχου:** Πριν το Βήμα 1
- **Ενέργεια συστήματος:** ErrorUI → "Invalid QR code"
- **Αποτέλεσμα:** Καμία πρόσβαση, κανένα request

A2: Το τραγούδι δεν είναι διαθέσιμο

- **Σημείο ελέγχου:** Βήμα 5
- **Ενέργεια συστήματος:** ErrorUI → "Song not available"
- **Αποτέλεσμα:** Request δεν καταχωρείται, ο Guest καλείται να επιλέξει άλλο

USE CASE 10: SYNC LIGHTING WITH MUSIC

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Αρχικοποίηση συγχρονισμού

- **Προϋπόθεση:** Υπάρχει ενεργή playlist
- **System → SyncController:** initiate

Βήμα 2: Λήψη τρέχοντος τραγουδιού

- **SyncController → Playlist:** get current song
- **Playlist → SyncController:** Επιστρέφει το Song

Βήμα 3: Ανάλυση beats

- **SyncController → BeatAnalyzer:** analyze beat / extract beat
- **BeatAnalyzer → Song:** Εξάγει beat data

Βήμα 4: Αντιστοίχιση σε χρώματα

- **BeatAnalyzer → DataMapper:** map to colors
- **DataMapper:** Παράγει lighting pattern

Βήμα 5: Μετάδοση εντολών

- **DataMapper → TransmissionController:** send commands
- **TransmissionController → NetworkInterface:** transmit

Βήμα 6: Επικοινωνία με LightingSystem

- **NetworkInterface → LightingSystem:** Αποστολή εντολών
- **LightingSystem:** Εκτελεί τον συγχρονισμό

Βήμα 7: Αποθήκευση κατάστασης

- **TransmissionController → PersistenceController:** save
- **PersistenceController:** Αποθηκεύει την τελευταία lighting state

Preconditions

1. Υπάρχει ενεργή playlist
2. Υπάρχει τρέχον τραγούδι που παίζει
3. Το LightingSystem είναι συνδεδεμένο

Postconditions

1. Ο φωτισμός είναι συγχρονισμένος με τη μουσική
2. Η κατάσταση έχει αποθηκευτεί

USE CASE 11: MANAGE SUBSCRIPTION/PURCHASE

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Πρόσβαση στο Subscription UI

- **Προϋπόθεση:** Ο χρήστης έχει λογαριασμό (authenticated)

- **Ενέργεια ηθοποιού:** PartyHost ανοίγει SubscriptionUI

Βήμα 2: Επιλογή προγράμματος και υποβολή

- **Ενέργεια ηθοποιού:** Ο Host επιλέγει πρόγραμμα
→ submit
- **UI → RequestHandler:** submit

Βήμα 3: Επεξεργασία αιτήματος

- **RequestHandler:** process request

Βήμα 4: Validation

- **RequestHandler → ValidationController:** validate
- **Αν invaliddata:** ErrorUI → τερματισμός

Βήμα 5: Λήψη λεπτομερειών προγράμματος

- **ValidationController (valid) → PricingService:** getdetails
- **PricingService:** plan

Βήμα 6: Εκκίνηση διαδικασίας πληρωμής

- **PricingService → SubscriptionController:** validated plan
- **SubscriptionController → PaymentOrchestrator:** execute payment request

Βήμα 7: Πληρωμή

- **PaymentOrchestrator → PaymentUI:** payment
- **PaymentUI → Host:** Ολοκλήρωση πληρωμής

Βήμα 8: Αποθήκευση συνδρομής

- **PaymentOrchestrator (payment success) → SubscriptionController:** payment
- **SubscriptionController → PersistenceController:** store → save
- **PersistenceController:** Subscription saved

Βήμα 9: Επιβεβαίωση

- **PersistenceController → ConfirmationUI:** showsuccess
 - **ConfirmationUI → PartyHost:** "Subscription activated"
-

Preconditions

1. Ο χρήστης έχει λογαριασμό (authenticated)
2. Ο χρήστης είναι PartyHost
3. Υπάρχουν διαθέσιμα προγράμματα

Postconditions

1. Η συνδρομή ενημερώνεται / δημιουργείται
2. Η πληρωμή έχει καταγραφεί
3. Ο Host έχει πρόσβαση στις νέες υπηρεσίες

USE CASE 12: PROCESS PAYMENT

Αναλυτικά βήματα (Main Success Scenario)

Βήμα 1: Λήψη αιτήματος πληρωμής

- **Προϋπόθεση:** Έχει ζητηθεί πληρωμή (από Use Case 11)
- **PaymentService → PaymentAPI:** prepare response (acknowledgment)

Βήμα 2: Μορφοποίηση δεδομένων

- **PaymentAPI → RequestBuilder:** format data
- **RequestBuilder:** Προετοιμάζει το αίτημα για το gateway

Βήμα 3: Προώθηση αιτήματος

- **RequestBuilder → ExternalGatewayAdapter:** forward request
- **ExternalGatewayAdapter → ExternalPaymentGateway:** authorize

Βήμα 4: Επεξεργασία από gateway

- **ExternalPaymentGateway:** authorize → success ή failure

Βήμα 5: Επεξεργασία απάντησης

- **ExternalPaymentGateway → ResponseHandler:** success / failure

Βήμα 6: Καταγραφή αποτελέσματος

- **ResponseHandler** → **PaymentController**: record / update
- **PaymentController** → **Payment**: Ενημερώνει την πληρωμή
- **PaymentController** → **Subscription**: Ενημερώνει τη συνδρομή

Βήμα 7: Αποθήκευση

- **PaymentController** → **PersistenceController**: store / save
- **PersistenceController**: Αποθηκεύει τις αλλαγές

Βήμα 8: Επιβεβαίωση

- **PersistenceController** → **ConfirmationUI**: show
- **ConfirmationUI**: "Payment processed successfully"

(Αν αποτυχία) Βήμα 8a: Error handling

- **ResponseHandler (failure)** → **ErrorUI**: show
- **ErrorUI**: "Payment failed"

Preconditions

1. Έχει ζητηθεί πληρωμή (από Manage Subscription)
2. Υπάρχει valid Payment object
3. Το ExternalPaymentGateway είναι διαθέσιμο

Postconditions

1. Η πληρωμή έχει καταγραφεί (επιτυχία ή αποτυχία)
2. Το Subscription έχει ενημερωθεί ανάλογα
3. Τα δεδομένα έχουν αποθηκευτεί

3) DOMAIN MODEL

1. Οντότητες (Entities) και Συσχετίσεις

1.1 User (Χρήστης)

Ο **User** αποτελεί τη βάση για όλους τους χρήστες του συστήματος. Κάθε χρήστης έχει λογαριασμό και μπορεί να λειτουργεί είτε ως **PartyHost** είτε ως **PartyGuest**.

Χαρακτηριστικά:

- Μοναδικό ID
- Στοιχεία ταυτοποίησης (username, password, email)
- Ρόλος (Host ή Guest)

1.2 PartyHost (Διοργανωτής Πάρτι)

Ο **PartyHost** είναι ένας εξειδικευμένος τύπος χρήστη που έχει τη δυνατότητα να:

- **Δημιουργεί νέα πάρτι** (Create Party Event - UC1)
- **Επιλέγει το μουσικό theme** του πάρτι (Select Party Theme - UC2)
- **Ορίζει τη διάρκεια** του πάρτι (Set Party Duration - UC3)
- **Δημιουργεί αυτόματα playlist** με βάση theme και διάρκεια (Generate Playlist - UC4)
- **Τροποποιεί την υπάρχουσα playlist** (Modify Playlist - UC5)
- **Επεκτείνει τη διάρκεια** του πάρτι (Extend Party Duration - UC6)
- **Ενεργοποιεί τη λειτουργία λήξης** (Activate Closing Mode - UC7)
- **Διαχειρίζεται συνδρομή και αγορές** (Manage Subscription - UC11)

Συσχετίσεις:

- Ένας PartyHost **hosts** (διοργανώνει) από 0 έως πολλά (0..*) πάρτι
- Ένας PartyHost συνδέεται με μία (1) συνδρομή (Subscription)

1.3 PartyGuest (Καλεσμένος Πάρτι)

Ο **PartyGuest** είναι ένας τύπος χρήστη που συμμετέχει σε πάρτι. Μπορεί να:

- **Σκανάρει QR code** για να αποκτήσει πρόσβαση (Scan QR Code - UC8)
- **Υποβάλλει αιτήματα για τραγούδια** (Submit Song Request - UC9)

Συσχετίσεις:

- Ένας PartyGuest **makes** (κάνει) από 0 έως πολλά (0..*) αιτήματα τραγουδιών (SongRequest)

1.4 Party (Πάρτι)

Το **Party** αποτελεί την κεντρική οντότητα του συστήματος. Κάθε πάρτι:

- Δημιουργείται από έναν PartyHost
- Έχει μοναδικό ID, όνομα, περιγραφή
- Βρίσκεται σε συγκεκριμένη κατάσταση (draft, active, closing, ended)
- Έχει επιλεγμένο **theme** (π.χ., Rock, Pop, Jazz)
- Έχει ορισμένη **διάρκεια** (π.χ., 120 λεπτά)
- Διαθέτει μία **playlist** με τραγούδια
- Μπορεί να έχει πολλά **αιτήματα τραγουδιών** (SongRequest)
- Μπορεί να έχει ένα **QR code** για πρόσβαση

- Συνδέεται με **περιβάλλον** (Environment) και **ηχητικές ρυθμίσεις** (AudioSettings)

Συσχετίσεις:

- Ένα Party **hosts** (διοργανώνεται από) έναν (1) PartyHost
- Ένα Party **contains** (περιέχει) μία (1) Playlist
- Ένα Party **has** (έχει) από 0 έως πολλά (0..*) SongRequest
- Ένα Party **has** (έχει) ένα (1) QRCode
- Ένα Party **has** (έχει) ένα (1) Environment
- Ένα Party **has** (έχει) ένα (1) AudioSettings

1.5 Playlist (Λίστα Αναπαραγωγής)

Η **Playlist** είναι η συλλογή τραγουδιών που θα παιχτούν στο πάρτι. Δημιουργείται αυτόματα βάσει theme και διάρκειας (UC4) και μπορεί να τροποποιηθεί από τον Host (UC5).

Συσχετίσεις:

- Μία Playlist **contains** (περιέχει) από 0 έως πολλά (0..*) τραγούδια (Song)
- Μία Playlist ανήκει σε ένα (1) Party

1.6 Song (Τραγούδι)

Το **Song** είναι μια μουσική οντότητα που μπορεί να:

- Υπάρχει στη βάση δεδομένων του συστήματος (από SongService)
- Προστίθεται σε playlists
- Ζητείται από Guests μέσω SongRequest
- Αναλύεται για συγχρονισμό φωτισμού (beat analysis)

Χαρακτηριστικά:

- Τίτλος, καλλιτέχνης, διάρκεια, BPM, genre/theme

Συσχετίσεις:

- Ένα Song **is contained in** (περιέχεται σε) μία (1) Playlist (μέσω της σχέσης contains)
- Ένα Song μπορεί να σχετίζεται με πολλά (0..*) SongRequest

1.7 SongRequest (Αίτημα Τραγουδιού)

Το **SongRequest** αντιπροσωπεύει την ενέργεια ενός Guest να ζητήσει ένα τραγούδι. Κάθε αίτημα:

- Υποβάλλεται από έναν Guest (UC9)
- Αφορά ένα συγκεκριμένο τραγούδι
- Ανήκει σε ένα συγκεκριμένο πάρτι
- Μπορεί να έχει κατάσταση (pending, approved, rejected, played)

Συσχετίσεις:

- Ένα SongRequest **is made by** (γίνεται από) έναν (1) PartyGuest
 - Ένα SongRequest αντιστοιχεί σε ένα (1) Song
 - Ένα SongRequest ανήκει σε ένα (1) Party
-

1.8 QRCode

Το **QRCode** είναι μια οντότητα που:

- Παράγεται από το σύστημα για κάθε πάρτι
- Επιτρέπει στους Guests να αποκτούν πρόσβαση (UC8)
- Περιέχει κωδικοποιημένα δεδομένα (π.χ., party ID, εκδότης, ημερομηνία λήξης)

Συσχετίσεις:

- Ένα QRCode ανήκει σε ένα (1) Party
-

1.9 LightingSystem (Σύστημα Φωτισμού)

Το **LightingSystem** είναι ένα εξωτερικό σύστημα που:

- Συγχρονίζεται με τη μουσική που παίζει (Sync Lighting With Music - UC10)
- Λαμβάνει εντολές μέσω δικτύου (NetworkInterface)
- Παράγει φωτιστικά εφέ βασισμένα σε beat analysis

Συσχετίσεις:

- Ένα LightingSystem συνδέεται με ένα (1) σύστημα (Environment)
-

1.10 Subscription (Συνδρομή)

Η **Subscription** αντιπροσωπεύει τη σχέση συνδρομής ενός PartyHost με το σύστημα. Μέσω αυτής:

- Ο Host αποκτά πρόσβαση σε premium υπηρεσίες
- Υπάρχει σύνδεση με πληρωμές (Payment)
- Μπορεί να έχει διαφορετικά προγράμματα (π.χ., Basic, Pro, VIP)

Συσχετίσεις:

- Μία Subscription ανήκει σε έναν (1) PartyHost
 - Μία Subscription σχετίζεται με 0 έως πολλές (0..*) πληρωμές (Payment)
-

1.11 Payment (Πληρωμή)

Η **Payment** καταγράφει κάθε συναλλαγή που πραγματοποιείται για:

- Αγορά ή αναβάθμιση συνδρομής (UC11, UC12)
- Περιλαμβάνει ποσό, ημερομηνία, μέθοδο πληρωμής, κατάσταση (επιτυχής / αποτυχημένη)

Συσχετίσεις:

- Μία Payment ανήκει σε μία (1) Subscription
-

1.12 Environment (Περιβάλλον)

Το **Environment** αντιπροσωπεύει τον φυσικό ή ψηφιακό χώρο όπου λαμβάνει χώρα το πάρτι. Περιλαμβάνει:

- Παραμέτρους χώρου (π.χ., μέγεθος, ακουστική)
- Συνδέσεις με εξωτερικά συστήματα (π.χ., LightingSystem)

Συσχετίσεις:

- Ένα Environment ανήκει σε ένα (1) Party
-

1.13 AudioSettings (Ηχητικές Ρυθμίσεις)

Οι **AudioSettings** περιλαμβάνουν παραμέτρους ήχου για το πάρτι:

- Ένταση, equalizer, εφέ
- Ρυθμίσεις αναπαραγωγής (π.χ., fade in/out)

Συσχετίσεις:

- Οι AudioSettings ανήκουν σε ένα (1) Party
-

2. Συνοπτικός Πίνακας Συσχετίσεων

Από	Προς	Cardinality	Περιγραφή
PartyHost	Party	1 → 0..*	Ένας Host διοργανώνει πολλά πάρτι
PartyGuest	SongRequest	1 → 0..*	Ένας Guest κάνει πολλά requests
Party	Playlist	1 → 1	Κάθε πάρτι έχει μία playlist
Playlist	Song	1 → 0..*	Μία playlist περιέχει πολλά τραγούδια
Party	SongRequest	1 → 0..*	Ένα πάρτι έχει πολλά requests

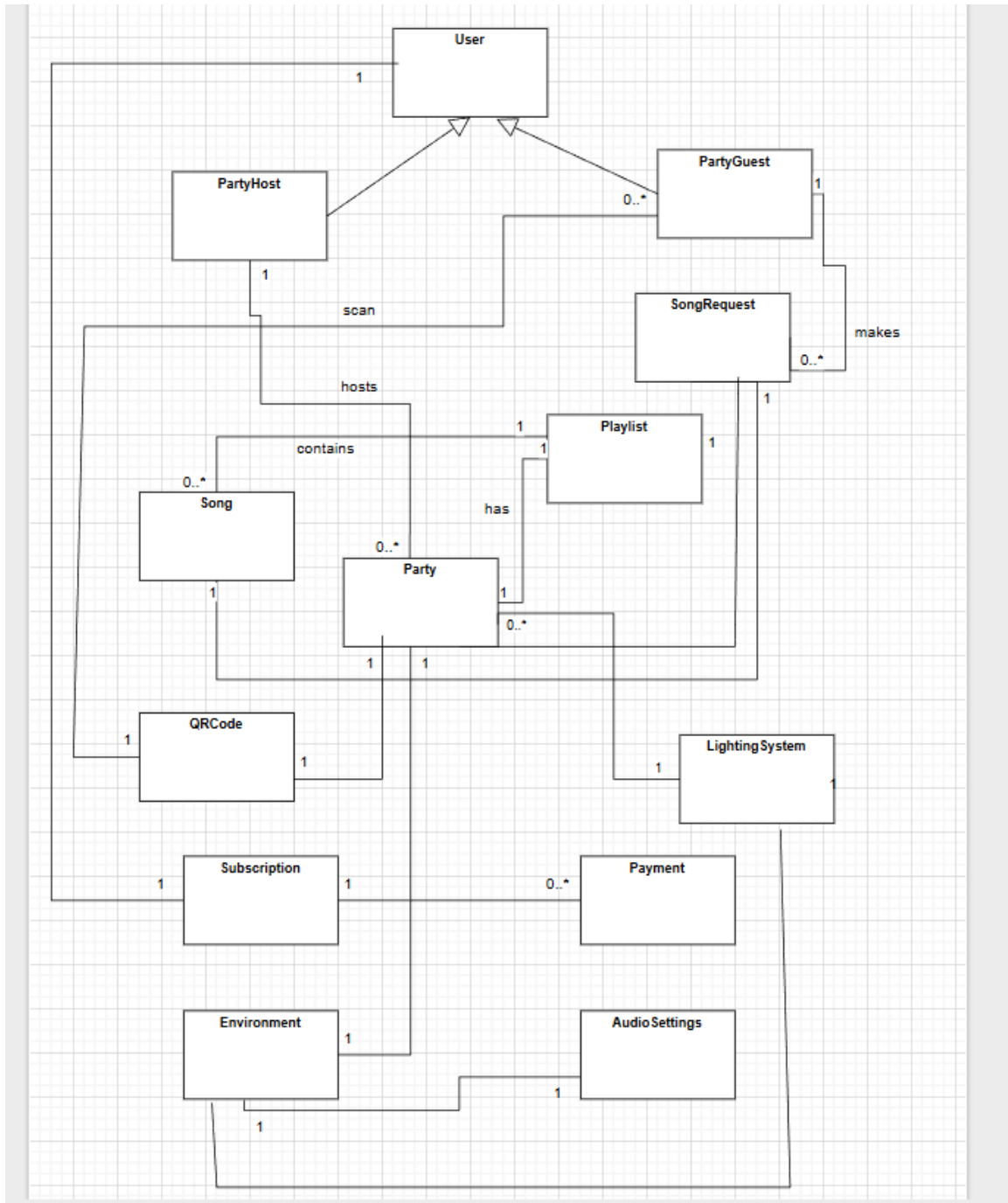
Από	Προς	Cardinalit y	Περιγραφή
Party	QRCode	1 → 1	Κάθε πάρτι έχει ένα QR code
Party	Environment	1 → 1	Κάθε πάρτι έχει ένα περιβάλλον
Party	AudioSettings	1 → 1	Κάθε πάρτι έχει ηχητικές ρυθμίσεις
SongRequest	Song	1 → 1	Κάθε request αφορά ένα τραγούδι
SongRequest	PartyGuest	1 → 1	Κάθε request γίνεται από έναν Guest

Από	Προς	Cardinalit y	Περιγραφή
Subscription	PartyHost	1 → 1	Κάθε Host έχει μία συνδρομή
Subscription	Payment	1 → 0..*	Μία συνδρομή έχει πολλές πληρωμές
LightingSystem	Environment	1 → 1	Ένα σύστημα φωτισμού συνδέεται με ένα περιβάλλον

3. Αντιστοίχιση Use Cases με Οντότητες

Use Case	Εμπλεκόμενες Οντότητες
UC1: Create Party Event	PartyHost, Party, PersistenceController
UC2: Select Party Theme	PartyHost, Party, ThemeService

Use Case	Εμπλεκόμενες Οντότητες
UC3: Set Party Duration	PartyHost, Party, BusinessRulesController
UC4: Generate Playlist	Party, Playlist, Song, ThemeService
UC5: Modify Playlist	PartyHost, Playlist, Song
UC6: Extend Party Duration	PartyHost, Party
UC7: Activate Closing Mode	PartyHost, Party
UC8: Scan QR Code	PartyGuest, Party, QRCode
UC9: Submit Song Request	PartyGuest, SongRequest, Song, Playlist
UC10: Sync Lighting With Music	Playlist, Song, LightingSystem, Environment
UC11: Manage Subscription	PartyHost, Subscription, Payment
UC12: Process Payment	Payment, Subscription



4) ROBUSTNESS DIAGRAMS

USE CASE 1: CREATE PARTY EVENT

1. Στοιχεία Use Case

Στοιχείο	Τιμή
Όνομα Use Case	Create Party Event
Primary Actor	Party Host
Secondary Actors	AuthorizationController, PersistenceController
Στόχος	Ο Party Host δημιουργεί ένα νέο party event στο σύστημα
Τύπος	Βασικός / Δημιουργίας

2. Σύντομη Περιγραφή (Brief Description)

Ο **Party Host** δημιουργεί ένα νέο πάρτι μέσω του συστήματος. Συμπληρώνει μια φόρμα με τα απαραίτητα στοιχεία (όνομα, περιγραφή κ.λπ.), το σύστημα ελέγχει τα δικαιώματά του και την εγκυρότητα των δεδομένων και στη συνέχεια δημιουργεί και αποθηκεύει το νέο αντικείμενο **Party**. Τέλος, το party συνδέεται με τον Host που το δημιούργησε και εμφανίζεται μήνυμα επιτυχίας.

3. Προϋποθέσεις (Preconditions)

1. Ο χρήστης είναι **authenticated** στο σύστημα (έχει συνδεθεί)
2. Ο χρήστης έχει ρόλο **PartyHost** (εξουσιοδότηση)
3. Το σύστημα είναι διαθέσιμο (λειτουργικό)
4. Η διεπαφή `CreatePartyFormUI` είναι προσβάσιμη

4. Βασική Ροή Επιτυχίας (Main Success Scenario)

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
1	Ο PartyHost συμπληρώνει τα στοιχεία του νέου party (όνομα, περιγραφή κ.λπ.) στη φόρμα Create PartyFormUI	Η φόρμα συλλέγει τα δεδομένα εισόδου	CreatePartyFormUI
2	Ο PartyHost πατάει το κουμπί "Submit" / "Create Party"	Τα δεδομένα μεταφέρονται στο InputHandler για περαιτέρω επεξεργασία	InputHandler
3	-	Το InputHandler επικοινωνεί με το AuthorizationController για έλεγχο	AuthorizationController → check role

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		δικαιωμάτων (check role)	
4	-	Αν ο χρήστης δεν είναι εξουσιοδοτημένος (not authorized), εμφανίζεται μήνυμα σφάλματος και η ροή τερματίζεται	not authorized
5	-	Αν ο χρήστης είναι εξουσιοδοτημένος (Authorized), το σύστημα προχωρά στον έλεγχο εγκυρότητας των δεδομένων μέσω ValidationController (valid input)	ValidationController → valid input
6	-	Σε περίπτωση αποτυχίας validation, το ValidationMessageUI εμφανίζει μήνυμα σφάλματος και ζητάει διόρθωση	ValidationMessage UI

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
7	-	Εφόσον τα δεδομένα είναι έγκυρα, το ValidationController καλεί το PartyFactory για να δημιουργήσει νέο αντικείμενο Party (create())	PartyFactory → create()
8	-	Το PartyFactory δημιουργεί το νέο Party object	Party
9	-	Το PartyFactory συνδέει το νέο party με τον PartyHost (link to host)	link to host → PartyHost
10	-	Το PartyFactory καλεί το PersistenceController για αποθήκευση (save)	PersistenceController → save
11	-	Το PersistenceController αποθηκεύει το νέο party στη βάση δεδομένων	-

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
12	-	Το PersistenceController εμφανίζει μήνυμα επιτυχίας μέσω ConfirmationUI (show confirmation)	ConfirmationUI → s how confirmation
13	Ο PartyHost βλέπει μήνυμα ότι το party δημιουργήθηκε επιτυχώς	-	-

5. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 4	Μη εξουσιοδοτημένος χρήστης – Αν ο χρήστης δεν έχει ρόλο PartyHost, το σύστημα εμφανίζει μήνυμα σφάλματος και δίνει προχωρά στη δημιουργία party
A2	Βήμα 6	Αποτυχία validation – Αν τα δεδομένα είναι ελλιπή ή μη έγκυρα (π.χ., κενό όνομα), το ValidationMessageUI εμφανίζει μήνυμα και ο Host πρέπει να διορθώσει τα στοιχεία

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A3	Βήμα 10	Αποτυχία αποθήκευσης – Αν το PersistenceController δεν μπορεί να αποθηκεύσει (π.χ., βάση δεδομένων offline) εμφανίζεται μήνυμα σφάλματος και το party δεν δημιουργείται

6. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case:

1. **Νέο Party object** έχει δημιουργηθεί στο σύστημα
2. Το party έχει **μοναδικό ID** (αναγνωριστικό)
3. Το party είναι **συνδεδεμένο** με τον συγκεκριμένο PartyHost που το δημιούργησε
4. Το party βρίσκεται σε αρχική κατάσταση (π.χ., **"draft"** ή **"pending"**), όχι ακόμα ενεργό
5. Όλα τα δεδομένα έχουν **αποθηκευτεί** στο persistence layer (βάση δεδομένων)
6. Ο Host έχει λάβει **οπτική επιβεβαίωση** επιτυχίας

7. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας	Περιγραφή
BR1	Μόνο χρήστες με ρόλο PartyHost μπορούν να δημιουργήσουν πάρτι
BR2	Το όνομα του party είναι υποχρεωτικό και δεν μπορεί να είναι κενό

Κανόνας	Περιγραφή
BR3	Η περιγραφή του party είναι προαιρετική
BR4	Κάθε party αποθηκεύεται αυτόματα με κατάσταση "draft" μέχρι να ολοκληρωθεί η ρύθμισή του

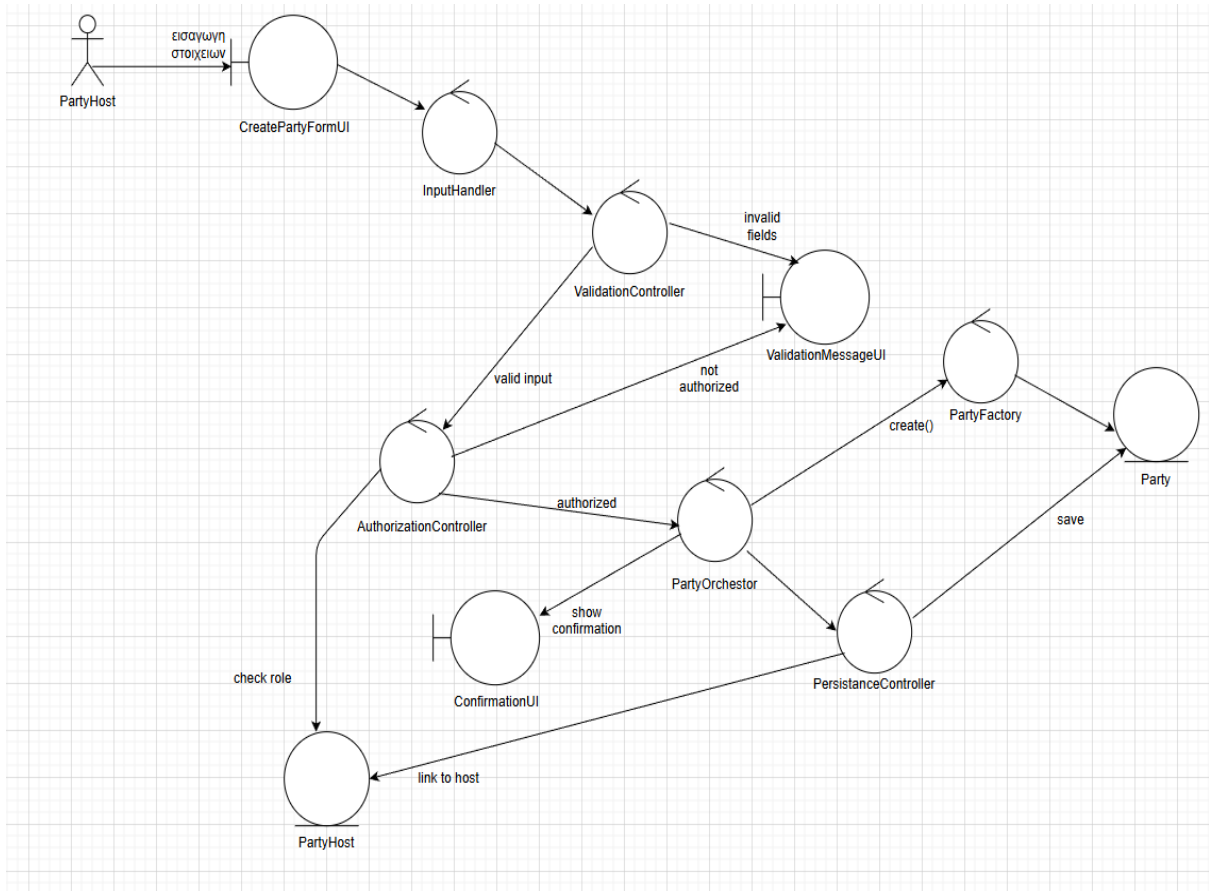
8. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία	Απαίτηση
Απόδοση	Η δημιουργία party πρέπει να ολοκληρώνεται σε λιγότερο από 2 δευτερόλεπτα
Ασφάλεια	Ο έλεγχος ρόλου γίνεται πριν από οποιαδήποτε άλλη ενέργεια
Διαθεσιμότητα	Το σύστημα πρέπει να είναι διαθέσιμο για δημιουργία party 24/7
Σφάλματα	Σε περίπτωση αποτυχίας, το σύστημα εμφανίζει κατανοητό μήνυμα σφάλματος

9. Διάγραμμα Αντιστοίχισης (Component Mapping)

Component στο Robustness Diagram	Ρόλος / Ευθύνη
CreatePartyFormUI	Συλλογή δεδομένων εισόδου από τον Host

Component στο Robustness Diagram	Ρόλος / Ευθύνη
InputHandler	Αρχική επεξεργασία και δρομολόγηση αιτήματος
AuthorizationController	Έλεγχος ρόλου και δικαιωμάτων χρήστη (check)
ValidationController	Έλεγχος εγκυρότητας δεδομένων (valid input)
ValidationMessageUI	Εμφάνιση μηνυμάτων σφάλματος validation
PartyFactory	Δημιουργία νέου αντικειμένου Party (create())
Party	Οντότητα που αναπαριστά το πάρτι
PersistenceController	Αποθήκευση δεδομένων (save)
ConfirmationUI	Επιβεβαίωση επιτυχούς ολοκλήρωσης
PartyHost (ηθοποιός)	Σύνδεση του δημιουργού με το νέο party (link to host)



USE CASE 2: SELECT PARTY THEME

1. Στοιχεία Use Case

Στοιχείο

Τιμή

Όνομα Use Case

Select Party Theme

Primary Actor

Party Host

Secondary Actors

ThemeService, PersistenceController

Στοιχείο	Τιμή
Στόχος	Ο Party Host επιλέγει το μουσικό theme του πάρτι (π.χ., Rock, Pop, Jazz, Electronic)
Τύπος	Βασικός / Διαμόρφωσης

2. Σύντομη Περιγραφή (Brief Description)

Ο Party Host επιλέγει ένα μουσικό θέμα (theme) για το πάρτι που έχει ήδη δημιουργήσει. Το σύστημα επικυρώνει την επιλογή, εμφανίζει προεπισκόπηση (sample playlist) και στη συνέχεια αποθηκεύει το theme στο αντικείμενο Party. Το theme αυτό θα χρησιμοποιηθεί αργότερα για την αυτόματη δημιουργία playlist.

3. Προϋποθέσεις (Preconditions)

1. Υπάρχει party event (έχει δημιουργηθεί από το UC1)
 2. Το party βρίσκεται σε κατάσταση που επιτρέπει αλλαγή theme (π.χ., δεν είναι active ή closing)
 3. Ο χρήστης είναι PartyHost και έχει πρόσβαση στο συγκεκριμένο party
 4. Το ThemeService είναι διαθέσιμο (περιέχει διαθέσιμα themes)
 5. Η διεπαφή ThemeSelectionUI είναι προσβάσιμη
-

4. Βασική Ροή Επιτυχίας (Main Success Scenario)

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
1	Ο PartyHost ανοίγει τη φόρμα επιλογής theme (ThemeSelectionUI)	Το σύστημα εμφανίζει τα διαθέσιμα themes	ThemeSelectionUI
2	Ο PartyHost επιλέγει ένα theme (π.χ., "Rock", "Pop", "Jazz")	Το UI συλλέγει την επιλογή και τη μεταφέρει στο InputHandler	InputHandler
3	-	Το InputHandler επικοινωνεί με το ThemeValidationController για έλεγχο εγκυρότητας του theme (valid)	ThemeValidationController → valid
4	-	Αν το theme δεν υπάρχει (invalid theme): το ErrorUI εμφανίζει	ErrorUI → invalid theme

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		μήνυμα σφάλματος και η ροή επιστρέφει στο Βήμα 1	
5	-	Αν το theme είναι έγκυρο (valid): το σύστημα προχωρά στην προεπισκόπηση	-
6	-	To ThemeValidationController καλεί το ThemePreviewUI για να εμφανίσει προεπισκόπηση του theme (showpreview)	ThemePreviewUI → showpreview
7	-	To ThemePreviewUI εμφανίζει στον PartyHost μια δειγματική playlist ή ενδεικτικό περιεχόμενο του theme	-
8	-	To ThemeValidationController επικοινωνεί με το PartyController για να ορίσει το theme (settheme)	PartyController → set theme

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
9	-	Το PartyController ενημερώνει το αντικείμενο Party με το επιλεγμένο theme	Party
10	-	Το PartyController καλεί το PersistenceController για αποθήκευση της αλλαγής (save)	PersistenceController → save
11	-	Το PersistenceController αποθηκεύει το theme στη βάση δεδομένων	-
12	-	Το PersistenceController εμφανίζει μήνυμα επιτυχίας μέσω ConfirmationUI (η επιβεβαίωση φαίνεται στο diagram ως show από PersistenceController προς ConfirmationUI)	ConfirmationUI → show
13	Ο PartyHost βλέπει μήνυμα ότι	-	

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
	το theme επιλέχθηκε επιτυχώς ("Theme selected successfull y")		

5. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 4	Μη έγκυρο theme – Αν ο Host επιλέξει ένα theme που δεν υπάρχει στο ThemeService (π.χ., πληκτρολογήσ αναγνωρίσιμη τιμή), το ErrorUI εμφανίζει "Invalid theme" και η ροή επιστρέφει για νέα επιλογή
A2	Βήμα 5-6	ThemeService μη διαθέσιμο – Αν το ThemeService είναι offline ή δεν ανταποκρίνεται,

Κωδικός	Σημείο Απόκλισης	Περιγραφή
		το σύστημα εμφανίζει μήνυμα σφάλματος και δεν προχωράει στην προεπισκόπηση
A3	Βήμα 10	Αποτυχία αποθήκευσης – Αν το PersistenceController δεν μπορεί να αποθηκεύσει το theme (π.χ., βάση δεδομένων offline), εμφανίζεται μήνυμα σφάλματος και το theme δεν ενημερώνεται

6. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case:

1. Το theme έχει οριστεί στο αντικείμενο Party
2. Η αλλαγή έχει αποθηκευτεί στο persistence layer
3. Το party είναι πλέον έτοιμο για τη δημιουργία playlist (μέσω UC4)
4. Ο Host έχει λάβει οπτική επιβεβαίωση της επιλογής του
5. Το ThemePreviewUI έχει εμφανίσει προεπισκόπηση (αν το σύστημα υποστηρίζει αυτή τη λειτουργία)

7. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας	Περιγραφή
BR1	Το theme μπορεί να αλλάξει μόνο πριν την ενεργοποίηση του πάρτι ή όσο το πάρτι δεν βρίσκεται σε closing mode
BR2	Μόνο ένα theme μπορεί να είναι ενεργό ανά πάρτι
BR3	Τα διαθέσιμα themes παρέχονται από το ThemeService και είναι προκαθορισμένα από το σύστημα
BR4	Η προεπισκόπηση του theme είναι προαιρετική αλλά συνιστάται για καλύτερη εμπειρία χρήστη
BR5	Η αλλαγή theme δεν επηρεάζει ήδη υπάρχοντα song requests, αλλά θα επηρεάσει τη μελλοντική δημιουργία playlist

8. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία	Απαίτηση
Απόδοση	Η εμφάνιση της προεπισκόπησης πρέπει να γίνεται σε λιγότερο από 1 δευτερόλεπτο
Χρηστικότητα	Η λίστα διαθέσιμων themes πρέπει να είναι σαφής και περιγραφική
Διαθεσιμότητα	Το ThemeService πρέπει να είναι διαθέσιμο 99.9% του χρόνου

Κατηγορία	Απαίτηση
Ανθεκτικότητα	Σε περίπτωση αποτυχίας του ThemeService, το σύστημα πρέπει να εμφανίζει φιλικό μήνυμα σφάλματος

9. Διάγραμμα Αντιστοίχισης (Component Mapping)

Component στο Robustness Diagram	Ρόλος / Ευθύνη
PartyHost (ηθοποιός)	Επιλέγει το theme μέσω του UI
ThemeSelectionUI	Παρουσιάζει τα διαθέσιμα themes και συλλέγει την επιλογή
InputHandler	Αρχική επεξεργασία και δρομολόγηση της επιλογής
ThemeValidationController	Ελέγχει αν το επιλεγμένο theme υπάρχει και είναι έγκυρο (valid / invalid theme)
ErrorUI	Εμφανίζει μήνυμα σφάλματος όταν το theme είναι άκυρο
ThemeService	Παρέχει τη λίστα των διαθέσιμων themes και τα δεδομένα τους
ThemePreviewUI	Εμφανίζει προεπισκόπηση του theme

Component στο Robustness
Diagram

Ρόλος / Ευθύνη

(π.χ., sample playlist)

PartyController

Διαχειρίζεται το αντικείμενο Party και ενημερώνει
theme (settheme)

Party

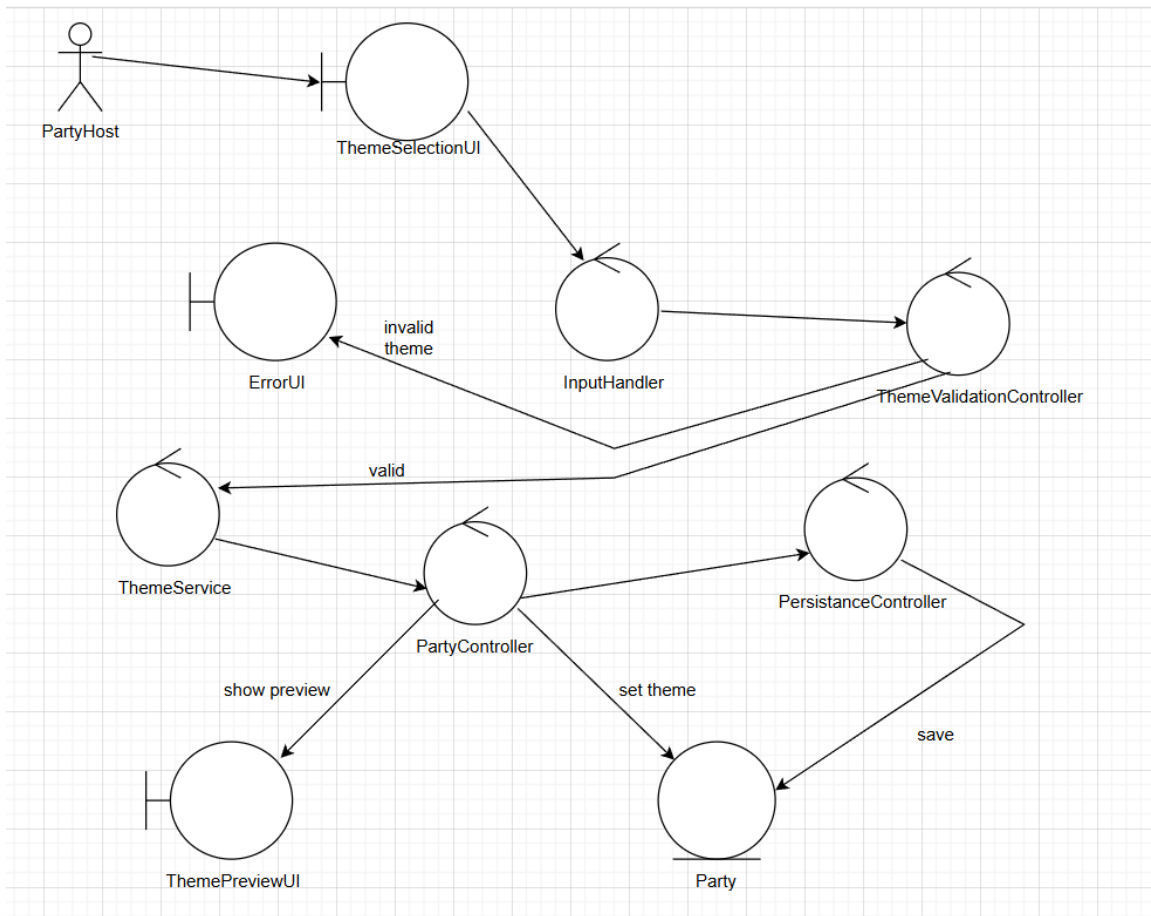
Οντότητα που αποθηκεύει το επιλεγμένο theme

PersistenceController

Αποθηκεύει την αλλαγή (save)

ConfirmationUI

Επιβεβαιώνει την επιτυχή αποθήκευση



USE CASE 3: SET PARTY DURATION

1. Στοιχεία Use Case

Στοιχείο

Τιμή

Όνομα Use Case

Set Party Duration

Primary Actor

Party Host

Στοιχείο	Τιμή
Secondary Actors	BusinessRulesController, PersistenceController
Στόχος	Ο Party Host ορίζει τη διάρκεια του πάρτι (π.χ., σε λεπτά ή ώρες)
Τύπος	Βασικός / Διαμόρφωσης

2. Σύντομη Περιγραφή (Brief Description)

Ο **Party Host** ορίζει τη συνολική διάρκεια του πάρτι (π.χ., 120 λεπτά, 2 ώρες, 4 ώρες). Το σύστημα επικυρώνει την τιμή (μη αρνητική, αριθμητική) και στη συνέχεια εφαρμόζει επιχειρηματικούς κανόνες (π.χ., ελάχιστη και μέγιστη επιτρεπόμενη διάρκεια). Εφόσον η διάρκεια είναι αποδεκτή, ενημερώνεται το αντικείμενο **Party** και αποθηκεύεται η αλλαγή.

3. Προϋποθέσεις (Preconditions)

1. **Υπάρχει party event** (έχει δημιουργηθεί από το UC1)
2. Το party **δεν είναι ακόμα active** (δεν έχει ξεκινήσει) – η διάρκεια μπορεί να οριστεί μόνο πριν την έναρξη
3. Ο χρήστης είναι **PartyHost** και έχει πρόσβαση στο συγκεκριμένο party
4. Το **BusinessRulesController** είναι διαθέσιμο (περιέχει τους κανόνες για ελάχιστη/μέγιστη διάρκεια)
5. Η διεπαφή **DurationInputUI** είναι προσβάσιμη

4. Βασική Ροή Επιτυχίας (Main Success Scenario)

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
1	Ο PartyHost ανοίγει τη φόρμα εισαγωγής διάρκειας (DurationInputUI)	Το σύστημα εμφανίζει την κατάλληλη διεπαφή για εισαγωγή διάρκειας	DurationInputUI
2	Ο PartyHost εισάγει τη διάρκεια (π.χ., 120 λεπτά, 2 ώρες) και υποβάλλει (submit data)	Το UI συλλέγει τα δεδομένα και τα μεταφέρει στο InputHandler	InputHandler → submit data
3	-	Το InputHandler επικοινωνεί με το DurationValidationController για βασικό	DurationValidationController → check duration

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		έλεγχο της διάρκειας (check duration)	
		Έλεγχοι βασικού validation (στο DurationValidationController): μη αρνητική τιμή, μη μηδενική (αν δεν επιτρέπεται), αριθμητική τιμή	
4	-		-
		Αν αποτύχει (invalid duration): το ErrorUI εμφανίζει μήνυμα σφάλματος και η ροή επιστρέφει στο Βήμα 1	
5	-		ErrorUI → invalid duration
		Αν περάσει τον βασικό έλεγχο (valid): το DurationValidationController προχωρά στο BusinessRulesController	
6	-		BusinessRulesController → valid
		Το BusinessRulesController εφαρμόζει τους επιχειρηματικούς	
7	-		-

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		κανόνες: ελέγχει αν η διάρκεια βρίσκεται εντός των ορίων (π.χ., ελάχιστο 30 λεπτά, μέγιστο 480 λεπτά / 8 ώρες)	
8	-	Αν η διάρκεια είναι εκτός ορίων: το ErrorUI εμφανίζει σχετικό μήνυμα και η ροή επιστρέφει στο Βήμα 1 Αν η διάρκεια είναι εντός ορίων: το BusinessRulesController επικοινωνεί με το PartyController για να ορίσει τη διάρκεια (setDuration)	-
9	-	Το PartyController ενημερώνει το αντικείμενο Party με τη νέα διάρκεια	PartyController → setDuration
10	-		Party

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
11	-	Το PartyController καλεί το PersistenceController για αποθήκευση (store → save)	PersistenceController → store → save
12	-	Το PersistenceController αποθηκεύει τη διάρκεια στη βάση δεδομένων	-
13	-	Το PersistenceController εμφανίζει μήνυμα επιτυχίας μέσω ConfirmationUI (show)	ConfirmationUI → show
14	Ο PartyHost βλέπει μήνυμα ότι η διάρκεια ορίστηκε επιτυχώς ("Duration set	-	

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
	successfully")		

5. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 5	Μη έγκυρη μορφή διάρκειας – Αν ο Host εισάγει μη αριθμητική τιμή (π.χ., "δύο ώρες"), αρνητική τιμή (π.χ., -30) ή μηδέν (απαγορεύεται), το ErrorUI επιτρέπεται, το ErrorUI εμφανίζει μήνυμα και ζητάει νέα εισαγωγή Παραβίαση επιχειρηματικών κανόνων – Αν η διάρκεια είναι μικρότερη από το ελάχιστο όριο (π.χ., λιγότερο από 30 λεπτά) ή μεγαλύτερη από το μέγιστο όριο (π.χ., πάνω από 8 ώρες), το ErrorUI εμφανίζει μήνυμα και η ροή επιστρέφει Αποτυχία αποθήκευσης – Αν το PersistenceController δεν μπορεί να αποθηκεύσει τη διάρκεια (π.χ., βάση δεδομένων offline), εμφανίζεται μήνυμα σφάλματος και η διάρκεια δεν ενημερώνεται
A2	Βήμα 7-8	
A3	Βήμα 11	

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A4	Βήμα 1	Απουσία party – Αν δεν υπάρχει party event (προϋπόθεση αποτυχάνει), το σύστημα εμφανίζει μήνυμα σφάλματος και το use case τερματίζεται

6. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case:

1. Η **διάρκεια έχει οριστεί** στο αντικείμενο **Party**
2. Η διάρκεια είναι **αποθηκευμένη** στο persistence layer
3. Το party είναι πλέον **έτοιμο** για τη δημιουργία playlist (UC4), καθώς η διάρκεια αποτελεί βασική παράμετρο για τον υπολογισμό του αριθμού τραγουδιών
4. Ο Host έχει λάβει **οπτική επιβεβαίωση** ότι η διάρκεια καταχωρήθηκε επιτυχώς
5. Η διάρκεια μπορεί αργότερα να **επεκταθεί** (μέσω UC6: Extend Party Duration)

7. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας	Περιγραφή
BR1	Η διάρκεια πρέπει να είναι θετικός αριθμός (μεγαλύτερος του μηδενός)
BR2	Υπάρχει ελάχιστη επιτρεπόμενη διάρκεια (π.χ., 30 λεπτά) –

Κανόνας	Περιγραφή
	ορίζεται από το BusinessRulesController
BR3	Υπάρχει μέγιστη επιτρεπόμενη διάρκεια (π.χ., 480 λεπτά / 8 ώρες) – ορίζεται από το BusinessRulesController
BR4	Η διάρκεια μπορεί να οριστεί μόνο πριν την ενεργοποίηση του πάρτι (δεν μπορεί να αλλάξει ενώ το πάρτι είναι σε εξέλιξη, παρά μόνο να επεκταθεί μέσω UC6)
BR5	Η διάρκεια εκφράζεται σε λεπτά (κοινή μονάδα μέτρησης για όλους τους υπολογισμούς)

8. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία	Απαίτηση
Απόδοση	Η επικύρωση και αποθήκευση της διάρκειας πρέπει να γίνεται σε λιγότερο από 1 δευτερόλεπτο
Χρηστικότητα	Το UI πρέπει να παρέχει σαφή οδηγία για τη μονάδα μέτρησης (π.χ., "Εισάγετε διάρκεια σε λεπτά")
Ανθεκτικότητα	Το σύστημα πρέπει να αποθηκεύει την τελευταία έγκυρη διάρκεια που εισήχθη

Κατηγορία	Απαίτηση
Ευελιξία	Οι επιχειρηματικοί κανόνες (min/max) πρέπει να είναι ρυθμιζόμενοι το σύστημα χωρίς αλλαγή κώδικα

9. Διάγραμμα Αντιστοίχισης (Component Mapping)

Component στο Robustness Diagram	Ρόλος / Ευθύνη
PartyHost (ηθοποιός)	Εισάγει τη διάρκεια και υποβάλλει το αίτημα
DurationInputUI	Παρουσιάζει τη φόρμα εισαγωγής διάρκειας και συλλέγει τα δεδομένα (submit data)
InputHandler	Αρχική επεξεργασία και δρομολόγηση της εισόδου
DurationValidationController	Εκτελεί βασικό έλεγχο εγκυρότητας: μη αρνητική τ αριθμητική, μη μηδενική (check duration)
ErrorUI	Εμφανίζει μήνυμα σφάλματος (invalid duration)

Component στο Robustness
Diagram

Ρόλος / Ευθύνη

BusinessRulesController

Εφαρμόζει επιχειρηματικούς κανόνες (ελάχιστη/μ
διάρκεια) και επιστρέφει valid

PartyController

Διαχειρίζεται το αντικείμενο Party και ενημερώνει
διάρκεια (setDuration)

Party

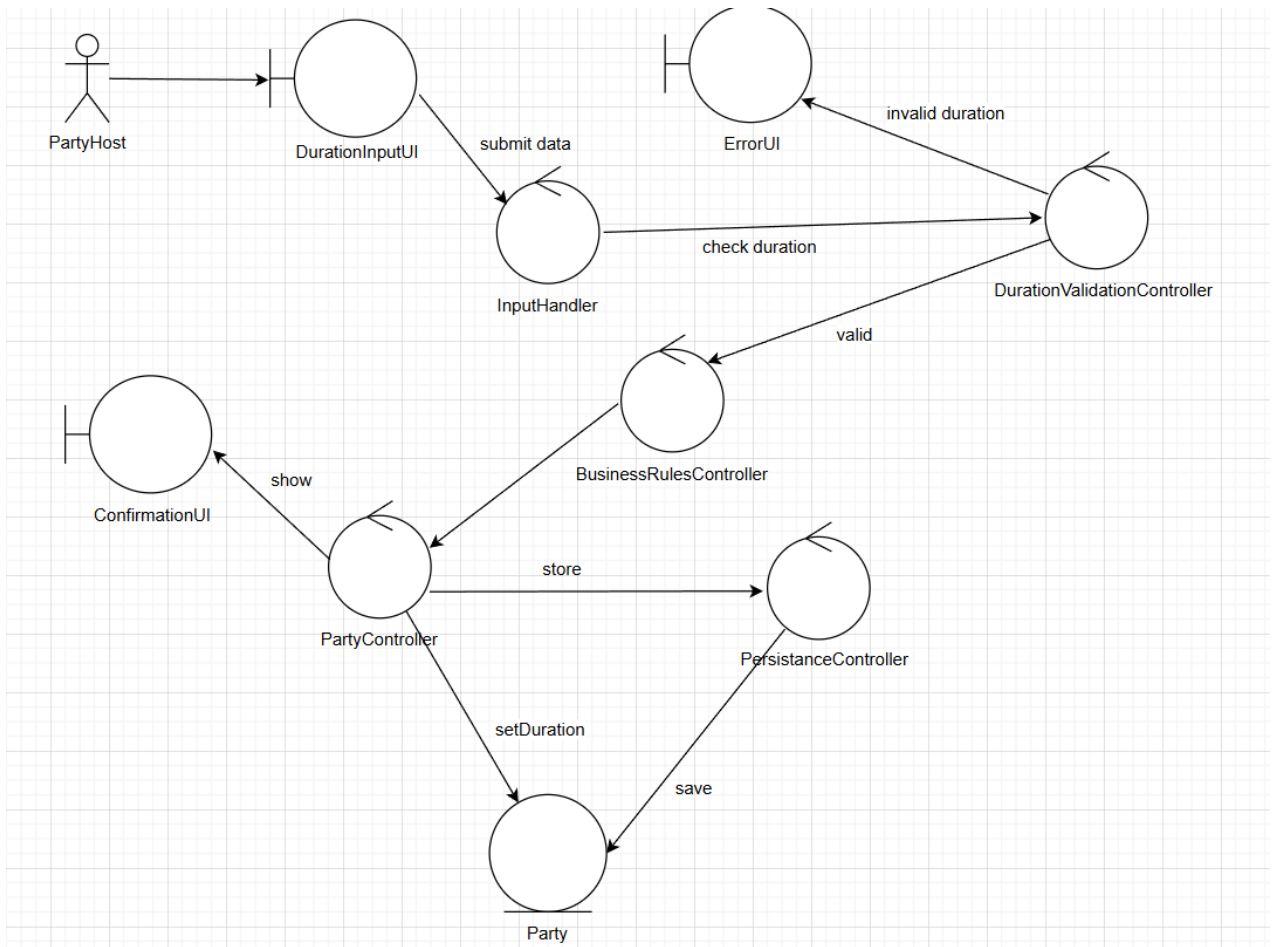
Οντότητα που αποθηκεύει τη διάρκεια

PersistenceController

Αποθηκεύει την αλλαγή (store / save)

ConfirmationUI

Επιβεβαιώνει την επιτυχή αποθήκευση
(show)



USE CASE 4: GENERATE PLAYLIST

1. Στοιχεία Use Case

Στοιχείο	Τιμή
Όνομα Use Case	Generate Playlist
Primary Actor	Party Host
Secondary Actors	PlaylistGenerator, PersistenceController, Song
Στόχος	Ο Party Host δημιουργεί αυτόματα μια playlist για το πάρτι, με βάση το επιλεγμένο theme και τη διάρκεια του event
Τύπος	Βασικός / Δημιουργίας

2. Σύντομη Περιγραφή (Brief Description)

Ο **Party Host** επιλέγει τη λειτουργία αυτόματης δημιουργίας playlist. Το σύστημα ελέγχει ότι έχουν προηγηθεί οι απαραίτητες ενέργειες (επιλογή theme και ορισμός διάρκειας). Στη συνέχεια, χρησιμοποιεί το theme και τη διάρκεια για να επιλέξει κατάλληλα τραγούδια, δημιουργεί μια νέα playlist, την αποθηκεύει και την παρουσιάζει στον Host για περαιτέρω επεξεργασία.

3. Προϋποθέσεις (Preconditions)

- Έχει δημιουργηθεί party event (από UC1)
- Έχει επιλεγεί party theme (από UC2)

3. Έχει οριστεί διάρκεια **party** (από UC3)
4. Ο χρήστης είναι **PartyHost** και έχει πρόσβαση στο συγκεκριμένο party
5. Το **PlaylistGenerator** είναι διαθέσιμο
6. Υπάρχουν **διαθέσιμα τραγούδια** για το επιλεγμένο theme

4. Βασική Ροή Επιτυχίας (Main Success Scenario)

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
1	Ο PartyHost επιλέγει τη λειτουργία " Generate Playlist " μέσω PlaylistUI	Το UI στέλνει σήμα generate στο σύστημα	PlaylistUI → generate
2	-	Το PlaylistUI επικοινωνεί με το CheckPreconditions για να επαληθεύσει ότι πληρούνται οι προϋποθέσεις	CheckPreconditions
3	-	Το CheckPreconditions ελέγχει αν έχει επιλεγεί theme (check theme)	check theme

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
4	-	Το CheckPreconditions ελέγχει αν έχει οριστεί διάρκεια (check duration)	check duration
5	-	Αν δεν έχει επιλεγεί theme: το σύστημα εμφανίζει μήνυμα missing theme μέσω PlaylistUI και η ροή τερματίζεται	missing theme
6	-	Αν δεν έχει οριστεί διάρκεια: το σύστημα εμφανίζει μήνυμα missing duration μέσω PlaylistUI και η ροή τερματίζεται	missing duration
7	-	Αν πληρούνται και οι δύο προϋποθέσεις: το CheckPreconditions επικοινωνεί με	OK → PlaylistController

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		το PlaylistController (OK)	
8	-	Το PlaylistController καλεί το PlaylistGenerator για δημιουργία playlist (create)	PlaylistGenerator → create
9	-	Το PlaylistGenerator λαμβάνει δεδομένα (get data) από το Party (theme και duration)	get data → Party
10	-	Το PlaylistGenerator αναζητά και επιλέγει κατάλληλα τραγούδια (select songs) από το σύστημα	select songs → Song
11	-	Το PlaylistGenerator δημιουργεί το αντικείμενο Playlist	Playlist
12	-	Το PlaylistGenerator αποθηκεύει την playlist	PersistenceController → store

Βήμα	Ενέργεια	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
	Ηθοποιού	μέσω PersistenceController (store)	
13	-	Το PersistenceController αποθηκεύει τη playlist (save)	save
14	-	Το PlaylistController παρουσιάζει την playlist στον Host μέσω PlaylistUI (show playlist)	show playlist
15	Ο PartyHost βλέπει τη νέα playlist που δημιουργήθηκε	-	

5. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 5	Δεν έχει επιλεγεί theme – Το σύστημα εμφανίζει μήνυμα "missing theme" μέσω PlaylistUI και ζητά από τον Host να επιλέξει theme πρώτα (redirect στο UC2)
A2	Βήμα 6	Δεν έχει οριστεί διάρκεια – Το σύστημα εμφανίζει μήνυμα "missing duration" μέσω PlaylistUI και ζητά από τον Host να ορίσει διάρκεια πρώτα (redirect στο UC3)
A3	Βήμα 10-11	Αποτυχία δημιουργίας playlist – Αν το PlaylistGenerator μπορεί να δημιουργήσει playlist (π.χ., δεν υπάρχουν τραγούδια για το theme, η διάρκεια είναι πολύ μικρή ή πολύ μεγάλη), το σύστημα εμφανίζει μήνυμα Generation failed μέσω PlaylistUI

6. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case:

1. **Έχει δημιουργηθεί playlist object** στο σύστημα
2. Η playlist **περιέχει τραγούδια** που ταιριάζουν στο επιλεγμένο theme

3. Η **συνολική διάρκεια** της playlist προσεγγίζει τη διάρκεια του πάρτι
 4. Η playlist είναι **αποθηκευμένη** στο persistence layer
 5. Η playlist είναι **διαθέσιμη** για αναπαραγωγή και περαιτέρω επεξεργασία (μέσω UC5)
 6. Ο Host έχει **οπτική επιβεβαίωση** της playlist που δημιουργήθηκε
-

7. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας Περιγραφή

BR1 Η playlist **δεν μπορεί να δημιουργηθεί** αν δεν έχει επιλεγεί theme ή δεν έχει οριστεί διάρκεια

BR2 Τα τραγούδια που επιλέγονται πρέπει να **αντιστοιχούν στο theme** (π.χ., Rock theme → rock τραγούδια)

BR3 Η συνολική διάρκεια της playlist πρέπει να είναι **όσο το δυνατόν πλησιέστερη** στη διάρκεια του πάρτι (χωρίς υπέρβαση ή με μικρή ανοχή)

BR4 Δεν επιτρέπονται **διπλότυπα τραγούδια** στην ίδια playlist

BR5 Η σειρά των τραγουδιών μπορεί να είναι **τυχαία** ή να βασίζεται σε αλγόριθμο (π.χ., BPM, δημοτικότητα)

8. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία	Απαίτηση
Απόδοση	Η δημιουργία playlist για 100 τραγούδια πρέπει να γίνεται σε λιγότερο από 3 δευτερόλεπτα
Ακρίβεια	Η συνολική διάρκεια της playlist πρέπει να αποκλίνει το πολύ ±5% από τη διάρκεια του πάρτι
Διαθεσιμότητα	Το PlaylistGenerator πρέπει να είναι διαθέσιμο 99.9% του χρόνου
Επεκτασιμότητα	Το σύστημα πρέπει να υποστηρίζει νέα themes χωρίς αλλαγή του αλγορίθμου δημιουργίας playlist

9. Διάγραμμα Αντιστοίχισης (Component Mapping)

Component στο Robustness Diagram	Ρόλος / Ευθύνη
PartyHost (ηθοποιός)	Επιλέγει τη λειτουργία "Generate Playlist"
PlaylistUI	Παρουσιάζει τη διεπαφή, στέλνει σήμα generate, εμφανίζει μηνύματα (missing theme, missing duration, Generation failed) και δείχνει την τελική playlist (show playlist)
CheckPreconditions	Ελέγχει αν έχουν επιλεγεί theme και

Component στο Robustness Diagram

Ρόλος / Ευθύνη

διάρκεια (check theme, check duration)

PlaylistController

Διαχειρίζεται τη ροή δημιουργίας playlist (OK, create)

Δημιουργεί τη playlist (create), λαμβάνει

PlaylistGenerator

δεδομένα (get data), επιλέγει τραγούδια
(select songs)

Party

Παρέχει το theme και τη διάρκεια (get data)

Song

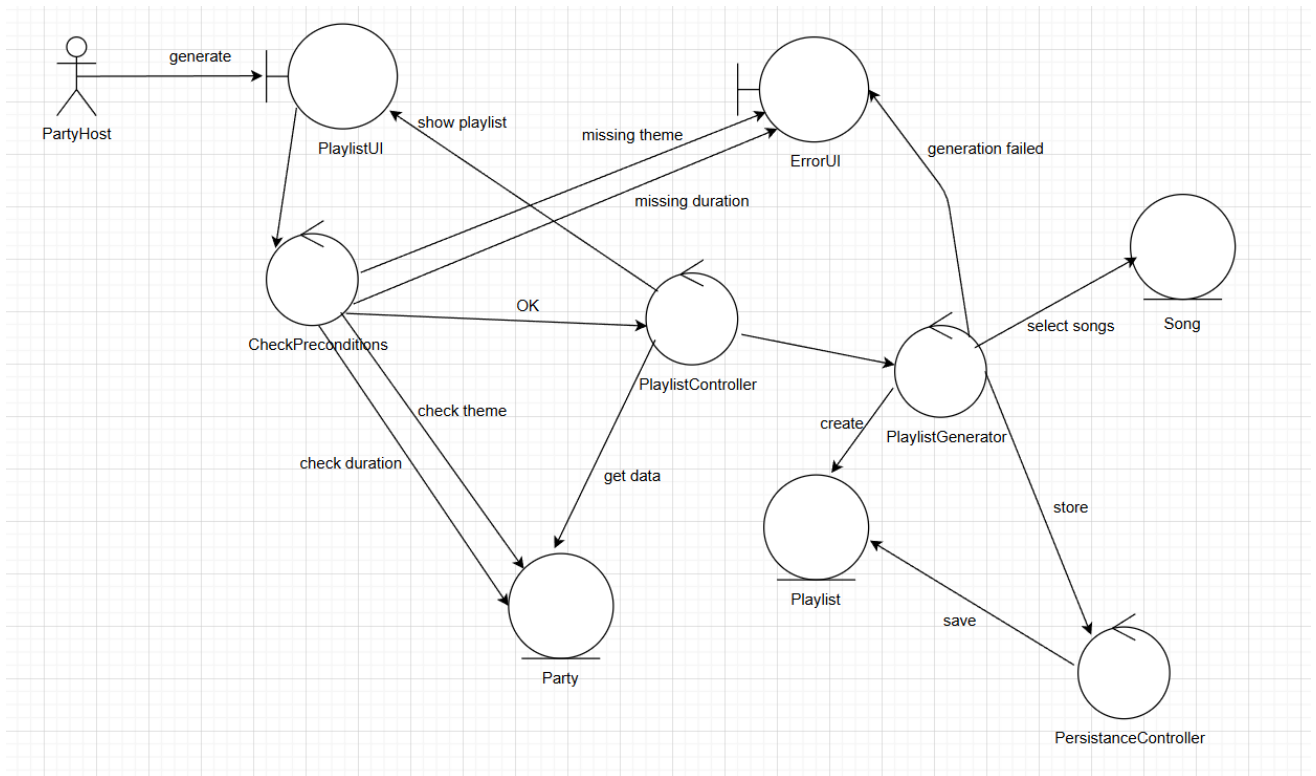
Παρέχει τα διαθέσιμα τραγούδια για
επιλογή (select songs)

Playlist

Οντότητα που δημιουργείται

PersistenceController

Αποθηκεύει την playlist (store, save)



USE CASE 5: MODIFY PLAYLIST

1. Στοιχεία Use Case

Στοιχείο

Τιμή

Όνομα Use Case

Modify Playlist

Primary Actor

Party Host

Στοιχείο	Τιμή
Secondary Actors	SongService, ValidationController, PersistenceController
Στόχος	Ο Party Host τροποποιεί την υπάρχουσα playlist (προσθήκη ή αφαίρεση τραγουδιών)
Τύπος	Βασικός / Διαχείρισης

2. Σύντομη Περιγραφή (Brief Description)

Ο **Party Host** επεξεργάζεται την υπάρχουσα playlist του πάρτι. Μπορεί είτε να **προσθέσει** ένα νέο τραγούδι (αναζητώντας το μέσω SongService) είτε να **αφαιρέσει** ένα υπάρχον. Το σύστημα επικυρώνει το αίτημα, ενημερώνει την playlist, αποθηκεύει τις αλλαγές και εμφανίζει επιβεβαίωση στον Host.

3. Προϋποθέσεις (Preconditions)

1. **Υπάρχει playlist** (έχει δημιουργηθεί από το UC4)
 2. Το party **δεν βρίσκεται σε closing mode** (ή ανάλογα επιχειρηματικά rules)
 3. Ο χρήστης είναι **PartyHost** και έχει πρόσβαση στο συγκεκριμένο party
 4. Το **SongService** είναι διαθέσιμο (για αναζήτηση τραγουδιών)
 5. Η διεπαφή **PlaylistEditorUI** είναι προσβάσιμη
-

4. Βασική Ροή Επιτυχίας (Main Success Scenario)

Β
ή
μ
α

Ενέργεια
Ηθοποιού

Ενέργεια
Συστήματος

Αντιστοίχιση Diagram

1 Ο PartyHost
ανοίγει τη
διεπαφή
επεξεργασίας
playlist
(**PlaylistEditorUI**)

Το σύστημα
εμφανίζει την
τρέχουσα playlist

PlaylistEditorUI

2 Ο PartyHost
επιλέγει **ενέργεια** (action):
προσθήκη ή
αφαίρεση
τραγουδιού

Η επιλογή
μεταφέρεται
στο **EditRequestHandler**

EditRequestHandler → action

3 **Για
προσθήκη:**
Ο Host
αναζητά
τραγούδι

Το **EditRequestHandler** καλεί
το **SongService** για
αναζήτηση (fetch
song)

fetch
song → SongService

4 -

Το **SongService** ανα
ζητά το τραγούδι και

Song

B
ή
μ
α

Ενέργεια
Ηθοποιού

Ενέργεια
Συστήματος

Αντιστοίχιση Diagram

επιστρέφει
το **Song** (αν βρεθεί)

**Αν το τραγούδι δεν
βρεθεί:**

5

-

το **ErrorUI** εμφανίζει
ι μήνυμα invalid και
η ροή επιστρέφει

invalid → ErrorUI

6

Ο Host
υποβάλλει
το αίτημα
(submit
request)

Το UI στέλνει το
αίτημα
στο **EditRequestHa
ndler**

submit request

7

-

Το **EditRequestHan
dler** επικοινωνεί με
το **ValidationContr
oller** για έλεγχο
εγκυρότητας
(request)

ValidationController → r
equest

8

-

Το **ValidationContr
oller** ελέγχει αν η
ενέργεια
επιτρέπεται (π.χ.,

-

Β
ή
μ
α

Ενέργεια
Ηθοποιού

Ενέργεια
Συστήματος

Αντιστοίχιση Diagram

όχι σε closing
mode, όχι duplicate
προσθήκη)

9

-

**Αν η ενέργεια δεν
επιτρέπεται:**
το **ErrorUI** εμφανίζει
ι μήνυμα invalid

invalid → ErrorUI

10

-

**Αν η ενέργεια είναι
έγκυρη (valid):** το
ValidationController
προχωρά
στο **PlaylistControl
ler**

valid → PlaylistControlle
r

11

-

Το **PlaylistControll
er** εκτελεί την
ενέργεια
(add/remove song)
στην **Playlist**

PlaylistController → add
/remove → song → Playlis
t

12

-

Το **PlaylistControll
er** καλεί
το **PersistenceCon**

PersistanceController →
store

B
ή
μ
α

Ενέργεια
Ηθοποιού

Ενέργεια
Συστήματος

Αντιστοίχιση Diagram

troller για
αποθήκευση (store)

13

-

Το **PersistenceCont
roller** αποθηκεύει
τις αλλαγές (save)

save

14

-

Το **PersistenceCont
roller** εμφανίζει
μήνυμα επιτυχίας
μέσω **Confirmation
UI** (showconfirmati
on)

ConfirmationUI → show
confirmation

15

Ο PartyHost
βλέπει
μήνυμα ότι η
playlist
ενημερώθη
κε επιτυχώς
("Playlist
updated
successfully
")

-

5. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 5	Τραγούδι δεν βρέθηκε – Αν το τραγούδι που αναζητήσε δεν υπάρχει στο SongService, το ErrorUI εμφανίζει μήνυμα "Song not found" και ο Host καλείται να δοκιμάσει άλλο τραγούδι
A2	Βήμα 9	Απαγορευμένη ενέργεια – Αν η ενέργεια δεν επιτρέπεται το party βρίσκεται σε closing mode, ή ο Host προσπαθεί να προσθέσει τραγούδι που ήδη υπάρχει), το ErrorUI εμφανίζει κατάλληλο μήνυμα
A3	Βήμα 12-13	Αποτυχία αποθήκευσης – Αν το PersistenceController δεν μπορεί να αποθηκεύσει τις αλλαγές (π.χ., βάση δεδομένων offline), εμφανίζεται μήνυμα σφάλματος και η playlist παραμένει ως είχε
A4	Βήμα 2	Αφαίρεση τραγουδιού – Στην περίπτωση αφαίρεσης, παραλείπεται η αναζήτηση τραγουδιού (Βήματα 3-5) και σύστημα προχωρά απευθείας στην αφαίρεση

6. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case:

1. Η **playlist έχει ενημερωθεί** (προσθήκη ή αφαίρεση τραγουδιού)
2. Οι αλλαγές έχουν **αποθηκευτεί** στο persistence layer
3. Η playlist είναι **έτοιμη για αναπαραγωγή**
4. Ο Host έχει λάβει **οπτική επιβεβαίωση** ότι η τροποποίηση ολοκληρώθηκε
5. Σε περίπτωση προσθήκης, το νέο τραγούδι είναι πλέον διαθέσιμο στην playlist

7. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας	Περιγραφή
---------	-----------

BR1	Δεν επιτρέπεται η προσθήκη duplicate τραγουδιού στην ίδια playlist
------------	---

BR2	Δεν επιτρέπεται η τροποποίηση playlist όταν το party βρίσκεται σε closing mode
------------	---

BR3	Η αφαίρεση τραγουδιού είναι πάντα επιτρεπτή (εκτός αν είναι το τελευταίο τραγούδι – προαιρετικά)
------------	--

BR4	Το τραγούδι που προστίθεται πρέπει να υπάρχει στο SongService
------------	--

BR5	Ο Host μπορεί να προσθέσει οποιοδήποτε τραγούδι
------------	--

Κανόνας Περιγραφή

(δεν περιορίζεται από το theme του πάρτι)

8. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία Απαίτηση

Απόδοση Η αναζήτηση τραγουδιού πρέπει να ολοκληρώνεται σε **λιγότερο από δευτερόλεπτο**

Χρηστικότητα Το PlaylistEditorUI πρέπει να παρέχει **σαφή διάκριση** μεταξύ προσθήκης και αφαίρεσης

Συνέπεια Σε περίπτωση αποτυχίας αποθήκευσης, η playlist πρέπει να **παραμείνει αμετάβλητη**

Διαθεσιμότητα Το SongService πρέπει να είναι διαθέσιμο **99.9%** του χρόνου

9. Διάγραμμα Αντιστοίχισης (Component Mapping)

Component στο Robustness
Diagram

Ρόλος / Ευθύνη

PartyHost (ηθοποιός)

Επιλέγει ενέργεια (add/remove), αναζητά τραγούδι
υποβάλλει αίτημα

Component στο Robustness Diagram

Ρόλος / Ευθύνη

PlaylistEditorUI

Παρουσιάζει την playlist, συλλέγει ενέργειες και υποβολές

EditRequestHandler

Χειρίζεται το αίτημα τροποποίησης (action, submit request)

SongService

Αναζητά τραγούδια (fetch song)

Song

Οντότητα τραγουδιού που επιστρέφεται

ValidationController

Ελέγχει αν η ενέργεια επιτρέπεται (request → valid / invalid)

ErrorUI

Εμφανίζει μηνύματα σφάλματος (invalid)

PlaylistController

Εκτελεί την ενέργεια (add/remove song)

Playlist

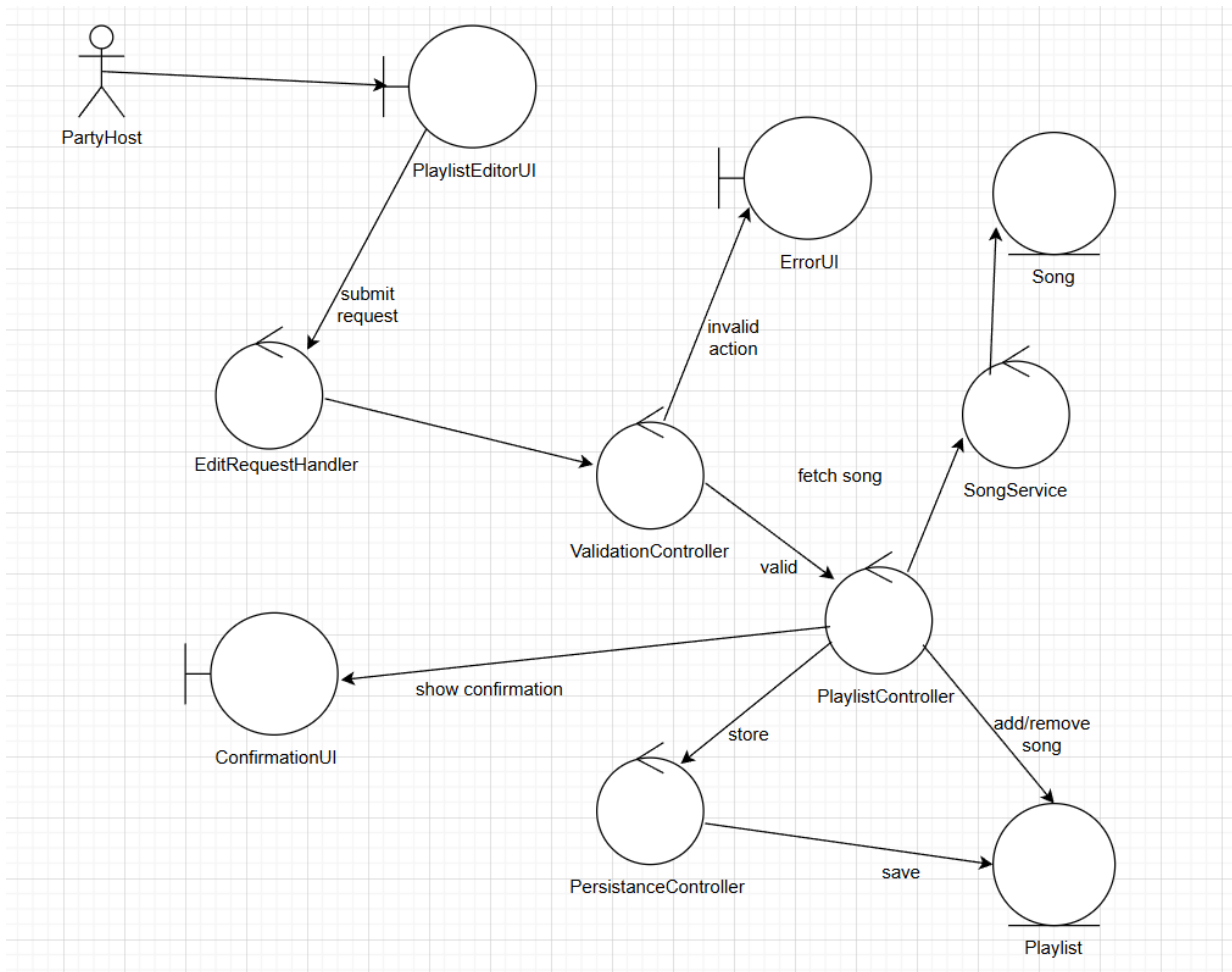
Οντότητα που τροποποιείται

PersistenceController

Αποθηκεύει τις αλλαγές (store, save)

ConfirmationUI

Επιβεβαιώνει την επιτυχή αποθήκευση (showconfirmation)



USE CASE 6: EXTEND PARTY DURATION

1. Στοιχεία Use Case

Στοιχείο

Τιμή

Όνομα Use Case

Extend Party Duration

Primary Actor

Party Host

Στοιχείο	Τιμή
Secondary Actors	RulesController, PersistenceController, ValidationController
Στόχος	Ο Party Host επεκτείνει τη διάρκεια ενός ήδη ενεργού πάρτι (προσθέτει επιπλέον χρόνο)
Τύπος	Βασικός / Διαχείρισης

2. Σύντομη Περιγραφή (Brief Description)

Ο **Party Host** επιθυμεί να παρατείνει τη διάρκεια ενός πάρτι που βρίσκεται ήδη σε εξέλιξη. Εισάγει την επιπλέον διάρκεια (π.χ., +30 λεπτά, +1 ώρα) μέσω ειδικής φόρμας επέκτασης. Το σύστημα επικυρώνει την τιμή, εφαρμόζει επιχειρηματικούς κανόνες (π.χ., μέγιστο συνολικό όριο διάρκειας) και, εφόσον είναι αποδεκτή, ενημερώνει τη διάρκεια του πάρτι και αποθηκεύει την αλλαγή.

3. Προϋποθέσεις (Preconditions)

1. Έχει οριστεί **διάρκεια** (από UC3 – Set Party Duration)
 2. Το **party** είναι **ενεργό** (βρίσκεται σε εξέλιξη)
 3. Ο χρήστης είναι **PartyHost** και έχει πρόσβαση στο συγκεκριμένο party
 4. Το **RulesController** είναι διαθέσιμο (περιέχει τους κανόνες για μέγιστη συνολική διάρκεια)
 5. Η διεπαφή **DurationExtensionUI** είναι προσβάσιμη
-

4. Βασική Ροή Επιτυχίας (Main Success Scenario)

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
1	Ο PartyHost ανοίγει τη φόρμα επέκτασης διάρκειας (DurationExtensionUI)	Το σύστημα εμφανίζει τη διεπαφή για εισαγωγή επιπλέον διάρκειας	DurationExtensionUI
2	Ο PartyHost εισάγει την επιπλέον διάρκεια (π.χ., 30 λεπτά, 1 ώρα) και πατάει submit	Το UI συλλέγει τα δεδομένα και τα μεταφέρει στο InputHandler	InputHandler → submit
3	-	Το InputHandler επικοινωνεί με το ValidationController για έλεγχο εγκυρότητας (validate)	ValidationController → validate
4	-	Έλεγχος validation: μη αρνητική τιμή,	-

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		αριθμητική τιμή, όχι υπερβολικά μεγάλη (λογικό όριο)	
5	-	Αν αποτύχει (invalid duration): το ErrorUI εμφανίζει μήνυμα σφάλματος και η ροή επιστρέφει στο Βήμα 1 Αν περάσει τον validation (valid): το ValidationController προχωρά στο RulesController (apply)	ErrorUI → invalid → duration
6	-	Το RulesController εφαρμόζει τους επιχειρηματικούς κανόνες: ελέγχει αν η νέα συνολική διάρκεια (τρέχουσα + επέκταση) υπερβαίνει το μέγιστο επιτρεπόμενο όριο	RulesController → apply
7	-		-

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
8	-	Αν υπερβαίνει το μέγιστο όριο: το ErrorUI εμφανίζει μήνυμα (π.χ., "Cannot extend beyond maximum duration") και η ροή επιστρέφει	-
9	-	Αν είναι εντός ορίων: το RulesController επικοινωνεί με το PartyController (extendDuration)	PartyController → extendDuration
10	-	Το PartyController ενημερώνει το αντικείμενο Party, αυξάνοντας τη διάρκεια	Party
11	-	Το PartyController καλεί το PersistenceController για αποθήκευση (store → save)	PersistenceController → store → save
12	-	Το PersistenceController αποθηκεύει τη νέα	-

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		διάρκεια στη βάση δεδομένων	
13	-	Το PersistenceController εμφανίζει μήνυμα επιτυχίας μέσω ConfirmationUI (showconfirmation)	ConfirmationUI → showconfirmation
14	Ο PartyHost βλέπει μήνυμα ότι η διάρκεια επεκτάθηκε επιτυχώς ("Duration extended successfully")	-	

5. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 5	Μη έγκυρη τιμή επέκτασης – Αν ο Host εισάγει αρνητική (π.χ., -10), μη αριθμητική τιμή ή μηδέν, το ErrorUI εμφανίζει μήνυμα και ζητάει νέα εισαγωγή
A2	Βήμα 7-8	Υπέρβαση μέγιστης συνολικής διάρκειας – Αν η τρέχουσα διάρκεια είναι ήδη κοντά στο μέγιστο όριο και η επέκταση το υπερβαίνει, το ErrorUI εμφανίζει μήνυμα και η επέκταση απορρίπτεται
A3	Βήμα 11	Αποτυχία αποθήκευσης – Αν το PersistenceController δεν μπορεί να αποθηκεύσει (π.χ., βάση δεδομένων offline), εμφανίζεται μήνυμα σφάλματος και η διάρκεια παραμένει ως είχε
A4	Βήμα 3	Το party δεν είναι ενεργό – Αν το party δεν βρίσκεται σε κατάσταση active, η επέκταση απορρίπτεται. ValidationController ή RulesController το αντιλαμβάνεται

6. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case:

1. Η **διάρκεια έχει αυξηθεί** κατά την επιπλέον τιμή που εισήγαγε ο Host

2. Η **νέα συνολική διάρκεια** είναι αποθηκευμένη στο persistence layer
 3. Το party παραμένει **ενεργό** για το επιπλέον χρονικό διάστημα
 4. Ο Host έχει λάβει **οπτική επιβεβαίωση** ότι η επέκταση ολοκληρώθηκε
 5. Η επέκταση **καταγράφεται** στο σύστημα (για ιστορικό / αναφορές)
-

7. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας	Περιγραφή
BR1	Η επέκταση μπορεί να γίνει μόνο όταν το party είναι ενεργό (active)
BR2	Η τιμή επέκτασης πρέπει να είναι θετικός αριθμός (μεγαλύτερος του μηδενός)
BR3	Υπάρχει μέγιστη συνολική διάρκεια (π.χ., 8 ώρες) που δεν μπορεί να ξεπεραστεί, ακόμα και με διαδοχικές επεκτάσεις
BR4	Δεν υπάρχει ελάχιστο όριο επέκτασης (μπορεί να είναι και 5 λεπτά)
BR5	Η επέκταση δεν επηρεάζει την υπάρχουσα playlist ή τα song requests

Κανόνας	Περιγραφή
BR6	Κάθε επέκταση καταγράφεται στο σύστημα για λόγους ελέγχου και χρέωσης (αν υπάρχει σχετική πολιτική)

8. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία	Απαίτηση
Απόδοση	Η επικύρωση, εφαρμογή και αποθήκευση της επέκτασης πρέπει να σε λιγότερο από 1 δευτερόλεπτο
Ακρίβεια	Το σύστημα πρέπει να ελέγχει σωστά το μέγιστο όριο χωρίς σφάλματα στρογγυλοποίησης
Διαθεσιμότητα	Το RulesController πρέπει να είναι διαθέσιμο 99.9% του χρόνου
Συνέπεια	Σε περίπτωση αποτυχίας αποθήκευσης, η διάρκεια δεν πρέπει να αλλάζει

9. Διάγραμμα Αντιστοίχισης (Component Mapping)

Component στο Robustness Diagram	Ρόλος / Ευθύνη
PartyHost (ηθοποιός)	Εισάγει την επιπλέον διάρκεια και υποβάλλει το αίτημα επέκτασης

Component στο Robustness Diagram

Ρόλος / Ευθύνη

DurationExtensionUI

Παρουσιάζει τη φόρμα επέκτασης και συλλέγει τα δεδομένα (submit)

InputHandler

Αρχική επεξεργασία και δρομολόγηση της εισόδου

ValidationController

Εκτελεί βασικό έλεγχο εγκυρότητας: μη αρνητική τιμή, αριθμητική, λογικό όριο (validate)

ErrorUI

Εμφανίζει μήνυμα σφάλματος (invalid duration)

RulesController

Εφαρμόζει επιχειρηματικούς κανόνες: έλεγχος μέγιστης συνολικής διάρκειας (apply)

PartyController

Διαχειρίζεται το αντικείμενο Party και επεκτείνει τη διάρκεια (extendDuration)

Party

Οντότητα που αποθηκεύει τη νέα διάρκεια

PersistenceController

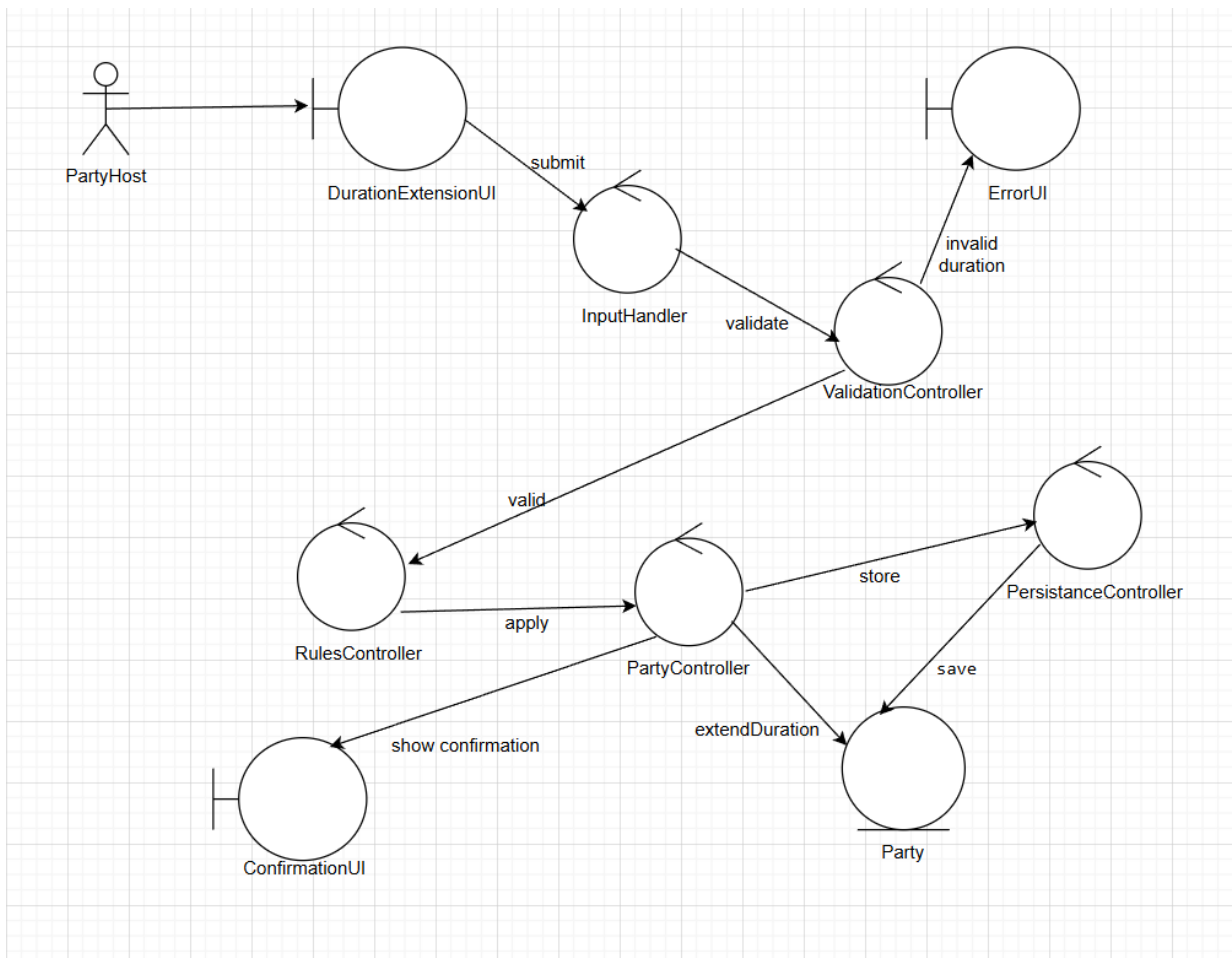
Αποθηκεύει την αλλαγή (store, save)

Component στο Robustness Diagram

Ρόλος / Ευθύνη

ConfirmationUI

Επιβεβαιώνει την επιτυχή επέκταση
(showconfirmation)



USE CASE 7: ACTIVATE CLOSING MODE

1. Στοιχεία Use Case

Στοιχείο	Τιμή
Όνομα Use Case	Activate Closing Mode
Primary Actor	Party Host
Secondary Actors	ModeController, PartyController, PersistenceController
Στόχος	Ο Party Host ενεργοποιεί τη λειτουργία λήξης (closing mode) του πάρτι, προετοιμάζοντας το σύστημα για τον τερματισμό.
Τύπος	Βασικός / Διαχείρισης

2. Σύντομη Περιγραφή (Brief Description)

Ο **Party Host** ενεργοποιεί τη λειτουργία λήξης (closing mode) για ένα πάρτι που βρίσκεται σε εξέλιξη. Αυτή η ενέργεια αλλάζει τη συμπεριφορά του συστήματος: συνήθως σταματάει την αποδοχή νέων αιτημάτων τραγουδιών, προσαρμόζει την playlist (π.χ., παίζει ειδικά τραγούδια λήξης) και προετοιμάζει το πάρτι για τερματισμό. Η αλλαγή αποθηκεύεται και το σύστημα μεταβαίνει σε κατάσταση closing mode.

3. Προϋποθέσεις (Preconditions)

1. Το **party** είναι ενεργό (active)
 2. Ο χρήστης είναι **PartyHost** και έχει πρόσβαση στο συγκεκριμένο party
 3. Το party **δεν βρίσκεται ήδη σε closing mode**
 4. Το **ModeController** είναι διαθέσιμο
 5. Η διεπαφή **ControlPanelUI** είναι προσβάσιμη
-

4. Βασική Ροή Επιτυχίας (Main Success Scenario)

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
1	Ο PartyHost ανοίγει το Control PanelUI	Το σύστημα εμφανίζει τον πίνακα ελέγχου με τις διαθέσιμες ενέργειες	ControlPanelUI
2	Ο PartyHost	Το UI συλλέγει την επιλογή και τη	ActionHandler → submit

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
	επιλέγει την ενέργεια " Activate Closing Mode " και πατάει submit	μεταφέρει στο ActionHandler	
3	-	Το ActionHandler επικοινωνεί με το ModeController για επεξεργασία της ενέργειας (process action)	ModeController → process action
4	-	Το ModeController προετοιμάζει την αλλαγή mode και καλεί το PartyController για εφαρμογή (apply mode)	apply mode → PartyController
5	-	Το PartyController εκτελεί την εντολή setClosingM	PartyController → setClosingMode → Party

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		ode στο αντικείμενο Party	
6	-	Το Party ενημερώνει την κατάσταση του σε closing mode	Party
7	-	Το PartyController κλείει το PersistenceController για αποθήκευση (store)	PersistenceController → store
8	-	Το PersistenceController αποθηκεύει την αλλαγή κατάστασης (save)	save
9	-	Το PersistenceController εμφανίζει μήνυμα επιτυχίας μέσω ConfirmationUI	ConfirmationUI
10	Ο PartyHost βλέπει μήνυμα	-	

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
	ότι το closing mode ενεργοποιήθηκε επιτυχώς ("Closing mode activated")		

5. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 3-4	Το party δεν είναι ενεργό – Αν το party δεν βρίσκεται σε κατάσταση active, το ModeController απορρίπτει την ενέργεια και εμφανίζεται μήνυμα σφάλματος
A2	Βήμα 3-4	Το closing mode είναι ήδη ενεργό – Αν το party βρίσκεται σε closing mode, το σύστημα εμφανίζει μήνυμα "Closing mode already active" και η ροή τερματίζεται

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A3	Βήμα 7-8	Αποτυχία αποθήκευσης – Αν το PersistenceController δεν μπορεί να αποθηκεύσει (π.χ., βάση δεδομένων offline), εμφανίζεται μήνυμα σφάλματος και η κατάσταση δεν αλλάζει (ή αλλάζει in-memory)
		Ακύρωση από τον Host – Αν ο Host αλλάξει γνώμη και ακυρώσει πριν την επιβεβαίωση, καμία αλλαγή δεν γίνεται

6. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case:

1. Το **σύστημα λειτουργεί σε closing mode**
 2. Το αντικείμενο **Party** έχει ενημερωθεί με την κατάσταση closing mode
 3. Η αλλαγή έχει **αποθηκευτεί** στο persistence layer
 4. **Νέα αιτήματα τραγουδιών** δεν γίνονται δεκτά (ή υπάρχει ειδική συμπεριφορά)
 5. Η **playlist** μπορεί να προσαρμοστεί αυτόματα για το κλείσιμο (π.χ., ειδικά τραγούδια λήξης)
 6. Ο Host έχει λάβει **οπτική επιβεβαίωση** ότι το closing mode ενεργοποιήθηκε
-

7. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας Περιγραφή

BR1 Το closing mode μπορεί να ενεργοποιηθεί **μόνο όταν το party είναι ενεργό** (active)

BR2 Η ενεργοποίηση closing mode είναι **μη αναστρέψιμη** (ή απαιτεί ειδική ενέργεια για επαναφορά)

BR3 Μετά την ενεργοποίηση closing mode, **δεν γίνονται δεκτά νέα song requests**

BR4 Ο Host μπορεί να ενεργοποιήσει closing mode **οποιαδήποτε στιγμή** κατά τη διάρκεια του πάρτι

BR5 Το closing mode **δεν επηρεάζει** τα ήδη αποθηκευμένα song requests (παραμένουν στην playlist)

BR6 Το σύστημα μπορεί να έχει **χρονικό όριο** για το closing mode (π.χ., 15 λεπτά) μετά το οποίο το πάρτι τερματίζεται αυτόματα

8. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία	Απαίτηση
Απόδοση	Η ενεργοποίηση closing mode πρέπει να ολοκληρώνεται σε λιγότερο από 1 δευτερόλεπτο
Αξιοπιστία	Η αλλαγή κατάστασης πρέπει να είναι ατομική (atomic) – δεν επιτρέπεται μερική αποθήκευση
Διαθεσιμότητα	Το ModeController πρέπει να είναι διαθέσιμο 99.9% του χρόνου
Ασφάλεια	Μόνο PartyHost μπορεί να ενεργοποιήσει closing mode (έλεγχος ρόλου)

9. Διάγραμμα Αντιστοίχισης (Component Mapping)

Component στο Robustness Diagram	Ρόλος / Ευθύνη
PartyHost (ηθοποιός)	Επιλέγει την ενέργεια "Activate Closing Mode" και την υποβάλλει
ControlPanelUI	Παρουσιάζει τον πίνακα ελέγχου και συλλέγει την ενέργεια (submit)
ActionHandler	Δέχεται το αίτημα (submit) και το προωθεί στο ModeController
ModeController	Επεξεργάζεται την ενέργεια αλλαγής

Component στο Robustness Diagram

Ρόλος / Ευθύνη

mode (process action, apply mode)

PartyController

Εκτελεί την αλλαγή κατάστασης στο
Party (setClosingMode)

Party

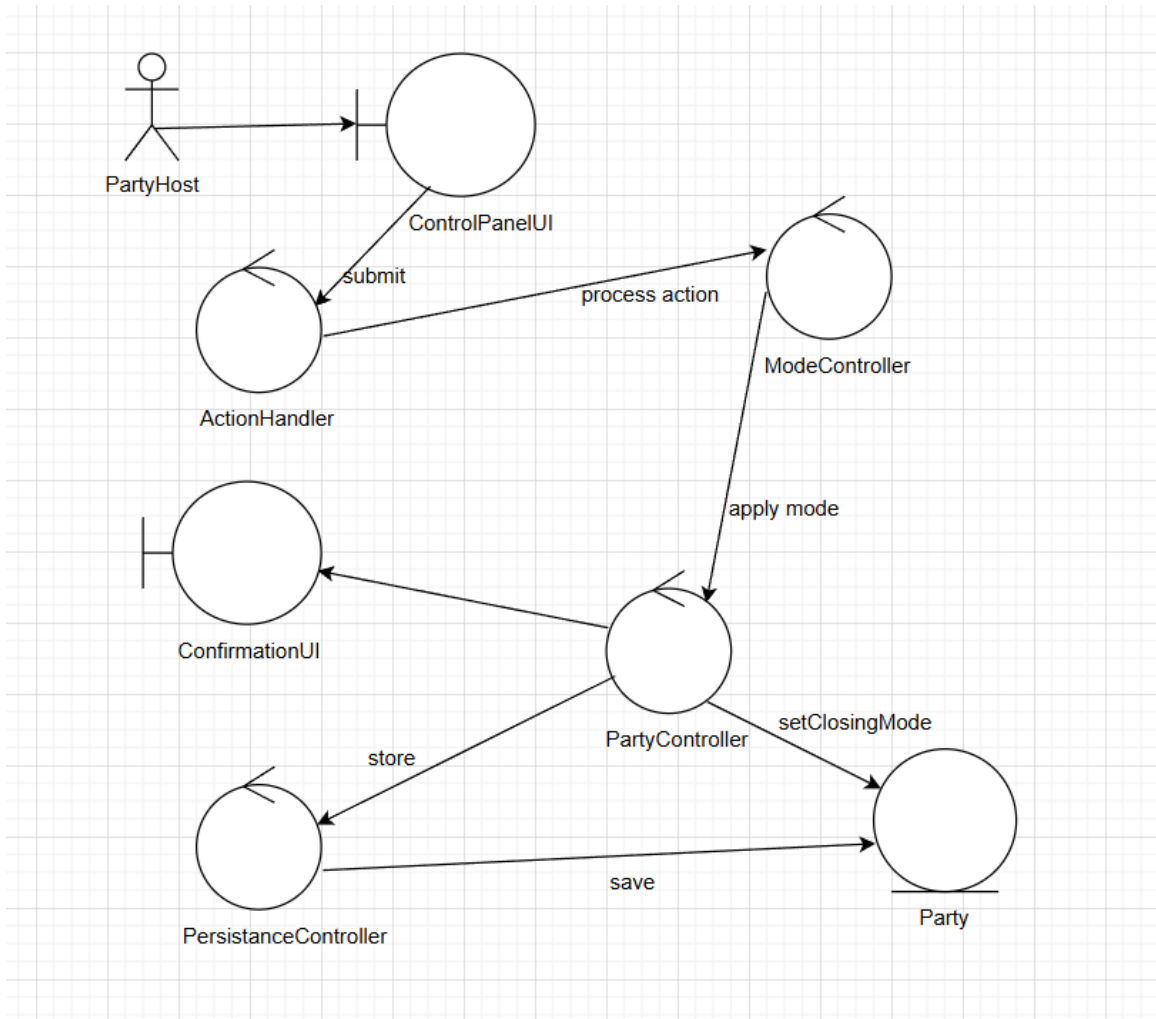
Οντότητα που ενημερώνεται σε
closing mode

PersistenceController

Αποθηκεύει την αλλαγή (store,
save)

ConfirmationUI

Επιβεβαιώνει την επιτυχή
ενεργοποίηση



USE CASE 8: SCAN QR CODE

1. Στοιχεία Use Case

Στοιχείο	Τιμή
Όνομα Use Case	Scan QR Code
Primary Actor	Party Guest
Secondary Actors	QRDecoder, ValidationController, AccessController, Party
Στόχος	Ο Party Guest σκανάρει το QR code του πάρτι για να αποκτήσει πρόσβαση στο σύστημα
Τύπος	Βασικός / Πρόσβασης

2. Σύντομη Περιγραφή (Brief Description)

Ο **Party Guest** χρησιμοποιεί τη συσκευή του (κάμερα) για να σκανάρει ένα QR code που σχετίζεται με ένα ενεργό πάρτι. Το σύστημα αποκωδικοποιεί το QR code, το επικυρώνει και, εφόσον είναι έγκυρο, χορηγεί πρόσβαση στον Guest. Μετά την επιτυχή πρόσβαση, ο Guest μπορεί να υποβάλλει αιτήματα για τραγούδια και να συμμετέχει ενεργά στο πάρτι.

3. Προϋποθέσεις (Preconditions)

- 1. Υπάρχει ενεργό party (το πάρτι βρίσκεται σε κατάσταση active)

2. Το **QR code** έχει εκδοθεί από το σύστημα για το συγκεκριμένο πάρτι
3. Ο Guest έχει **συσκευή με κάμερα** και δυνατότητα σάρωσης QR
4. Το QR code είναι **ορατό** και προσβάσιμο από τον Guest
5. Η διεπαφή **QRScannerUI** είναι προσβάσιμη

4. Βασική Ποή Επιτυχίας (Main Success Scenario)

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
1	Ο PartyGuest ανοίγει τον σαρωτή QR code (QRScannerUI → open scanner)	Το σύστημα ενεργοποιεί την κάμερα και προετοιμάζεται για σάρωση	QRScannerUI → open scanner
2	Ο PartyGuest σκανάρει το QR code (scan)	Η συσκευή καταγράφει την εικόνα του QR code και τη μεταφέρει στο ScanController	ScanController → scan
3	-	Το ScanController καλεί το QRDecoder για	QRDecoder → decode

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		αποκωδικοποίηση (decode)	
4	-	Το QRDecoder αποκωδικοποιεί το QR code και εξάγει τα δεδομένα (π.χ., party ID, κλειδί πρόσβασης)	-
5	-	Το QRDecoder επικοινωνεί με το ValidationController για επικύρωση (validate)	ValidationController → validate
6	-	Το ValidationController ελέγχει: (α) το QR code είναι έγκυρο, (β) αντιστοιχεί σε υπάρχον party, (γ) το party είναι ενεργό	-
7	-	Αν το QR code είναι άκυρο (invalid QR): το ErrorUI εμφανίζει μήνυμα σφάλματος	ErrorUI → invalid QR

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		και η ροή τερματίζεται	
8	-	Αν το QR code είναι έγκυρο (valid): το ValidationController προχωρά στο AccessController (verify)	AccessController → verify
9	-	Το AccessController επαληθεύει τα στοιχεία πρόσβασης και επικοινωνεί με το Party για να επιβεβαιώσει ότι το party είναι ενεργό	Party
10	-	Το AccessController χορηγεί πρόσβαση στον Guest (grant)	grant → PartyGuest
11	-	Το AccessController καλεί το AccessGrantedUI για να εμφανίσει	AccessGrantedUI → show success

Βήμα	Ενέργεια Ηθοποιού	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
		μήνυμα επιτυχίας (show success)	
12	Ο PartyGuest βλέπει μήνυμα ότι η πρόσβαση δόθηκε επιτυχώς και μπορεί πλέον να χρησιμοποιήσει το σύστημα	-	-

5. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 7	Άκυρο QR code – Το QR code δεν αναγνωρίζεται από το σύστημα (δεν έχει εκδοθεί, είναι κατεστραμμένο ή αντιστοιχεί σε κανένα party). Το ErrorUI εμφανίζει μήνυμα "Invalid QR

Κωδικός	Σημείο Απόκλισης	Περιγραφή
		code"
A2	Βήμα 9	Το party δεν είναι ενεργό – Το QR code είναι έγκυρο αλλά το party έχει λήξει ή δεν είναι σε κατάσταση α Το σύστημα εμφανίζει μήνυμα "Party is not active"
A3	Βήμα 2	Αποτυχία σάρωσης – Η κάμερα δεν μπορεί να διαβάσει το code (θολό, κακός φωτισμός, κατεστραμμένο). Το σύστημα εμφανίζει μήνυμα και ζητάει επανάληψη
A4	Βήμα 3	Αποτυχία αποκωδικοποίησης – Το QR code είναι ορατό αλλά δεν μπορεί να αποκωδικοποιηθεί. Το ErrorUI εμφανίζει μήνυμα "Failed to decode QR code"
A5	Βήμα 1	Ο Guest δεν έχει δώσει δικαιώματα κάμερας – Το σύστημα ζητάει δικαιώματα. Αν δεν δοθούν, το use case τερματίζεται

6. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case:

1. Ο **χρήστης έχει πρόσβαση** στο σύστημα

2. Ο Guest **ταυτοποιείται** από το σύστημα (συνδέεται με το συγκεκριμένο party)
 3. Ο Guest μπορεί πλέον να **υποβάλλει song requests** (UC9)
 4. Το σύστημα **καταγράφει** την είσοδο του Guest (για αναλυτικά στοιχεία / μετρήσεις)
 5. Ο Guest έχει λάβει **οπτική επιβεβαίωση** (AccessGrantedUI)
-

7. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας Περιγραφή

BR1 Το QR code πρέπει να είναι **μοναδικό** ανά πάρτι

BR2 Το QR code έχει **περιορισμένη διάρκεια ζωής** (π.χ., λήγει με το πάρτι)

BR3 Η πρόσβαση δίνεται **μόνο εφόσον το party είναι ενεργό** (active)

BR4 Ένας Guest μπορεί να σκανάρει το QR code **πολλές φορές** – το σύστημα αναγνωρίζει ότι έχει ήδη πρόσβαση

BR5 Το QR code **δεν απαιτεί λογαριασμό** χρήστη – η πρόσβαση είναι ανώνυμη και συνδεδεμένη με τη συσκευή/session

Κανόνας	Περιγραφή
BR6	Σε περίπτωση που το closing mode είναι ενεργό, η σάρωση QR code μπορεί να απαγορεύεται (ανάλογα πολιτική)

8. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία	Απαίτηση
Απόδοση	Η αποκωδικοποίηση και επικύρωση του QR code πρέπει να γίνεται σε λιγότερο από 2 δευτερόλεπτα
Ασφάλεια	Το QR code πρέπει να περιέχει κρυπτογραφημένα ή μοναδικά δεδομένα για αποφυγή πλαστογράφησης
Χρηστικότητα	Το QRScannerUI πρέπει να παρέχει σαφή οδηγία και οπτική ανατροφοδότηση (π.χ., επιτυχής/αποτυχημένη σάρωση)
Διαθεσιμότητα	Το σύστημα επικύρωσης QR πρέπει να είναι διαθέσιμο 99.9% του χρόνου

9. Διάγραμμα Αντιστοίχισης (Component Mapping)

Component στο Robustness Diagram	Ρόλος / Ευθύνη
PartyGuest (ηθοποιός)	Ανοίγει τον σαρωτή και σκανάρει το QR code
QRScannerUI	Παρουσιάζει τη διεπαφή σάρωσης, συλλέγει το σήμα open scanner και scan
ScanController	Διαχειρίζεται τη διαδικασία σάρωσης (scan)
QRDecoder	Αποκωδικοποιεί το QR code (decode)
ValidationController	Επικυρώνει το QR code (validate, valid / invalid QR)
ErrorUI	Εμφανίζει μήνυμα σφάλματος (invalid QR)
AccessController	Επαληθεύει την πρόσβαση (verify) και

Component στο Robustness Diagram

Ρόλος / Ευθύνη

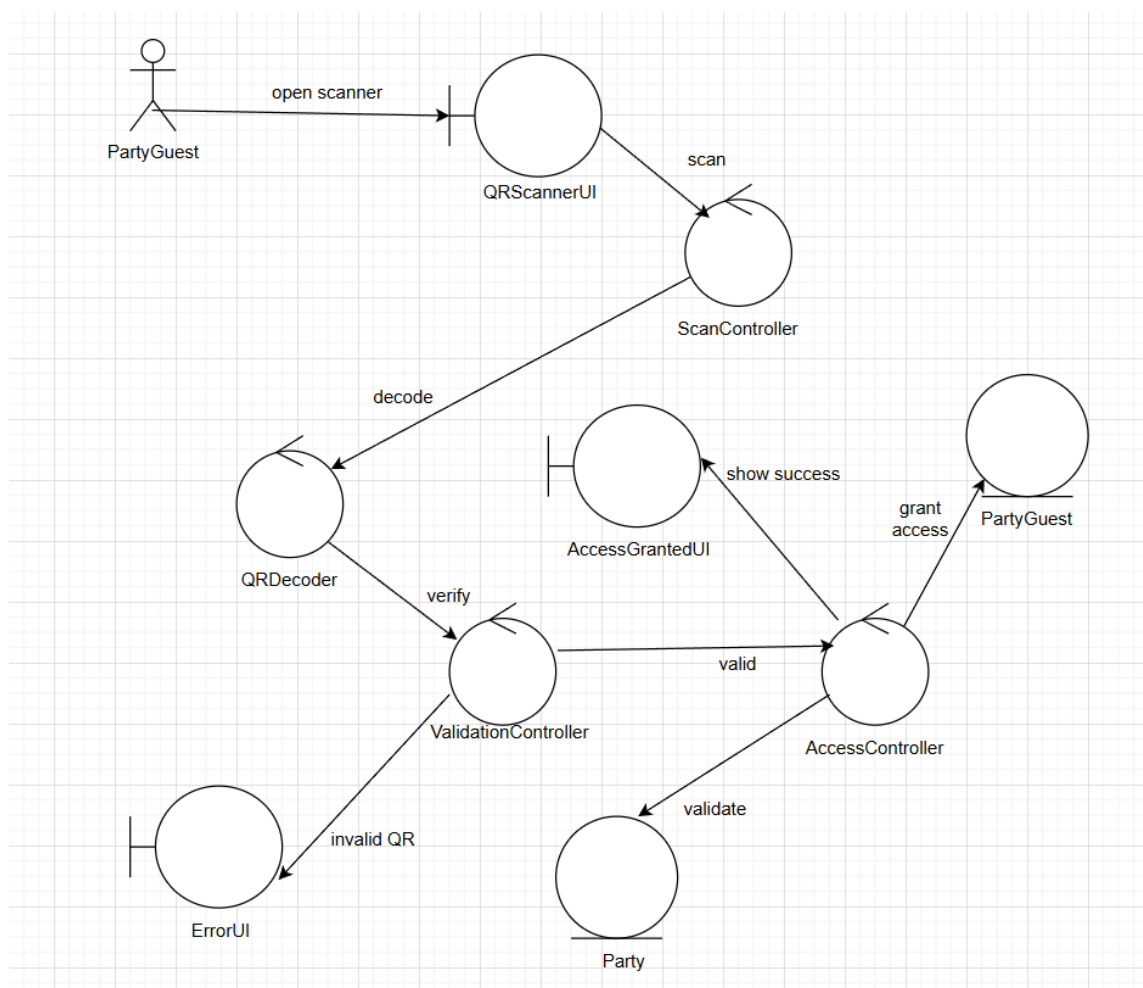
τη χορηγεί (grant)

Party

Παρέχει πληροφορίες για την κατάσταση τ
πάρτι (ενεργό / μη ενεργό)

AccessGrantedUI

Εμφανίζει μήνυμα επιτυχίας (show succes



USE CASE 9: SUBMIT SONG REQUEST

1. Σκοπός του Συστήματος

Το σύστημα επιτρέπει σε έναν χρήστη/καλεσμένο (PartyGuest) να υποβάλει αίτημα για κάποιο τραγούδι.

Το αίτημα:

- ελέγχεται ως προς τα δικαιώματα πρόσβασης,
- γίνεται αναζήτηση του τραγουδιού,
- επιβεβαιώνεται η εγκυρότητα,
- προστίθεται στην ουρά αναπαραγωγής,
- αποθηκεύεται μόνιμα,
- και εμφανίζεται μήνυμα επιβεβαίωσης ή σφάλματος.

2. Στοιχεία του Diagram

Το robustness diagram χρησιμοποιεί τρεις βασικούς τύπους αντικειμένων:

Τύπος	Ρόλος
Boundary Objects	Επικοινωνία με χρήστη
Control Objects	Επιχειρησιακή λογική
Entity Objects	Δεδομένα συστήματος

3. Boundary Objects (UI)

3.1 PartyGuest

Ο εξωτερικός χρήστης του συστήματος.

Ρόλος:

- Υποβάλλει αίτημα τραγουδιού.
-

3.2 SongRequestUI

Διεπαφή μέσω της οποίας ο χρήστης εισάγει το τραγούδι.

Λειτουργίες:

- λήψη στοιχείων request,
- submit request.

Επικοινωνεί με:

- RequestHandler.
-

3.3 ErrorUI

Εμφανίζει μηνύματα αποτυχίας.

Παραδείγματα:

- No access
 - Song not found
-

3.4 ConfirmationUI

Εμφανίζει επιβεβαίωση επιτυχούς καταχώρησης.

Παράδειγμα:

- “Το τραγούδι προστέθηκε επιτυχώς στην ουρά.”
-

4. Control Objects

4.1 RequestHandler

Πρώτο control αντικείμενο που λαμβάνει το request από το UI.

Ρόλος:

- Παραλαμβάνει το submit.
- Ξεκινά τη διαδικασία validation.

Επικοινωνεί με:

- AccessValidationController.
-

4.2 AccessValidationController

Ελέγχει αν ο χρήστης έχει δικαίωμα να κάνει request.

Πιθανοί έλεγχοι:

- authentication,
- request limits,
- banned users,
- premium access.

Έξοδοι:

- Αν αποτύχει → ErrorUI (“no access”)
 - Αν επιτύχει → SongSearchController
-

4.3 SongSearchController

Αναζητά το τραγούδι στο σύστημα.

Ρόλος:

- εκτελεί search,

- επιστρέφει αποτελέσματα.

Επικοινωνεί με:

- Song entity,
- SongValidationController.

Έξοδοι:

- Αν δεν βρεθεί τραγούδι → ErrorUI (“not found”)
 - Αν βρεθεί → validation process.
-

4.4 SongValidationController

Ελέγχει αν το τραγούδι μπορεί να προστεθεί.

Πιθανοί έλεγχοι:

- duplicate song,
- inappropriate content,
- already queued,
- unsupported format.

Έξοδος:

- valid → RequestController
-

4.5 RequestController

Κεντρικός controller διαχείρισης request.

Ρόλος:

- δημιουργεί SongRequest,
- ζητά προσθήκη στην queue,

- εμφανίζει confirmation.

Επικοινωνεί με:

- QueueManager,
 - ConfirmationUI.
-

4.6 QueueManager

Διαχειρίζεται την ουρά τραγουδιών.

Λειτουργίες:

- add to queue,
- update playlist,
- save request.

Επικοινωνεί με:

- Playlist,
 - PersistenceController,
 - SongRequest.
-

4.7 PersistenceController

(πιθανό typo → PersistenceController)

Ρόλος:

- αποθήκευση δεδομένων σε database ή storage.

Λειτουργίες:

- save queue,
- save requests,

- persist playlist updates.
-

5. Entity Objects

5.1 Song

Αναπαριστά τραγούδι στο σύστημα.

Περιέχει:

- title,
- artist,
- duration,
- metadata.

Χρησιμοποιείται από:

- SongSearchController.
-

5.2 Playlist

Αναπαριστά την ενεργή playlist.

Ενημερώνεται από:

- QueueManager.
-

5.3 SongRequest

Αναπαριστά το αίτημα τραγουδιού.

Περιέχει:

- requester,
- requested song,

- timestamp,
- status.

Δημιουργείται από:

- RequestController.

Αποθηκεύεται μέσω:

- PersistenceController.

6. Αναλυτική Ροή Εκτέλεσης

Βήμα 1 — Υποβολή Request

Ο PartyGuest χρησιμοποιεί το SongRequestUI και κάνει submit request.

Ροή:

PartyGuest → SongRequestUI → RequestHandler

Βήμα 2 — Έλεγχος Πρόσβασης

Ο RequestHandler στέλνει το αίτημα στον AccessValidationController.

Έλεγχος:

- αν επιτρέπεται το request.

Περίπτωση αποτυχίας:

AccessValidationController → ErrorUI

Μήνυμα:

- “No access”

Περίπτωση επιτυχίας:

AccessValidationController → SongSearchController

Βήμα 3 — Αναζήτηση Τραγουδιού

Ο SongSearchController ψάχνει το τραγούδι μέσω του Song entity.

Ροή:

SongSearchController ↔ Song

Αν δεν βρεθεί:

SongValidationController → ErrorUI

Μήνυμα:

- “Song not found”

Αν βρεθεί:

- επιστρέφονται αποτελέσματα.

Βήμα 4 — Validation Τραγουδιού

Ο SongValidationController ελέγχει:

- duplicates,
- restrictions,
- queue conflicts.

Αν είναι valid:

SongValidationController → RequestController

Βήμα 5 — Δημιουργία Request

Ο RequestController:

- δημιουργεί νέο SongRequest,

- ζητά προσθήκη στην ουρά.

Ροή:

RequestController → QueueManager

RequestController → SongRequest

Βήμα 6 — Queue Management

Ο QueueManager:

- προσθέτει το τραγούδι στην ουρά,
- ενημερώνει την Playlist,
- καλεί persistence.

Ροή:

QueueManager → Playlist

QueueManager → PersistenceController

QueueManager → SongRequest

Βήμα 7 — Αποθήκευση

Ο PersistenceController αποθηκεύει:

- queue state,
 - request,
 - playlist changes.
-

Βήμα 8 — Επιβεβαίωση

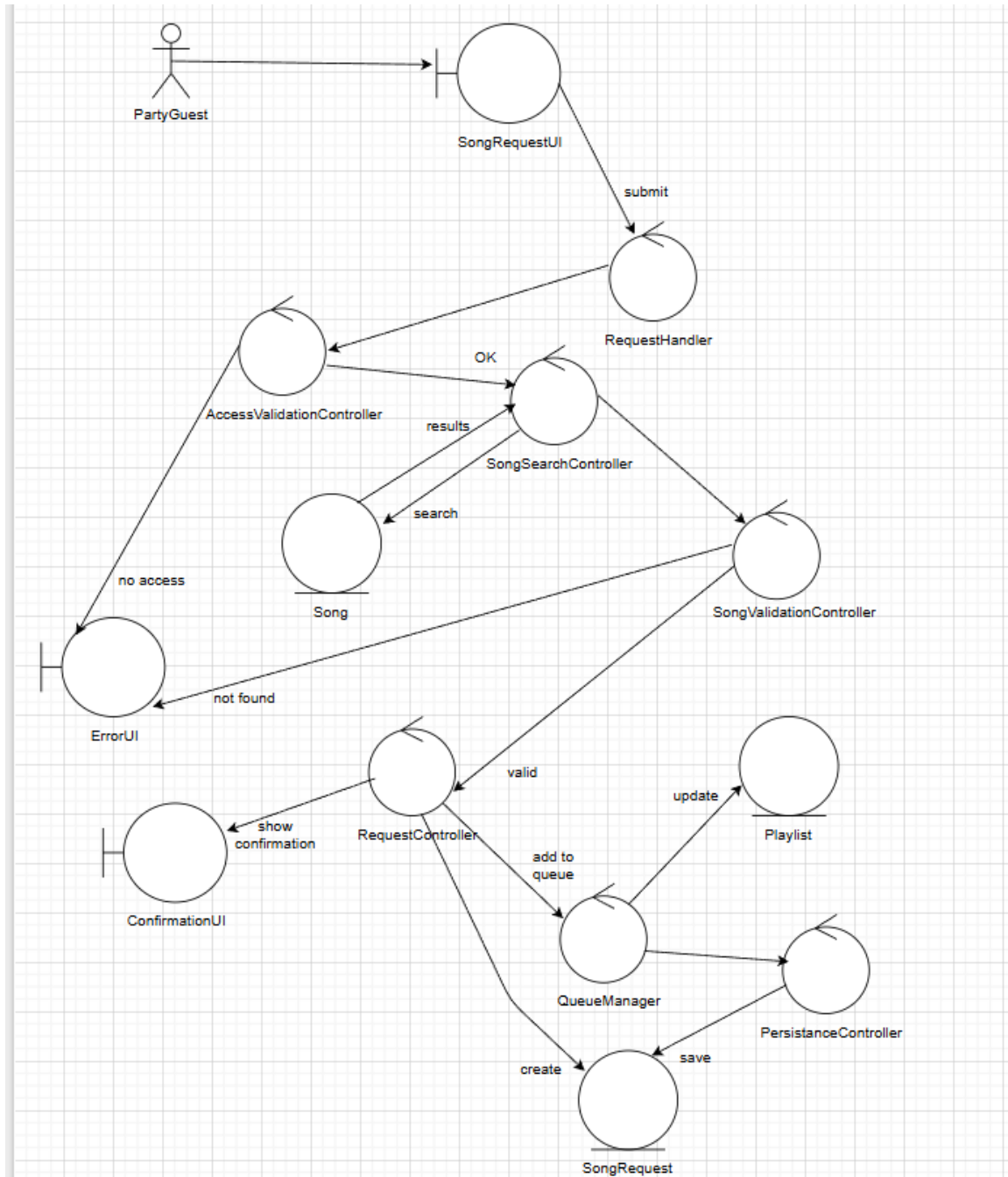
Ο RequestController ενημερώνει το ConfirmationUI.

Ροή:

RequestController → ConfirmationUI

Μήνυμα:

- “Request submitted successfully”



USE CASE 10: SYNC LIGHTING WITH MUSIC

1. Στοιχεία Use Case

Στοιχείο	Τιμή
Όνομα Use Case	Sync Lighting With Music
Primary Actor	System (αυτοματοποιημένο σύστημα)
Secondary Actors	LightingSystem (εξωτερικό σύστημα φωτισμού), BeatAnalyzer, DataMapper, TransmissionController
Στόχος	Το σύστημα συγχρονίζει τον φωτισμό με τη μουσική που παίζει, αναλύοντας τα beats του τραγουδιού και στέλνοντας αντίστοιχες εντολές στο εξωτερικό σύστημα φωτισμού
Τύπος	Βασικός / Αυτοματοποιημένος

2. Σύντομη Περιγραφή (Brief Description)

Το **σύστημα** αυτόματα συγχρονίζει τον φωτισμό με τη μουσική που αναπαράγεται κατά τη διάρκεια του πάρτι. Αρχικοποιεί τη διαδικασία

συγχρονισμού, λαμβάνει το τρέχον τραγούδι από την playlist, αναλύει τα beats του, αντιστοιχεί τα beat δεδομένα σε χρωματικά μοτίβα (patterns) και στέλνει εντολές στο εξωτερικό σύστημα φωτισμού (LightingSystem). Το αποτέλεσμα είναι ότι τα φώτα "χορεύουν" στον ρυθμό της μουσικής.

3. Προϋποθέσεις (Preconditions)

- 1. Υπάρχει ενεργή playlist (από UC4)
 - 2. Υπάρχει τρέχον τραγούδι που παίζει (current song)
 - 3. Το **LightingSystem** είναι συνδεδεμένο και διαθέσιμο μέσω δικτύου
 - 4. Το **BeatAnalyzer** είναι διαθέσιμο (μπορεί να αναλύει beats τραγουδιών)
 - 5. Το σύστημα έχει πρόσβαση στο **NetworkInterface** για επικοινωνία
-

4. Βασική Ροή Επιτυχίας (Main Success Scenario)

Βήμα	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
1	Το System αρχικοποιεί τη διαδικασία συγχρονισμού (initiate)	System → initiate
2	Το System επικοινωνεί με το SyncController για να ξεκινήσει ο συγχρονισμός	SyncController
3	Το SyncController ζητά το τρέχον τραγούδι από την Playlist (get current song)	SyncController → get current song → Playlist

Βήμα	Ενέργεια Συστήματος	Αντιστοίχιση Diagram
4	Η Playlist επιστρέφει το τρέχον Song	Song
5	Το SyncController καλεί το BeatAnalyzer για ανάλυση beats (analyze beat)	BeatAnalyzer → analyze beat
6	Το BeatAnalyzer εξάγει τα beat δεδομένα από το τραγούδι (extract beat)	extract beat → Song
7	Το BeatAnalyzer επικοινωνεί με το DataManager για αντιστοίχιση beats σε χρώματα (map to colors)	DataManager → map to colors
8	Το DataManager παράγει ένα χρωματικό μοτίβο (lighting pattern) και το μεταφέρει στο TransmissionController (send commands)	TransmissionController → commands
9	Το TransmissionController στέλνει τις εντολές μέσω NetworkInterface (transmit)	NetworkInterface → transmit
10	Το NetworkInterface επικοινωνεί με το LightingSystem για εκτέλεση	LightingSystem
11	Το LightingSystem εκτελεί τις εντολές – τα φώτα συγχρονίζονται με τη μουσική	-
12	Το TransmissionController αποθηκεύει την κατάσταση φωτισμού (save)	save

5. Σημειώσεις για την αντιστοίχιση

To robustness diagram που δόθηκε έχει την εξής ροή:

Σειρά	Component → Component	Ενέργεια
1	System → SyncController	initiate
2	SyncController → Playlist	get current song
3	Playlist → Song	(returns current song)
4	SyncController → BeatAnalyzer	analyze beat
5	BeatAnalyzer → Song	extract beat
6	BeatAnalyzer → DataMapper	map to colors
7	DataManager → TransmissionController	send commands
8	TransmissionController → NetworkInterface	transmit
9	NetworkInterface → LightingSystem	(commands delivered)
10	TransmissionController → save	(state saved)

6. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 3-4	Δεν υπάρχει τρέχον τραγούδι – Η playlist είναι άδεια ή σε παύση. Το SyncController περιμένει ή τερματίζει τον συγχρονισμό
A2	Βήμα 5-6	Αποτυχία ανάλυσης beats – Το BeatAnalyzer δεν μπορεί να αναλύσει το τραγούδι (μη υποστηριζόμενη μορφή, κατεστραμμένο αρχείο). Το σύστημα χρησιμοποιεί default lighting pattern
A3	Βήμα 7-8	Αποτυχία αντιστοίχισης χρωμάτων – Το DataMapper δεν μπορεί να παράγει έγκυρο pattern. Χρησιμοποιείται default pattern
A4	Βήμα 9-10	Αποτυχία επικοινωνίας με LightingSystem – Το LightingSystem είναι offline ή δεν απαντά. Το σύστημα καταγράφει σφάλμα και προσπαθεί ξανά (retry)
A5	Βήμα 12	Αποτυχία αποθήκευσης – Δεν μπορεί να αποθηκευτεί η κατάσταση φωτισμού. Ο συγχρονισμός συνεχίζεται κανονικά, αλλά η κατάσταση δεν διατηρείται

7. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case:

1. Ο **φωτισμός είναι συγχρονισμένος** με τη μουσική
 2. Τα φώτα ανταποκρίνονται στα beats του τραγουδιού που παίζει
 3. Η **τελευταία κατάσταση φωτισμού** έχει αποθηκευτεί (για πιθανή επαναφορά)
 4. Το σύστημα είναι έτοιμο για τον επόμενο συγχρονισμό (π.χ., όταν αλλάξει τραγούδι)
-

8. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας Περιγραφή

BR1	Ο συγχρονισμός γίνεται σε πραγματικό χρόνο – τα φώτα πρέπει να ανταποκρίνονται άμεσα στα beats
BR2	Σε περίπτωση απουσίας beat data, χρησιμοποιείται default pattern (π.χ., σταθερό χρώμα ή αργό fade)
BR3	Η αντιστοίχιση beats σε χρώματα γίνεται βάσει προκαθορισμένου αλγόριθμου (π.χ., δυνατά beats → φωτεινά χρώματα)
BR4	Ο συγχρονισμός σταματά όταν σταματήσει η μουσική (παύση, τέλος τραγουδιού)
BR5	Το σύστημα υποστηρίζει πολλαπλά lighting profiles (π.χ., party

Κανόνας	Περιγραφή
---------	-----------

	mode, chill mode, closing mode)
--	---------------------------------

9. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία	Απαίτηση
-----------	----------

Απόδοση	
----------------	--

	Η καθυστέρηση (latency) από την ανίχνευση beat έως την εκτέλεση εντολής πρέπει να είναι < 50ms
--	--

Αξιοπιστία	
-------------------	--

	Το σύστημα πρέπει να συνεχίζει τον συγχρονισμό χωρίς διακοπές ο και με μικρές αποτυχίες
--	---

Επεκτασιμότητα	
-----------------------	--

	Πρέπει να υποστηρίζονται διαφορετικά μοντέλα LightingSystem του ExternalGatewayAdapter
--	---

Ανθεκτικότητα	
----------------------	--

	Σε περίπτωση αποτυχίας δικτύου, το σύστημα προσπαθεί αυτόματα επανάληψη (retry)
--	--

10. Διάγραμμα Αντιστοίχισης (Component Mapping)

Component στο Robustness Diagram

Ρόλος / Ευθύνη

System (Primary Actor)

Αρχικοποιεί τη διαδικασία συγχρονισμού (initiate)

SyncController

Ελέγχει τη συνολική ροή συγχρονισμού

Playlist

Παρέχει το τρέχον τραγούδι (get current song)

Song

Οντότητα τραγουδιού που αναλύεται

BeatAnalyzer

Αναλύει τα beats του τραγουδιού (analyze beat, extract beat)

DataMapper

Αντιστοιχεί beat δεδομένα σε χρωματικά μοτίβα (map beat to colors)

TransmissionController

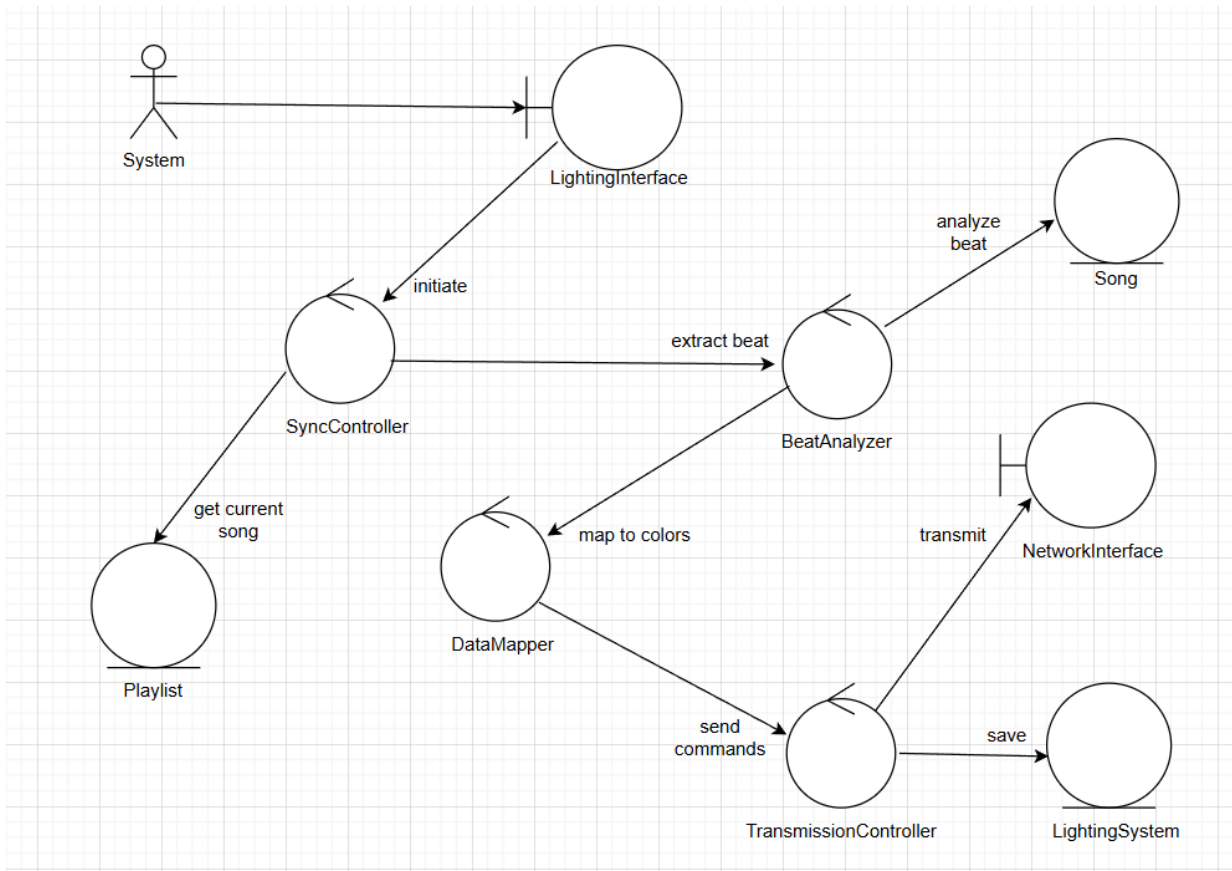
Στέλνει εντολές (send commands) και αποθηκεύει κατάσταση (save)

NetworkInterface

Διαχειρίζεται την επικοινωνία μέσω δικτύου (transmit)

LightingSystem

Εξωτερικό σύστημα φωτισμού που εκτελεί τις εντολές



USE CASE 11: MANAGE SUBSCRIPTION / PURCHASE

1. Σκοπός του Συστήματος

Το σύστημα επιτρέπει σε έναν χρήστη τύπου **PartyHost** να:

- επιλέξει συνδρομητικό πλάνο,
- υποβάλει αίτημα συνδρομής,
- γίνει validation των δεδομένων,

- υπολογιστεί το pricing,
 - πραγματοποιηθεί πληρωμή,
 - αποθηκευτεί η συνδρομή,
 - και να εμφανιστεί επιβεβαίωση ή μήνυμα σφάλματος.
-

2. Κατηγορίες Αντικειμένων

Το robustness diagram ακολουθεί τη διάκριση:

Τύπος	Ρόλος
Boundary Objects	Επικοινωνία με χρήστη
Control Objects	Επιχειρησιακή λογική
Entity Objects	Δεδομένα / domain objects

3. Boundary Objects (UI)

3.1 PartyHost

Ο βασικός actor του συστήματος.

Ρόλος:

- ξεκινά τη διαδικασία αγοράς συνδρομής.
-

3.2 SubscriptionUI

Η διεπαφή μέσω της οποίας ο χρήστης:

- επιλέγει πλάνο,
- εισάγει στοιχεία,

- υποβάλλει αίτημα συνδρομής.

Λειτουργίες:

- collect subscription data,
- submit request.

Επικοινωνεί με:

- RequestHandler.
-

3.3 PaymentUI

Διεπαφή πληρωμής.

Ρόλος:

- συλλογή payment details,
- ολοκλήρωση πληρωμής.

Ενεργοποιείται από:

- PaymentOrchestrator.
-

3.4 ConfirmationUI

Εμφανίζει μήνυμα επιτυχίας.

Παράδειγμα:

Subscription activated successfully

3.5 ErrorUI

Εμφανίζει σφάλματα.

Πιθανές περιπτώσεις:

- invalid data,
 - payment failed.
-

4. Control Objects

4.1 RequestHandler

Πρώτο control object που δέχεται το request από το UI.

Ρόλος:

- παραλαμβάνει το submit,
- προωθεί το request για validation.

Επικοινωνεί με:

- ValidationController.
-

4.2 ValidationController

Ελέγχει την εγκυρότητα των δεδομένων.

Πιθανοί έλεγχοι:

- required fields,
- valid email,
- selected plan exists,
- valid billing information.

Αν τα δεδομένα είναι invalid:

ValidationController → ErrorUI

Αν είναι valid:

ValidationController → PricingService

4.3 PricingService

Υπολογίζει το κόστος της συνδρομής.

Ρόλος:

- ανάκτηση στοιχείων πλάνου,
- pricing calculation,
- discounts/taxes handling.

Επικοινωνεί με:

- Plan entity,
- SubscriptionController.

Λειτουργίες:

- get details από Plan,
 - επιστροφή validated plan.
-

4.4 SubscriptionController

Ο βασικός controller της διαδικασίας subscription.

Ρόλος:

- διαχείριση validated subscription,
- εκκίνηση πληρωμής,
- αποθήκευση subscription,
- εμφάνιση επιβεβαίωσης.

Επικοινωνεί με:

- PaymentOrchestrator,
 - PersistenceController,
 - ConfirmationUI.
-

4.5 PaymentOrchestrator

Διαχειρίζεται τη διαδικασία πληρωμής.

Ρόλος:

- request payment,
- execute payment,
- χειρισμός αποτυχημένων συναλλαγών.

Επικοινωνεί με:

- PaymentUI,
- ErrorUI.

Περιπτώσεις:

Επιτυχία:

επιστρέφει success στον SubscriptionController.

Αποτυχία:

PaymentOrchestrator → ErrorUI

4.6 PersistenceController

Υπεύθυνος για μόνιμη αποθήκευση.

Ρόλος:

- save subscription,

- persist data στη βάση.

Επικοινωνεί με:

- Subscription entity.
-

5. Entity Objects

5.1 Plan

Αναπαριστά συνδρομητικό πλάνο.

Περιέχει:

- name,
- price,
- duration,
- features.

Χρησιμοποιείται από:

- PricingService.
-

5.2 Subscription

Αναπαριστά ενεργή συνδρομή χρήστη.

Περιέχει:

- subscriber,
- selected plan,
- payment status,
- activation date,

- expiration date.

Αποθηκεύεται μέσω:

- PersistenceController.
-

6. Αναλυτική Ροή Εκτέλεσης

Βήμα 1 — Έναρξη Subscription

Ο PartyHost χρησιμοποιεί το SubscriptionUI.

Ροή:

PartyHost → SubscriptionUI

Ο χρήστης:

- επιλέγει πλάνο,
 - εισάγει στοιχεία,
 - πατά submit.
-

Βήμα 2 — Υποβολή Request

Το SubscriptionUI αποστέλλει το request.

Ροή:

SubscriptionUI → RequestHandler

Βήμα 3 — Validation Δεδομένων

Ο RequestHandler προωθεί το request στον ValidationController.

Ροή:

RequestHandler → ValidationController

Έλεγχοι:

- validity,
 - completeness,
 - billing info.
-

Περίπτωση αποτυχίας

Αν τα δεδομένα είναι invalid:

ValidationController → ErrorUI

Παράδειγμα:

- “Invalid subscription data”
-

Περίπτωση επιτυχίας

Αν το validation περάσει:

ValidationController → PricingService

Βήμα 4 — Pricing & Plan Retrieval

Ο PricingService:

- ανακτά στοιχεία του Plan,
- υπολογίζει κόστος,
- εφαρμόζει taxes/discounts.

Ροή:

PricingService ↔ Plan

Στη συνέχεια:

PricingService → SubscriptionController

Βήμα 5 — Εκτέλεση Πληρωμής

Ο SubscriptionController ξεκινά payment process.

Ροή:

SubscriptionController → PaymentOrchestrator

Ο PaymentOrchestrator:

- ζητά πληρωμή από PaymentUI.

PaymentOrchestrator → PaymentUI

Βήμα 6 — Payment Handling

Αν η πληρωμή αποτύχει:

PaymentOrchestrator → ErrorUI

Παράδειγμα:

- “Payment failed”
-

Αν η πληρωμή πετύχει:

επιστρέφει επιτυχία στον SubscriptionController.

Βήμα 7 — Αποθήκευση Συνδρομής

Ο SubscriptionController ζητά persistence.

Ροή:

SubscriptionController → PersistenceController

PersistenceController → Subscription

Η συνδρομή αποθηκεύεται στη βάση.

Βήμα 8 — Επιβεβαίωση

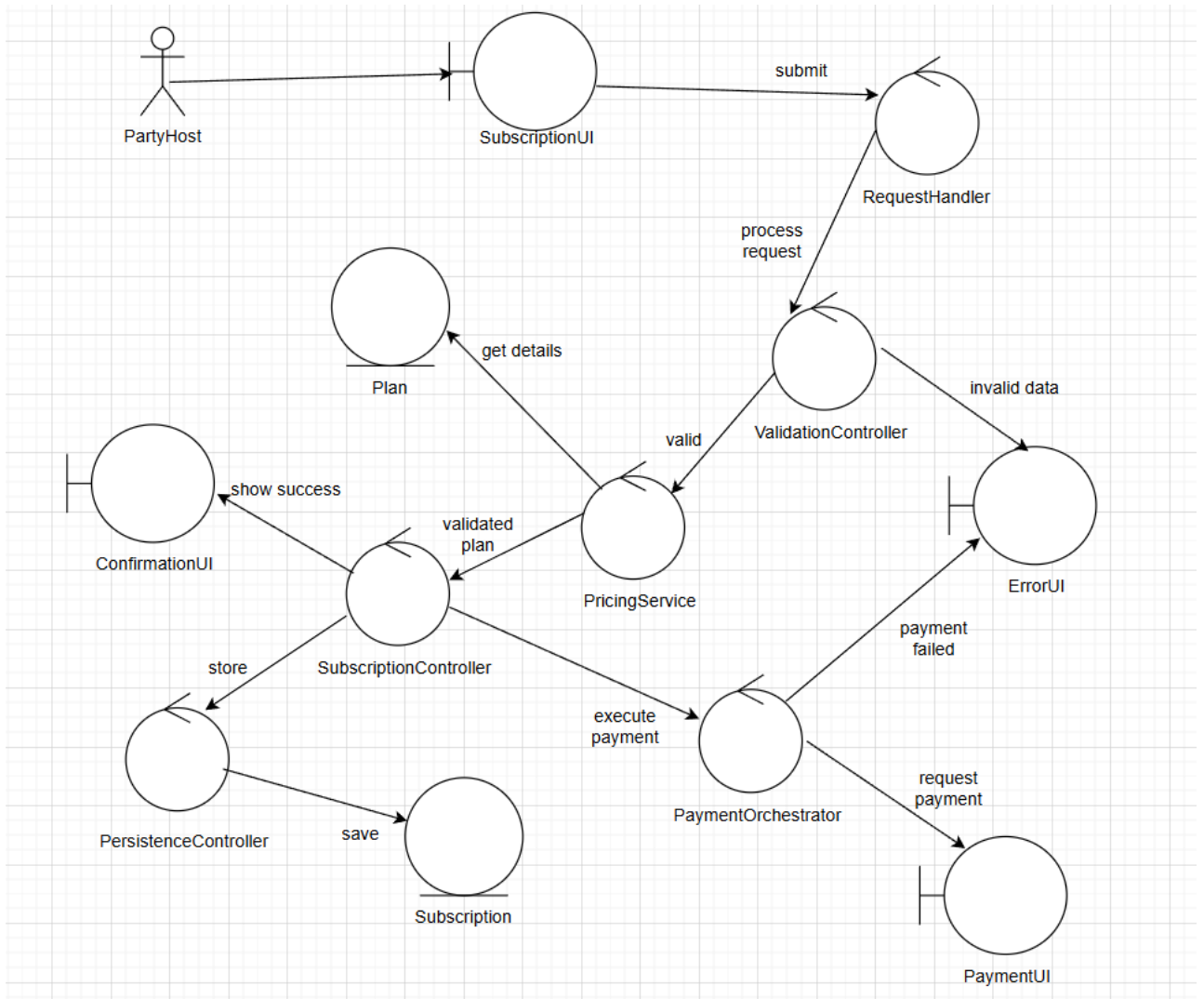
Ο SubscriptionController ενημερώνει το ConfirmationUI.

Ροή:

SubscriptionController → ConfirmationUI

Μήνυμα:

Subscription activated successfully



USE CASE 12: PROCESS PAYMENT

1. Στοιχεία Use Case

Στοιχείο	Τιμή
Όνομα Use Case	Process Payment
Primary Actor	Payment Service (εξωτερικό σύστημα πληρωμών)
Secondary Actors	ExternalPaymentGateway, PaymentController, PersistenceController, ResponseHandler
Στόχος	Το εξωτερικό σύστημα πληρωμών επεξεργάζεται μια πληρωμή που έχει ζητηθεί από το σύστημα, ενημερώνει το εσωτερικό σύστημα για το αποτέλεσμα (επιτυχία ή αποτυχία) και καταγράφει τη συναλλαγή
Τύπος	Βασικός / Εξωτερικής Ολοκλήρωσης

2. Σύντομη Περιγραφή (Brief Description)

Το **Payment Service** (εξωτερικό σύστημα πληρωμών) λαμβάνει ένα αίτημα πληρωμής από το εσωτερικό σύστημα μέσω του **PaymentAPI**. Το αίτημα μορφοποιείται κατάλληλα μέσω του **RequestBuilder** και προωθείται στο **ExternalPaymentGateway** για εξουσιοδότηση (authorize). Το gateway επιστρέφει success ή failure. Το **ResponseHandler** επεξεργάζεται την απάντηση, το **PaymentController** ενημερώνει την πληρωμή και τη συνδρομή, και το **PersistenceController** αποθηκεύει όλες τις αλλαγές. Τέλος,

εμφανίζεται επιβεβαίωση μέσω **ConfirmationUI** ή μήνυμα σφάλματος μέσω **ErrorUI**.

3. Προϋποθέσεις (Preconditions)

1. Έχει ζητηθεί πληρωμή (από το UC11: Manage Subscription / Purchase)
 2. Υπάρχει **έγκυρο Payment object** (με ποσό, νόμισμα, μέθοδο πληρωμής)
 3. Υπάρχει **Subscription** που σχετίζεται με την πληρωμή
 4. Το **ExternalPaymentGateway** είναι διαθέσιμο και λειτουργικό
 5. Το **PaymentAPI** είναι προσβάσιμο
-

4. Βασική Ροή Επιτυχίας (Main Success Scenario)

Βήμα	Ενέργεια / Αλληλεπίδραση	Αντιστοίχιση Diagram
1	Το PaymentService (εξωτερικό σύστημα) λαμβάνει το αίτημα πληρωμής	PaymentService
2	Το PaymentService επικοινωνεί με το PaymentAPI και προετοιμάζει απάντηση (prepare response)	PaymentAPI → prepare response
3	Το PaymentAPI καλεί το RequestBuilder για μορφοποίηση δεδομένων (format data)	RequestBuilder → format data

Βήμα	Ενέργεια / Αλληλεπίδραση	Αντιστοίχιση Diagram
4	Το RequestBuilder κατασκευάζει το αίτημα προς το gateway (forward request)	forward request
5	Το RequestBuilder προωθεί το αίτημα στο ExternalGatewayAdapter	ExternalGatewayAdapter
6	Το ExternalGatewayAdapter επικοινωνεί με το ExternalPaymentGateway για εξουσιοδότηση (authorize)	authorize → ExternalPayment
7	Το ExternalPaymentGateway επεξεργάζεται την πληρωμή και επιστρέφει success ή failure	success / failure
8	Το ResponseHandler λαμβάνει την απάντηση (success / failure)	ResponseHandler
9	Αν success: Το ResponseHandler καλεί το PaymentController για καταγραφή (record, update)	PaymentController → record / update
10	Το PaymentController ενημερώνει το Payment object (κατάσταση, ημερομηνία, transaction ID)	Payment
11	Το PaymentController ενημερώνει το Subscription (π.χ., ενεργοποίηση, παράταση, αναβάθμιση)	Subscription

Βήμα	Ενέργεια / Αλληλεπίδραση	Αντιστοίχιση Diagram
12	Το PaymentController καλεί το PersistenceController για αποθήκευση (store / save)	PersistenceController → store / save
13	Το PersistenceController αποθηκεύει όλες τις αλλαγές	-
14	Το PersistenceController εμφανίζει μήνυμα επιτυχίας μέσω ConfirmationUI (show)	ConfirmationUI → show
15	Αν failure (από Βήμα 8): Το ResponseHandler καλεί το ErrorUI για εμφάνιση μηνύματος αποτυχίας	ErrorUI

5. Εναλλακτικές Ροές (Alternative Flows)

Κωδικός	Σημείο Απόκλισης	Περιγραφή
A1	Βήμα 7	Αποτυχία πληρωμής – Το ExternalPaymentGateway επιστρέφει failure (π.χ., ανεπαρκές υπόλοιπο, ληγμένη κάρτα, λανθασμένα στοιχεία). Το ErrorUI εμφανίζει μήνυμα και η πληρωμή δεν καταγράφεται ως επιτυχής
A2	Βήμα 6-7	Timeout / μη διαθέσιμο gateway – Το ExternalPaymentGateway δεν ανταποκρίνεται εντός

Κωδικός	Σημείο Απόκλισης	Περιγραφή
		χρονικού ορίου. Το σύστημα κάνει retry ή καταγράφει αποτυχία
A3	Βήμα 12-13	Αποτυχία αποθήκευσης – Το PersistenceController δεν μπορεί να αποθηκεύσει (π.χ., DB offline). Η πληρωμή έχει ολοκληρωθεί αλλά ΔΕΝ καταγράφηκε → ανάγκη για reconciliation
A4	Βήμα 4-5	Μη έγκυρη μορφοποίηση δεδομένων – Το RequestBuilder αποτυγχάνει να δημιουργήσει έγκυρο αίτημα. Το ErrorUI εμφανίζει μήνυμα
A5	Βήμα 2-3	Αποτυχία λήψης αιτήματος – Το PaymentAPI δεν λαμβάνει σωστό αίτημα. Η διαδικασία τερματίζεται

6. Μετασυνθήκες (Postconditions)

Μετά την επιτυχή ολοκλήρωση του use case (ροή επιτυχίας):

1. Η **πληρωμή έχει καταγραφεί** στο σύστημα (με κατάσταση success)
2. Το **Subscription έχει ενημερωθεί** ανάλογα (ενεργοποιήθηκε, ανανεώθηκε, αναβαθμίστηκε)
3. Το **Payment object** περιέχει όλα τα απαραίτητα στοιχεία (transaction ID, ποσό, ημερομηνία)
4. Όλες οι αλλαγές έχουν **αποθηκευτεί** στο persistence layer

5. Ο χρήστης (Host) έχει λάβει **επιβεβαίωση** (ConfirmationUI)

Μετά από ροή αποτυχίας:

1. Η πληρωμή **δεν καταγράφεται** ως επιτυχής
2. Το Subscription **δεν ενημερώνεται**
3. Εμφανίζεται **μήνυμα σφάλματος** (ErrorUI)

7. Επιχειρηματικοί Κανόνες (Business Rules)

Κανόνας	Περιγραφή
---------	-----------

BR1	Κάθε πληρωμή πρέπει να έχει μοναδικό transaction ID
------------	--

BR2	Η πληρωμή θεωρείται ολοκληρωμένη μόνο μετά από επιτυχή authorize από το ExternalPaymentGateway
------------	---

BR3	Σε περίπτωση success αλλά αποτυχίας save, η ασυνέπεια πρέπει να καταγράφεται για ανθρώπινη παρέμβαση
------------	---

BR4	Δεν επιτρέπονται διπλότυπες πληρωμές για το ίδιο αίτημα (idempotency)
------------	--

BR5	Η πληρωμή συνδέεται υποχρεωτικά με μία Subscription
------------	--

BR6	Το PaymentAPI πρέπει να επικυρώνει όλες τις εισερχόμενες αιτήσεις (π.χ., υπογραφή, timestamp)
------------	--

8. Απαιτήσεις Μη-Λειτουργικότητας (Non-Functional Requirements)

Κατηγορία	Απαίτηση
Απόδοση	Η επεξεργασία πληρωμής (από authorize έως confirmation) πρέπει να γίνεται σε λιγότερο από 5 δευτερόλεπτα
Ασφάλεια	Όλα τα δεδομένα πληρωμής πρέπει να είναι κρυπτογραφημένα (PCI DSS compliance)
Αξιοπιστία	Σε περίπτωση αποτυχίας, το σύστημα πρέπει να υποστηρίζει retry mechanism
Συνέπεια	Η ενημέρωση Payment + Subscription πρέπει να είναι ατομική (atomic) – και οι δύο επιτυχείς ή καμία
Διαθεσιμότητα	Το PaymentAPI πρέπει να είναι διαθέσιμο 99.95% του χρόνου

9. Διάγραμμα Αντιστοίχησης (Component Mapping)

Component στο Robustness Diagram	Ρόλος / Ευθύνη
PaymentService (Primary Actor)	Εξωτερικό σύστημα που εκκινεί τη διαδικασία πληρωμής
PaymentAPI	Υποδοχή αιτημάτων πληρωμής (prepare response)
RequestBuilder	Μορφοποίηση δεδομένων (format data) και

Component στο Robustness Diagram

Ρόλος / Ευθύνη

προώθηση αιτήματος (forward request)

ExternalGatewayAdapter

Προσαρμογέας για επικοινωνία με
ExternalPaymentGateway

ExternalPaymentGateway

Εξωτερική πύλη πληρωμών (π.χ., Stripe, PayPal)
εκτελεί authorize

ResponseHandler

Επεξεργάζεται την απάντηση (success / failure)

PaymentController

Ενημερώνει Payment και Subscription (record,
update)

Payment

Οντότητα πληρωμής

Subscription

Οντότητα συνδρομής (ενημερώνεται βάσει
πληρωμής)

PersistenceController

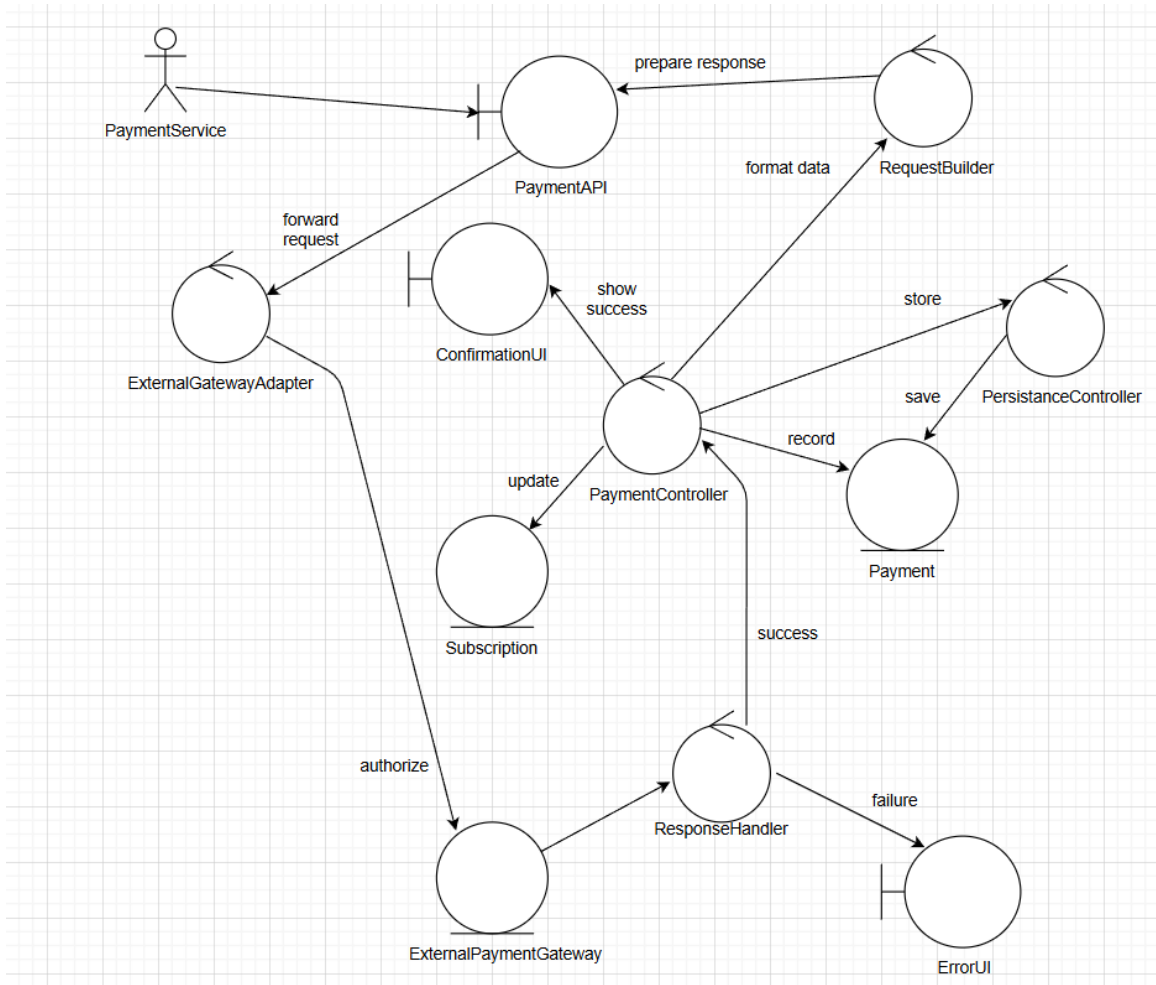
Αποθηκεύει αλλαγές (store, save)

ConfirmationUI

Επιβεβαίωση επιτυχίας (show)

ErrorUI

Εμφάνιση μηνύματος αποτυχίας



5) SEQUENCE DIAGRAMS

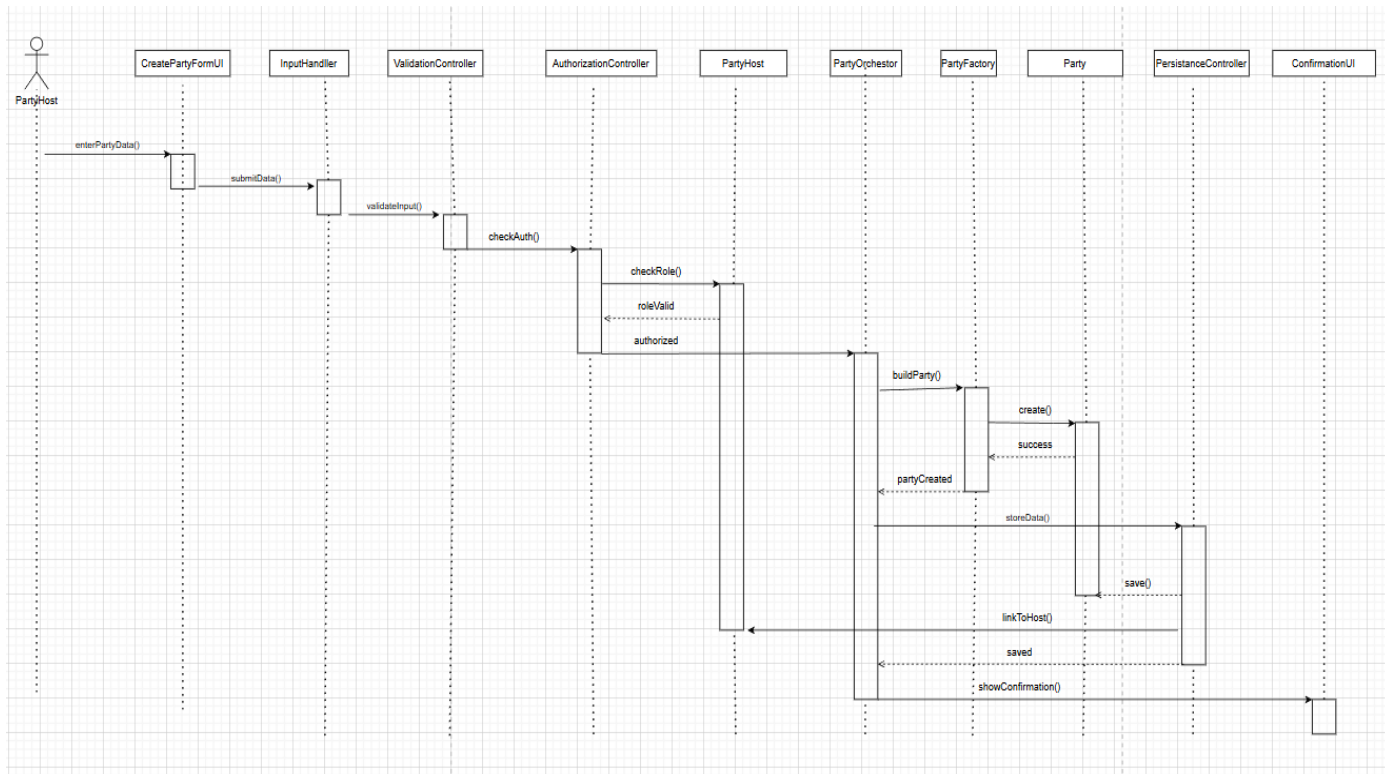
USE CASE 1: CREATE PARTY EVENT

Αντικείμενα που συμμετέχουν: PartyHost (ηθοποιός), createPartyFormUI, inputHandler, validationController, authorizationController, partyFactory, party, persistenceController, confirmationUI

Βήμα προς βήμα περιγραφή:

1. Ο PartyHost καλεί το createPartyFormUI με το μήνυμα enterPartyData(), εισάγοντας τα στοιχεία του νέου πάρτι.
2. Ο PartyHost καλεί το createPartyFormUI με το μήνυμα submitData(), υποβάλλοντας τη φόρμα.
3. Το createPartyFormUI καλεί το inputHandler με το μήνυμα submitData(), προωθώντας τα δεδομένα.
4. Το inputHandler καλεί το validationController με το μήνυμα validationInput(), ζητώντας έλεγχο εγκυρότητας.
5. Το validationController καλεί το authorizationController με το μήνυμα checkAuth(), ζητώντας έλεγχο δικαιωμάτων.
6. Το authorizationController καλεί τον εαυτό του με το μήνυμα checkRole(), εκτελώντας εσωτερικό έλεγχο ρόλου.
7. Το authorizationController επιστρέφει στο validationController με το μήνυμα roleValid(), επιβεβαιώνοντας ότι ο ρόλος είναι έγκυρος.
8. Το validationController επιστρέφει στο inputHandler με το μήνυμα authorized(), δηλώνοντας ότι ο χρήστης είναι εξουσιοδοτημένος.

9. Το `inputHandler` καλεί το `partyFactory` με το μήνυμα `buildParty()`, ζητώντας τη δημιουργία του πάρτι.
10. Το `partyFactory` καλεί τον εαυτό του με το μήνυμα `create()`, δημιουργώντας το αντικείμενο `Party`.
11. Το `partyFactory` επιστρέφει στο `inputHandler` με το μήνυμα `success()`, επιβεβαιώνοντας την επιτυχή δημιουργία.
12. Το `inputHandler` καλεί το `partyFactory` με το μήνυμα `partyCreated()`, επιβεβαιώνοντας ότι το πάρτι δημιουργήθηκε.
13. Το `partyFactory` καλεί το `persistenceController` με το μήνυμα `storeData()`, ζητώντας αποθήκευση.
14. Το `persistenceController` καλεί τον εαυτό του με το μήνυμα `save()`, εκτελώντας την αποθήκευση.
15. Το `persistenceController` επιστρέφει στο `partyFactory` με το μήνυμα `save()`, επιβεβαιώνοντας την αποθήκευση.
16. Το `partyFactory` καλεί το `party` με το μήνυμα `linkToHost()`, συνδέοντας το νέο πάρτι με τον `PartyHost`.
17. Το `partyFactory` καλεί ξανά το `persistenceController` με το μήνυμα `save()`, αποθηκεύοντας τη σύνδεση.
18. Το `persistenceController` καλεί το `confirmationUI` με το μήνυμα `showConfirmation()`, εμφανίζοντας μήνυμα επιτυχίας.
19. Ο `PartyHost` βλέπει την οπτική επιβεβαίωση μέσω του `confirmationUI`.



USE CASE 2: SELECT PARTY THEME

Αντικείμενα που συμμετέχουν: PartyHost (ηθοποιός), ThemeSelectionUI, InputHandler, ThemeValidationController, ThemeService, PartyController, Party, PersistenceController, ThemePreviewUI

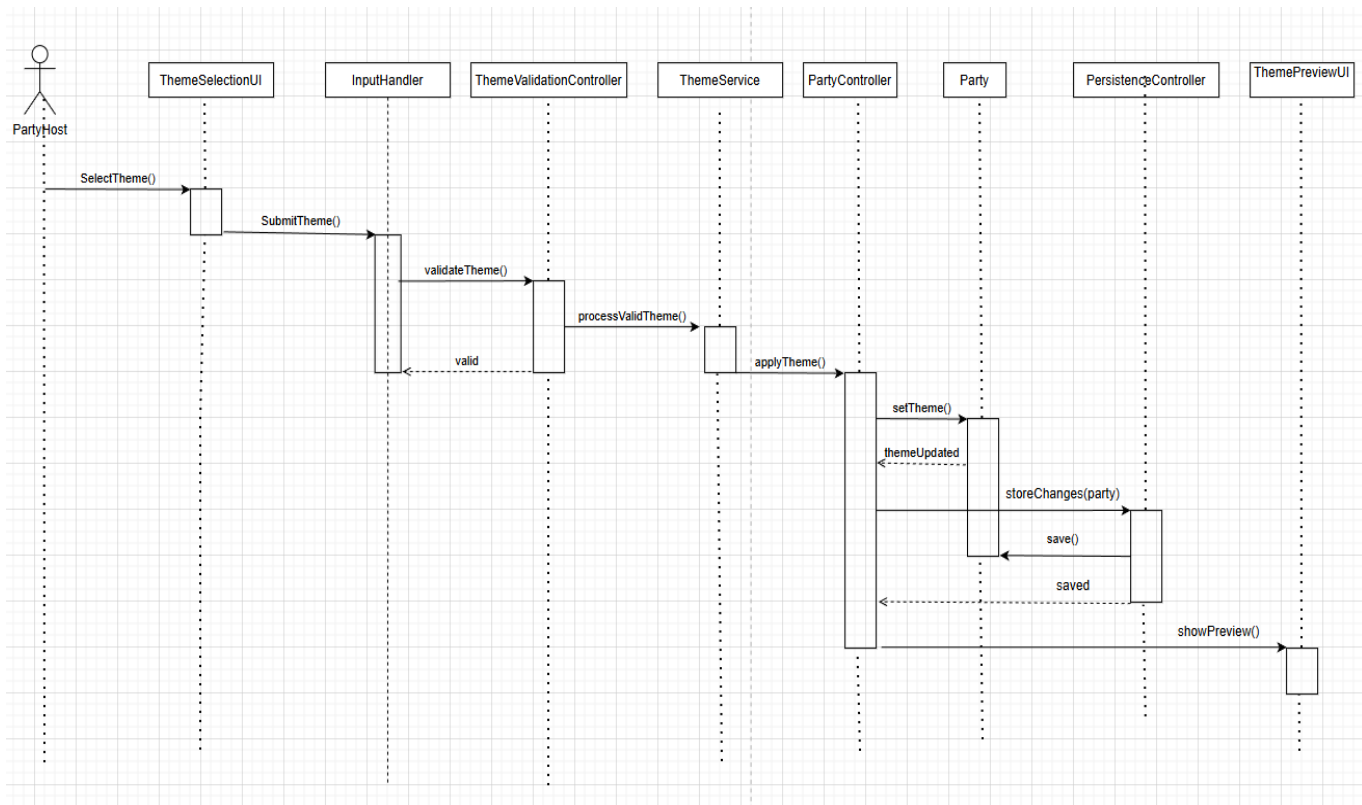
Βήμα προς βήμα περιγραφή:

- Ο PartyHost καλεί το ThemeSelectionUI με το μήνυμα SelectTheme(), επιλέγοντας ένα μουσικό θέμα (π.χ., Rock, Pop, Jazz)
- Ο PartyHost καλεί το ThemeSelectionUI με το μήνυμα submitTheme(), υποβάλλοντας την επιλογή

- Το ThemeSelectionUI καλεί το InputHandler με το μήνυμα submitTheme(), προωθώντας την επιλογή
 - Το InputHandler καλεί το ThemeValidationController με το μήνυμα validateTheme(), ζητώντας έλεγχο εγκυρότητας του theme
 - Το ThemeValidationController καλεί τον εαυτό του με το μήνυμα processValidateTheme(), εκτελώντας εσωτερική επεξεργασία επικύρωσης
 - Το ThemeValidationController καλεί το ThemeService με το μήνυμα applyTheme(), ζητώντας να εφαρμοστεί το theme
 - Το ThemeService καλεί το PartyController με το μήνυμα setTheme(), ζητώντας να οριστεί το theme στο πάρτι
 - Το PartyController καλεί το Party με το μήνυμα themeUpdated(), ενημερώνοντας το αντικείμενο Party
 - Το PartyController καλεί το PersistenceController με το μήνυμα storeChanges(party), ζητώντας αποθήκευση των αλλαγών
 - Το PersistenceController καλεί τον εαυτό του με το μήνυμα save(), εκτελώντας την αποθήκευση
 - Το PersistenceController επιστρέφει στο PartyController με το μήνυμα save(), επιβεβαιώνοντας την αποθήκευση
 - Το ThemeValidationController καλεί το ThemePreviewUI με το μήνυμα showPreview(), εμφανίζοντας προεπισκόπηση του theme (π.χ., sample playlist)
 - Ο PartyHost βλέπει την προεπισκόπηση και την επιβεβαίωση επιτυχίας
-

Σημειώσεις:

- Το `processValidateTheme()` είναι αυτο-κλήση (self-call) – υποδηλώνει εσωτερική λογική επικύρωσης
- Το `ThemePreviewUI` εμφανίζεται αφού ολοκληρωθεί η αποθήκευση, προσφέροντας οπτική ανατροφοδότηση
- Το `ThemeService` λειτουργεί ως ενδιάμεσος για την εφαρμογή του theme στο Party μέσω του `PartyController`
- Η ροή δείχνει ότι πρώτα γίνεται validation, μετά εφαρμογή, μετά αποθήκευση και τέλος preview



USE CASE 3: SET PARTY DURATION

Αντικείμενα που συμμετέχουν: PartyHost (ηθοποιός), DurationInputUI, InputHandler, DurationValidationController, BusinessRulesController, PartyController, Party, PersistenceController, ConfirmationUI

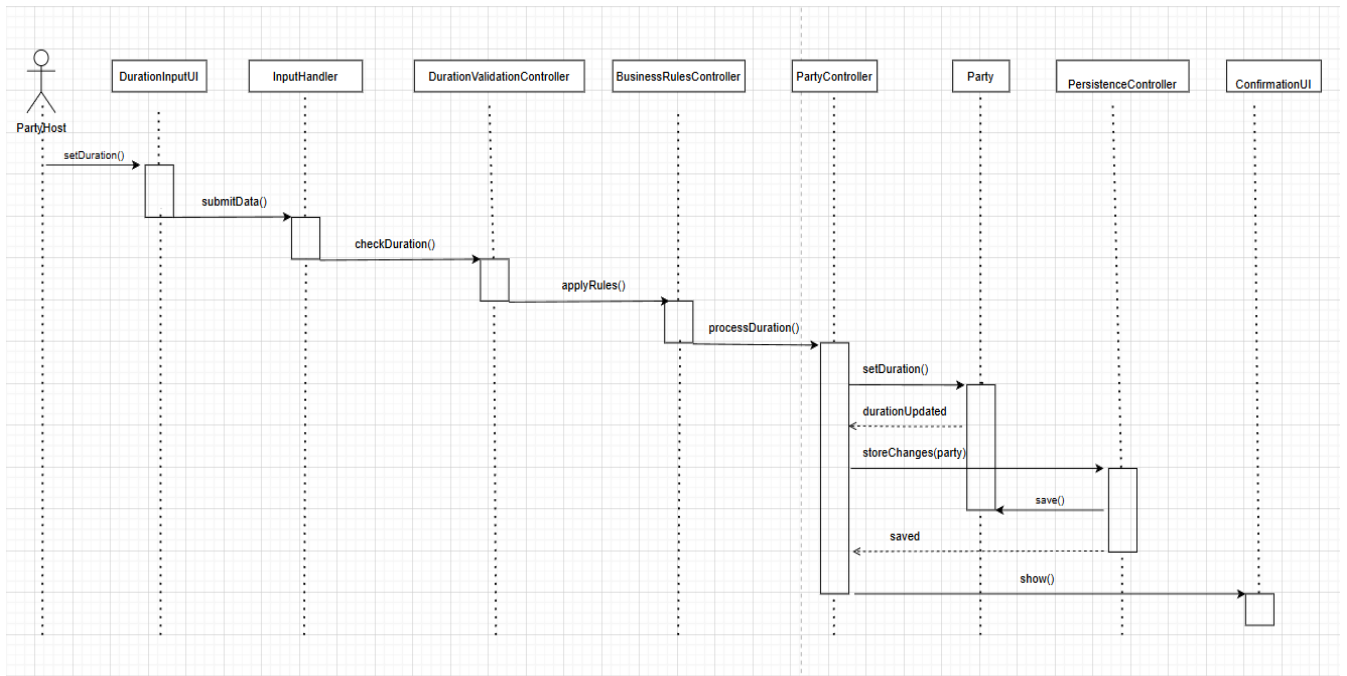
Βήμα προς βήμα περιγραφή:

- Ο PartyHost καλεί το DurationInputUI με το μήνυμα setDuration(), εισάγοντας τη διάρκεια του πάρτι (π.χ., 120 λεπτά)
- Ο PartyHost καλεί το DurationInputUI με το μήνυμα submitData(), υποβάλλοντας τη διάρκεια
- Το DurationInputUI καλεί το InputHandler με το μήνυμα submitData(), προωθώντας τα δεδομένα
- Το InputHandler καλεί το DurationValidationController με το μήνυμα checkDuration(), ζητώντας βασικό έλεγχο εγκυρότητας (μη αρνητική, αριθμητική, μη μηδενική)
- Το DurationValidationController καλεί τον εαυτό του με το μήνυμα applyRules(), εκτελώντας εσωτερική εφαρμογή κανόνων
- Το DurationValidationController καλεί το BusinessRulesController με το μήνυμα processDuration(), ζητώντας έλεγχο επιχειρηματικών κανόνων (ελάχιστη και μέγιστη διάρκεια)
- Το BusinessRulesController καλεί το PartyController με το μήνυμα setDuration(), ζητώντας να οριστεί η διάρκεια στο πάρτι
- Το PartyController καλεί το Party με το μήνυμα durationUpdated(), ενημερώνοντας το αντικείμενο Party με τη νέα διάρκεια
- Το PartyController καλεί το PersistenceController με το μήνυμα storeChanges(party), ζητώντας αποθήκευση των αλλαγών

- Το PersistenceController καλεί τον εαυτό του με το μήνυμα save(), εκτελώντας την αποθήκευση
 - Το PersistenceController επιστρέφει στο PartyController με το μήνυμα save(), επιβεβαιώνοντας την αποθήκευση
 - Το PartyController καλεί το ConfirmationUI με το μήνυμα show(), εμφανίζοντας μήνυμα επιτυχίας
 - Ο PartyHost βλέπει την επιβεβαίωση ότι η διάρκεια ορίστηκε επιτυχώς
-

Σημειώσεις:

- Το applyRules() είναι αυτο-κλήση (self-call) – υποδηλώνει εσωτερική εφαρμογή κανόνων επικύρωσης
- Η ροή δείχνει δύο επίπεδα validation: πρώτα το DurationValidationController (βασικός έλεγχος) και μετά το BusinessRulesController (επιχειρηματικοί κανόνες)
- Το storeChanges(party) μεταβιβάζει ολόκληρο το αντικείμενο Party για αποθήκευση
- Η αποθήκευση γίνεται μέσω save() δύο φορές (μία ως self-call και μία ως επιστροφή)
- Το ConfirmationUI εμφανίζεται μετά την επιτυχή αποθήκευση



USE CASE 4: GENERATE PLAYLIST

Αντικείμενα που συμμετέχουν: PartyHost (ηθοποιός), PlaylistUI / PartyController, Party, PlaylistGenerator, Song, Playlist, PersistenceController

Βήμα προς βήμα περιγραφή:

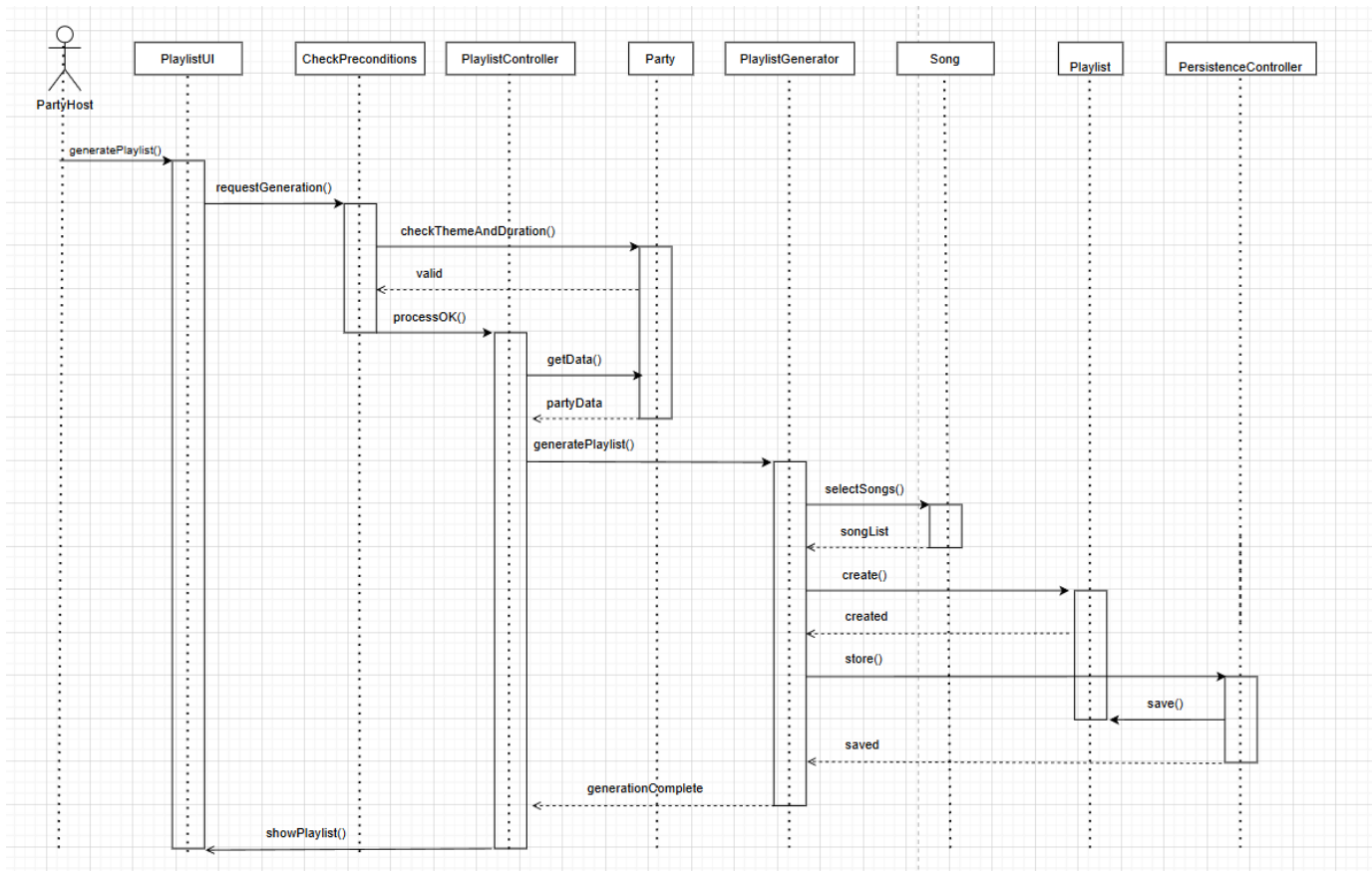
- Ο PartyHost καλεί το PlaylistUI (ή PartyController) με το μήνυμα `generatePlaylist()`, επιλέγοντας τη λειτουργία δημιουργίας playlist
- Ο PartyHost καλεί το PlaylistUI με το μήνυμα `requestGeneration()`, υποβάλλοντας το αίτημα

- Το PlaylistUI καλεί τον εαυτό του (ή το PartyController) με το μήνυμα checkThemeAndDuration(), ελέγχοντας αν έχουν επιλεγεί theme και διάρκεια
- Σε περίπτωση επιτυχίας, επιστρέφει valid
- Το σύστημα προχωρά με το μήνυμα processOK(), επιβεβαιώνοντας ότι πληρούνται οι προϋποθέσεις
- Το PartyController καλεί το Party με το μήνυμα getData(), ζητώντας τα δεδομένα του πάρτι (theme και διάρκεια)
- Το Party επιστρέφει partyData (τα δεδομένα του πάρτι)
- Το PartyController καλεί το PlaylistGenerator με το μήνυμα generatePlaylist(), ζητώντας τη δημιουργία playlist
- Το PlaylistGenerator καλεί το Song με το μήνυμα selectSongs(), αναζητώντας τραγούδια που ταιριάζουν στο theme
- Το Song επιστρέφει songList (λίστα με τα διαθέσιμα τραγούδια)
- Το PlaylistGenerator καλεί τον εαυτό του με το μήνυμα create(), δημιουργώντας το αντικείμενο Playlist
- Επιστρέφει created (επιβεβαίωση δημιουργίας)
- Το PlaylistGenerator καλεί το PersistenceController με το μήνυμα store(), ζητώντας αποθήκευση της playlist
- Το PersistenceController επιστρέφει saved, επιβεβαιώνοντας την αποθήκευση
- Το PlaylistGenerator επιστρέφει στο PartyController με το μήνυμα save(), επιβεβαιώνοντας την ολοκλήρωση
- Το PersistenceController καλεί τον εαυτό του με το μήνυμα save(), εκτελώντας την τελική αποθήκευση
- Επιστρέφει saved

- Το PartyController καλεί το PlaylistUI με το μήνυμα showPlaylist(), εμφανίζοντας τη νέα playlist στον Host
 - Ο PartyHost βλέπει την playlist που δημιουργήθηκε
-

Σημειώσεις:

- Το checkThemeAndDuration() είναι αυτο-κλήση που ελέγχει τις προϋποθέσεις πριν τη δημιουργία
- Το processOK() είναι το σήμα ότι όλες οι προϋποθέσεις πληρούνται
- Το PlaylistGenerator είναι το κεντρικό component για τη δημιουργία playlist
- Η ροή selectSongs() → songList δείχνει ότι τα τραγούδια επιλέγονται από το Song με βάση το theme
- Η δημιουργία του Playlist γίνεται με αυτο-κλήση create() μέσα στο PlaylistGenerator
- Η αποθήκευση γίνεται σε δύο επίπεδα: store() από PlaylistGenerator προς PersistenceController και save() ως self-call
- Το showPlaylist() εμφανίζει το τελικό αποτέλεσμα στον Host



USE CASE 5: MODIFY PLAYLIST

Αντικείμενα που συμμετέχουν: PartyHost (ηθοποιός), PlaylistEditorUI, EditRequestHandler, ValidationController, SongService, PlaylistController, Playlist, PersistenceController, ConfirmationUI

Βήμα προς βήμα περιγραφή:

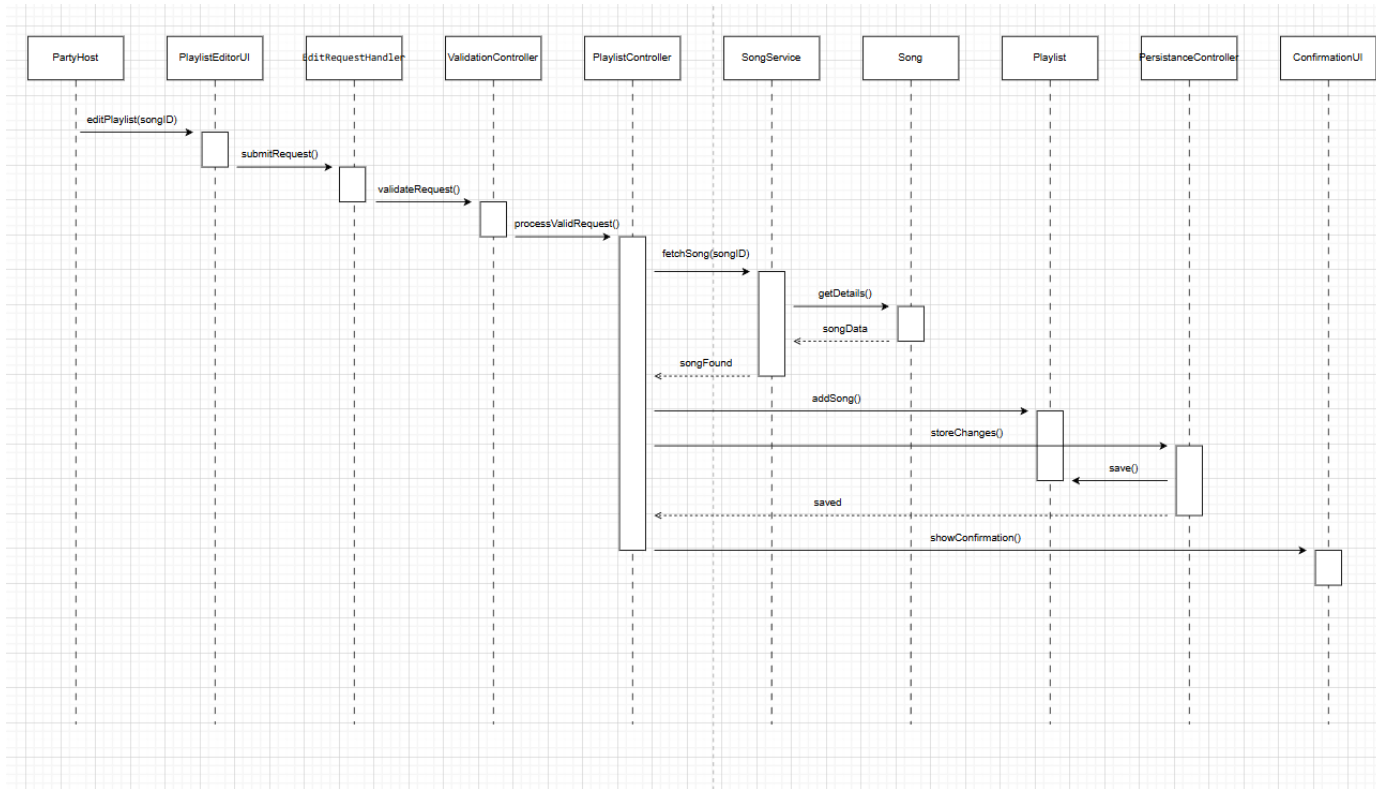
- Ο PartyHost καλεί το PlaylistEditorUI με το μήνυμα editPlaylist(songID), επιλέγοντας ένα τραγούδι για προσθήκη ή αφαίρεση

- Ο PartyHost καλεί το PlaylistEditorUI με το μήνυμα submitRequest(), υποβάλλοντας το αίτημα τροποποίησης
- Το PlaylistEditorUI καλεί το EditRequestHandler με το μήνυμα submitRequest(), προωθώντας το αίτημα
- Το EditRequestHandler καλεί το ValidationController με το μήνυμα validateRequest(), ζητώντας έλεγχο εγκυρότητας
- Το ValidationController καλεί τον εαυτό του με το μήνυμα processValidateRequest(), εκτελώντας εσωτερική επεξεργασία επικύρωσης
- Το ValidationController καλεί το SongService με το μήνυμα fetchSong(songID), ζητώντας το τραγούδι με το συγκεκριμένο ID
- Το SongService καλεί τον εαυτό του με το μήνυμα getDetails(), ανακτώντας τις λεπτομέρειες του τραγουδιού
- Το SongService επιστρέφει στο ValidationController με το μήνυμα songData, στέλνοντας τα δεδομένα του τραγουδιού
- Το ValidationController επιστρέφει στο EditRequestHandler με το μήνυμα songFound, επιβεβαιώνοντας ότι το τραγούδι βρέθηκε
- Το EditRequestHandler καλεί το PlaylistController με το μήνυμα addSong(), ζητώντας προσθήκη του τραγουδιού στην playlist
- Το PlaylistController καλεί το Playlist με το μήνυμα storeChanges(), ενημερώνοντας την playlist
- Το PlaylistController καλεί το PersistenceController με το μήνυμα save(), ζητώντας αποθήκευση
- Το PersistenceController καλεί τον εαυτό του με το μήνυμα save(), εκτελώντας την αποθήκευση

- Το PersistenceController επιστρέφει στο PlaylistController με το μήνυμα save(), επιβεβαιώνοντας την αποθήκευση
 - Το PlaylistController καλεί το ConfirmationUI με το μήνυμα showConfirmation(), εμφανίζοντας μήνυμα επιτυχίας
 - Ο PartyHost βλέπει την επιβεβαίωση ότι η playlist τροποποιήθηκε επιτυχώς
-

Σημειώσεις:

- Το processValidateRequest() είναι αυτο-κλήση – υποδηλώνει εσωτερική λογική επικύρωσης
- Το getDetails() είναι αυτο-κλήση μέσα στο SongService για ανάκτηση λεπτομερειών τραγουδιού
- Η ροή fetchSong(songID) → songData → songFound δείχνει την αναζήτηση και επιβεβαίωση του τραγουδιού
- Το addSong() είναι η κεντρική ενέργεια τροποποίησης (για αφαίρεση θα υπήρχε removeSong())
- Η αποθήκευση γίνεται με save() από PlaylistController προς PersistenceController και αυτο-κλήση save() στο PersistenceController
- Το storeChanges() ενημερώνει το αντικείμενο Playlist πριν την αποθήκευση
- Το ConfirmationUI εμφανίζεται μετά την επιτυχή αποθήκευση



USE CASE 6: EXTEND PARTY DURATION

Αντικείμενα που συμμετέχουν: PartyHost (ηθοποιός), DurationExtensionUI, InputHandler, ValidationController, RulesController, PartyController, Party, PersistenceController, ConfirmationUI

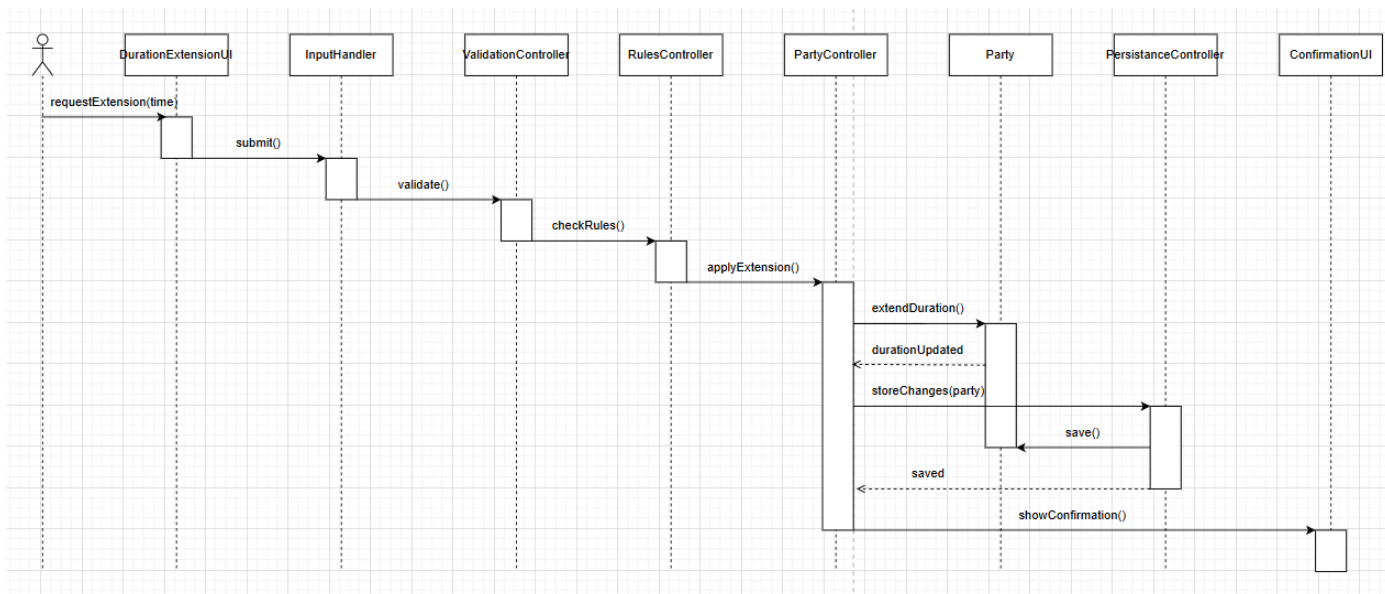
Βήμα προς βήμα περιγραφή:

- Ο PartyHost καλεί το DurationExtensionUI με το μήνυμα requestExtension(time), εισάγοντας την επιπλέον διάρκεια (π.χ., +30 λεπτά)

- Ο PartyHost καλεί το DurationExtensionUI με το μήνυμα submit(), υποβάλλοντας το αίτημα επέκτασης
 - Το DurationExtensionUI καλεί το InputHandler με το μήνυμα submit(), προωθώντας το αίτημα
 - Το InputHandler καλεί το ValidationController με το μήνυμα validate(), ζητώντας βασικό έλεγχο εγκυρότητας (μη αρνητική, αριθμητική τιμή)
 - Το ValidationController καλεί το RulesController με το μήνυμα checkRules(), ζητώντας έλεγχο επιχειρηματικών κανόνων (μέγιστο συνολικό όριο)
 - Το RulesController καλεί το PartyController με το μήνυμα applyExtension(), ζητώντας να εφαρμοστεί η επέκταση
 - Το PartyController καλεί το Party με το μήνυμα extendDuration(), ενημερώνοντας το αντικείμενο Party για την αύξηση διάρκειας
 - Το Party επιστρέφει στο PartyController με το μήνυμα durationUpdated, επιβεβαιώνοντας την ενημέρωση
 - Το PartyController καλεί το PersistenceController με το μήνυμα storeChanges(party), ζητώντας αποθήκευση των αλλαγών
 - Το PersistenceController καλεί τον εαυτό του με το μήνυμα save(), εκτελώντας την αποθήκευση
 - Το PersistenceController επιστρέφει στο PartyController με το μήνυμα save, επιβεβαιώνοντας την αποθήκευση
 - Το PartyController καλεί το ConfirmationUI με το μήνυμα showConfirmation(), εμφανίζοντας μήνυμα επιτυχίας
 - Ο PartyHost βλέπει την επιβεβαίωση ότι η διάρκεια επεκτάθηκε επιτυχώς
-

Σημειώσεις:

- Το `requestExtension(time)` περιλαμβάνει την παράμετρο `time` (επιπλέον λεπτά/ώρες)
- Το `ValidationController` εκτελεί βασικό έλεγχο (μη αρνητικό, αριθμητικό)
- Το `RulesController` ελέγχει αν η νέα συνολική διάρκεια υπερβαίνει το μέγιστο όριο
- Το `applyExtension()` είναι η εντολή για εφαρμογή της επέκτασης
- Το `extendDuration()` και η επιστροφή `durationUpdated` δείχνουν την ενημέρωση του αντικειμένου `Party`
- Το `storeChanges(party)` μεταβιβάζει ολόκληρο το αντικείμενο `Party` για αποθήκευση
- Η αποθήκευση γίνεται με `save()` από `PartyController` προς `PersistenceController` και αυτο-κλήση `save()` στο `PersistenceController`
- Το `ConfirmationUI` εμφανίζεται μετά την επιτυχή αποθήκευση



USE CASE 7: ACTIVATE CLOSING MODE

Αντικείμενα που συμμετέχουν: PartyHost (ηθοποιός), ControlPanelUI, ActionHandler, ModeController, PartyController, Party, PersistenceController, ConfirmationUI

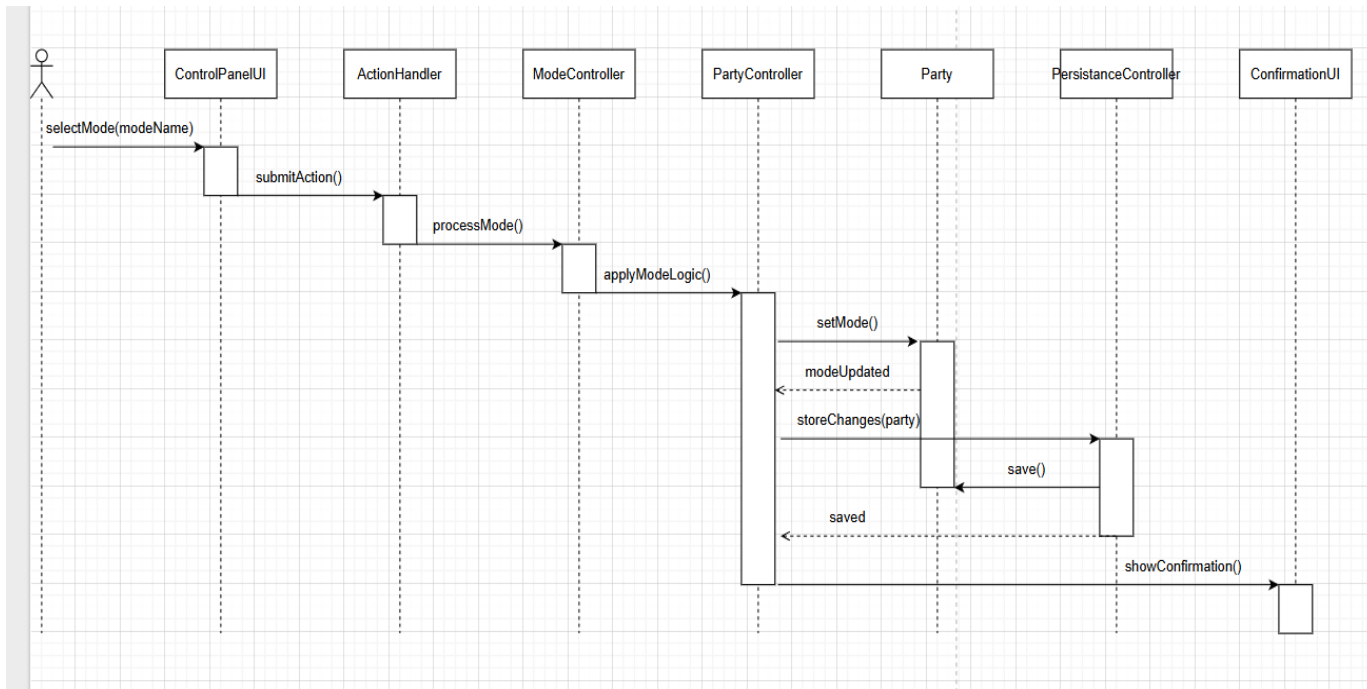
Βήμα προς βήμα περιγραφή:

- Ο PartyHost καλεί το ControlPanelUI με το μήνυμα selectMode(modeName), επιλέγοντας τη λειτουργία "Closing Mode"
- Ο PartyHost καλεί το ControlPanelUI με το μήνυμα submitAction(), υποβάλλοντας την ενέργεια
- Το ControlPanelUI καλεί το ActionHandler με το μήνυμα submitAction(), προωθώντας την ενέργεια
- Το ActionHandler καλεί το ModeController με το μήνυμα processMode(), ζητώντας επεξεργασία της αλλαγής mode
- Το ModeController καλεί τον εαυτό του με το μήνυμα applyModeLogic(), εκτελώντας εσωτερική λογική εφαρμογής του mode
- Το ModeController καλεί το PartyController με το μήνυμα setMode(), ζητώντας να οριστεί το closing mode στο πάρτι
- Το PartyController καλεί το Party με το μήνυμα modeUpdated, ενημερώνοντας το αντικείμενο Party για την αλλαγή κατάστασης
- Το PartyController καλεί το PersistenceController με το μήνυμα storeChanges(party), ζητώντας αποθήκευση των αλλαγών
- Το PersistenceController καλεί τον εαυτό του με το μήνυμα save(), εκτελώντας την αποθήκευση

- Το PersistenceController επιστρέφει στο PartyController με το μήνυμα save, επιβεβαιώνοντας την αποθήκευση
 - Το PartyController καλεί το ConfirmationUI με το μήνυμα showConfirmation(), εμφανίζοντας μήνυμα επιτυχίας
 - Ο PartyHost βλέπει την επιβεβαίωση ότι το closing mode ενεργοποιήθηκε επιτυχώς
-

Σημειώσεις:

- Το selectMode(modeName) περιλαμβάνει την παράμετρο modeName (π.χ., "Closing")
- Το applyModeLogic() είναι αυτο-κλήση – υποδηλώνει εσωτερική λογική εφαρμογής κανόνων του mode
- Το setMode() είναι η εντολή για αλλαγή της κατάστασης του πάρτι
- Το modeUpdated είναι η επιβεβαίωση από το Party ότι η κατάσταση ενημερώθηκε
- Το storeChanges(party) μεταβιβάζει ολόκληρο το αντικείμενο Party για αποθήκευση
- Η αποθήκευση γίνεται με save() από PartyController προς PersistenceController και αυτο-κλήση save() στο PersistenceController
- Το ConfirmationUI εμφανίζεται μετά την επιτυχή αποθήκευση
- Μετά την ενεργοποίηση closing mode, νέα song requests δεν γίνονται δεκτά



USE CASE 8: SCAN QR CODE

Πλήρης Λεκτική Περιγραφή

Αντικείμενα που συμμετέχουν: PartyGuest (ηθοποιός), QRScannerUI, ScanController, QRDecoder, ValidationController, AccessController, Party, AccessGrantedUI

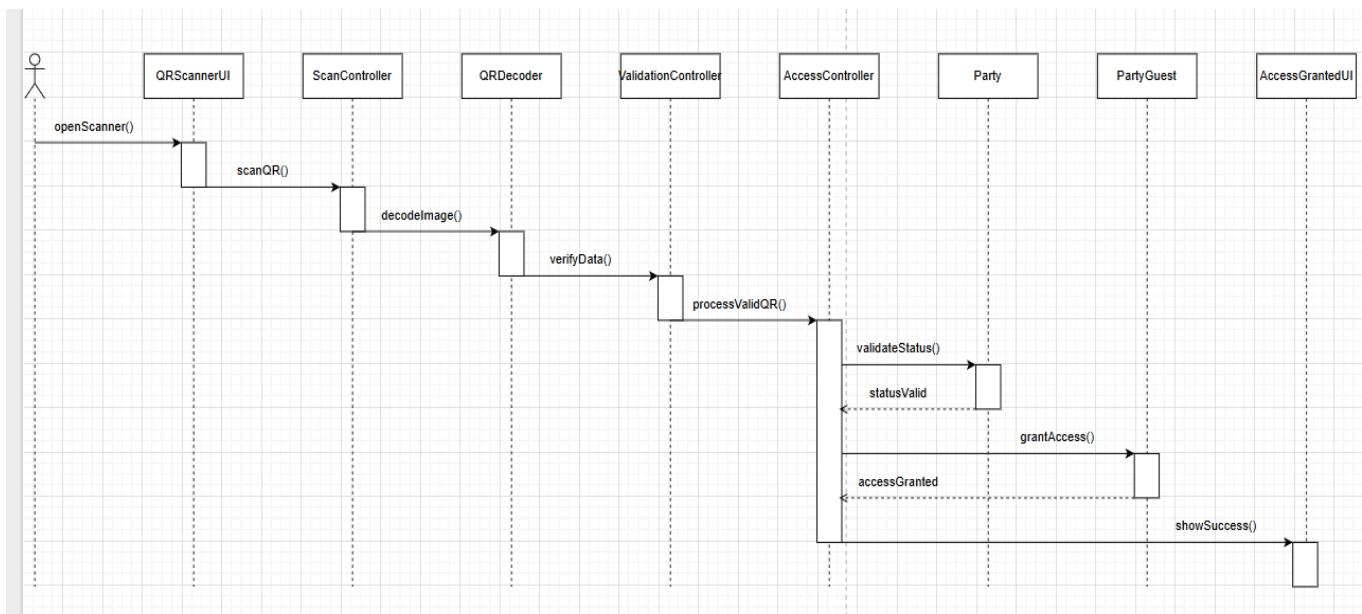
Βήμα προς βήμα περιγραφή:

- Ο PartyGuest καλεί το QRScannerUI με το μήνυμα openScanner(), ανοίγοντας τον σαρωτή QR code
- Ο PartyGuest καλεί το QRScannerUI με το μήνυμα scanQR(), σαρώνοντας το QR code

- Το QRScannerUI καλεί το ScanController με το μήνυμα scanQR(), προωθώντας την εικόνα ή τα δεδομένα του QR
 - Το ScanController καλεί το QRDecoder με το μήνυμα decodeImage(), ζητώντας αποκωδικοποίηση της εικόνας
 - Το QRDecoder καλεί τον εαυτό του με το μήνυμα verifyData(), εκτελώντας εσωτερική επαλήθευση των αποκωδικοποιημένων δεδομένων
 - Το QRDecoder καλεί το ValidationController με το μήνυμα processValidQR(), προωθώντας τα έγκυρα δεδομένα του QR
 - Το ValidationController καλεί το AccessController με το μήνυμα validateStatus(), ζητώντας έλεγχο κατάστασης (π.χ., το πάρτι είναι ενεργό)
 - Το AccessController καλεί το Party με το μήνυμα validateStatus(), επαληθεύοντας ότι το πάρτι είναι ενεργό
 - Το Party επιστρέφει στο AccessController με το μήνυμα statusValid, επιβεβαιώνοντας ότι το πάρτι είναι σε κατάσταση active
 - Το AccessController καλεί τον εαυτό του (ή επιστρέφει) και στη συνέχεια καλεί το QRDecoder (ή απευθείας) με το μήνυμα grantAccess(), χορηγώντας πρόσβαση
 - Το AccessController καλεί το PartyGuest (ηθοποιό) με το μήνυμα accessGranted, ενημερώνοντας ότι η πρόσβαση δόθηκε
 - Το AccessController καλεί το AccessGrantedUI με το μήνυμα showSuccess(), εμφανίζοντας μήνυμα επιτυχίας
 - Ο PartyGuest βλέπει την επιβεβαίωση ότι η πρόσβαση στο πάρτι δόθηκε επιτυχώς
-

Σημειώσεις:

- Το `verifyData()` είναι αυτο-κλήση – υποδηλώνει εσωτερική επαλήθευση των αποκωδικοποιημένων δεδομένων
- Το `validateStatus()` καλείται δύο φορές: μία από `AccessController` προς `ValidationController` και μία από `AccessController` προς `Party`
- Το `statusValid` είναι η επιβεβαίωση ότι το πάρτι είναι ενεργό
- Το `grantAccess()` χορηγεί την πρόσβαση στον `PartyGuest`
- Το `accessGranted` ενημερώνει απευθείας τον ηθοποιό `PartyGuest`
- Το `showSuccess()` εμφανίζει το οπτικό μήνυμα επιτυχίας μέσω `AccessGrantedUI`
- Δεν υπάρχει `PersistenceController` στο διάγραμμα – η πρόσβαση πιθανώς διατηρείται μέσω `session` χωρίς μόνιμη αποθήκευση



USE CASE 9: SUBMIT SONG REQUEST

Αντικείμενα που συμμετέχουν: PartyGuest (ηθοποιός), SongRequestUI, SearchController, RequestHandler, SongValidationController, RequestController, QueueManager, Song, SongRequest, Playlist, PersistenceController, ConfirmationUI

Βήμα προς βήμα περιγραφή:

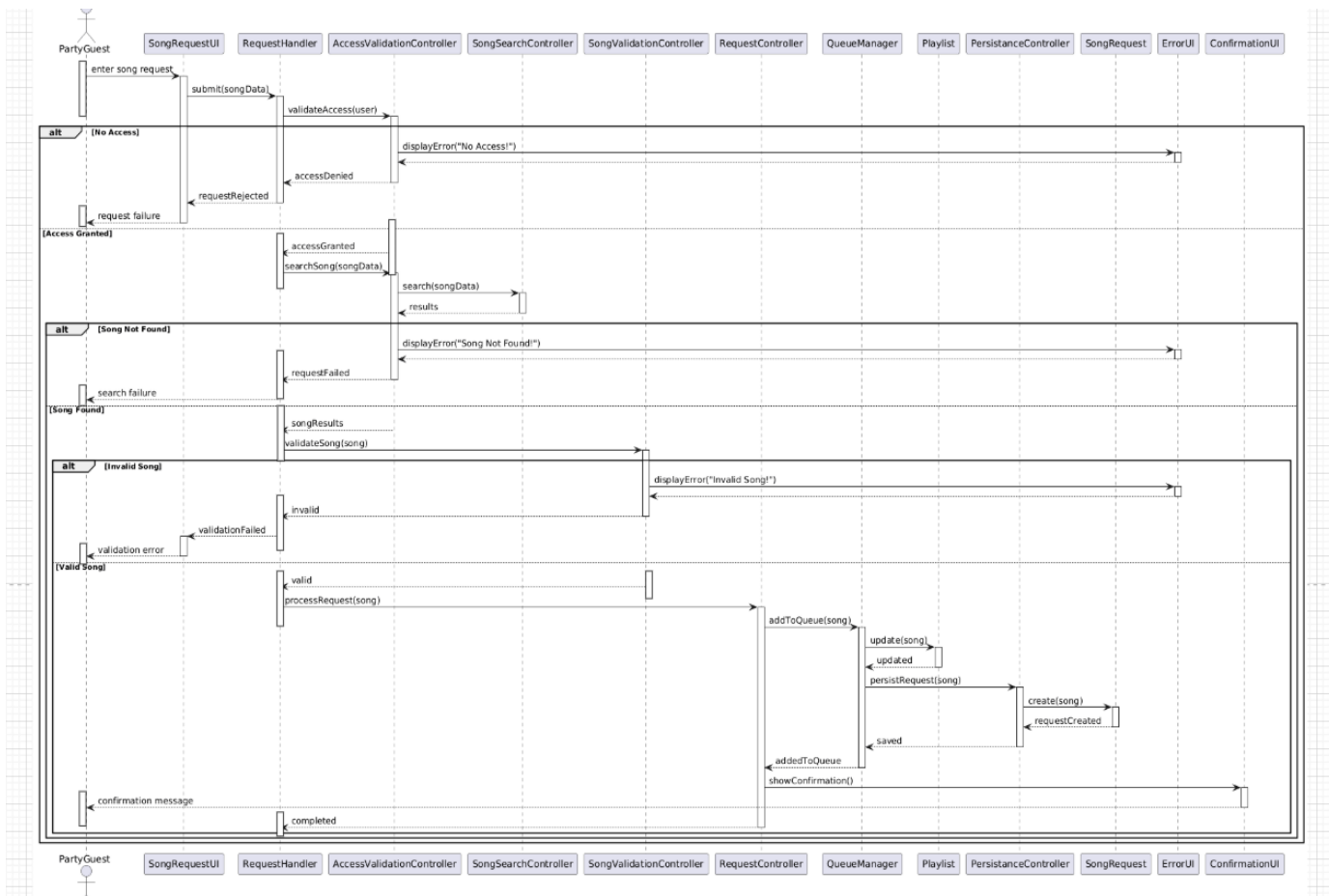
- Ο PartyGuest καλεί το SongRequestUI με το μήνυμα `openRequestUI()`, ανοίγοντας τη διεπαφή υποβολής αιτημάτων
- Ο PartyGuest καλεί το SongRequestUI με το μήνυμα `searchSong(keyword)`, αναζητώντας τραγούδι με λέξη-κλειδί
- Το SongRequestUI καλεί το SearchController με το μήνυμα `searchSong(keyword)`, προωθώντας την αναζήτηση
- Το SearchController καλεί το Song με το μήνυμα `getSongsByKeyword(keyword)`, αναζητώντας τραγούδια που ταιριάζουν
- Το Song επιστρέφει στο SearchController με το μήνυμα `songList`, στέλνοντας τη λίστα των αποτελεσμάτων
- Το SearchController επιστρέφει στο SongRequestUI με το μήνυμα `showResults(songList)`, εμφανίζοντας τα αποτελέσματα
- Ο PartyGuest επιλέγει τραγούδι και καλεί το SongRequestUI με το μήνυμα `selectSong(songId)`
- Ο PartyGuest καλεί το SongRequestUI με το μήνυμα `submitRequest()`, υποβάλλοντας το αίτημα
- Το SongRequestUI καλεί το RequestHandler με το μήνυμα `submitRequest(requestData)`, προωθώντας το αίτημα

- Το RequestHandler καλεί το SongValidationController με το μήνυμα validateSong(songId), ζητώντας έλεγχο διαθεσιμότητας του τραγουδιού
- Το SongValidationController καλεί το Song με το μήνυμα checkAvailability(songId), ελέγχοντας αν το τραγούδι υπάρχει και είναι διαθέσιμο
- Το Song επιστρέφει στο SongValidationController με το μήνυμα available, επιβεβαιώνοντας διαθεσιμότητα
- Το SongValidationController καλεί το RequestController με το μήνυμα processRequest(requestData), προωθώντας για επεξεργασία
- Το RequestController καλεί το QueueManager με το μήνυμα addToQueue(songId, guestId), ζητώντας προσθήκη στην ουρά
- Το QueueManager καλεί το Playlist με το μήνυμα updateQueue(songId), ενημερώνοντας την playlist
- Το QueueManager καλεί το SongRequest με το μήνυμα create(songId, guestId, partyId), δημιουργώντας νέο αντικείμενο SongRequest
- Το SongRequest επιστρέφει στο QueueManager με το μήνυμα created, επιβεβαιώνοντας τη δημιουργία
- Το QueueManager καλεί το PersistenceController με το μήνυμα save(requestData), ζητώντας αποθήκευση του αιτήματος
- Το PersistenceController καλεί τον εαυτό του με το μήνυμα save(), εκτελώντας την αποθήκευση
- Το PersistenceController επιστρέφει στο QueueManager με το μήνυμα saved, επιβεβαιώνοντας αποθήκευση

- Το QueueManager επιστρέφει στο RequestController με το μήνυμα success, επιβεβαιώνοντας επιτυχία
- Το RequestController καλεί το ConfirmationUI με το μήνυμα showConfirmation(), εμφανίζοντας μήνυμα επιτυχίας
- Ο PartyGuest βλέπει την επιβεβαίωση ότι το αίτημα υποβλήθηκε επιτυχώς

Σημειώσεις:

- Η αναζήτηση τραγουδιού (searchSong) γίνεται πριν την υποβολή για καλύτερη εμπειρία χρήστη
- Το checkAvailability(songId) ελέγχει αν το τραγούδι υπάρχει στη βάση και είναι διαθέσιμο για αναπαραγωγή
- Το addToQueue() προσθέτει το τραγούδι στην ουρά αναπαραγωγής
- Το create() δημιουργεί νέο αντικείμενο SongRequest που συνδέει τραγούδι, guest και πάρτι
- Η αποθήκευση γίνεται με save() από QueueManager προς PersistenceController και αυτο-κλήση save() στο PersistenceController
- Η ροή success → showConfirmation() δείχνει την επιτυχή ολοκλήρωση



USE CASE 10: SYNC LIGHTING WITH MUSIC

Αντικείμενα που συμμετέχουν: System (αυτοματοποιημένος δρομέας), LightingInterface, SyncController, Playlist, BeatAnalyzer, Song, DataMapper, TransmissionController, NetworkInterface, LightingSystem

Βήμα προς βήμα περιγραφή:

- Το System καλεί το LightingInterface με το μήνυμα Initialize Lighting Sync, εκκινώντας τη διαδικασία συγχρονισμού φωτισμού

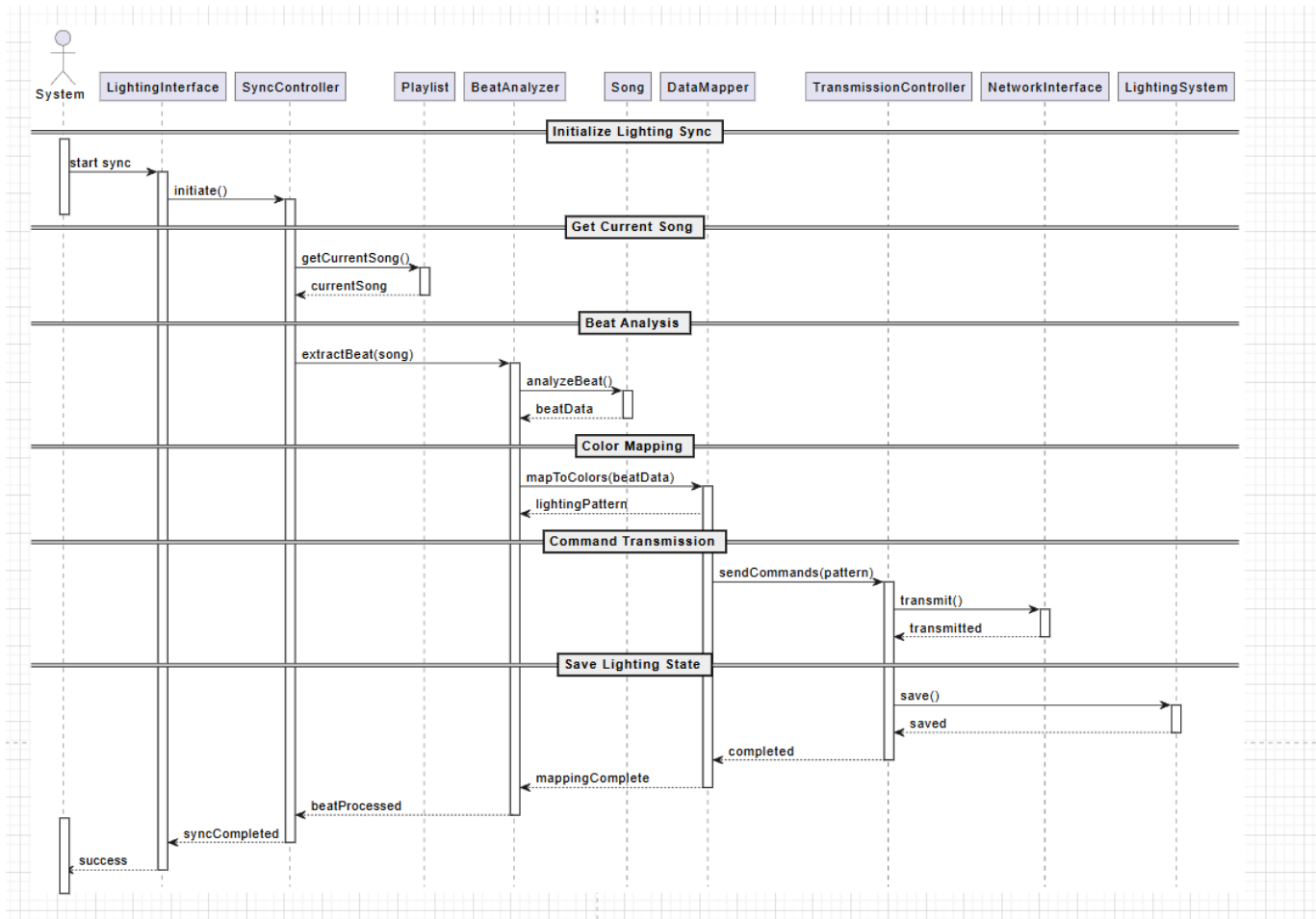
- Το `LightingInterface` καλεί το `SyncController` με το μήνυμα `start sync`, ξεκινώντας τον συγχρονισμό
- Το `SyncController` καλεί τον εαυτό του με το μήνυμα `initiate()`, προετοιμάζοντας εσωτερικά τη διαδικασία
- Το `SyncController` καλεί το `Playlist` με το μήνυμα `Get Current Song`, ζητώντας το τρέχον τραγούδι που παίζει
- Το `Playlist` επιστρέφει στο `SyncController` με το μήνυμα `getCurrentSong()`, στέλνοντας το τρέχον τραγούδι
- Το `SyncController` επιστρέφει `currentSong` και προχωρά στη φάση `Beat Analysis`
- Το `SyncController` καλεί το `BeatAnalyzer` με το μήνυμα `extractBeat(song)`, ζητώντας εξαγωγή των `beat` δεδομένων
- Το `BeatAnalyzer` καλεί τον εαυτό του με το μήνυμα `analyzeBeat()`, εκτελώντας ανάλυση των `beats`
- Το `BeatAnalyzer` επιστρέφει `beatData` (δομημένα δεδομένα `beats`) και προχωρά στη φάση `Color Mapping`
- Το `BeatAnalyzer` καλεί το `DataMapper` με το μήνυμα `mapToColors(beatData)`, ζητώντας αντιστοίχιση `beats` σε χρώματα
- Το `DataMapper` επιστρέφει `lightingPattern` (το χρωματικό μοτίβο φωτισμού) και προχωρά στη φάση `Command Transmission`
- Το `DataMapper` καλεί το `TransmissionController` με το μήνυμα `sendCommands(pattern)`, ζητώντας αποστολή εντολών
- Το `TransmissionController` καλεί το `NetworkInterface` με το μήνυμα `transmit()`, μεταδίδοντας τις εντολές μέσω δικτύου
- Το `NetworkInterface` στέλνει τις εντολές στο `LightingSystem` (εξωτερικό σύστημα)

- Το `TransmissionController` λαμβάνει επιβεβαίωση `transmitted` ότι οι εντολές στάλθηκαν και προχωρά στη φάση `Save Lighting State`
- Το `TransmissionController` καλεί τον εαυτό του (ή το σύστημα αποθήκευσης) με το μήνυμα `save()`, αποθηκεύοντας την κατάσταση φωτισμού
- Επιστρέφει `saved` και ολοκληρώνεται η διαδικασία
- Σταδιακά επιστρέφονται μηνύματα επιτυχίας: `completed`, `mappingComplete`, `beatProcessed`, `syncCompleted`, `success`
- Το `SyncController` ενημερώνεται ότι ο συγχρονισμός ολοκληρώθηκε επιτυχώς

Σημειώσεις:

- Το `initiate()` είναι αυτο-κλήση – υποδηλώνει εσωτερική προετοιμασία του `SyncController`
- Το `analyzeBeat()` είναι αυτο-κλήση – υποδηλώνει εσωτερική ανάλυση των `beat` δεδομένων
- Η ροή χωρίζεται σε πέντε φάσεις: Αρχικοποίηση, Λήψη τραγουδιού, Ανάλυση Beats, Αντιστοίχιση Χρωμάτων, Μετάδοση Εντολών, Αποθήκευση Κατάστασης
- Το `lightingPattern` είναι το αποτέλεσμα της αντιστοίχισης `beats` σε χρώματα
- Το `transmitted` είναι η επιβεβαίωση ότι οι εντολές στάλθηκαν μέσω δικτύου
- Τα μηνύματα `completed`, `mappingComplete`, `beatProcessed`, `syncCompleted`, `success` είναι διαδοχικές επιβεβαιώσεις για κάθε φάση

- Δεν υπάρχει αλληλεπίδραση με χρήστη – το use case είναι πλήρως αυτοματοποιημένο



USE CASE 11: MANAGE SUBSCRIPTION / PURCHASE

Αντικείμενα που συμμετέχουν: PartyHost (ηθοποιός), SubscriptionUI, RequestHandler, ValidationController, PricingService,

SubscriptionController, PaymentOrchestrator, PaymentUI, PersistenceController, Subscription, ConfirmationUI, ErrorUI

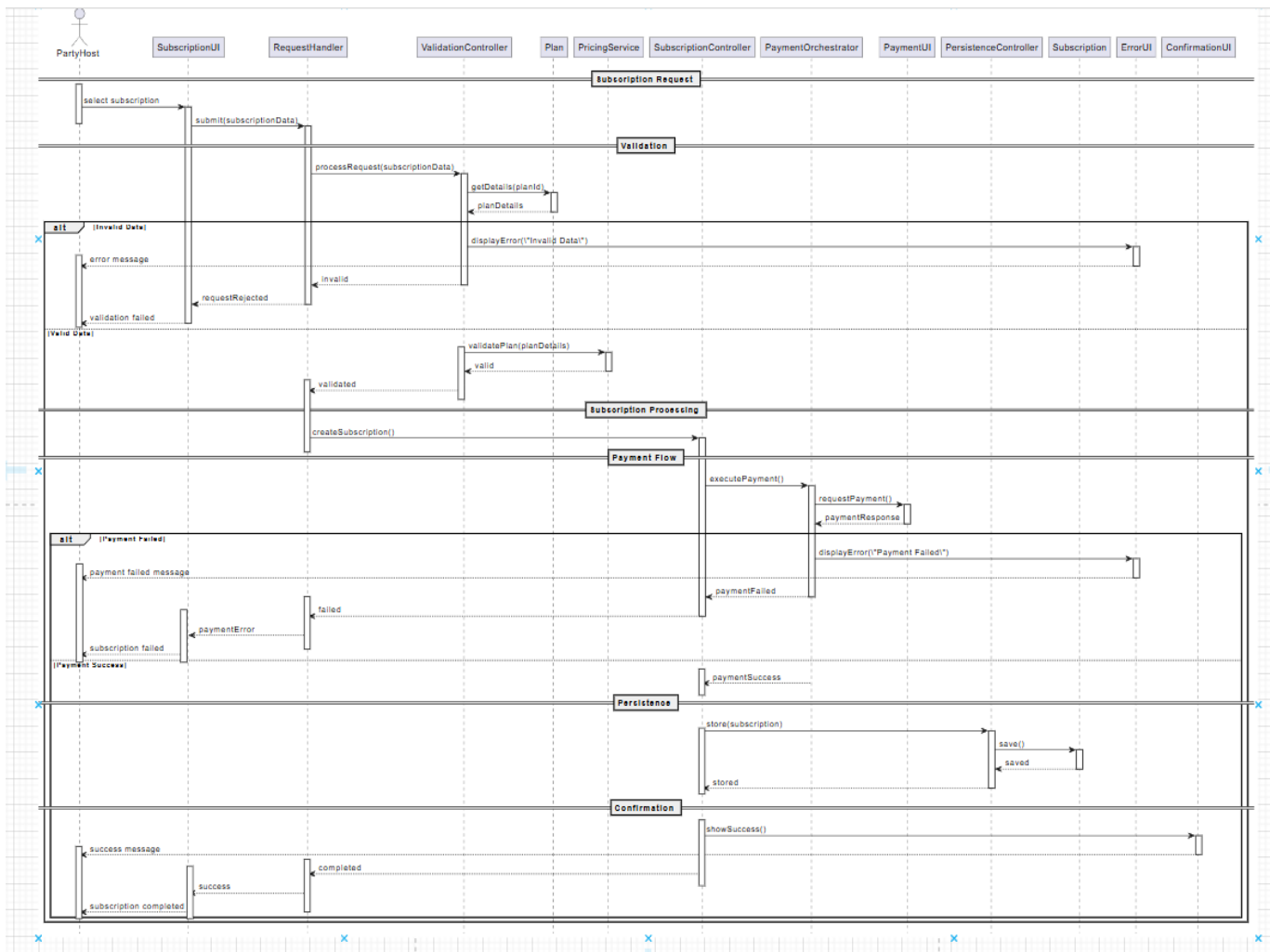
Βήμα προς βήμα περιγραφή:

- Ο PartyHost καλεί το SubscriptionUI με το μήνυμα selectPlan(planId), επιλέγοντας ένα πρόγραμμα συνδρομής (π.χ., Premium, Pro)
- Ο PartyHost καλεί το SubscriptionUI με το μήνυμα submit(), υποβάλλοντας το αίτημα αγοράς
- Το SubscriptionUI καλεί το RequestHandler με το μήνυμα submit(), προωθώντας το αίτημα
- Το RequestHandler καλεί τον εαυτό του με το μήνυμα process(), επεξεργαζόμενο το αίτημα
- Το RequestHandler καλεί το ValidationController με το μήνυμα validate(requestData), ζητώντας έλεγχο εγκυρότητας
- Το ValidationController επιστρέφει valid (αν τα δεδομένα είναι έγκυρα) ή invalid (αν όχι)
- Σε περίπτωση invalid, το ErrorUI εμφανίζεται και η ροή τερματίζεται
- Το ValidationController καλεί το PricingService με το μήνυμα getDetails(planId), ζητώντας λεπτομέρειες του προγράμματος
- Το PricingService επιστρέφει plan, περιέχοντας τιμή, διάρκεια, χαρακτηριστικά
- Το PricingService καλεί το SubscriptionController με το μήνυμα processSubscription(plan, userData), ζητώντας επεξεργασία συνδρομής

- Το SubscriptionController καλεί τον εαυτό του με το μήνυμα validateSubscription(), ελέγχοντας αν ο χρήστης έχει ήδη ενεργή συνδρομή
 - Το SubscriptionController καλεί το PaymentOrchestrator με το μήνυμα executePayment(paymentDetails), ζητώντας εκτέλεση πληρωμής
 - Το PaymentOrchestrator καλεί το PaymentUI με το μήνυμα showPaymentScreen(), εμφανίζοντας οθόνη πληρωμής
 - Ο PartyHost εισάγει στοιχεία πληρωμής και υποβάλλει
 - Το PaymentUI επιστρέφει στο PaymentOrchestrator με το μήνυμα paymentCompleted, επιβεβαιώνοντας επιτυχή πληρωμή
 - Το PaymentOrchestrator επιστρέφει στο SubscriptionController με το μήνυμα paymentSuccess, επιβεβαιώνοντας επιτυχία
 - Το SubscriptionController καλεί το PersistenceController με το μήνυμα store(subscriptionData), ζητώντας αποθήκευση της συνδρομής
 - Το PersistenceController καλεί τον εαυτό του με το μήνυμα save(), εκτελώντας αποθήκευση
 - Το PersistenceController δημιουργεί το αντικείμενο Subscription και το αποθηκεύει
 - Το PersistenceController επιστρέφει στο SubscriptionController με το μήνυμα saved, επιβεβαιώνοντας αποθήκευση
 - Το SubscriptionController καλεί το ConfirmationUI με το μήνυμα showSuccess(), εμφανίζοντας μήνυμα επιτυχίας
 - Ο PartyHost βλέπει την επιβεβαίωση ότι η συνδρομή ενεργοποιήθηκε επιτυχώς
-

Σημειώσεις:

- Το `process()` είναι αυτο-κλήση – υποδηλώνει εσωτερική επεξεργασία του αιτήματος
- Το `validateSubscription()` είναι αυτο-κλήση – ελέγχει αν υπάρχει ήδη ενεργή συνδρομή
- Σε περίπτωση `invalid validation`, η ροή κατευθύνεται στο `ErrorUI`
- Το `PricingService` παρέχει τις λεπτομέρειες του επιλεγμένου προγράμματος
- Το `PaymentOrchestrator` συντονίζει τη διαδικασία πληρωμής μέσω `PaymentUI`
- Η αποθήκευση γίνεται με `store()` από `SubscriptionController` προς `PersistenceController` και αυτο-κλήση `save()`
- Το `Subscription` object δημιουργείται και αποθηκεύεται μετά από επιτυχή πληρωμή
- Το `ConfirmationUI` εμφανίζεται μετά την επιτυχή αποθήκευση



USE CASE 12: PROCESS PAYMENT

Αντικείμενα που συμμετέχουν: PaymentService (εξωτερικός δρομέας), PaymentAPI, RequestBuilder, ExternalGatewayAdapter, ExternalPaymentGateway, ResponseHandler, PaymentController, PersistenceController, Payment, Subscription, ConfirmationUI, ErrorUI

Βήμα προς βήμα περιγραφή:

Φάση 1: Λήψη και προετοιμασία αιτήματος

- Το PaymentService (εξωτερικό σύστημα) στέλνει Payment Request στο PaymentAPI
- Το PaymentAPI καλεί τον εαυτό του με το μήνυμα prepareResponse(), προετοιμάζοντας απάντηση
- Το PaymentAPI επιστρέφει responsePrepared
- Το PaymentAPI καλεί το RequestBuilder με το μήνυμα forwardRequest(), προωθώντας το αίτημα
- Το RequestBuilder καλεί τον εαυτό του με το μήνυμα formatData(), μορφοποιώντας τα δεδομένα για το gateway
- Το RequestBuilder επιστρέφει formattedData

Φάση 2: Εξουσιοδότηση πληρωμής

- Το RequestBuilder καλεί το ExternalGatewayAdapter με το μήνυμα authorize(), ζητώντας εξουσιοδότηση
- Το ExternalGatewayAdapter καλεί το ExternalPaymentGateway με το μήνυμα authorize()
- Το ExternalPaymentGateway επεξεργάζεται την πληρωμή
- **Ροή επιτυχίας:** Το ExternalPaymentGateway επιστρέφει gatewayResponse (επιτυχία) στο ExternalGatewayAdapter
- **Ροή αποτυχίας:** Το ExternalPaymentGateway επιστρέφει failure στο ExternalGatewayAdapter

Φάση 3α: Ροή Αποτυχίας (Payment Failure)

- Σε περίπτωση failure, το ExternalGatewayAdapter καλεί το ResponseHandler με το μήνυμα failure

- Το `ResponseHandler` καλεί το `PaymentController` με το μήνυμα `paymentFailed`
- Το `PaymentController` καλεί το `ErrorUI` με το μήνυμα `displayError('Payment Failed!')`
- Η ροή τερματίζεται χωρίς αποθήκευση

Φάση 3β: Ροή Επιτυχίας (Payment Success)

- Σε περίπτωση `gatewayResponse` (επιτυχία), το `ExternalGatewayAdapter` καλεί το `ResponseHandler` με το μήνυμα `success`
- Το `ResponseHandler` καλεί το `PaymentController` με το μήνυμα `paymentCompleted`
- Το `PaymentController` καλεί τον εαυτό του με το μήνυμα `recordPayment()`, καταγράφοντας την πληρωμή
- Το `PaymentController` καλεί το `PersistenceController` με το μήνυμα `record()`, ζητώντας καταγραφή
- Το `PersistenceController` ενημερώνει το `Payment` object
- Επιστρέφει `paymentRecorded`

Φάση 4: Ενημέρωση Subscription

- Το `PaymentController` καλεί τον εαυτό του με το μήνυμα `updateSubscription()`, ενημερώνοντας τη συνδρομή
- Το `PaymentController` καλεί το `Subscription` με το μήνυμα `updateStatus()`, αλλάζοντας κατάσταση (π.χ., `active`, `renewed`)
- Επιστρέφει `subscriptionUpdated`

Φάση 5: Αποθήκευση

- Το PaymentController καλεί το PersistenceController με το μήνυμα store(payment), ζητώντας αποθήκευση της πληρωμής
- Το PersistenceController καλεί τον εαυτό του με το μήνυμα save(), εκτελώντας αποθήκευση
- Το PersistenceController επιστρέφει stored
- Το PersistenceController επιστρέφει saved στο PaymentController

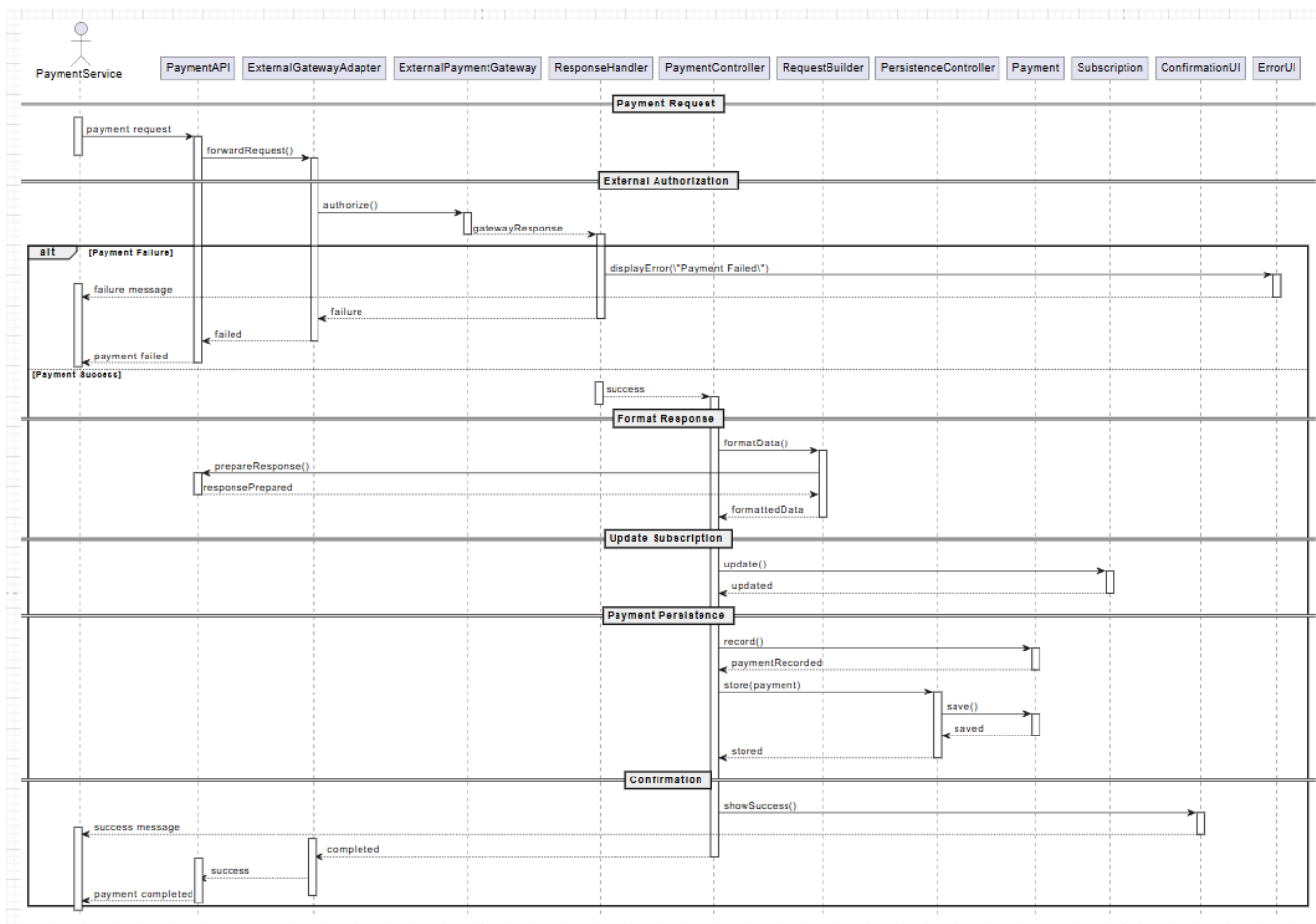
Φάση 6: Επιβεβαίωση

- Το PaymentController καλεί τον εαυτό του με το μήνυμα confirmation
- Το PaymentController καλεί το ConfirmationUI με το μήνυμα showSuccess(), εμφανίζοντας μήνυμα επιτυχίας
- Το ConfirmationUI εμφανίζει success message ("payment completed")
- Η ροή ολοκληρώνεται με success

Σημειώσεις:

- Το prepareResponse() και formatData() είναι αυτο-κλήσεις για προετοιμασία δεδομένων
- Υπάρχουν δύο διακριτές ροές: μία για επιτυχία και μία για αποτυχία
- Σε περίπτωση αποτυχίας, η ροή κατευθύνεται στο ErrorUI και τερματίζεται χωρίς αποθήκευση
- Σε περίπτωση επιτυχίας, ακολουθούν διαδοχικά: καταγραφή πληρωμής, ενημέρωση subscription, αποθήκευση, επιβεβαίωση

- Το record() και store(payment) είναι δύο διακριτές κλήσεις προς PersistenceController
- Το updateSubscription() είναι αυτο-κλήση για ενημέρωση της συνδρομής
- Το confirmation είναι αυτο-κλήση πριν την εμφάνιση μηνύματος επιτυχίας
- Η ροή αυτή είναι το backend κομμάτι που εκτελείται μετά την υποβολή αιτήματος από το UC11



6) CLASS DIAGRAM

Εισαγωγή

Το class diagram περιγράφει τη στατική δομή του συστήματος διαχείρισης πάρτι. Περιλαμβάνει οντότητες που σχετίζονται με χρήστες (Hosts, Guests), πάρτι, playlist, τραγούδια, αιτήματα τραγουδιών, συνδρομές, πληρωμές, περιβάλλον, ηχητικές ρυθμίσεις, σύστημα φωτισμού και QR codes.

Κλάση User

Η κλάση User αποτελεί τη βάση για όλους τους χρήστες του συστήματος.

Ιδιότητες (Attributes):

- userId (int) – μοναδικό αναγνωριστικό χρήστη
- name (String) – όνομα χρήστη
- email (String) – ηλεκτρονική διεύθυνση
- password (String) – κωδικός πρόσβασης

Κανένα δημόσιο method στο διάγραμμα (κληρονομείται).

Κλάση PartyHost (κληρονομεί από User)

Ο PartyHost είναι ένας εξειδικευμένος χρήστης που διοργανώνει πάρτι.

Ιδιότητες (Attributes):

- hostRating (double) – βαθμολογία του διοργανωτή
- hostedPartiesCount (int) – αριθμός πάρτι που έχει διοργανώσει

- `subscriptionLevel (String)` – επίπεδο συνδρομής (π.χ., Basic, Pro, VIP)
- `totalRevenue (double)` – συνολικά έσοδα

Μέθοδοι (Methods):

- `createParty()` – δημιουργία νέου πάρτι
 - `modifyPlaylist()` – τροποποίηση playlist
 - `activateClosingMode()` – ενεργοποίηση λειτουργίας λήξης
 - `manageSubscription()` – διαχείριση συνδρομής
-

Κλάση PartyGuest (κληρονομεί από User)

Ο PartyGuest είναι ένας χρήστης που συμμετέχει σε πάρτι.

Ιδιότητες (Attributes):

- `nickname (String)` – ψευδώνυμο
- `requestLimit (int)` – όριο αιτημάτων ανά πάρτι

Μέθοδοι (Methods):

- `joinParty()` – συμμετοχή σε πάρτι
 - `submitSongRequest()` – υποβολή αιτήματος για τραγούδι
 - `scanQRCode()` – σάρωση QR code για πρόσβαση
-

Κλάση Party

Η κλάση Party αντιπροσωπεύει μια εκδήλωση πάρτι.

Ιδιότητες (Attributes):

- `partyId (int)` – μοναδικό αναγνωριστικό πάρτι

- theme (String) – μουσικό θέμα (π.χ., Rock, Pop)
- location (String) – τοποθεσία πάρτι
- startTime (DateTime) – ώρα έναρξης
- endTime (DateTime) – ώρα λήξης
- status (String) – κατάσταση (draft, active, closing, ended)
- qrEnabled (boolean) – αν το QR code είναι ενεργοποιημένο

Σχέσεις:

- Ένα PartyHost hosts (διοργανώνει) ένα ή πολλά Party
- Ένα Party has (έχει) ένα QRCode
- Ένα Party has (έχει) ένα Environment
- Ένα Party has (έχει) ένα AudioSettings
- Ένα Party contains (περιέχει) μία Playlist
- Ένα Party has (έχει) πολλά SongRequest

Κλάση Playlist

Η κλάση Playlist αντιπροσωπεύει τη λίστα αναπαραγωγής του πάρτι.

Ιδιότητες (Attributes):

- playlistId (int) – μοναδικό αναγνωριστικό playlist
- generated (boolean) – αν η playlist δημιουργήθηκε αυτόματα
- totalSongs (int) – συνολικός αριθμός τραγουδιών

Μέθοδοι (Methods):

- addSong() – προσθήκη τραγουδιού
- removeSong() – αφαίρεση τραγουδιού

- reorderSongs() – αλλαγή σειράς τραγουδιών

Σχέσεις:

- Μία Playlist contains (περιέχει) πολλά Song
-

Κλάση Song

Η κλάση Song αντιπροσωπεύει ένα τραγούδι.

Ιδιότητες (Attributes):

- songId (int) – μοναδικό αναγνωριστικό τραγουδιού
- title (String) – τίτλος
- artist (String) – καλλιτέχνης
- genre (String) – είδος μουσικής
- duration (int) – διάρκεια σε δευτερόλεπτα
- popularity (double) – δημοτικότητα (βαθμολογία)

Μέθοδοι (Methods):

- playSong() – αναπαραγωγή τραγουδιού
- updateMetadata() – ενημέρωση μεταδεδομένων

Σχέσεις:

- Ένα Song περιέχεται σε μία Playlist (σχέση contains)
 - Ένα Song σχετίζεται με πολλά SongRequest
-

Κλάση SongRequest

Η κλάση SongRequest αντιπροσωπεύει ένα αίτημα για τραγούδι.

Ιδιότητες (Attributes):

- requestId (int) – μοναδικό αναγνωριστικό αιτήματος
- status (String) – κατάσταση (pending, approved, rejected, played)
- timestamp (DateTime) – χρονική στιγμή υποβολής
- queuePosition (int) – θέση στην ουρά

Μέθοδοι (Methods):

- validateRequest() – επικύρωση αιτήματος
- approveRequest() – έγκριση αιτήματος
- queueRequest() – τοποθέτηση στην ουρά

Σχέσεις:

- Πολλά SongRequest αντιστοιχούν σε ένα Song
- Πολλά SongRequest αντιστοιχούν σε ένα Party
- Πολλά SongRequest γίνονται από έναν PartyGuest

Κλάση Subscription

Η κλάση Subscription αντιπροσωπεύει τη συνδρομή ενός PartyHost.

Ιδιότητες (Attributes):

- subscriptionId (int) – μοναδικό αναγνωριστικό συνδρομής
- planType (String) – τύπος προγράμματος (Basic, Pro, VIP)
- status (String) – κατάσταση (active, expired, cancelled)
- monthlyCost (double) – μηνιαίο κόστος

Μέθοδοι (Methods):

- activate() – ενεργοποίηση συνδρομής
- renew() – ανανέωση

- cancel() – ακύρωση

Σχέσεις:

- Μία Subscription αντιστοιχεί σε έναν PartyHost
 - Μία Subscription σχετίζεται με πολλές Payment
-

Κλάση Payment

Η κλάση Payment αντιπροσωπεύει μια πληρωμή.

Ιδιότητες (Attributes):

- paymentId (int) – μοναδικό αναγνωριστικό πληρωμής
- amount (double) – ποσό
- paymentDate (DateTime) – ημερομηνία πληρωμής
- paymentStatus (String) – κατάσταση (success, failed, pending)

Μέθοδοι (Methods):

- processPayment() – επεξεργασία πληρωμής
- generateReceipt() – δημιουργία απόδειξης

Σχέσεις:

- Πολλές Payment αντιστοιχούν σε μία Subscription
-

Κλάση QRCode

Η κλάση QRCode αντιπροσωπεύει τον κωδικό QR για πρόσβαση.

Ιδιότητες (Attributes):

- qrCodeId (int) – μοναδικό αναγνωριστικό QR
- qrValue (String) – κωδικοποιημένη τιμή

- `expirationDate (DateTime)` – ημερομηνία λήξης
- `isValid (boolean)` – αν είναι έγκυρος

Μέθοδοι (Methods):

- `generateQR()` – δημιουργία QR code
- `validateQR()` – επικύρωση QR code

Σχέσεις:

- Ένα `QRCode` αντιστοιχεί σε ένα `Party`
-

Κλάση Environment

Η κλάση `Environment` αντιπροσωπεύει τον χώρο όπου γίνεται το πάρτι.

Ιδιότητες (Attributes):

- `environmentId (int)` – μοναδικό αναγνωριστικό
- `crowdDensity (int)` – πυκνότητα πλήθους
- `roomSize (int)` – μέγεθος δωματίου
- `detectedMood (String)` – ανιχνευμένη διάθεση (mood)

Μέθοδοι (Methods):

- `analyzeEnvironment()` – ανάλυση περιβάλλοντος
- `calculateMood()` – υπολογισμός διάθεσης

Σχέσεις:

- Ένα `Environment` αντιστοιχεί σε ένα `Party`
-

Κλάση AudioSettings

Η κλάση `AudioSettings` αντιπροσωπεύει τις ηχητικές ρυθμίσεις.

Ιδιότητες (Attributes):

- volume (int) – ένταση ήχου
- bassLevel (int) – επίπεδο μπάσων
- equalizerMode (String) – λειτουργία equalizer

Μέθοδοι (Methods):

- adjustVolume() – ρύθμιση έντασης
- muteAudio() – σίγαση ήχου

Σχέσεις:

- Ένα AudioSettings αντιστοιχεί σε ένα Party
-

Κλάση LightingSystem

Η κλάση LightingSystem αντιπροσωπεύει το σύστημα φωτισμού.

Ιδιότητες (Attributes):

- lightingMode (String) – λειτουργία φωτισμού (party, chill, closing)
- brightness (int) – φωτεινότητα
- colorScheme (String) – χρωματικό σχήμα

Μέθοδοι (Methods):

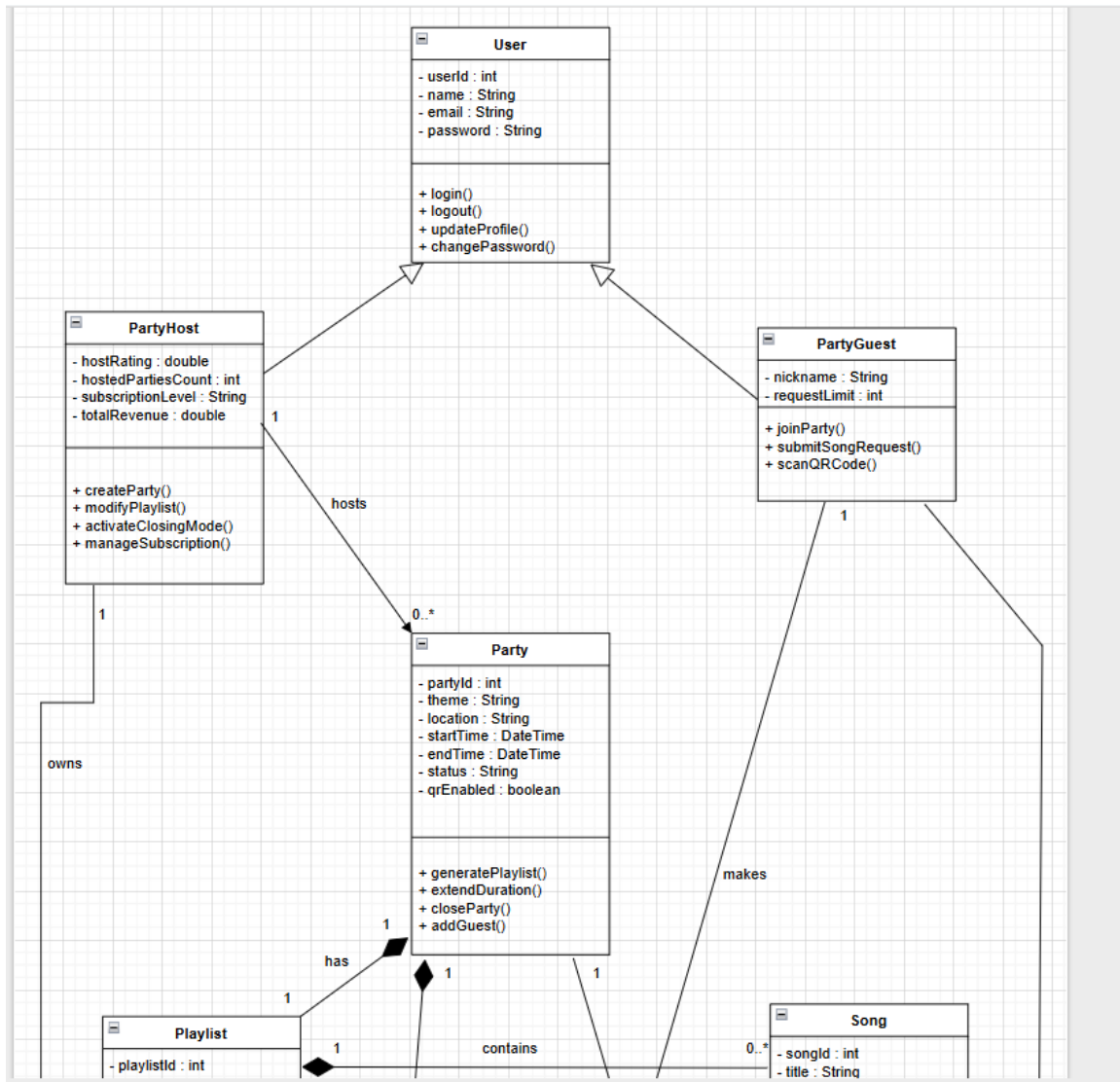
- syncLighting() – συγχρονισμός φωτισμού με μουσική
- changeColorScheme() – αλλαγή χρωματικού σχήματος

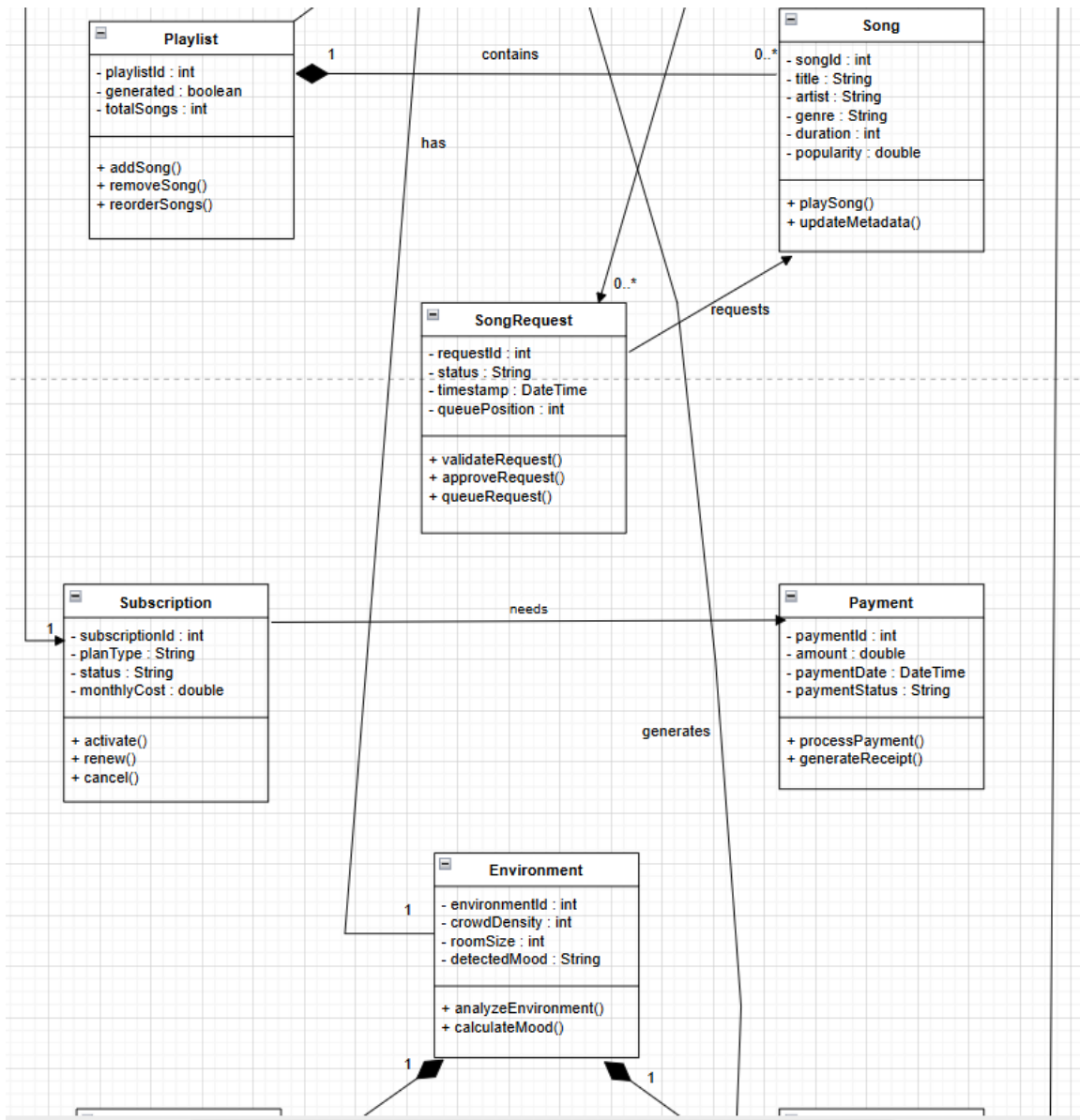
Σχέσεις:

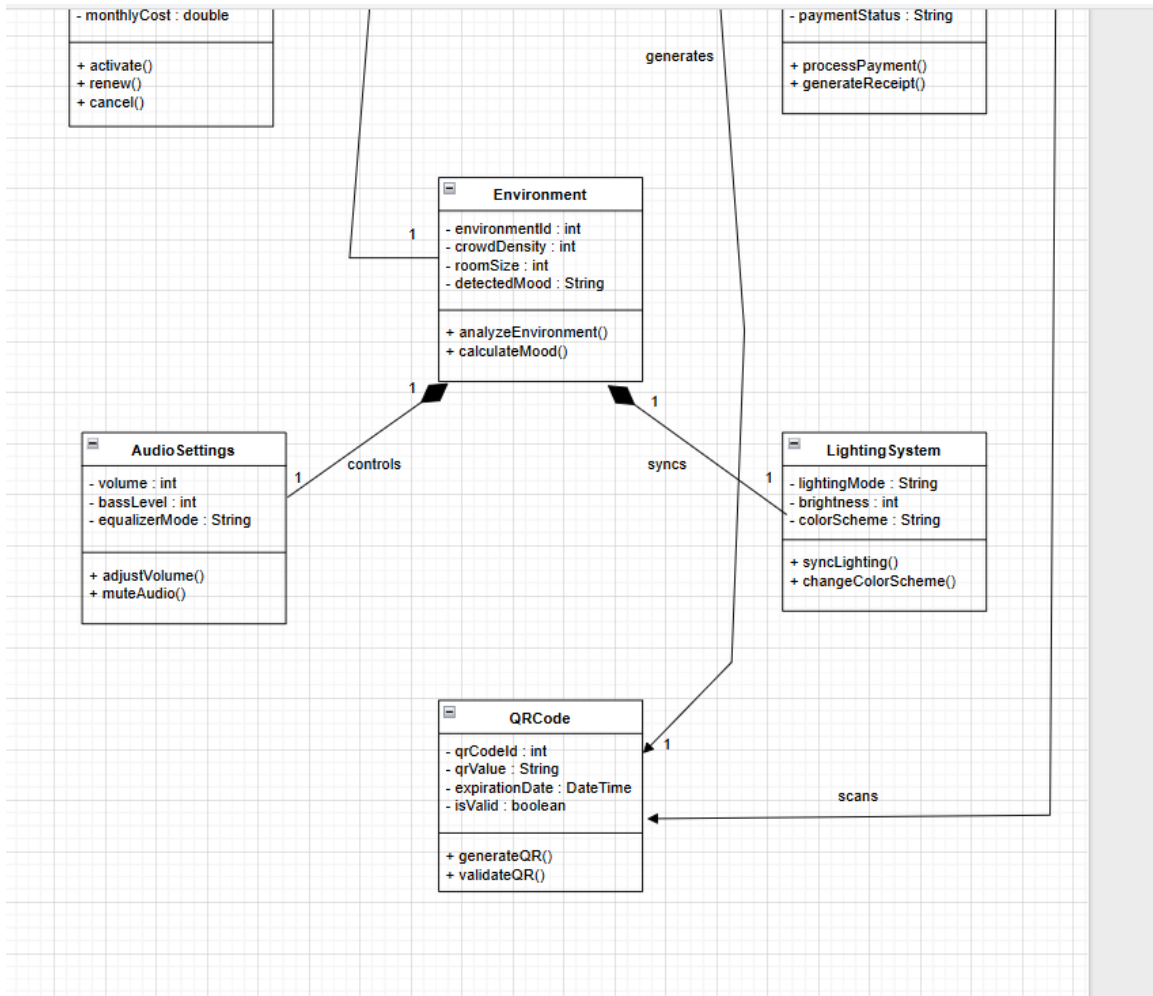
- Συνδέεται με το Environment (δεν φαίνεται ρητή σχέση στο διάγραμμα)
-

Πίνακας Συσχετίσεων (Cardinalities)

Από	Προς	Cardinality	Τύπος Σχέσης
PartyHost	Party	1 → *	hosts
PartyGuest	SongRequest	1 → *	makes
Party	Playlist	1 → 1	contains
Playlist	Song	1 → *	contains
Song	SongRequest	1 → *	(αντίστροφη)
Party	SongRequest	1 → *	has
Party	QRCode	1 → 1	has
Party	Environment	1 → 1	has
Party	AudioSettings	1 → 1	has
PartyHost	Subscription	1 → 1	(συσχέτιση)
Subscription	Payment	1 → *	(συσχέτιση)







7) ΚΩΔΙΚΑΣ

Link για Github:

<https://github.com/dervenisja/Technologia-logismikou>

Σύντομη περιγραφή του τι θα περιέχει η κάθε κλάση:

1. Κλάση User (Βάση για όλους τους χρήστες)

Η κλάση User είναι μια αφηρημένη (abstract) βασική κλάση που περιέχει τα κοινά χαρακτηριστικά όλων των χρηστών. Από αυτήν κληρονομούν οι PartyHost και PartyGuest.

Ιδιότητες (Properties):

- userId (int, private) – μοναδικό αναγνωριστικό χρήστη, auto-increment
- name (string, private) – ονοματεπώνυμο χρήστη
- email (string, private) – ηλεκτρονική διεύθυνση (unique)
- password (string, private) – κωδικός πρόσβασης (αποθηκεύεται hashed)

Μέθοδοι (Methods):

- + User(name, email, password) – constructor
- + getUserId() – επιστρέφει το userId
- + getName() – επιστρέφει το όνομα
- + getEmail() – επιστρέφει το email
- + setPassword(newPassword) – αλλάζει τον κωδικό
- + authenticate(password) – επαληθεύει τον κωδικό

2. Κλάση PartyHost (κληρονομεί από User)

Η κλάση PartyHost αντιπροσωπεύει τον διοργανωτή ενός πάρτι.

Ιδιότητες (Properties) – επιπλέον της User:

- hostRating (double, private) – βαθμολογία διοργανωτή (0.0 – 5.0)
- hostedPartiesCount (int, private) – αριθμός πάρτι που έχει διοργανώσει
- subscriptionLevel (string, private) – επίπεδο συνδρομής ("Basic", "Pro", "VIP")
- totalRevenue (double, private) – συνολικά έσοδα

Μέθοδοι (Methods):

- + PartyHost(name, email, password) – constructor
- + createParty(partyData) – δημιουργεί νέο πάρτι, επιστρέφει Party object
- + modifyPlaylist(playlistId, action, songId) – τροποποιεί την playlist
- + activateClosingMode(partyId) – ενεργοποιεί closing mode
- + manageSubscription(planType, action) – διαχειρίζεται συνδρομή
- + updateRating(newRating) – ενημερώνει τη βαθμολογία
- + getSubscriptionLevel() – επιστρέφει το επίπεδο συνδρομής

3. Κλάση PartyGuest (κληρονομεί από User)

Η κλάση PartyGuest αντιπροσωπεύει τον καλεσμένο που συμμετέχει σε ένα πάρτι.

Ιδιότητες (Properties) – επιπλέον της User:

- nickname (string, private) – ψευδώνυμο (προαιρετικό)
- requestLimit (int, private) – όριο αιτημάτων ανά πάρτι

Μέθοδοι (Methods):

- + PartyGuest(name, email, password, nickname) – constructor
 - + joinParty(partyId, qrCodeValue) – συμμετοχή σε πάρτι με QR code
 - + submitSongRequest(songId, partyId) – υποβολή αιτήματος για τραγούδι
 - + scanQRCode(qrImage) – σάρωση QR code
 - + getRemainingRequests() – επιστρέφει υπόλοιπα διαθέσιμα αιτήματα
-

4. Κλάση Party

Η κλάση Party είναι η κεντρική οντότητα του συστήματος.

Ιδιότητες (Properties):

- partyId (int, private) – μοναδικό αναγνωριστικό πάρτι
- theme (string, private) – μουσικό θέμα ("Rock", "Pop", "Jazz", κλπ.)
- location (string, private) – τοποθεσία
- startTime (DateTime, private) – ώρα έναρξης
- endTime (DateTime, private) – ώρα λήξης
- status (string, private) – "draft", "active", "closing", "ended"
- qrEnabled (bool, private) – αν επιτρέπεται είσοδος με QR

Μέθοδοι (Methods):

- + Party(hostId, theme, location, duration) – constructor

- + getPartyId() – επιστρέφει partyId
 - + setTheme(theme) – ορίζει μουσικό θέμα
 - + setDuration(minutes) – ορίζει διάρκεια
 - + setStatus(newStatus) – αλλάζει κατάσταση πάρτι
 - + activateClosingMode() – ενεργοποιεί closing mode
 - + isActive() – επιστρέφει true αν το πάρτι είναι ενεργό
 - + generateQRCode() – δημιουργεί QR code
-

5. Κλάση Playlist

Η κλάση Playlist διαχειρίζεται τη λίστα αναπαραγωγής.

Ιδιότητες (Properties):

- playlistId (int, private) – μοναδικό αναγνωριστικό
- generated (bool, private) – αν δημιουργήθηκε αυτόματα
- totalSongs (int, private) – αριθμός τραγουδιών στη playlist

Μέθοδοι (Methods):

- + Playlist(partyId) – constructor
 - + addSong(songId) – προσθήκη τραγουδιού (αν δεν υπάρχει ήδη)
 - + removeSong(songId) – αφαίρεση τραγουδιού
 - + reorderSongs(newOrderList) – αλλάζει σειρά τραγουδιών
 - + getTotalDuration() – επιστρέφει συνολική διάρκεια
 - + clearPlaylist() – διαγράφει όλα τα τραγούδια
 - + getSongs() – επιστρέφει λίστα τραγουδιών
-

6. Κλάση Song

Η κλάση Song αντιπροσωπεύει ένα τραγούδι.

Ιδιότητες (Properties):

- songId (int, private) – μοναδικό αναγνωριστικό
- title (string, private) – τίτλος
- artist (string, private) – καλλιτέχνης
- genre (string, private) – είδος μουσικής
- duration (int, private) – διάρκεια σε δευτερόλεπτα
- popularity (double, private) – βαθμολογία δημοτικότητας (0.0 – 100.0)

Μέθοδοι (Methods):

- + Song(title, artist, genre, duration) – constructor
- + getSongId() – επιστρέφει songId
- + playSong() – προσομοίωση αναπαραγωγής
- + updateMetadata(newTitle, newArtist, newGenre) – ενημέρωση
- + getDurationInMinutes() – επιστρέφει διάρκεια σε λεπτά
- + matchesTheme(theme) – ελέγχει αν το τραγούδι ταιριάζει σε theme

7. Κλάση SongRequest

Η κλάση SongRequest καταγράφει ένα αίτημα τραγουδιού.

Ιδιότητες (Properties):

- requestId (int, private) – μοναδικό αναγνωριστικό
- status (string, private) – "pending", "approved", "rejected", "played"

- timestamp (DateTime, private) – χρόνος υποβολής
- queuePosition (int, private) – θέση στην ουρά

Μέθοδοι (Methods):

- + SongRequest(songId, guestId, partyId) – constructor
 - + validateRequest() – επικύρωση (π.χ., duplicate, closing mode)
 - + approveRequest() – έγκριση αιτήματος
 - + queueRequest(position) – τοποθέτηση στην ουρά
 - + getStatus() – επιστρέφει κατάσταση
 - + getQueuePosition() – επιστρέφει θέση
-

8. Κλάση Subscription

Η κλάση Subscription διαχειρίζεται τη συνδρομή του Host.

Ιδιότητες (Properties):

- subscriptionId (int, private) – μοναδικό αναγνωριστικό
- planType (string, private) – "Basic", "Pro", "VIP"
- status (string, private) – "active", "expired", "cancelled"
- monthlyCost (double, private) – μηνιαίο κόστος

Μέθοδοι (Methods):

- + Subscription(hostId, planType) – constructor
- + activate() – ενεργοποιεί τη συνδρομή
- + renew() – ανανεώνει για άλλο μήνα
- + cancel() – ακυρώνει τη συνδρομή
- + isActive() – επιστρέφει true αν είναι ενεργή

- + getPlanType() – επιστρέφει τύπο προγράμματος
-

9. Κλάση Payment

Η κλάση Payment καταγράφει μια πληρωμή.

Ιδιότητες (Properties):

- paymentId (int, private) – μοναδικό αναγνωριστικό
- amount (double, private) – ποσό
- paymentDate (DateTime, private) – ημερομηνία
- paymentStatus (string, private) – "success", "failed", "pending"

Μέθοδοι (Methods):

- + Payment(subscriptionId, amount) – constructor
 - + processPayment(paymentDetails) – επεξεργασία πληρωμής
 - + generateReceipt() – δημιουργία απόδειξης
 - + getPaymentStatus() – επιστρέφει κατάσταση
 - + refund() – επιστροφή χρημάτων (αν υποστηρίζεται)
-

10. Κλάση QRCode

Η κλάση QRCode αντιπροσωπεύει τον κωδικό QR.

Ιδιότητες (Properties):

- qrCodeId (int, private) – μοναδικό αναγνωριστικό
- qrValue (string, private) – κωδικοποιημένη τιμή (π.χ., partyId + token)
- expirationDate (DateTime, private) – ημερομηνία λήξης

- isValid (bool, private) – αν είναι έγκυρος

Μέθοδοι (Methods):

- + QRCode(partyId) – constructor (δημιουργεί νέο QR)
 - + generateQR() – δημιουργεί QR code string
 - + validateQR(scannedValue) – επικύρωση σαρωμένου QR
 - + isExpired() – επιστρέφει true αν έχει λήξει
 - + refresh() – ανανέωση QR code
-

11. Κλάση Environment

Η κλάση Environment αναλύει τον χώρο του πάρτι.

Ιδιότητες (Properties):

- environmentId (int, private) – μοναδικό αναγνωριστικό
- crowdDensity (int, private) – πυκνότητα πλήθους (0 – 100)
- roomSize (int, private) – μέγεθος δωματίου σε τετραγωνικά μέτρα
- detectedMood (string, private) – ανιχνευμένη διάθεση ("calm", "energetic", κλπ.)

Μέθοδοι (Methods):

- + Environment(partyId) – constructor
 - + analyzeEnvironment(sensorData) – ανάλυση περιβάλλοντος
 - + calculateMood() – υπολογίζει διάθεση από πυκνότητα και μουσική
 - + updateCrowdDensity(newDensity) – ενημερώνει πυκνότητα
-

12. Κλάση AudioSettings

Η κλάση AudioSettings ρυθμίζει τον ήχο.

Ιδιότητες (Properties):

- volume (int, private) – ένταση (0 – 100)
- bassLevel (int, private) – επίπεδο μπάσων (0 – 100)
- equalizerMode (string, private) – "normal", "bass", "treble", "vocal"

Μέθοδοι (Methods):

- + AudioSettings(partyId) – constructor
 - + adjustVolume(newVolume) – ρύθμιση έντασης
 - + muteAudio() – σίγαση
 - + setEqualizerMode(mode) – αλλαγή equalizer
 - + getCurrentSettings() – επιστρέφει τρέχουσες ρυθμίσεις
-

13. Κλάση LightingSystem

Η κλάση LightingSystem ελέγχει τον φωτισμό.

Ιδιότητες (Properties):

- lightingMode (string, private) – "party", "chill", "closing"
- brightness (int, private) – φωτεινότητα (0 – 100)
- colorScheme (string, private) – χρωματικό σχήμα ("rainbow", "warm", "cold")

Μέθοδοι (Methods):

- + LightingSystem(environmentId) – constructor
- + syncLighting(beatData) – συγχρονισμός με beats τραγουδιού
- + changeColorScheme(newScheme) – αλλαγή χρωματικού σχήματος

- + setLightingMode(mode) – αλλαγή λειτουργίας
- + emergencyStop() – άμεση διακοπή φωτισμού

8) TEST CASES

Για να τα test cases μας, χρησιμοποιήσαμε blackbox

UCS1

TC-UC1: Create Party — Black Box Test Cases

TC-UC1-01 — Επιτυχής δημιουργία party (Happy Path)

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC1-01
Use Case	UC1 – Create Party
Preconditions	Ο PartyHost είναι συνδεδεμένος και έχει εξουσιοδοτημένο ρόλο
Input	Όλα τα πεδία συμπληρωμένα και έγκυρα (όνομα, ημερομηνία, τοποθεσία κ.λπ.)
Steps	1. Ο host ανοίγει το CreatePartyFormUI και εισάγει στοιχεία → 2. InputHandler παραλαμβάνει δεδομένα → 3. ValidationController επικυρώνει → 4. AuthorizationController ελέγχει ρόλο → 5. PartyOrchestrator → PartyFactory.create() → Party αποθηκεύεται → ConfirmationUI

Πεδίο	Περιεχόμενο
Expected Result	Party δημιουργείται, αποθηκεύεται, συνδέεται με τον host, εμφανίζεται ConfirmationUI
Type	Positive

TC-UC1-02 — Κενά υποχρεωτικά πεδία

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC1-02
Use Case	UC1 – Create Party
Preconditions	Ο PartyHost είναι συνδεδεμένος
Input	Ένα ή περισσότερα υποχρεωτικά πεδία κενά
Steps	1. Ο host υποβάλλει φόρμα με κενά πεδία → 2. InputHandler → 3. ValidationController ανιχνεύει invalid fields
Expected Result	Εμφανίζεται ValidationMessageUI με μήνυμα σφάλματος, δεν προχωράει σε AuthorizationController
Type	Negative

TC-UC1-03 — Μη έγκυρο format πεδίου

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC1-03
Use Case	UC1 – Create Party

Πεδίο	Περιεχόμενο
Preconditions	Ο PartyHost είναι συνδεδεμένος
Input	Πεδίο ημερομηνίας με λάθος format (π.χ. "32/13/2025") ή αριθμητικό πεδίο με κείμενο
Steps	1. Ο host υποβάλλει φόρμα → 2. ValidationController ανιχνεύει invalid format
Expected Result	ValidationMessageUI εμφανίζει συγκεκριμένο μήνυμα λάθους ανά πεδίο, δεν δημιουργείται party
Type	Negative

TC-UC1-04 — Μη εξουσιοδοτημένος χρήστης (wrong role)

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC1-04
Use Case	UC1 – Create Party
Preconditions	Ο χρήστης είναι συνδεδεμένος αλλά δεν έχει ρόλο PartyHost
Input	Έγκυρα δεδομένα φόρμας
Steps	1. Validation περνά → 2. AuthorizationController.checkRole() επιστρέφει "not authorized"
Expected Result	Εμφανίζεται μήνυμα "not authorized", δεν καλείται ο PartyOrchestrator, δεν δημιουργείται party
Type	Negative

TC-UC1-05 — Επαλήθευση σύνδεσης party με PartyHost (link to host)

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC1-05
Use Case	UC1 – Create Party
Preconditions	Επιτυχής ολοκλήρωση δημιουργίας party
Input	Έγκυρα δεδομένα, εξουσιοδοτημένος host
Steps	1. Party δημιουργείται → 2. PersistenceController.save() → 3. linkToHost() καλείται
Expected Result	Το Party αντικείμενο αποθηκεύεται με σωστό hostId, η σύνδεση host↔party επαληθεύεται στη βάση
Type	Positive

TC-UC1-06 — Αποτυχία αποθήκευσης (persistence failure)

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC1-06
Use Case	UC1 – Create Party
Preconditions	Validation και authorization περνούν, αλλά η βάση δεδομένων είναι μη διαθέσιμη
Input	Έγκυρα δεδομένα
Steps	1. PartyFactory.create() επιτυχώς → 2. PersistenceController.save() αποτυγχάνει

Πεδίο	Περιεχόμενο
Expected Result	Δεν εμφανίζεται ConfirmationUI, εμφανίζεται μήνυμα σφάλματος, δεν αποθηκεύεται party
Type	Negative

TC-UC1-07 — Επανυποβολή φόρμας μετά από αποτυχία validation

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC1-07
Use Case	UC1 – Create Party
Preconditions	Πρώτη υποβολή απέτυχε λόγω invalid fields
Input	Διορθωμένα, πλήρως έγκυρα δεδομένα στη δεύτερη υποβολή
Steps	1. Αποτυχία → ValidationMessageUI → 2. Ο host διορθώνει πεδία → 3. Επανυποβολή → Validation OK → Authorization OK → Party δημιουργείται
Expected Result	Το σύστημα επεξεργάζεται κανονικά τη δεύτερη υποβολή και δημιουργεί το party επιτυχώς
Type	Positive

TC-UC1-08 — Boundary: μέγιστο/ελάχιστο μήκος πεδίων

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC1-08
Use Case	UC1 – Create Party

Πεδίο	Περιεχόμενο
Preconditions	Ο PartyHost είναι συνδεδεμένος
Input	Πεδίο ονόματος: ακριβώς στο όριο (π.χ. 1 χαρακτήρας, max χαρακτήρες, max+1 χαρακτήρες)
Steps	1. Υποβολή με boundary τιμές → 2. ValidationController αξιολογεί
Expected Result	1 char → rejected / max chars → accepted / max+1 → rejected με ValidationMessageUI
Type	Boundary

UCS2

TC-UC2: Select Theme — Black Box Test Cases

TC-UC2-01 — Επιτυχής επιλογή θέματος (Happy Path)

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC2-01
Use Case	UC2 – Select Theme
Preconditions	Το party έχει δημιουργηθεί, ο PartyHost είναι συνδεδεμένος
Input	Έγκυρο θέμα επιλεγμένο από τη λίστα (π.χ. "Tropical")

Πεδίο	Περιεχόμενο
Steps	1. Host ανοίγει ThemeSelectionUI → 2. Επιλέγει θέμα → 3. InputHandler παραλαμβάνει → 4. ThemeValidationController: valid → 5. ThemeService → 6. PartyController.setTheme() → 7. Party ενημερώνεται → 8. PersistenceController.save() → 9. ThemePreviewUI εμφανίζεται
Expected Result	Το θέμα αποθηκεύεται στο Party, εμφανίζεται ThemePreviewUI με το επιλεγμένο θέμα, επιστρέφεται themeUpdated
Type	Positive

TC-UC2-02 — Μη έγκυρο / ανύπαρκτο θέμα

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC2-02
Use Case	UC2 – Select Theme
Preconditions	Το party έχει δημιουργηθεί, ο host είναι συνδεδεμένος
Input	Θέμα που δεν υπάρχει στη λίστα ή κενή επιλογή
Steps	1. Host υποβάλλει μη έγκυρο θέμα → 2. InputHandler → 3. ThemeValidationController: invalid theme → 4. ErrorUI
Expected Result	Εμφανίζεται ErrorUI με μήνυμα "invalid theme", δεν καλείται ThemeService, δεν αποθηκεύεται αλλαγή στο Party
Type	Negative

TC-UC2-03 — Εμφάνιση preview μετά από επιλογή

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC2-03
Use Case	UC2 – Select Theme
Preconditions	Έγκυρο θέμα επιλέχθηκε και αποθηκεύτηκε
Input	Έγκυρο θέμα
Steps	1. Validation OK → 2. PartyController.setTheme() → 3. storeChanges(party) → 4. showPreview() καλείται → 5. ThemePreviewUI φορτώνει
Expected Result	ThemePreviewUI εμφανίζει σωστά το επιλεγμένο θέμα με τα αντίστοιχα χρώματα/στυλ
Type	Positive

TC-UC2-04 — Αλλαγή θέματος σε ήδη επιλεγμένο party

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC2-04
Use Case	UC2 – Select Theme
Preconditions	Το party έχει ήδη αποθηκευμένο θέμα (π.χ. "Tropical")
Input	Νέο έγκυρο θέμα (π.χ. "Retro")
Steps	1. Host επιλέγει νέο θέμα → 2. Validation OK → 3. PartyController.setTheme() → 4. Party ενημερώνεται με νέο θέμα → 5. PersistenceController.save()

Πεδίο	Περιεχόμενο
Expected Result	Το παλιό θέμα αντικαθίσταται, το νέο αποθηκεύεται σωστά, ThemePreviewUI εμφανίζει το νέο θέμα
Type	Positive

TC-UC2-05 — Αποτυχία αποθήκευσης θέματος

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC2-05
Use Case	UC2 – Select Theme
Preconditions	Validation OK, αλλά η βάση δεδομένων δεν είναι διαθέσιμη
Input	Έγκυρο θέμα
Steps	1. ThemeValidationController: valid → 2. PartyController.setTheme() → 3. PersistenceController.save() αποτυγχάνει
Expected Result	Εμφανίζεται ErrorUI με μήνυμα αποτυχίας, το θέμα δεν αποθηκεύεται, το Party παραμένει στην προηγούμενη κατάσταση
Type	Negative

TC-UC2-06 — ThemeService δεν επιστρέφει διαθέσιμα θέματα

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC2-06

Πεδίο	Περιεχόμενο
Use Case	UC2 – Select Theme
Preconditions	Το ThemeService είναι μη διαθέσιμο ή επιστρέφει κενή λίστα
Input	Οποιαδήποτε επιλογή θέματος από τον host
Steps	1. Host ανοίγει ThemeSelectionUI → 2. ThemeService δεν επιστρέφει θέματα
Expected Result	Εμφανίζεται ErrorUI ή κενή λίστα θεμάτων, ο host δεν μπορεί να προχωρήσει
Type	Negative

TC-UC2-07 — Επαλήθευση storeChanges(party) — σωστή ενημέρωση entity

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC2-07
Use Case	UC2 – Select Theme
Preconditions	Επιτυχής επιλογή θέματος
Input	Θέμα "Jazz Night"
Steps	1. setTheme("Jazz Night") → 2. storeChanges(party) → 3. PersistenceController.save() → 4. saved επιστρέφεται
Expected Result	Το πεδίο theme του Party αντικειμένου έχει τιμή "Jazz Night" στη βάση δεδομένων
Type	Positive

TC-UC2-08 — Boundary: θέμα με ειδικούς χαρακτήρες ή πολύ μεγάλο όνομα

Πεδίο	Περιεχόμενο
Test Case ID	TC-UC2-08
Use Case	UC2 – Select Theme
Preconditions	Ο host είναι συνδεδεμένος
Input	Θέμα με ειδικούς χαρακτήρες (π.χ. <script>) ή υπερβολικά μεγάλο string (500+ chars)
Steps	1. Εισαγωγή μη φυσιολογικής τιμής → 2. ThemeValidationController αξιολογεί
Expected Result	Το σύστημα απορρίπτει την τιμή, εμφανίζεται ErrorUI, δεν αποθηκεύεται τίποτα
Type	Boundary / Security

UCS3

TC ID	Test Scenario	Input	Expected Output	Expected Result
TC-01	Valid duration input	120 minutes	Confirmation message displayed	PASS
TC-02	Minimum valid duration	30 minutes	Duration accepted and saved	PASS
TC-03	Maximum valid duration	720 minutes	Duration accepted and stored	PASS
TC-04	Duration below minimum	10 minutes	Error message: "Invalid Duration"	PASS
TC-05	Duration above maximum	1000 minutes	Error message displayed	PASS
TC-06	Empty duration field	null / empty	Validation error shown	PASS
TC-07	Negative duration	-50 minutes	Input rejected	PASS
TC-08	Non-numeric input	"abc"	Validation error displayed	PASS
TC-09	Decimal input	90.5 minutes	Error or automatic rounding	PASS
TC-10	Special characters input	"@#\$\$%"	Input rejected	PASS
TC-11	Boundary valid value	29 → 30 minutes	29 rejected / 30 accepted	PASS

TC ID	Test Scenario	Input	Expected Output	Expected Result
TC-12	Very large integer	999999	System rejects input safely	PASS
TC-13	Save operation success	Valid duration	Duration stored in database	PASS
TC-14	Persistence failure	Valid duration + DB failure	Error message shown	PASS
TC-15	Multiple submissions	Submit duration twice	Latest valid duration stored	PASS

✓ Equivalence Classes

Valid Inputs

30 – 720 minutes

Invalid Inputs

< 30

> 720

empty

negative

non-numeric

special characters

✓ Boundary Value Analysis

Boundary Test Value Expected

Minimum 29 Reject

Boundary Test Value Expected

Minimum 30	Accept
Maximum 720	Accept
Maximum 721	Reject

Expected System Behavior

Valid Flow

PartyHost

- enters duration
 - system validates input
 - duration stored
 - confirmation displayed
-

Invalid Flow

PartyHost

- enters invalid duration
- validation fails
- error message displayed
- duration NOT stored

UCS4

TC ID	Test Scenario	Input	Expected Output	Expected Result
TC-01	Valid playlist generation	Valid theme + valid duration	Playlist generated successfully	PASS
TC-02	Empty theme	Empty theme field	Error message displayed	PASS
TC-03	Invalid duration	Duration = 0	Validation error shown	PASS
TC-04	Valid party data	Theme + duration + guests	Playlist displayed	PASS
TC-05	Missing party data	Missing duration	Generation blocked	PASS
TC-06	Unsupported music theme	Unknown theme	Default playlist generated or error shown	PASS
TC-07	Song database unavailable	Valid request + DB failure	System displays generation failure	PASS
TC-08	No songs found	Rare/invalid genre	Empty playlist warning	PASS
TC-09	Playlist persistence success	Generated playlist	Playlist stored successfully	PASS
TC-10	Playlist persistence failure	DB storage failure	Error message displayed	PASS

TC ID	Test Scenario	Input	Expected Output	Expected Result
TC-11	Multiple generation requests	Generate playlist twice	New playlist replaces previous one	PASS
TC-12	Large playlist generation	500+ songs	Playlist generated without crash	PASS
TC-13	Null input request	Null party object	Request rejected safely	PASS
TC-14	Very long theme text	500-character theme	Validation error or truncation	PASS
TC-15	Successful completion flow	Valid generation process	Confirmation + playlist shown	PASS

Equivalence Classes

Valid Inputs

Valid theme

Valid duration

Existing party data

Supported genres

Invalid Inputs

Empty theme

Null data

Invalid duration

Unsupported genres
Database failure

✓ **Boundary Value Analysis**

Boundary	Test Value	Expected
Minimum duration	1 minute	Reject
Minimum valid duration	30 minutes	Accept
Maximum valid duration	720 minutes	Accept
Above maximum	721 minutes	Reject

✓ **Expected System Behavior**

Successful Flow

PartyHost

- requests playlist generation
 - system validates party data
 - songs selected
 - playlist created
 - playlist saved
 - playlist displayed
-

Failure Flow

PartyHost

- invalid request
- validation fails
- generation cancelled
- error displayed

UCS5

Test Case ID	Input / Action	Expected Output / Result
TC_01	Υπάρχον τραγούδι στη βάση, σωστό songID → submit request	showConfirmation(), το τραγούδι προστίθεται στο playlist, αποθηκεύεται (save)
TC_02	songID δεν υπάρχει στο SongService	invalid → ErrorUI, δεν προστίθεται τραγούδι
TC_03	songID άκυρη μορφή (π.χ., κενό, null, μη αριθμητικό)	invalid → ErrorUI
TC_04	Submit request ενώ το ίδιο τραγούδι υπάρχει ήδη στο playlist	invalid → ErrorUI (duplicate not allowed)
TC_05	Αποτυχία αποθήκευσης από PersistenceController	ErrorUI (π.χ., «δεν ήταν δυνατή η αποθήκευση»)
TC_06	Ο χρήστης ακυρώνει πριν την επιβεβαίωση	Δεν καλείται save, το playlist παραμένει το ίδιο (ή όπως είχε)
TC_07	Ο χρήστης επιβεβαιώνει με valid request	showConfirmation(), save → επιτυχής ενημέρωση

UCS6

TC_EXT_01 – Επιτυχής παράταση εντός ορίων

Πεδίο	Τιμή
Input	requestExtension(30) (λεπτά), υπάρχον πάρτι με τρέχουσα διάρκεια 120
Expected Output	validate() → checkRules() → applyExtension() → extendDuration() → store()
Αποτέλεσμα	Η νέα διάρκεια = 150 λεπτά, αποθηκεύεται, μήνυμα επιτυχίας

TC_EXT_02 – Υπέρβαση μέγιστης συνολικής διάρκειας

Πεδίο	Τιμή
Input	requestExtension(60), τρέχουσα διάρκεια = 200, max = 240
Expected Output	checkRules() αποτυχία → δεν γίνεται applyExtension, εμφάνιση σφάλματος (π.χ. μέσω ValidationController ή απευθείας μήνυμα)
Αποτέλεσμα	Η διάρκεια παραμένει 200, καμία αποθήκευση

TC_EXT_03 – Αρνητική ή μηδενική παράταση

Πεδίο	Τιμή
Input	requestExtension(0) ή requestExtension(-10)
Expected Output	validate() αποτυχία → απορρίπτεται πριν checkRules
Αποτέλεσμα	Κανένα extendDuration, εμφάνιση μηνύματος "μη έγκυρη παράταση"

TC_EXT_04 – Παράταση με μη αριθμητική τιμή

Πεδίο	Τιμή
Input	requestExtension("δύο ώρες") (αν το σύστημα δέχεται string)
Expected Output	Αποτυχία validation → error
Αποτέλεσμα	Δεν φτάνει ποτέ στο RulesController

TC_EXT_05 – Αποτυχία αποθήκευσης (PersistenceController fail)

Πεδίο	Τιμή
Input	valid requestExtension(30), όλοι οι κανόνες OK, αλλά το save() αποτυγχάνει (π.χ., δισκος γεμάτος, DB error)
Expected Output	storeChanges() καλείται, αλλά επιστρέφει σφάλμα. Δεν φτάνει σε showConfirmation() κανονικά, αλλά σε error UI

Πεδίο	Τιμή
Αποτέλεσμα	Το Party δεν έχει αλλάξει μόνιμα (ή αλλάζει in-memory αλλά χάνεται μ — ανάλογα με την υλοποίηση

TC_EXT_06 – Διαδοχικές παρατάσεις χωρίς refresh

Πεδίο	Τιμή
Input	1ο: +30 → OK. 2ο: +40 → με νέα βάση διάρκειας
Expected Output	Κάθε φορά ελέγχονται τα rules με την <i>updated</i> διάρκεια
Αποτέλεσμα	Αν το άθροισμα ξεπεράσει το max, το 2ο αποτυγχάνει

TC_EXT_07 – Παράταση μετά από λήξη πάρτι (αν υπάρχει concept λήξης)

Πεδίο	Τιμή
Input	Το πάρτι έχει ήδη λήξει (π.χ., current time > end time)
Expected Output	RulesController το απορρίπτει
Αποτέλεσμα	No extension, error message

TC_EXT_08 – Ο Host ακυρώνει πριν την επιβεβαίωση

Πεδίο	Τιμή
Input	Ο host πατάει submit, μετά ακυρώνει (αν υπάρχει confirmation d
Expected Output	Δεν καλείται ποτέ storeChanges/save
Αποτέλεσμα	Το πάρτι παραμένει ως είχε

UCS7

TC_CLOSE_01 – Επιτυχής ενεργοποίηση Closing Mode

Πεδίο	Τιμή
Precondition	Ενεργό πάρτι σε "Normal Mode" ή "Party Mode", Host authenticated
Input	Host πατάει "Activate Closing Mode"
Expected Output	Σύστημα: mode changes to "Closing", no new requests accepted (ή ειδικά requests), confirmation message στον Host
Postcondition	party.mode = "CLOSING", αποθηκεύεται, guests βλέπουν closing m

TC_CLOSE_02 – Closing Mode ήδη ενεργό (δεύτερη ενεργοποίηση)

Πεδίο	Τιμή
Precondition	Το πάρτι είναι ήδη σε Closing Mode
Input	Host ξαναπατάει "Activate Closing Mode"
Expected Output	Μήνυμα: "Το closing mode είναι ήδη ενεργό" – καμία αλλαγή
Postcondition	Το πάρτι παραμένει σε Closing Mode (idempotent)

TC_CLOSE_03 – Ο Host δεν είναι authenticated / unauthorized

Πεδίο	Τιμή
Precondition	Ο χρήστης είναι Guest (όχι Host)
Input	Προσπάθεια ενεργοποίησης closing mode
Expected Output	"Μη εξουσιοδοτημένη ενέργεια", καμία αλλαγή
Postcondition	Το πάρτι παραμένει στην προηγούμενη κατάσταση

TC_CLOSE_04 – Το πάρτι δεν είναι ενεργό (precondition fail)

Πεδίο	Τιμή
Precondition	Το πάρτι έχει ήδη λήξει (ended / cancelled)
Input	Host προσπαθεί να ενεργοποιήσει closing mode
Expected Output	"Το πάρτι δεν είναι ενεργό" ή ενέργεια μη διαθέσιμη
Postcondition	Καμία αλλαγή, error message

TC_CLOSE_05 – Αποτυχία αποθήκευσης (PersistenceController fail)

Πεδίο	Τιμή
Precondition	Όλα valid, αλλά το save() αποτυγχάνει (DB error, disk full)
Input	Host activates closing mode
Expected Output	Το σύστημα μπορεί να αλλάξει in-memory mode, αλλά error στην αποθήκευση → error UI, η αλλαγή χάνεται μετά από restart
Postcondition	In-memory: closing mode, persistent: παλιά κατάσταση → inconsistent

TC_CLOSE_06 – Ο Host ακυρώνει πριν την επιβεβαίωση

Πεδίο	Τιμή
Precondition	Το σύστημα ζητάει confirmation πριν activate

Πεδίο	Τιμή
Input	Host → Activate, μετά Cancel
Expected Output	Δεν γίνεται αλλαγή mode, καμία αποθήκευση
Postcondition	Το πάρτι παραμένει σε προηγούμενο mode

TC_CLOSE_07 – Συμπεριφορά κατά το closing mode (λειτουργικός έλεγχος)

Πεδίο	Τιμή
Precondition	Closing mode active
Input	Guest προσπαθεί να υποβάλει νέο song request
Expected Output	Απόρριψη: "Το πάρτι κλείνει – δεν γίνονται δεκτά νέα αιτήματα"
Postcondition	Το playlist δεν αλλάζει

TC_CLOSE_08 – Closing mode με χρονικό όριο (auto-termination)

Πεδίο	Τιμή
Precondition	Closing mode έχει διάρκεια 15 λεπτά (παράδειγμα)
Input	Host activates closing mode, περνούν 15 λεπτά

Πεδίο	Τιμή
Expected Output	Το σύστημα αυτόματα τερματίζει το πάρτι (ή μεταβαίνει σε ended)
Postcondition	Το πάρτι δεν είναι πλέον active

TC_CLOSE_09 – Closing mode με ειδική playlist

Πεδίο	Τιμή
Precondition	Υπάρχει "closing playlist" (π.χ., slow songs, last dance)
Input	Host activates closing mode
Expected Output	Το σύστημα φορτώνει/παίζει την closing playlist (αντί για κανονική)
Postcondition	Το τρέχον τραγούδι αλλάζει (ή επόμενο από closing playlist)

TC_CLOSE_10 – Ειδοποίηση προς guests

Πεδίο	Τιμή
Precondition	Closing mode activated
Expected Output	Όλοι οι guests λαμβάνουν μήνυμα/notification: "Το πάρτι κλείνει σ
Postcondition	Guests ενημερωμένοι

UCS8

TC_QR_01 – Επιτυχής σάρωση και πρόσβαση

Πεδίο	Τιμή
Precondition	Ενεργό πάρτι με ID = P001, έγκυρο QR code που αντιστοιχεί στο P
Input	Ο χρήστης σκανάρει το QR code
Expected Output	Σύστημα → "πρόσβαση χορηγήθηκε", ο χρήστης βλέπει επιβεβαίωση στο main UI
Postcondition	Ο χρήστης έχει πρόσβαση (π.χ., μπαίνει στο playlist editor)

TC_QR_02 – QR code άκυρο (δεν υπάρχει στο σύστημα)

Πεδίο	Τιμή
Precondition	Το QR code που σαρώθηκε δεν έχει εκδοθεί ποτέ από το σύστημα
Input	Σάρωση άκυρου QR
Expected Output	"Μη έγκυρο QR code" / "Access denied"

Πεδίο	Τιμή
Postcondition	Ο χρήστης δεν έχει πρόσβαση, παραμένει εκτός

TC_QR_03 – QR code ληγμένο (expired)

Πεδίο	Τιμή
Precondition	Το QR code έχει ημερομηνία λήξης (π.χ., 24 ώρες) και έχει λήξει
Input	Σάρωση ληγμένου QR
Expected Output	"Το QR code έχει λήξει"
Postcondition	Καμία πρόσβαση

TC_QR_04 – Το πάρτι δεν είναι ενεργό (π.χ., έχει τελειώσει)

Πεδίο	Τιμή
Precondition	Το πάρτι με ID = P002 είναι σε κατάσταση "ended" / "cancelled"
Input	Σάρωση QR code που αντιστοιχεί στο P002
Expected Output	"Το πάρτι δεν είναι ενεργό"
Postcondition	Καμία πρόσβαση

TC_QR_05 – Ο χρήστης έχει ήδη πρόσβαση (duplicate scan)

Πεδίο	Τιμή
Precondition	Ο ίδιος χρήστης (same device/session) έχει ήδη αποκτήσει πρόσβαση
Input	Σαρώνει ξανά το ίδιο QR code
Expected Output	"Έχετε ήδη πρόσβαση" / ανακατεύθυνση στο main UI (χωρίς νέα εγγραφή)
Postcondition	Παραμένει η ίδια πρόσβαση (κανένα duplicate entry)

TC_QR_06 – QR code αλλοιωμένο / κατεστραμμένο

Πεδίο	Τιμή
Precondition	Το QR code είναι φυσικά κατεστραμμένο (π.χ., μισό, θολό, κακή εκτύπωση)
Input	Προσπάθεια σάρωσης
Expected Output	"Δεν ήταν δυνατή η ανάγνωση QR code" / "προσπαθήστε ξανά"
Postcondition	Καμία πρόσβαση

TC_QR_07 – Αποτυχία σύνδεσης στο backend / database

Πεδίο	Τιμή
Precondition	Το QR code είναι έγκυρο, αλλά το σύστημα δεν μπορεί να επικοινωνήσει με το database / API
Input	Σάρωση
Expected Output	"Σφάλμα συστήματος, δοκιμάστε αργότερα"
Postcondition	Καμία πρόσβαση (graceful failure)

TC_QR_08 – Σάρωση από μη εξουσιοδοτημένη εφαρμογή

Πεδίο	Τιμή
Precondition	Ο χρήστης σαρώνει το QR με generic camera app (όχι με την επίσημη εφαρμογή) αν το σύστημα απαιτεί specific app
Input	Σάρωση
Expected Output	Ανάλογα: είτε redirect σε web, είτε "παρακαλώ χρησιμοποιήστε την εφαρμογή"
Postcondition	Πιθανή μερική ή καθόλου πρόσβαση

TC_QR_09 – Σάρωση πολύ γρήγορα μετά τη δημιουργία QR

Πεδίο	Τιμή
Precondition	Το QR code μόλις δημιουργήθηκε (< 1 sec)
Input	Σάρωση
Expected Output	Επιτυχής πρόσβαση (το σύστημα το βλέπει ως έγκυρο)
Postcondition	Πρόσβαση χορηγείται

TC_QR_10 – Σάρωση όταν δεν υπάρχει κανένα ενεργό πάρτι (global)

Πεδίο	Τιμή
Precondition	Κανένα πάρτι δεν είναι ενεργό (0 active parties)
Input	Σάρωση οποιουδήποτε QR code
Expected Output	"Δεν υπάρχει ενεργό πάρτι"
Postcondition	Καμία πρόσβαση

UCS9

TC_SONG_01 – Επιτυχής υποβολή αιτήματος (βασική ροή)

Πεδίο	Τιμή
Precondition	Ενεργό πάρτι, Guest έχει πρόσβαση, Closing Mode NOT active
Input	Guest: αναζήτηση → επιλογή τραγουδιού "Bohemian Rhapsody" → submit
Expected Output	Validation → success → καταχώρηση → "Το αίτημά σου υποβλήθηκε"
Postcondition	Το request υπάρχει στο σύστημα, συνδεδεμένο με το party και τον

TC_SONG_02 – Μη έγκυρο QR code (Alternative A1)

Πεδίο	Τιμή
Precondition	Ο χρήστης δεν έχει σκανάρει QR, ή το QR είναι άκυρο
Input	Guest προσπαθεί να υποβάλει request χωρίς valid session
Expected Output	"Μη έγκυρη πρόσβαση – παρακαλώ σκανάρετε το QR code"
Postcondition	Καμία καταχώρηση request

TC_SONG_03 – Το τραγούδι δεν είναι διαθέσιμο (Alternative A2)

Πεδίο	Τιμή
Precondition	Valid party access
Input	Guest υποβάλλει τραγούδι που ΔΕΝ υπάρχει στο SongService (π.χ "Fake Song 123")
Expected Output	"Το τραγούδι δεν βρέθηκε – παρακαλώ επιλέξτε άλλο"
Postcondition	Καμία καταχώρηση, το UI παραμένει για νέα επιλογή

TC_SONG_04 – Duplicate request (ίδιος χρήστης, ίδιο τραγούδι)

Πεδίο	Τιμή
Precondition	Ο Guest έχει ήδη υποβάλει το "Bohemian Rhapsody" στο ίδιο party
Input	Ξαναυποβάλλει το ίδιο τραγούδι
Expected Output	"Έχετε ήδη ζητήσει αυτό το τραγούδι"
Postcondition	Δεν καταχωρείται δεύτερο request (κανένα duplicate)

TC_SONG_05 – Το πάρτι βρίσκεται σε Closing Mode

Πεδίο	Τιμή
Precondition	Το πάρτι είναι σε Closing Mode (από προηγούμενο use case)
Input	Guest προσπαθεί να υποβάλει οποιοδήποτε τραγούδι
Expected Output	"Το πάρτι κλείνει – δεν γίνονται δεκτά νέα αιτήματα"
Postcondition	Καμία καταχώρηση

TC_SONG_06 – Το πάρτι δεν είναι ενεργό (precondition fail)

Πεδίο	Τιμή
Precondition	Το πάρτι έχει λήξει (ended)
Input	Guest προσπαθεί να υποβάλει request
Expected Output	"Το πάρτι δεν είναι ενεργό"
Postcondition	Καμία καταχώρηση

TC_SONG_07 – Αναζήτηση τραγουδιού (partial match)

Πεδίο	Τιμή
Precondition	Valid party access

Πεδίο	Τιμή
Input	Guest πληκτρολογεί "Bohem" στο search field
Expected Output	Το σύστημα εμφανίζει "Bohemian Rhapsody" (και άλλα matching)
Postcondition	Ο Guest μπορεί να επιλέξει από τα αποτελέσματα

TC_SONG_08 – Αναζήτηση χωρίς αποτελέσματα

Πεδίο	Τιμή
Precondition	Valid party access
Input	Guest αναζητά "XYZ123NonExistentSong"
Expected Output	"Δεν βρέθηκαν τραγούδια"
Postcondition	Ο Guest μπορεί να δοκιμάσει ξανά

TC_SONG_09 – Αποτυχία αποθήκευσης request (PersistenceController fail)

Πεδίο	Τιμή
Precondition	Όλα valid, αλλά το save() αποτυγχάνει
Input	Guest υποβάλλει valid τραγούδι

Πεδίο	Τιμή
Expected Output	"Σφάλμα συστήματος – δοκιμάστε ξανά αργότερα"
Postcondition	To request ΔΕΝ καταχωρείται

TC_SONG_10 – Πολλαπλά requests από διαφορετικούς guests

Πεδίο	Τιμή
Precondition	Ενεργό πάρτι, 3 διαφορετικοί guests
Input	Guest A: "Song A", Guest B: "Song B", Guest C: "Song C"
Expected Output	Και τα 3 requests καταχωρούνται ανεξάρτητα
Postcondition	3 requests στο σύστημα, όλα διαθέσιμα για playlist

TC_SONG_11 – Ο Guest ακυρώνει πριν την υποβολή

Πεδίο	Τιμή
Precondition	Valid party access, Guest έχει επιλέξει τραγούδι αλλά δεν πατά su
Input	Guest πατάει "Cancel" ή κλείνει το UI
Expected Output	Δεν γίνεται καμία καταχώρηση

Πεδίο	Τιμή
Postcondition	Κανένα νέο request

TC_SONG_12 – Υποβολή τραγουδιού χωρίς να επιλεγεί τραγούδι πρώτα

Πεδίο	Τιμή
Precondition	Valid party access
Input	Guest πατάει "Submit" χωρίς να έχει επιλέξει τραγούδι
Expected Output	"Παρακαλώ επιλέξτε τραγούδι πρώτα"
Postcondition	Καμία καταχώρηση

UCS10

TC_LIGHT_01 – Επιτυχής συγχρονισμός (βασική ροή)

Πεδίο	Τιμή
Precondition	Ενεργή playlist, current song = "Bohemian Rhapsody", LightingSystem
Input	System triggers start sync
Expected Output	getCurrentSong() → extractBeat() → analyzeBeat() → mapToColors() → s
Postcondition	LightingSystem synchronized, state saved

TC_LIGHT_02 – Δεν υπάρχει current song (Alternative A1)

Πεδίο	Τιμή
Precondition	Ενεργή playlist αλλά κανένα τραγούδι δεν παίζει (paused ή empty queue)
Input	start sync
Expected Output	getCurrentSong() returns null → sync stops ή περιμένει
Postcondition	Κανένα beat analysis, καμία εντολή στο LightingSystem

TC_LIGHT_03 – BeatAnalyzer αποτυγχάνει (Alternative A2)

Πεδίο	Τιμή
Precondition	Current song exists, αλλά το BeatAnalyzer δεν μπορεί να εξάγει beat (corrupted file, unsupported format)
Input	start sync
Expected Output	analyzeBeat() fails → fallback to default lighting pattern
Postcondition	Ο φωτισμός λειτουργεί με default pattern (π.χ., σταθερό χρώμα ή α fade)

TC_LIGHT_04 – LightingSystem offline / network failure (A3)

Πεδίο	Τιμή
Precondition	Current song exists, beat analysis OK, αλλά το LightingSystem δεν απαντάει
Input	sendCommands(pattern)
Expected Output	transmit() timeout ή error → retry (π.χ., 3 φορές) → logging error
Postcondition	Ο φωτισμός ΔΕΝ συγχρονίζεται, το σύστημα καταγράφει το σφάλμα

TC_LIGHT_05 – Αποτυχία αποθήκευσης κατάστασης (A4)

Πεδίο	Τιμή
Precondition	Όλα τα προηγούμενα βήματα επιτυχή, αλλά save() αποτυγχάνει (dis DB error)
Input	start sync → full success chain
Expected Output	save() fails → error logged, αλλά ο συγχρονισμός δεν διακόπτεται
Postcondition	Ο φωτισμός είναι συγχρονισμένος, αλλά η κατάσταση ΔΕΝ αποθηκεύτηκε

TC_LIGHT_06 – Τραγούδι χωρίς beats (π.χ., spoken word, ηχητικό τοπίο)

Πεδίο	Τιμή
Precondition	Current song has no detectable beats (ambient, classical piano so
Input	start sync
Expected Output	analyzeBeat() returns minimal beat data → mapToColors() produce slow/gentle pattern
Postcondition	Ο φωτισμός είναι ήπιος, όχι έντονο strobe

TC_LIGHT_07 – Αλλαγή τραγουδιού κατά τη διάρκεια sync

Πεδίο	Τιμή
Precondition	Sync in progress για το Song A
Input	To playlist αλλάζει σε Song B (π.χ., next track)
Expected Output	Το σύστημα σταματά το sync για Song A και ξεκινά νέο sync για Song B
Postcondition	Ο φωτισμός συγχρονίζεται με το Song B

TC_LIGHT_08 – Πολύ γρήγορο τραγούδι (high BPM)

Πεδίο	Τιμή
Precondition	Current song BPM = 180 (π.χ., drum and bass)
Input	start sync
Expected Output	extractBeat() → mapToColors() → sendCommands() με υψηλή συχνότητα
Postcondition	Ο φωτισμός αναβοσβήνει γρήγορα, συγχρονισμένος

TC_LIGHT_09 – Πολύ αργό τραγούδι (low BPM)

Πεδίο	Τιμή
Precondition	Current song BPM = 60 (π.χ., ballad)

Πεδίο	Τιμή
Input	start sync
Expected Output	mapToColors() produces slow transitions
Postcondition	Ο φωτισμός αλλάζει αργά, χωρίς έντονες εναλλαγές

TC_LIGHT_10 – DataMapper αστοχία (invalid color mapping)

Πεδίο	Τιμή
Precondition	Beat analysis OK, αλλά το DataMapper δεν μπορεί να μετατρέψει b σε colors (π.χ., config error)
Input	mapToColors(beatData)
Expected Output	Default color pattern (π.χ., λευκό) ή error fallback
Postcondition	Ο φωτισμός λειτουργεί με default, όχι beat-synced

TC_LIGHT_11 – Πολλαπλά sync requests ταυτόχρονα

Πεδίο	Τιμή
Precondition	Sync ήδη σε εξέλιξη

Πεδίο	Τιμή
Input	Νέο start sync καλείται
Expected Output	Το σύστημα είτε αγνοεί το δεύτερο, είτε σταματά το πρώτο και ξεκινάει νέο
Postcondition	Μόνο ένα ενεργό sync τη φορά

TC_LIGHT_12 – Αποθήκευση κατάστασης μετά από κάθε sync

Πεδίο	Τιμή
Precondition	Sync completed successfully
Expected Output	save() καλείται στο τέλος
Postcondition	Η τελευταία lighting state είναι αποθηκευμένη

UCS11

TC_SUB_01 – Επιτυχής αγορά συνδρομής (βασική ροή)

Πεδίο	Τιμή
Precondition	Host authenticated, valid payment method, Premium plan available
Input	Host selects "Premium Plan" → submit
Expected Output	<code>validate()</code> → <code>valid</code> → <code>getdetails()</code> → <code>plan</code> → <code>PaymentOrchestrator.execute()</code> → <code>pa</code>
Postcondition	Subscription active, host has premium features

TC_SUB_02 – Μη έγκυρα δεδομένα (Alternative A1)

Πεδίο	Τιμή
Precondition	Host authenticated
Input	Host submits with invalid credit card number (π.χ., 1234)
Expected Output	<code>ValidationController</code> returns <code>invaliddata</code> → <code>ErrorUI</code>
Postcondition	Καμία πληρωμή, καμία αποθήκευση, subscription unchanged

TC_SUB_03 – Αποτυχία πληρωμής (Alternative A2)

Πεδίο	Τιμή
Precondition	Valid data, but payment fails (π.χ., insufficient funds, card declined)

Πεδίο	Τιμή
Input	Host submits valid plan
Expected Output	<code>PaymentOrchestrator.execute()</code> returns <code>payment failed</code> → <code>ErrorUI</code>
Postcondition	No subscription saved, no charges (ή partially charged depending on system)

TC_SUB_04 – PricingService unavailable (A3)

Πεδίο	Τιμή
Precondition	PricingService is down / timeout
Input	Host submits request
Expected Output	<code>getdetails()</code> fails → <code>ErrorUI</code> ή retry mechanism
Postcondition	Subscription not processed

TC_SUB_05 – Ο Host ακυρώνει πριν την πληρωμή (A4)

Πεδίο	Τιμή
Precondition	Host reaches PaymentUI
Input	Host clicks "Cancel" instead of "Confirm Payment"
Expected Output	No <code>execute()</code> → no <code>store()</code> → no <code>save()</code>

Πεδίο	Τιμή
Postcondition	Subscription unchanged

TC_SUB_06 – Αναβάθμιση από existing subscription

Πεδίο	Τιμή
Precondition	Host has "Basic" subscription active
Input	Host upgrades to "Pro" plan
Expected Output	System processes prorated charge, updates subscription in PersistenceController
Postcondition	New "Pro" subscription, old "Basic" replaced

TC_SUB_07 – Αποτυχία αποθήκευσης (PersistenceController fail)

Πεδίο	Τιμή
Precondition	Payment successful, but <code>save()</code> fails (DB error)
Input	Valid subscription purchase
Expected Output	Payment taken (unfortunately), but system shows error → must reconcile manual refund
Postcondition	Subscription NOT saved in system (inconsistent state)

TC_SUB_08 – Duplicate subscription purchase

Πεδίο	Τιμή
Precondition	Host already has active "Premium" subscription
Input	Host tries to buy "Premium" again
Expected Output	Validation rejects: "Already subscribed" ή system offers upgrade path
Postcondition	No double charge, subscription unchanged

TC_SUB_09 – Μη έγκυρο πρόγραμμα (plan does not exist)

Πεδίο	Τιμή
Precondition	Host selects a plan ID that doesn't exist in system
Input	Submit with invalid plan ID
Expected Output	<code>getdetails()</code> returns null → <code>ErrorUI</code>
Postcondition	No subscription created

TC_SUB_10 – Payment timeout (network issue)

Πεδίο	Τιμή
Precondition	Valid data, but PaymentOrchestrator takes too long (>30s)
Input	Submit request
Expected Output	Timeout → <code>ErrorUI</code> : "Payment service not responding"
Postcondition	No subscription, no charge (assumed)

TC_SUB_11 – Επιτυχής πληρωμή με διαφορετική μέθοδο (π.χ., PayPal)

Πεδίο	Τιμή
Precondition	Host selects PayPal as payment method
Input	Submit → redirect to PayPal → completes payment
Expected Output	<code>PaymentOrchestrator</code> receives webhook/confirmation → <code>store()</code> → <code>save()</code> → <code>ConfirmationUI</code>
Postcondition	Subscription active

TC_SUB_12 – Ακύρωση συνδρομής (αντί για αγορά)

Πεδίο	Τιμή
Precondition	Host has active subscription
Input	Host selects "Cancel Subscription"
Expected Output	System updates subscription end date, no future charges
Postcondition	Subscription will expire at end of billing period

UCS12

TC_PAY_01 – Επιτυχής επεξεργασία πληρωμής (βασική ροή)

Πεδίο	Τιμή
Precondition	Valid payment request exists, ExternalPaymentGateway available
Input	PaymentService receives authorization request
Expected Output	prepare response → format data → forward request → authorize → success → record → update → save → store → Co
Postcondition	Payment recorded, Subscription updated (active), persistence saved

TC_PAY_02 – Αποτυχία πληρωμής (Alternative A1)

Πεδίο	Τιμή
Precondition	Valid request, but ExternalPaymentGateway returns failure (π.χ., declined card)
Input	Authorization request
Expected Output	authorize → failure → ResponseHandler → ErrorUI
Postcondition	No subscription update, no payment record (or payment marked as failed)

TC_PAY_03 – ExternalPaymentGateway timeout (A2)

Πεδίο	Τιμή
Precondition	Valid request, but gateway does not respond within timeout (π.χ., sec)
Input	Authorization request
Expected Output	Timeout → failure → ErrorUI ή retry logic
Postcondition	Payment not processed, subscription unchanged

TC_PAY_04 – Αποτυχία αποθήκευσης μετά από επιτυχή πληρωμή (A3)

Πεδίο	Τιμή
Precondition	Payment successful (success from gateway), but save() fails (DB e
Input	Authorization request
Expected Output	success → record → update → save() fails → inconsistency logged, possible ErrorUI
Postcondition	Payment taken but NOT recorded in system → reconciliation needed

TC_PAY_05 – Μη έγκυρα δεδομένα από PaymentAPI (A4)

Πεδίο	Τιμή
Precondition	Payment request with malformed data (π.χ., missing amount, invalid currency)
Input	Request reaches PaymentAPI
Expected Output	prepare response or format data fails → ErrorUI
Postcondition	No payment processed

TC_PAY_06 – Επιτυχής πληρωμή με ενημέρωση υπάρχουσας συνδρομής (renewal)

Πεδίο	Τιμή
Precondition	Existing subscription about to expire, renewal payment requested
Input	Authorization request for renewal
Expected Output	success → update extends subscription end date
Postcondition	Subscription renewed, new end date = old end date + billing period

TC_PAY_07 – Επιτυχής πληρωμή για αναβάθμιση (upgrade)

Πεδίο	Τιμή
Precondition	Existing "Basic" subscription, upgrade to "Pro" requested
Input	Payment for upgrade (prorated amount)
Expected Output	success → update changes subscription plan to "Pro"
Postcondition	Subscription plan = "Pro", active

TC_PAY_08 – Double charging prevention (idempotency)

Πεδίο	Τιμή
Precondition	Same payment request sent twice (network retry)

Πεδίο	Τιμή
Input	Two identical authorization requests with same idempotency key
Expected Output	First → success, second → ignored or returns cached result
Postcondition	Only one payment recorded, one subscription update

TC_PAY_09 – Αποτυχία λόγω ανεπαρκών υπολοίπων (insufficient funds)

Πεδίο	Τιμή
Precondition	Valid request, but card has insufficient funds
Input	Authorization request
Expected Output	authorize → failure (insufficient_funds) → ErrorUI
Postcondition	No payment recorded, subscription unchanged

TC_PAY_10 – Αποτυχία λόγω ληγμένης κάρτας

Πεδίο	Τιμή
Precondition	Card expiry date is in the past
Input	Authorization request

Πεδίο	Τιμή
Expected Output	authorize → failure (expired_card) → ErrorUI
Postcondition	Payment not processed

TC_PAY_11 – ExternalPaymentGateway returns partial response

Πεδίο	Τιμή
Precondition	Gateway returns success but missing transaction ID
Input	Authorization request
Expected Output	ResponseHandler detects malformed response → ErrorUI
Postcondition	Payment status unclear → 可能需要 manual review

TC_PAY_12 – Webhook από PaymentService (asynchronous flow)

Πεδίο	Τιμή
Precondition	Payment processed asynchronously
Input	Webhook received from PaymentService after completion
Expected Output	ResponseHandler → record → update → save

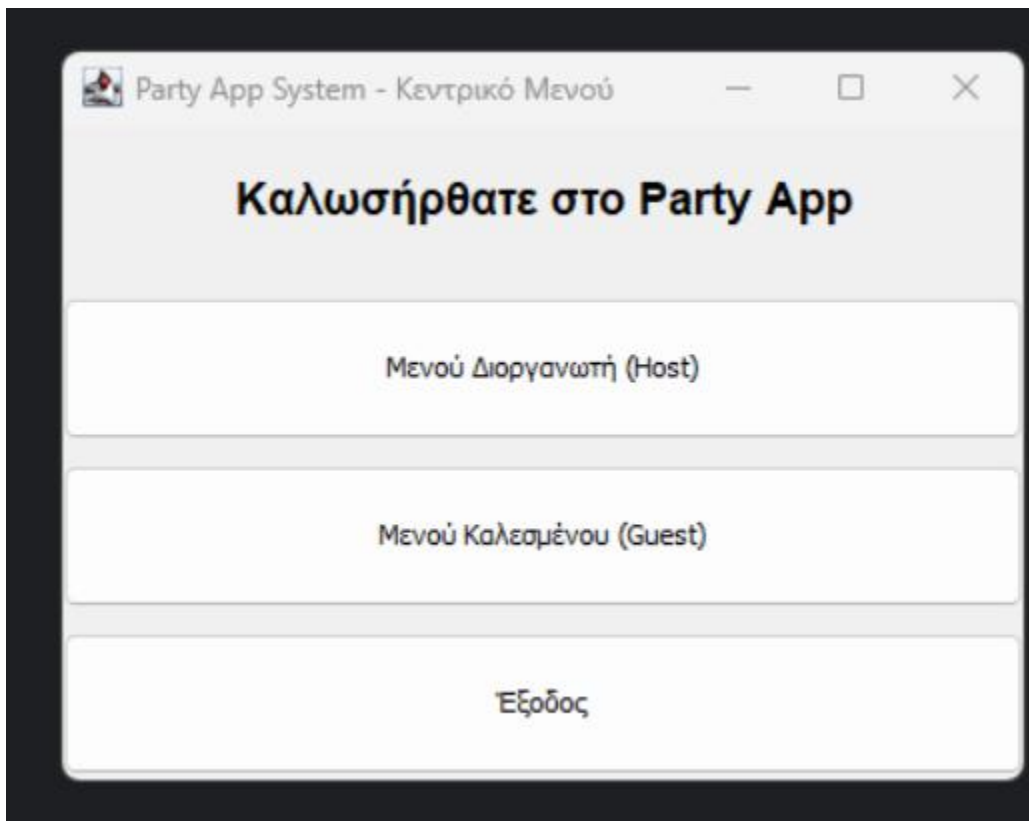
Πεδίο

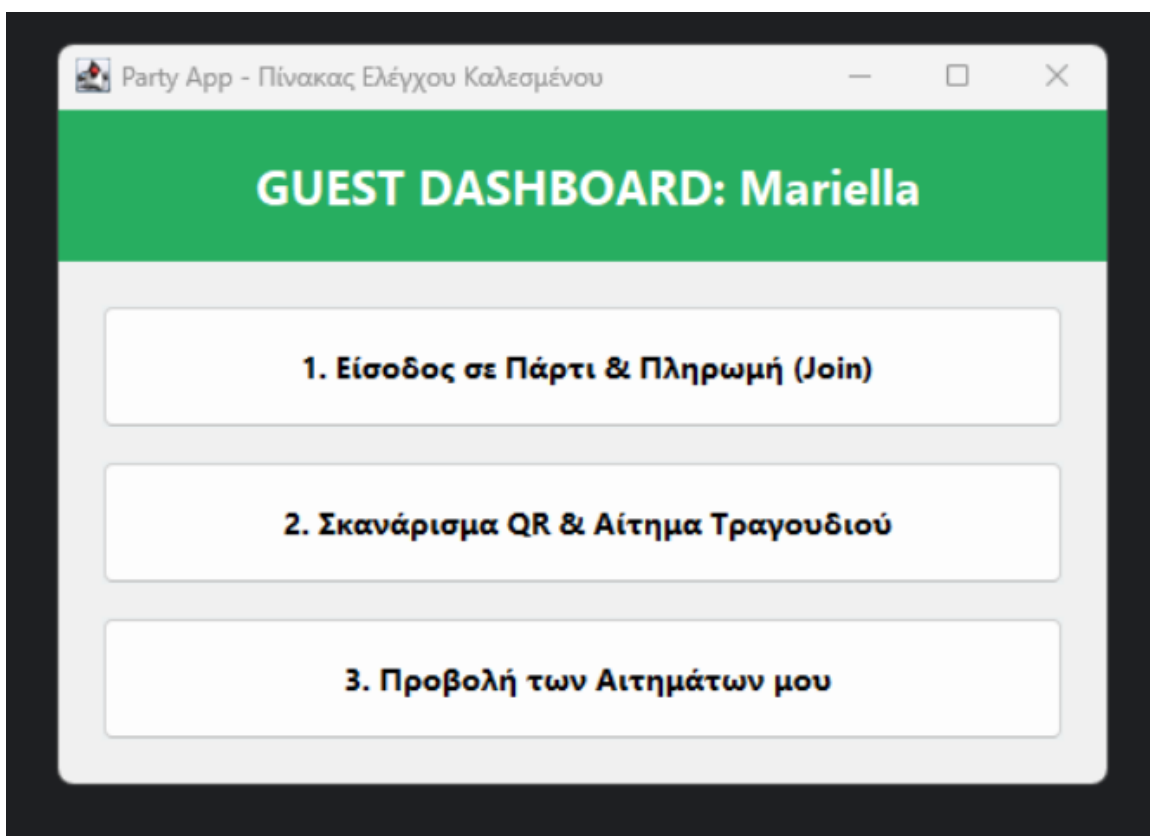
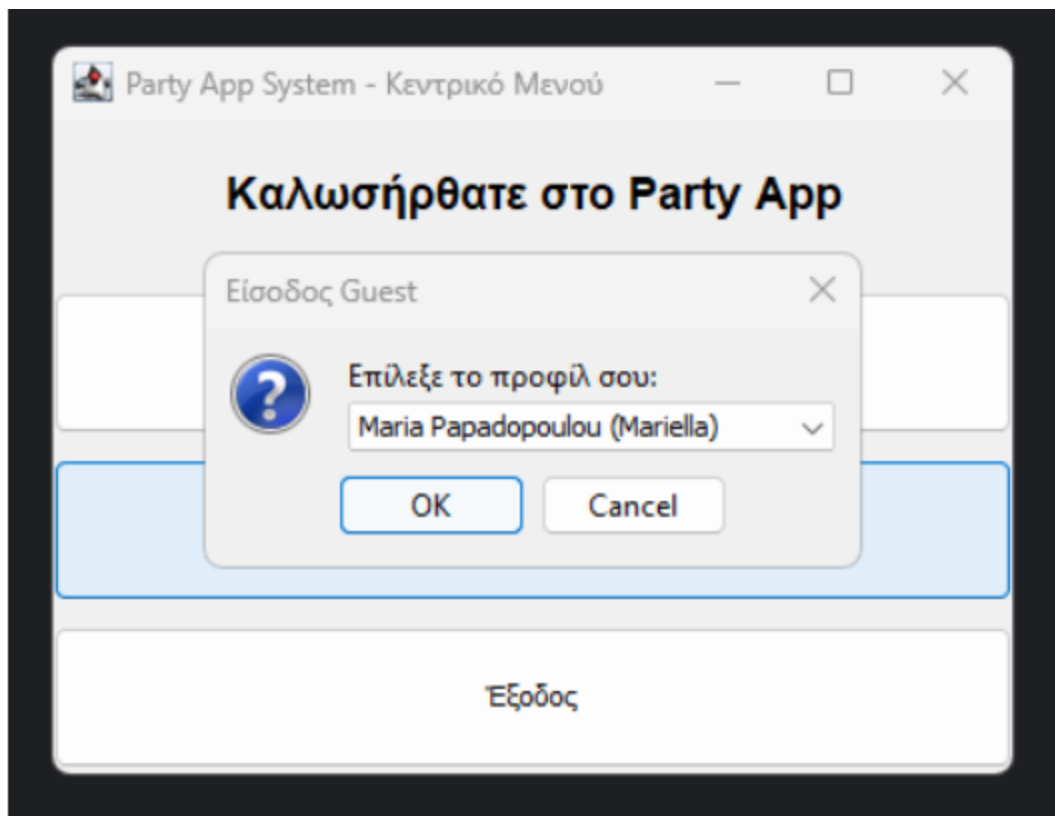
Τιμή

Postcondition

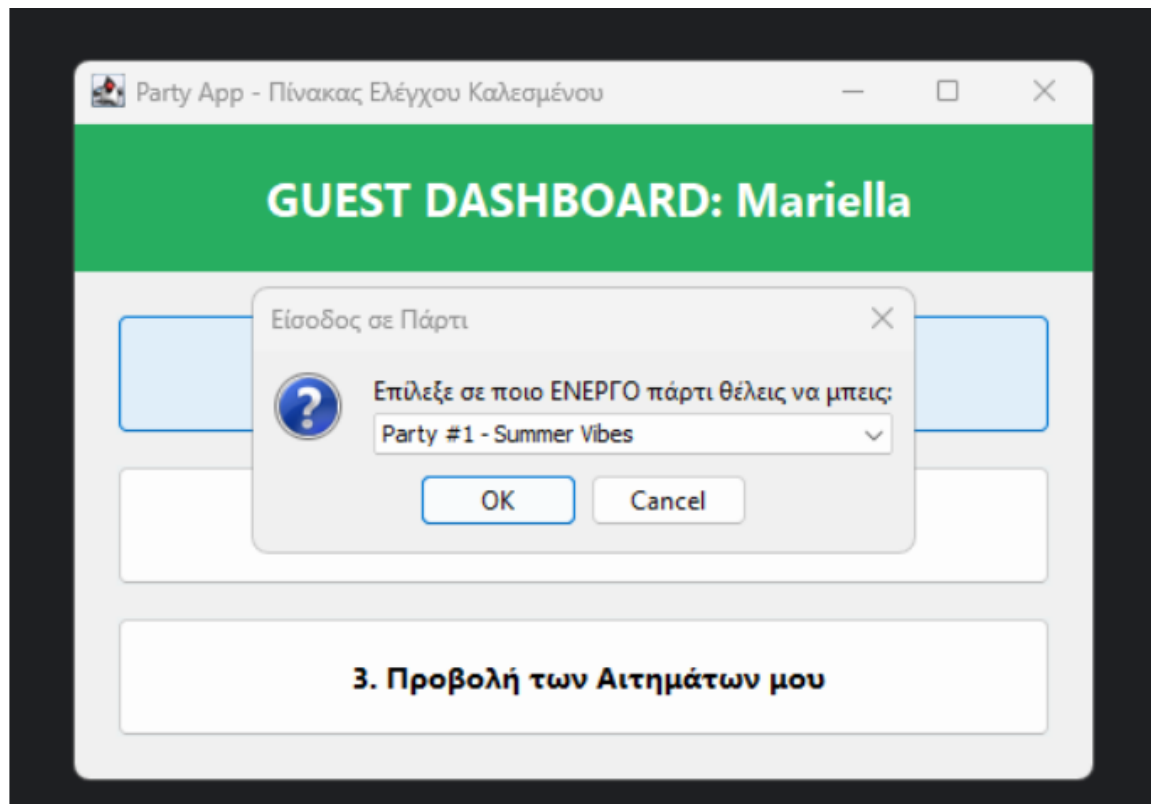
Payment recorded, subscription updated

9) SCREENSHOTS ΕΚΤΕΛΕΣΗΣ





Guest





GUEST DASHBOARD: Mariella



Πληρωμή Εισόδου



Απαιτείται πληρωμή εισόδου €15 για το πάρτι: Summer Vibes

Θέλετε να προχωρήσετε;

Yes

No

3. Προβολή των Αιτημάτων μου

GUEST DASHBOARD: Mariella

Επιτυχία



✓ Η πληρωμή ολοκληρώθηκε! Έγινες μέλος στο πάρτι.

OK

3. Προβολή των Αιτημάτων μου

GUEST DASHBOARD: Mariella

Επιβεβαίωση Παρουσίας



📷 ΣΚΑΝΑΡΙΣΜΑ QR CODE

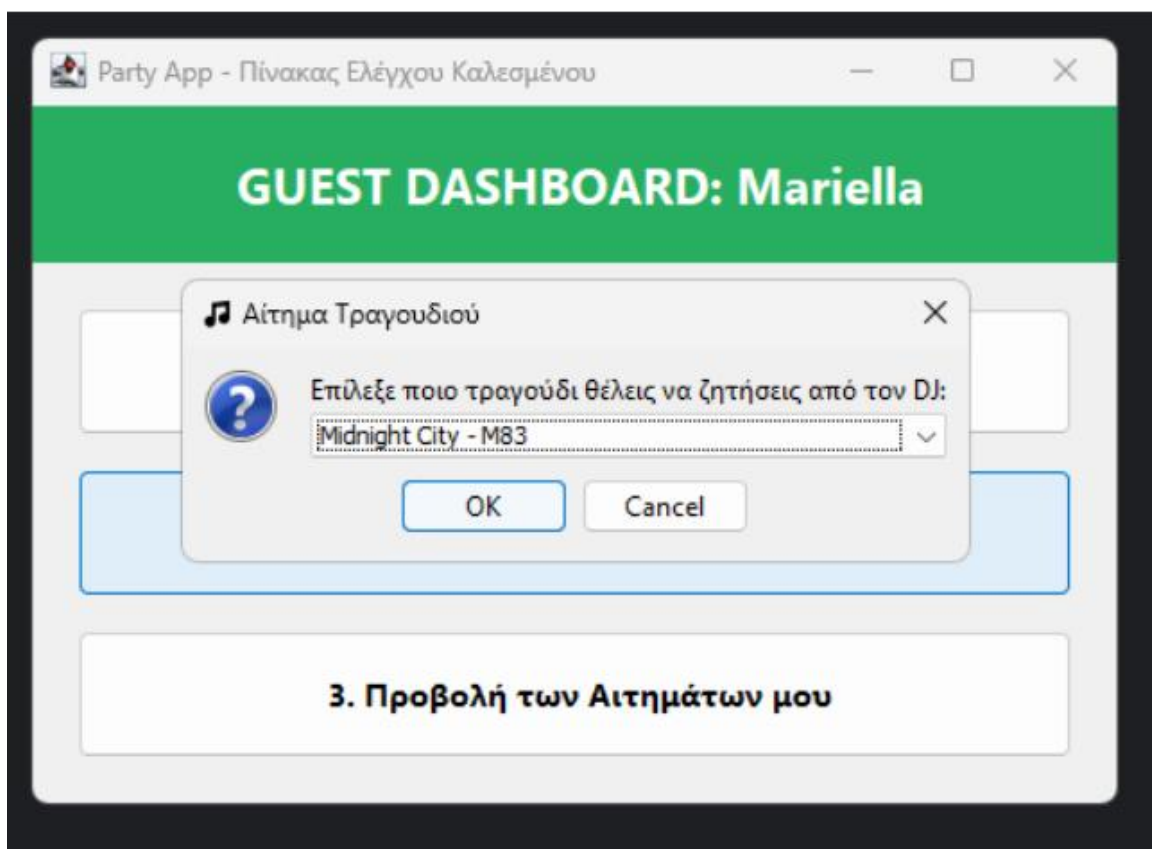
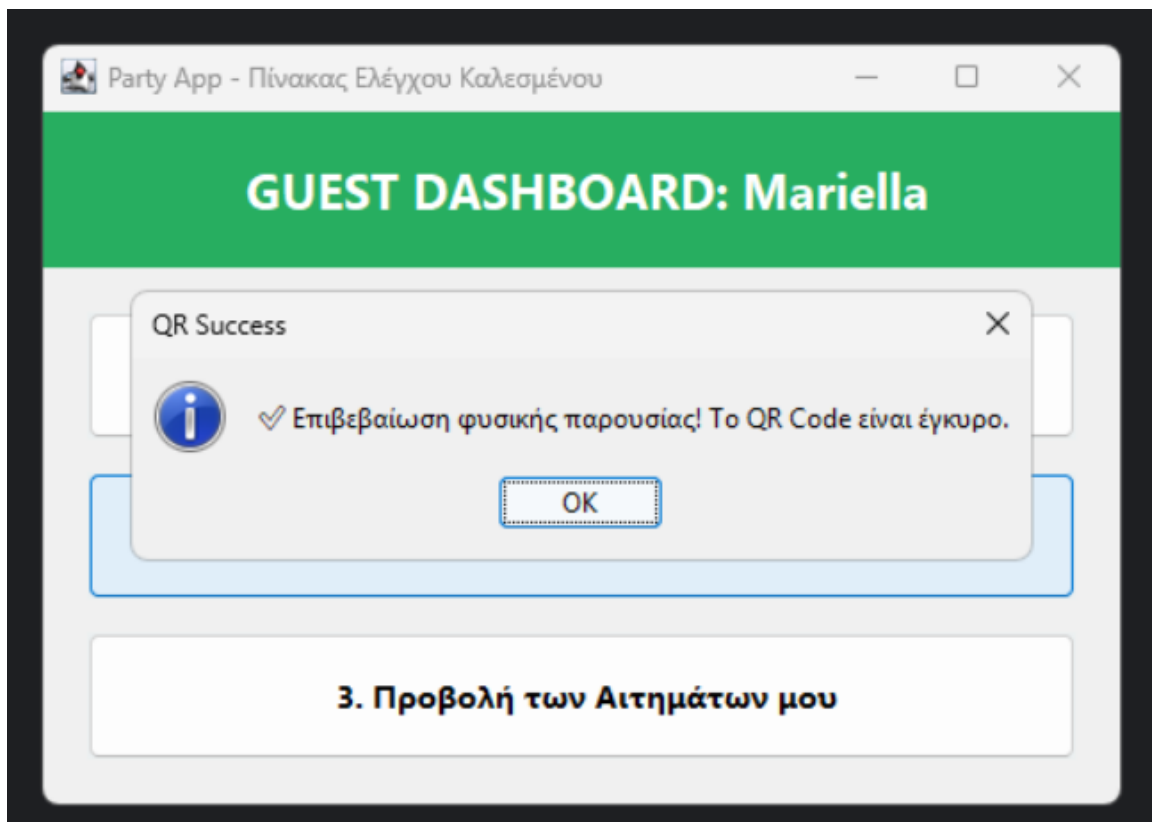
Βρίσκεσαι στο πάρτι: Summer Vibes

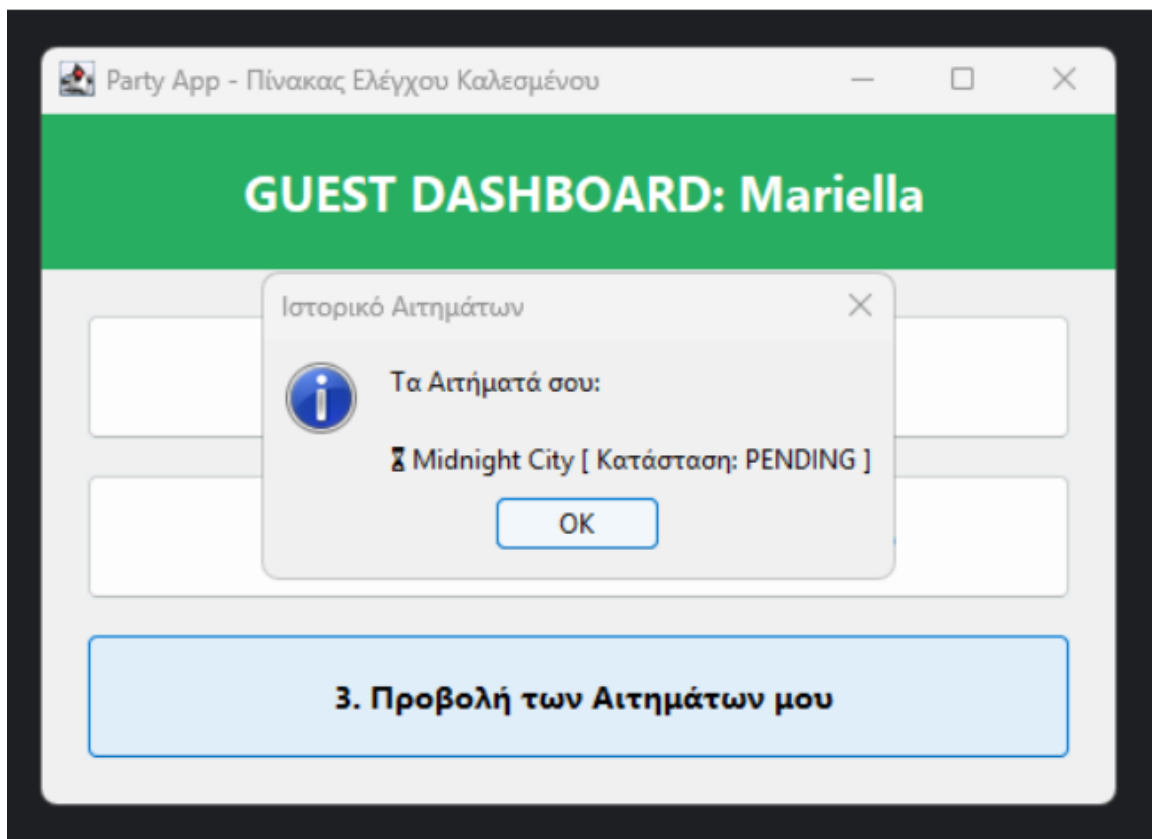
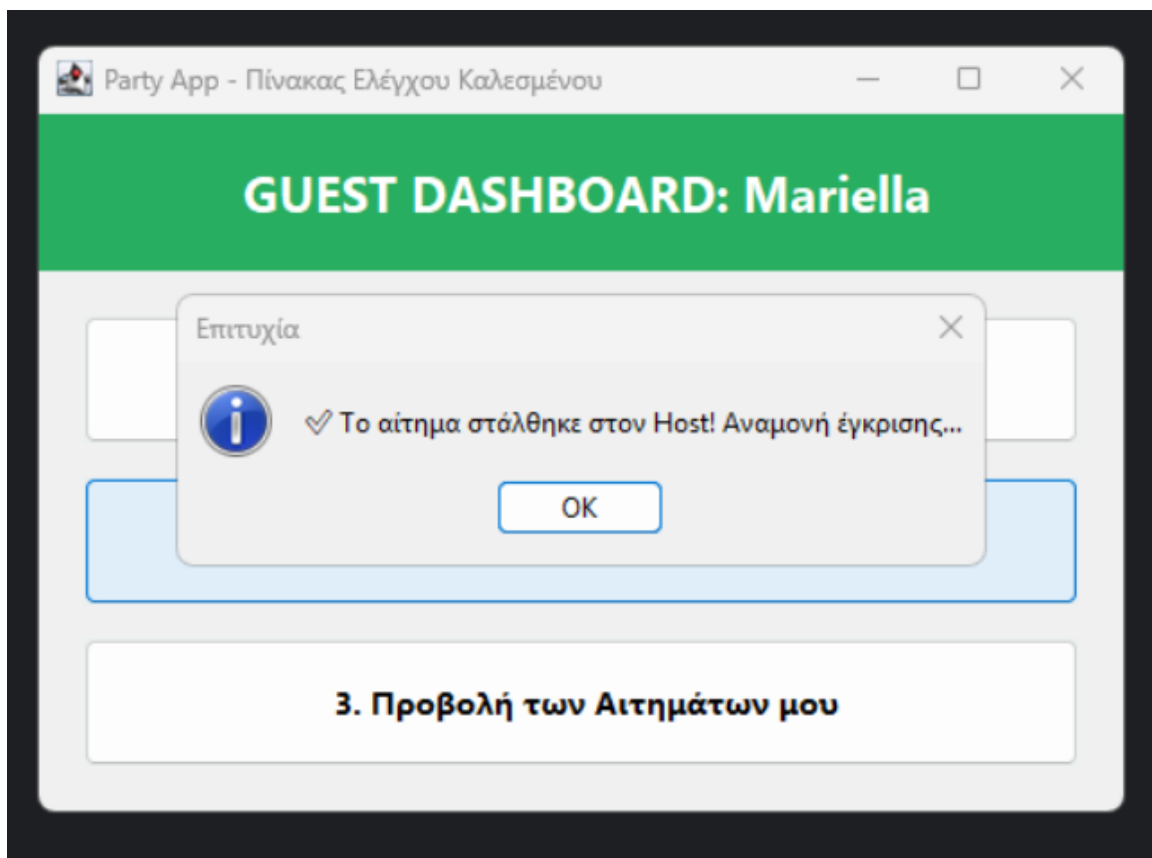
Πληκτρολόγησε το ID του πάρτι που βλέπεις στην οθόνη του μαγαζιού:

OK

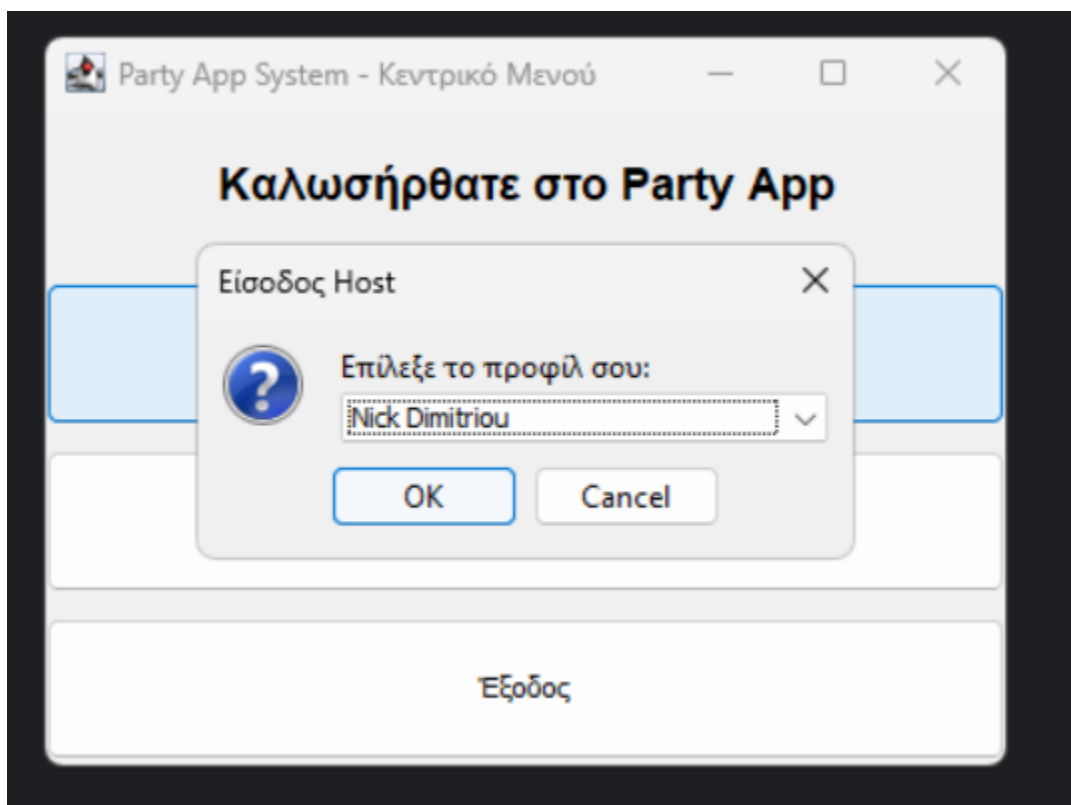
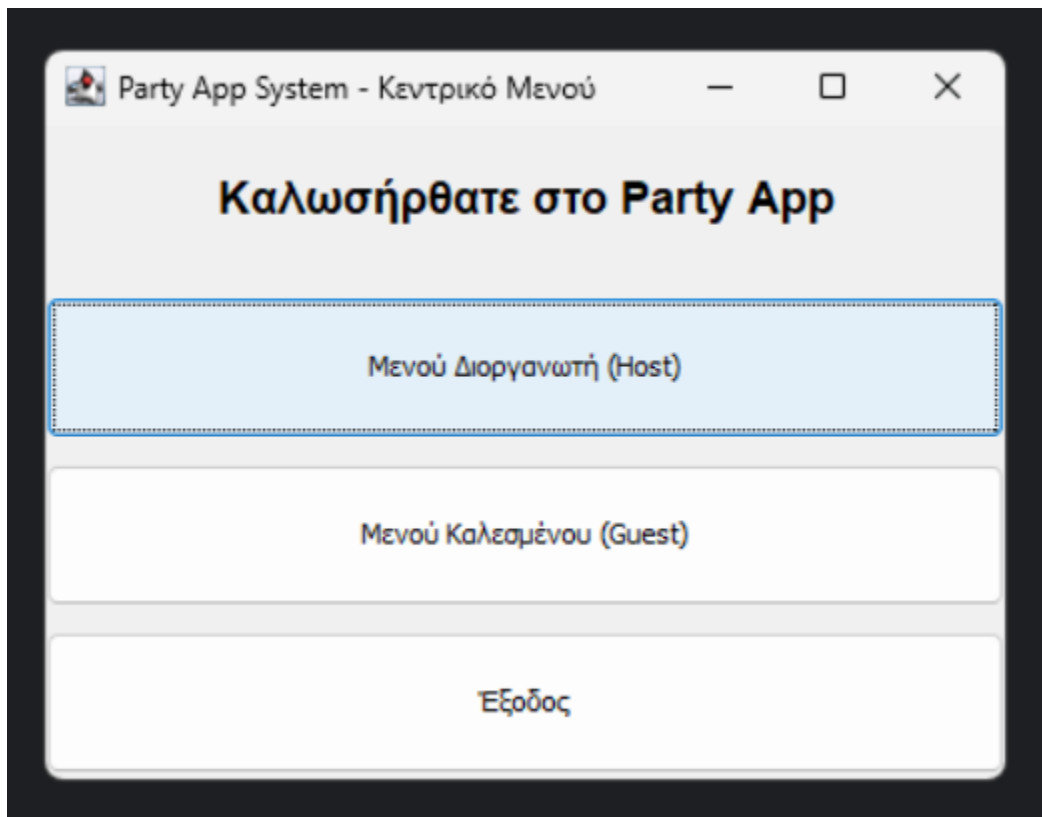
Cancel

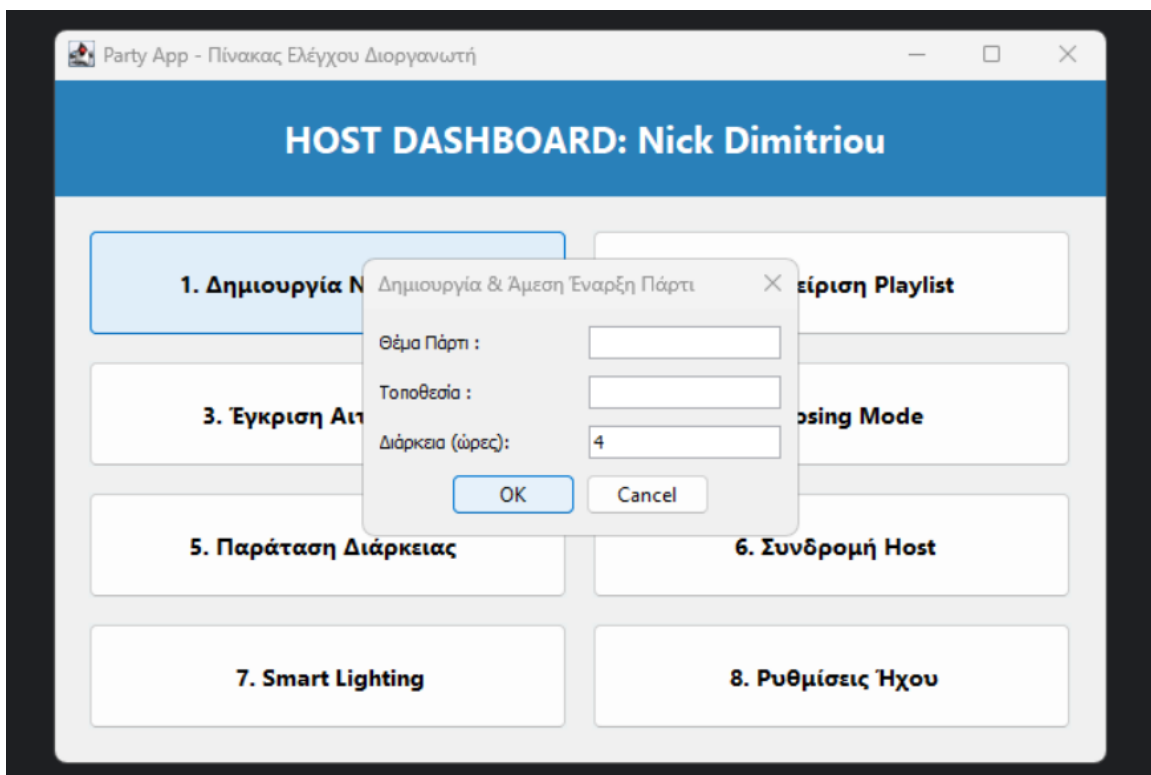
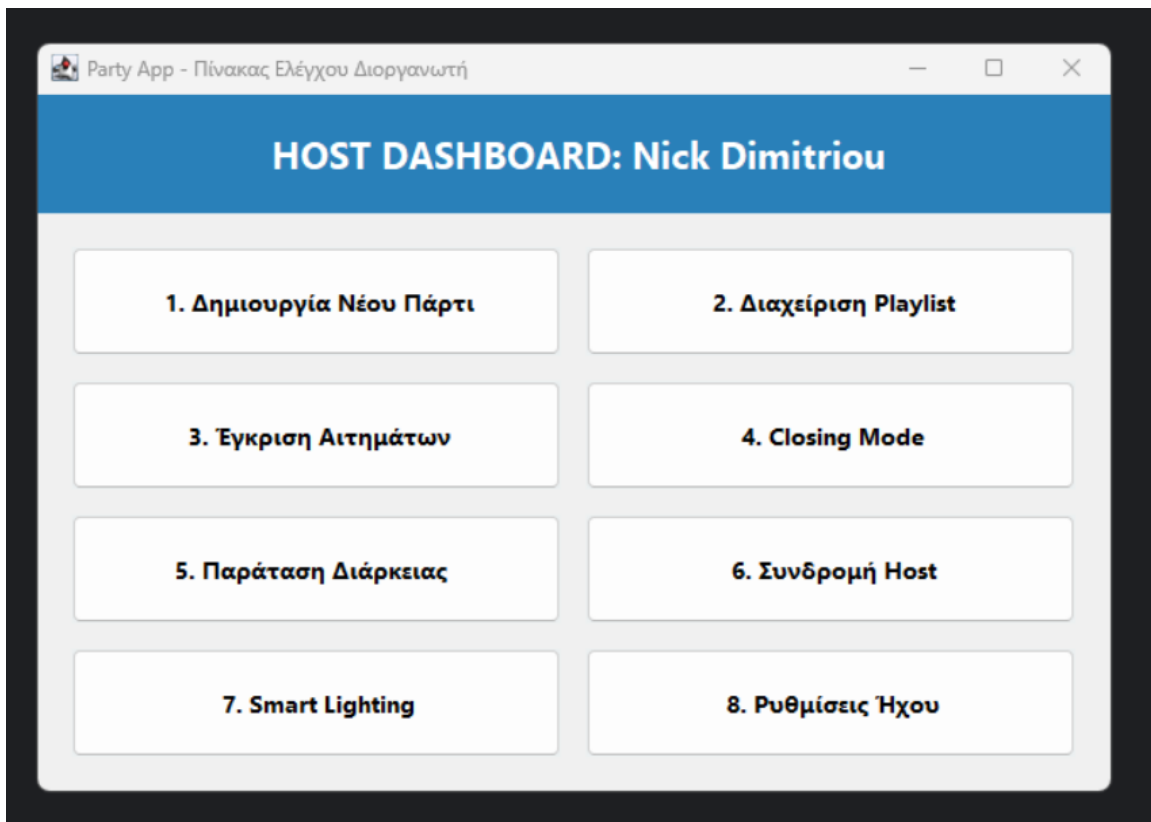
3. Προβολή των Αιτημάτων μου

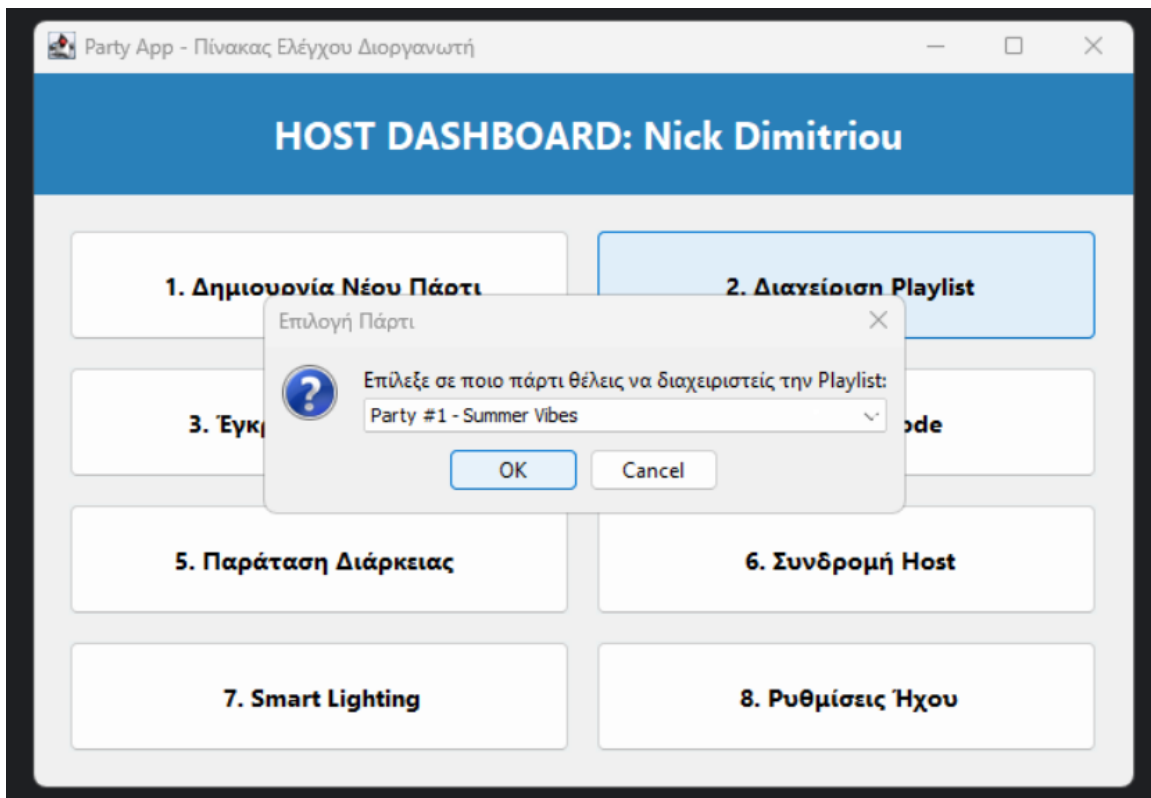
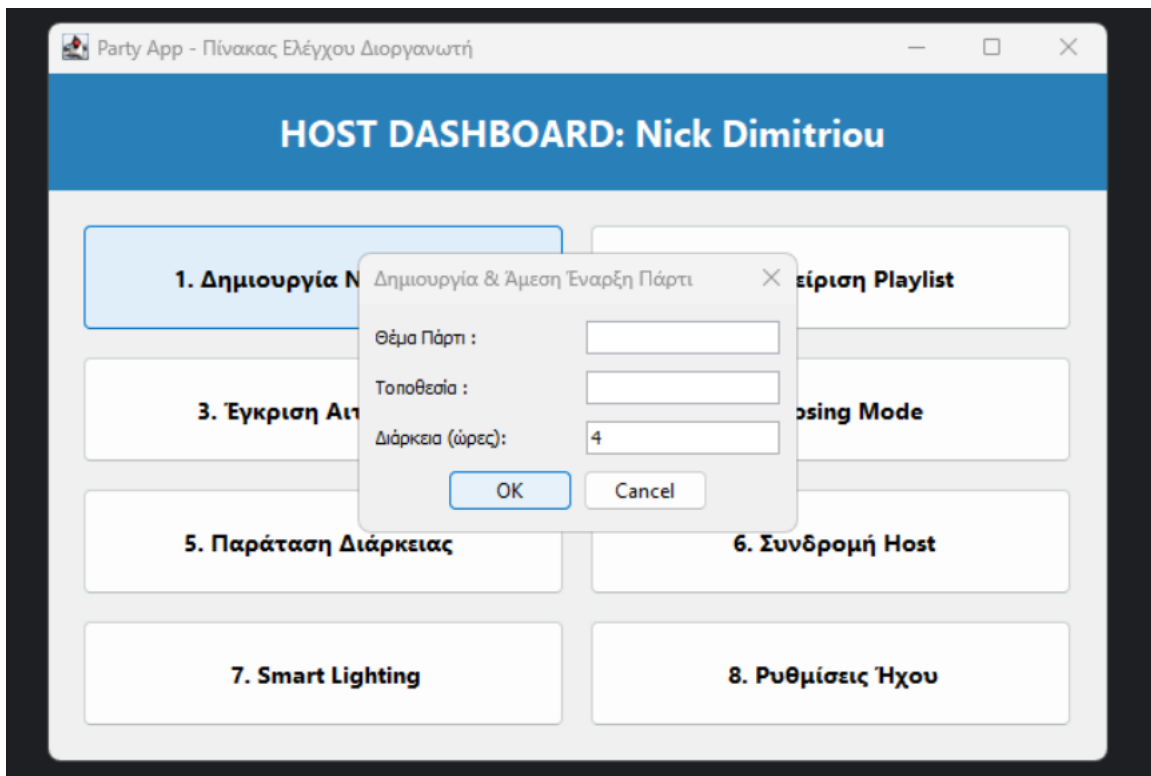


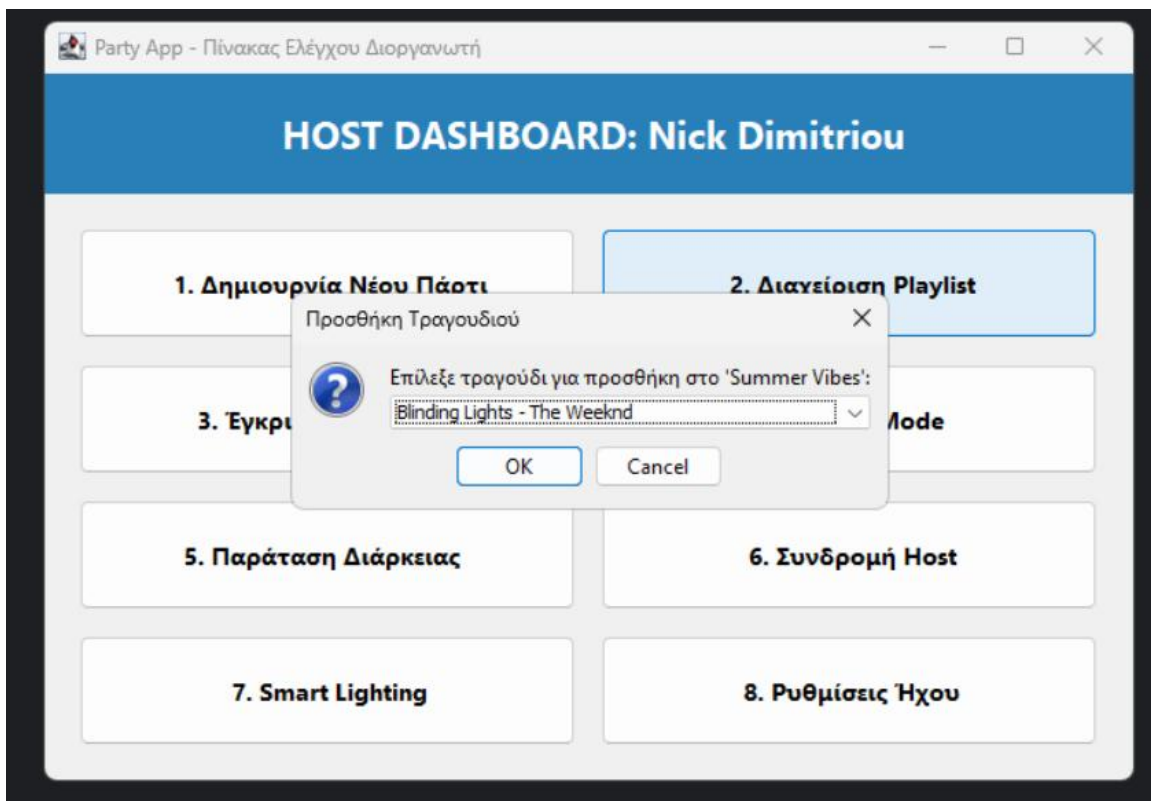
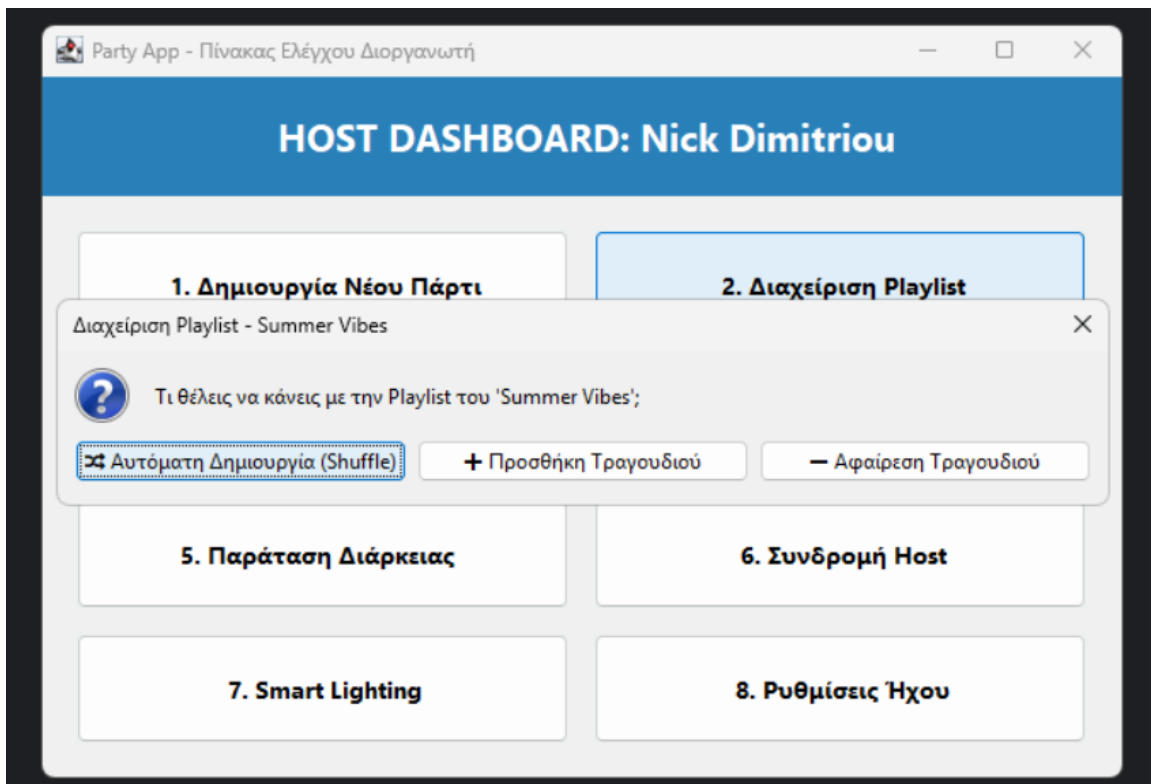


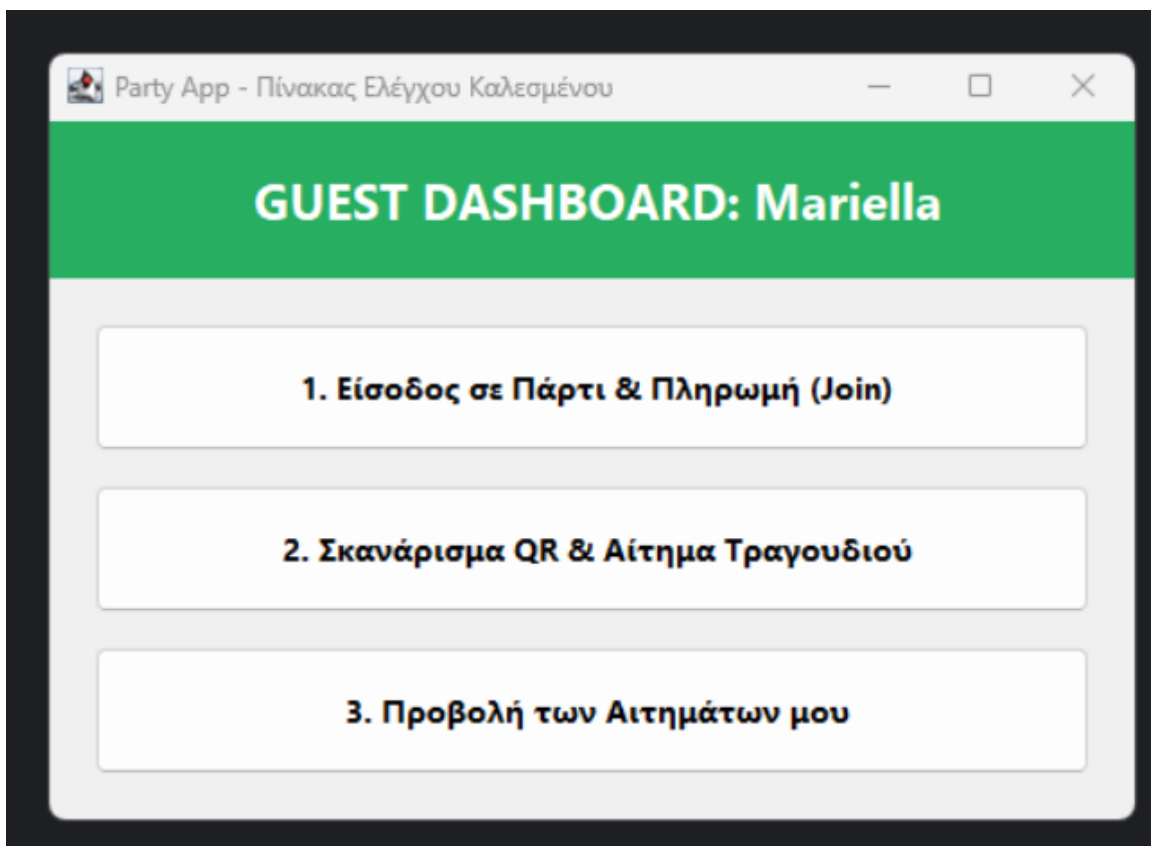
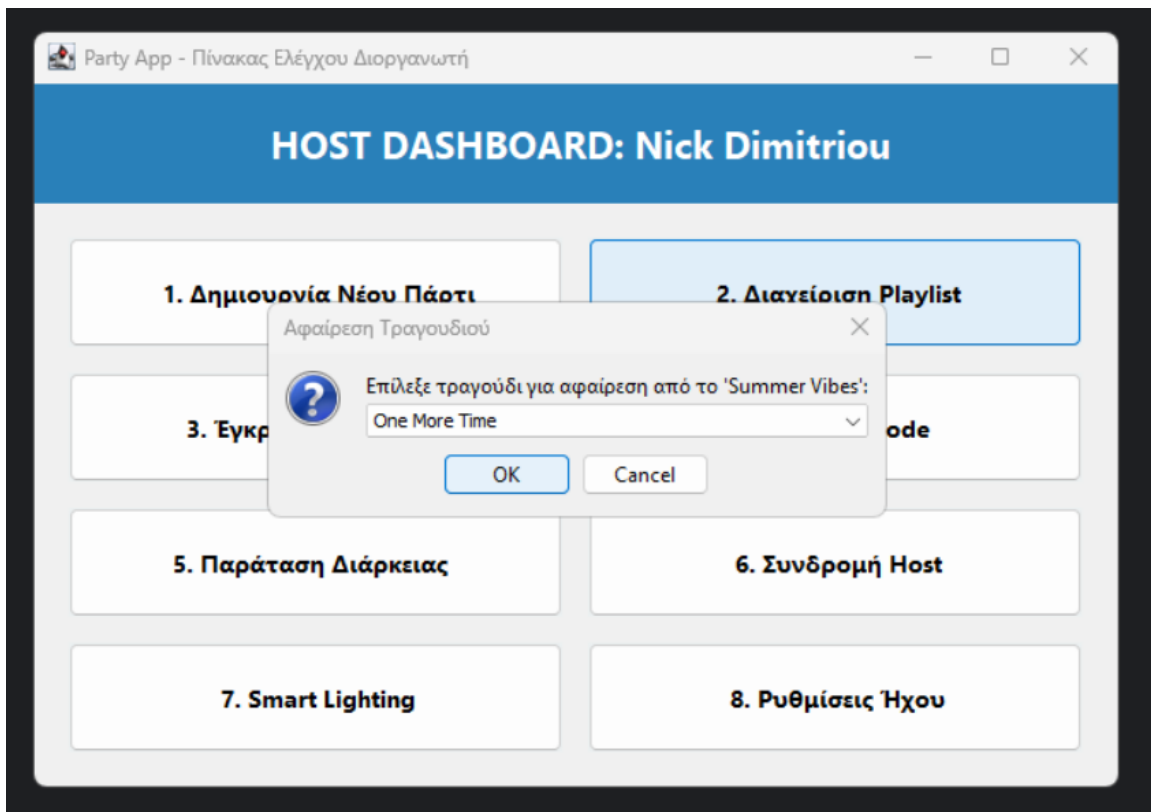
Host

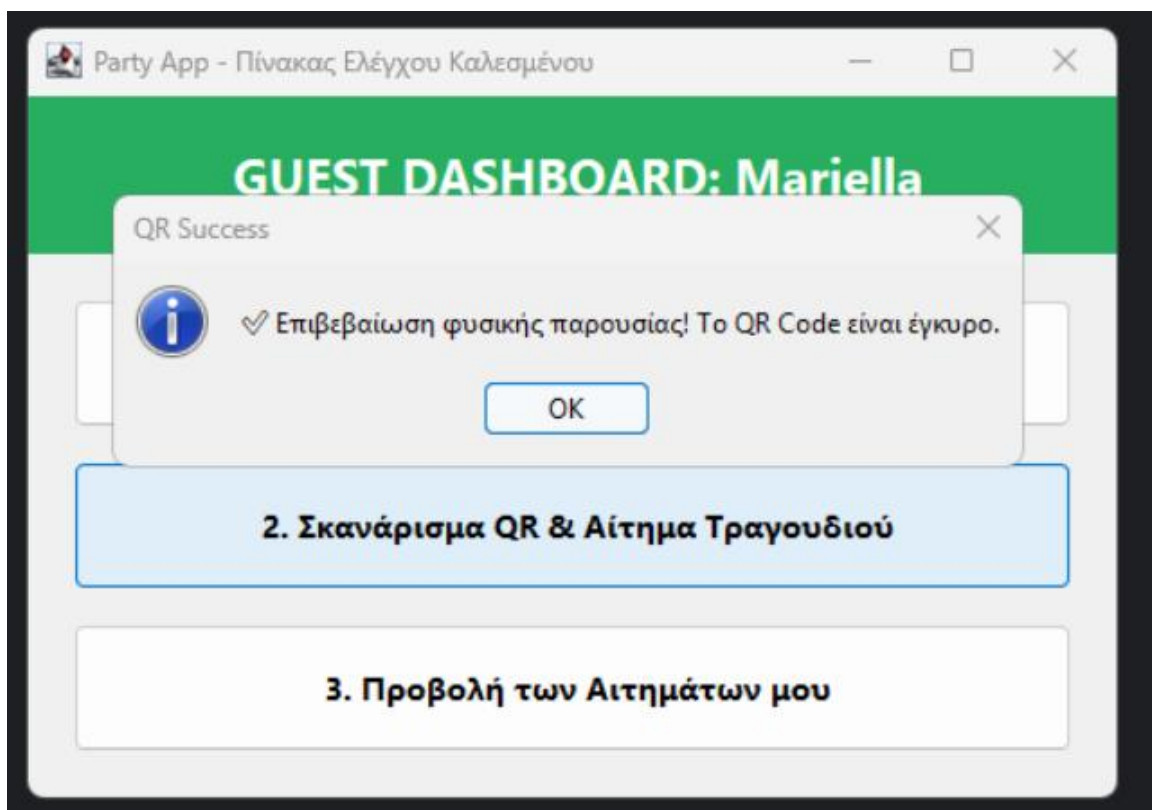
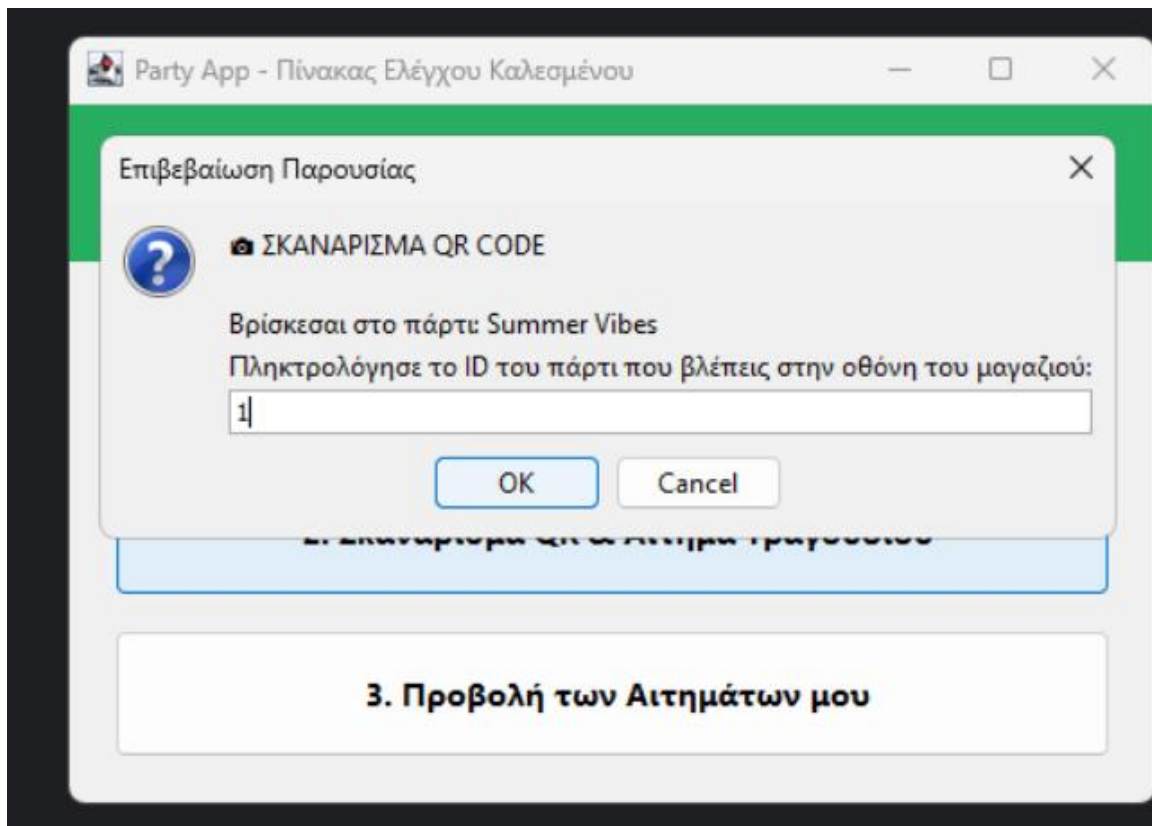


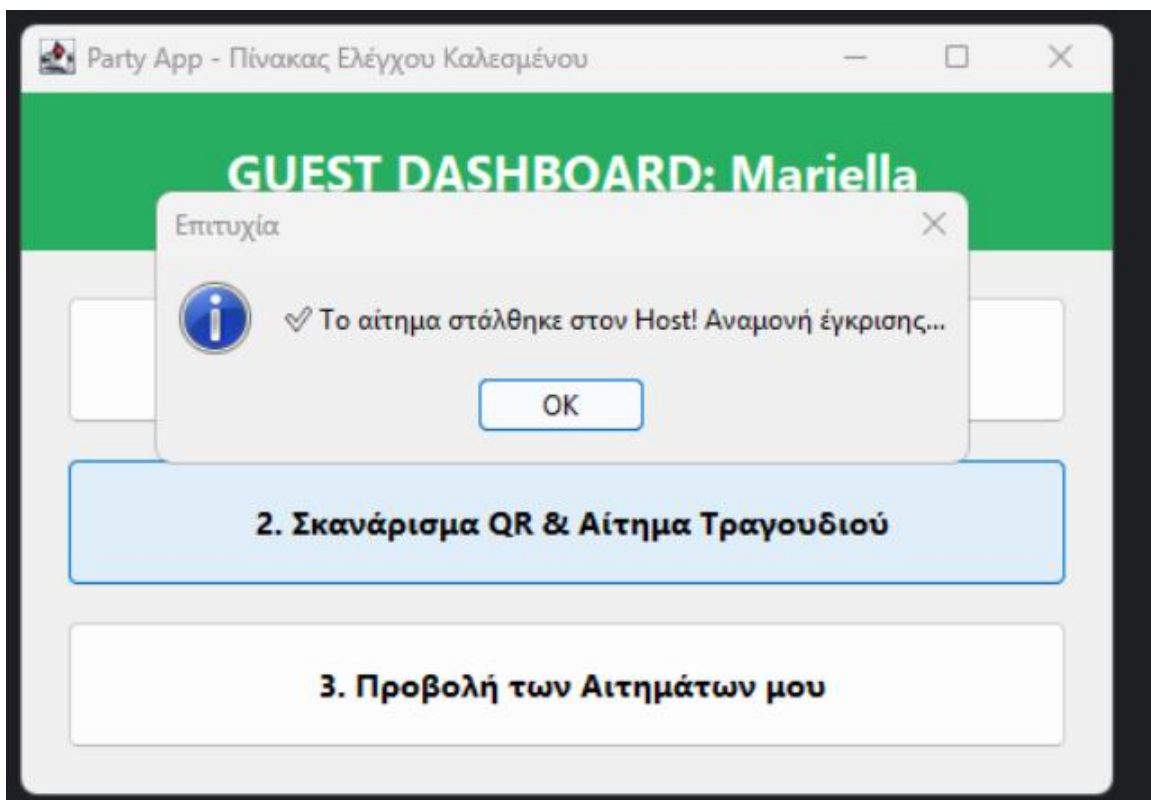
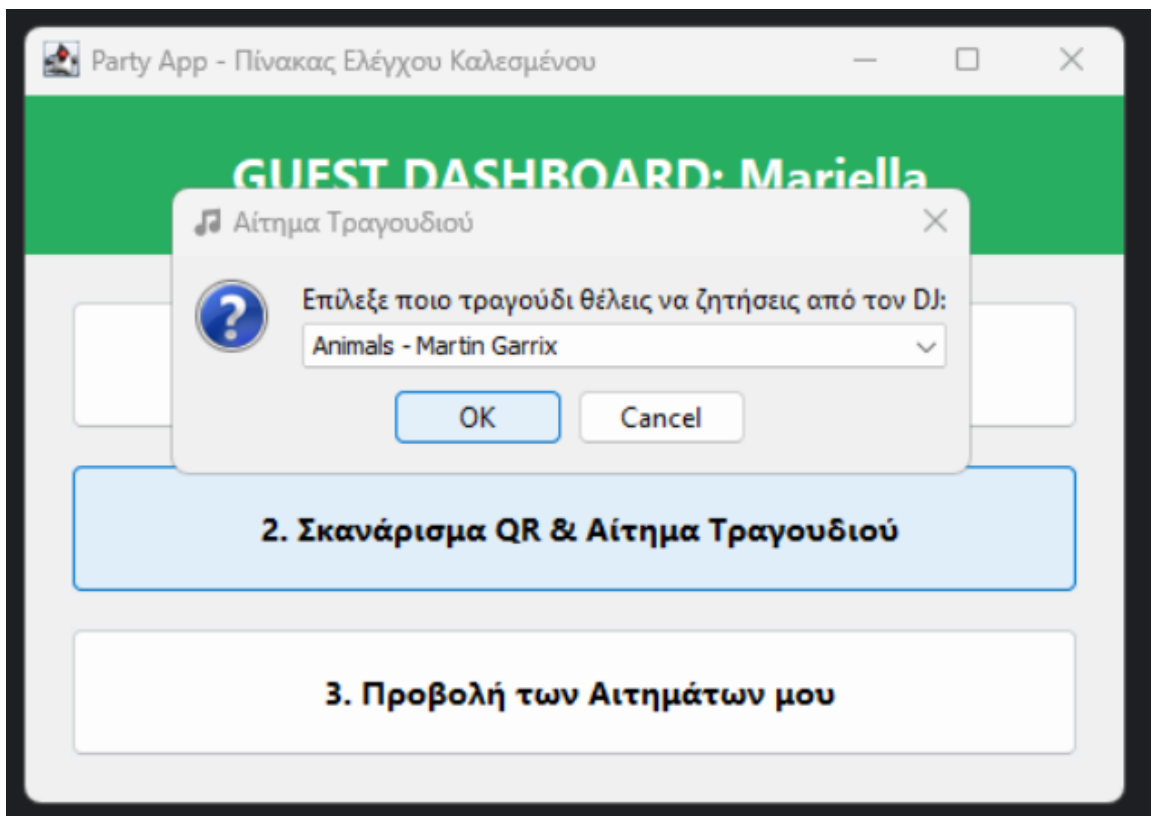


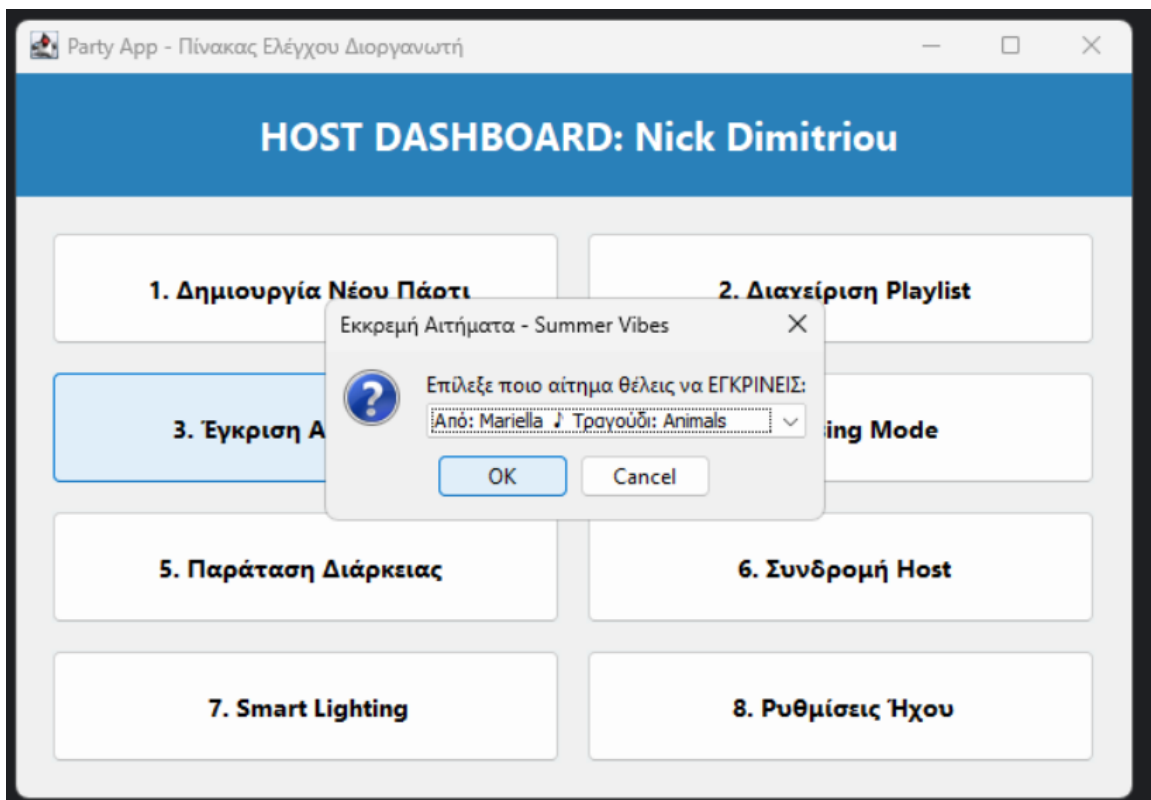
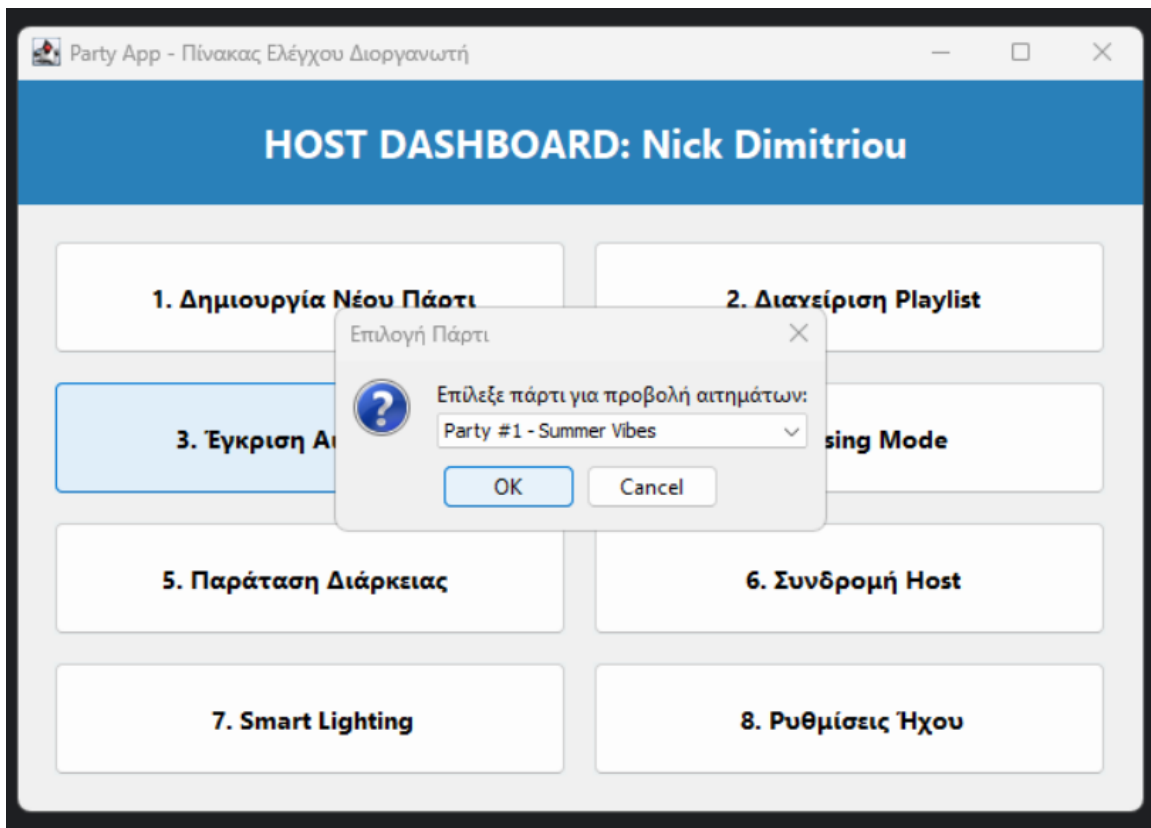


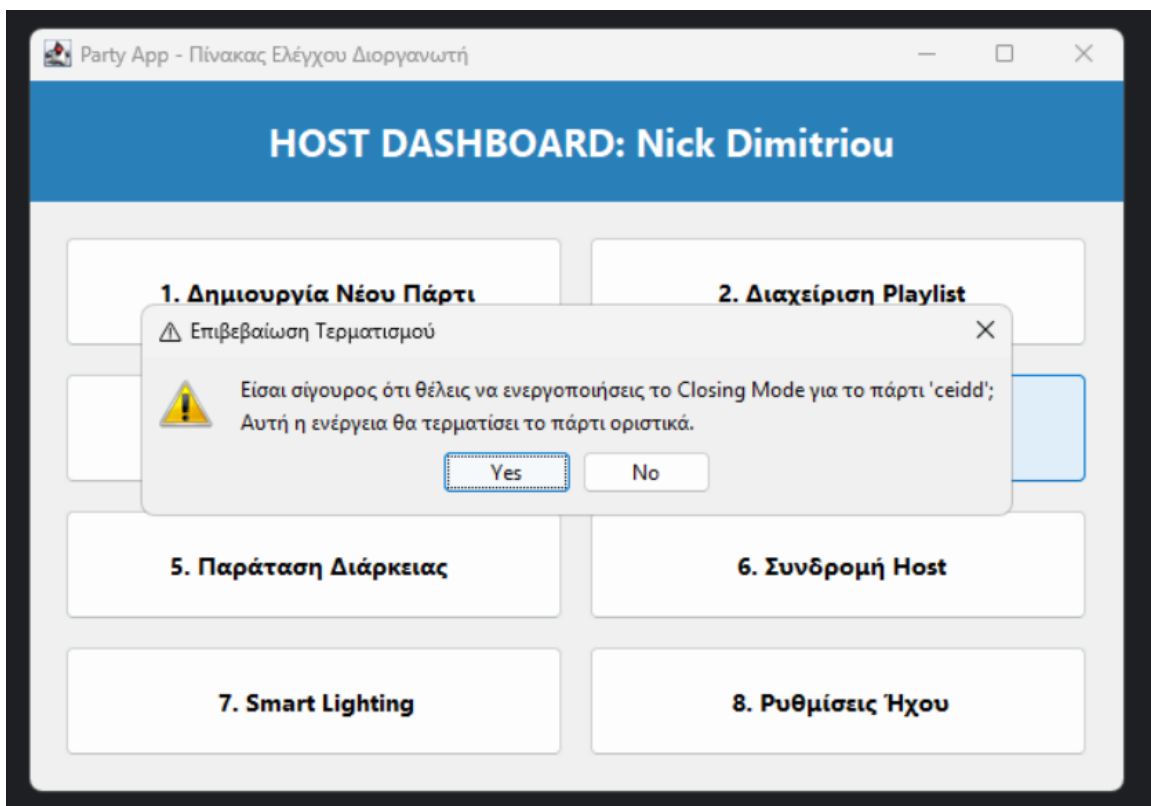
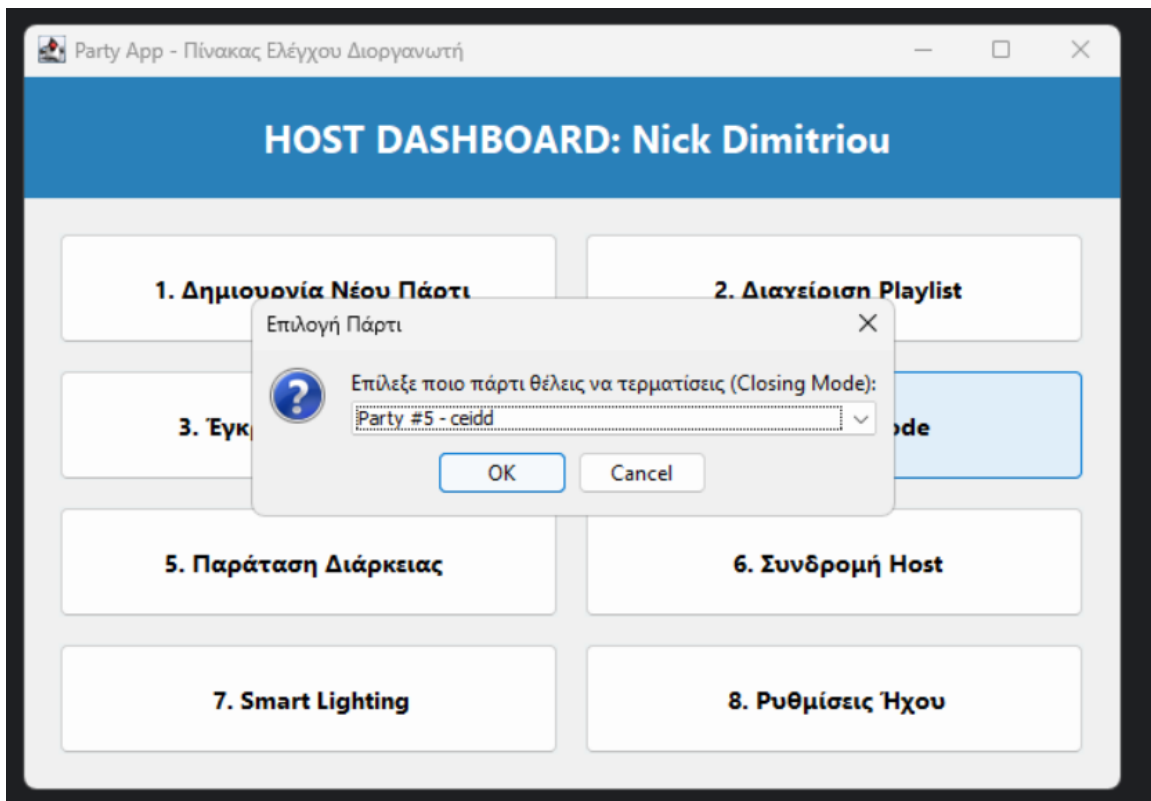


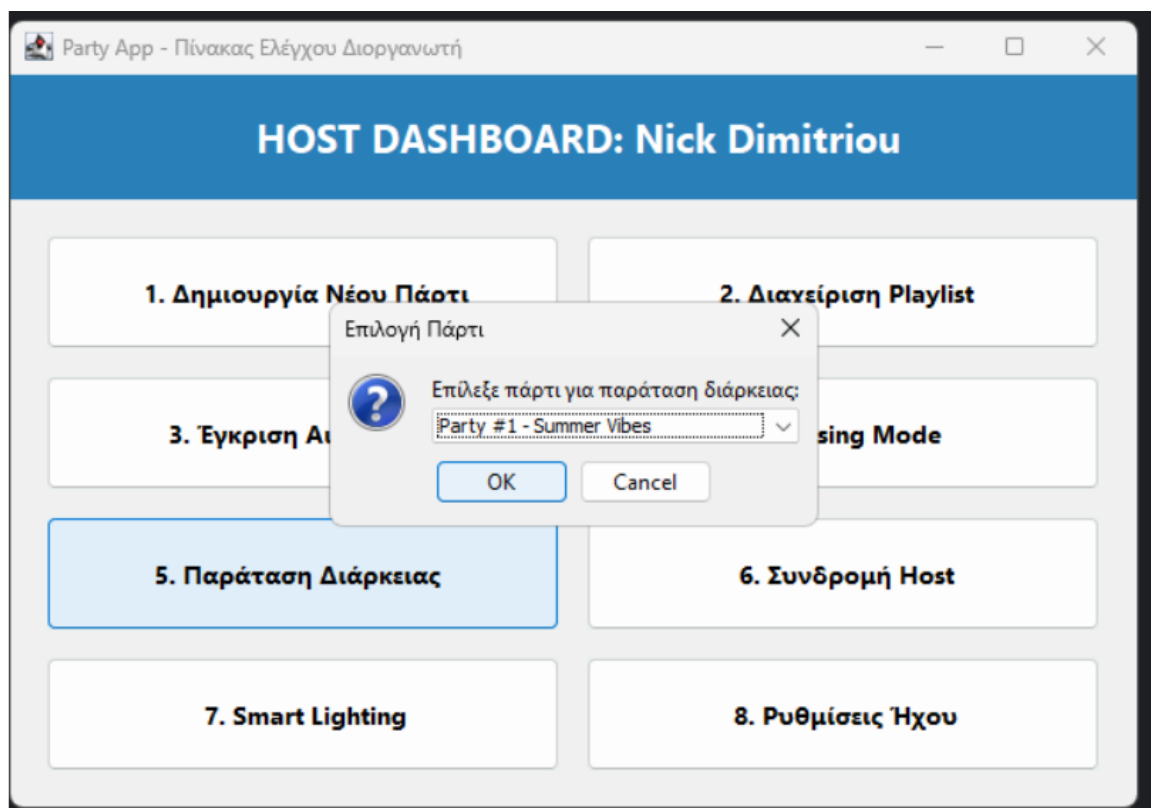
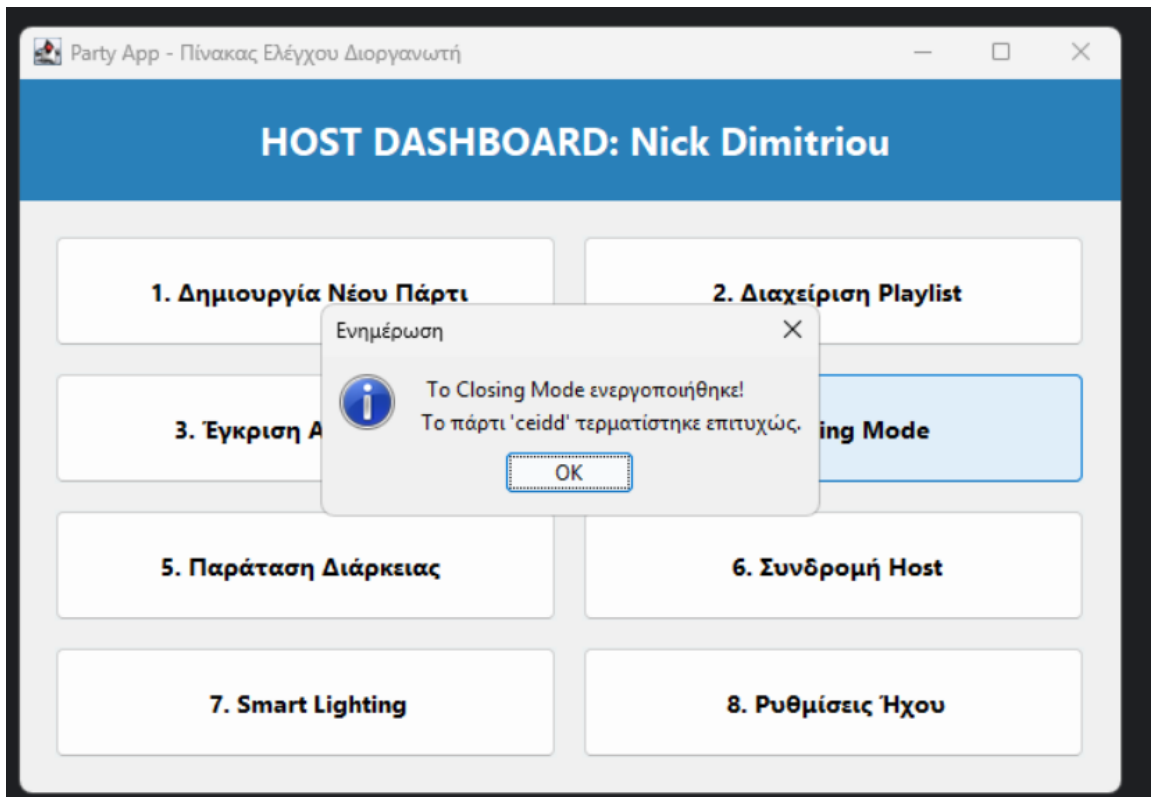


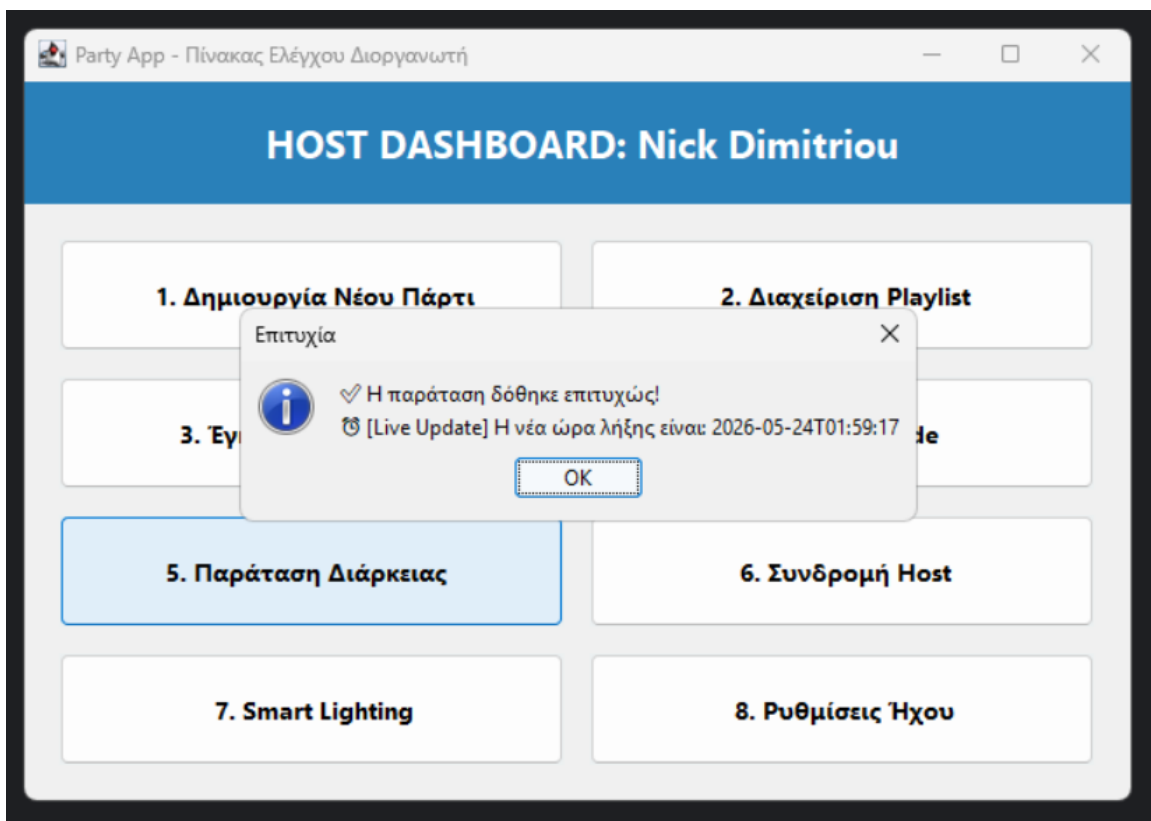
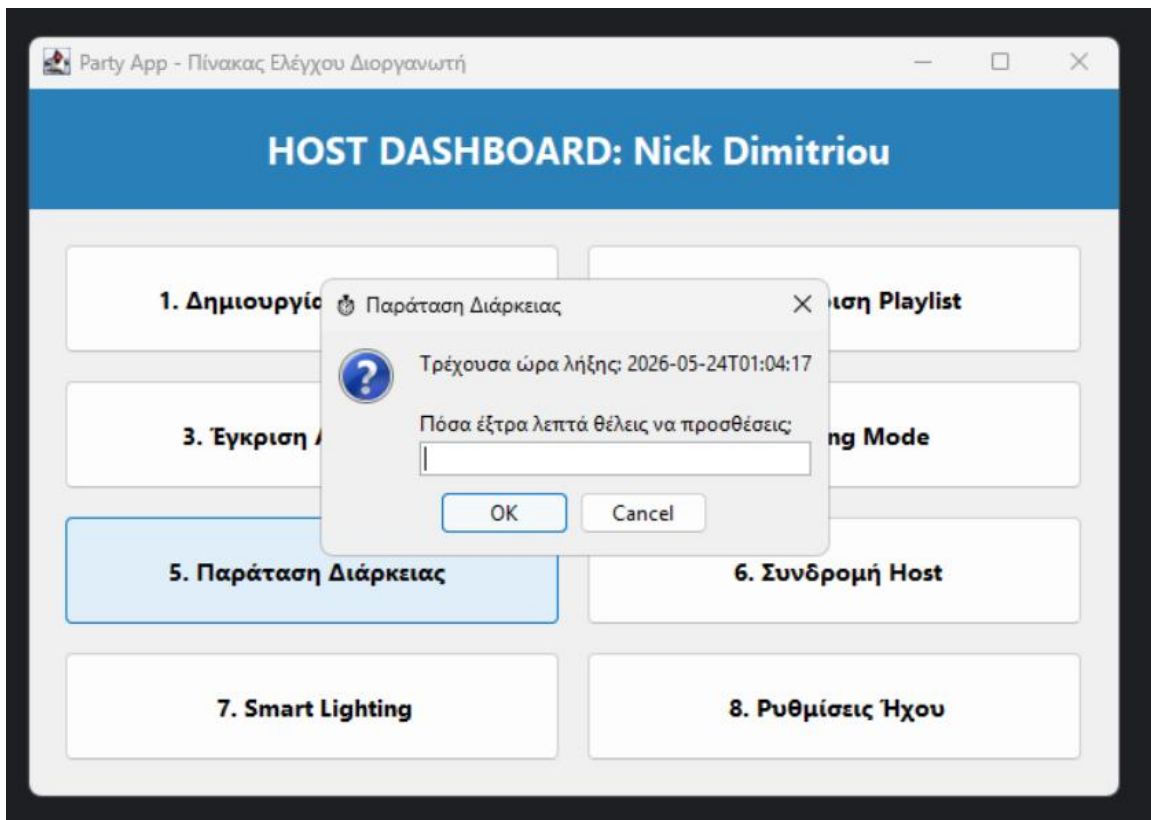


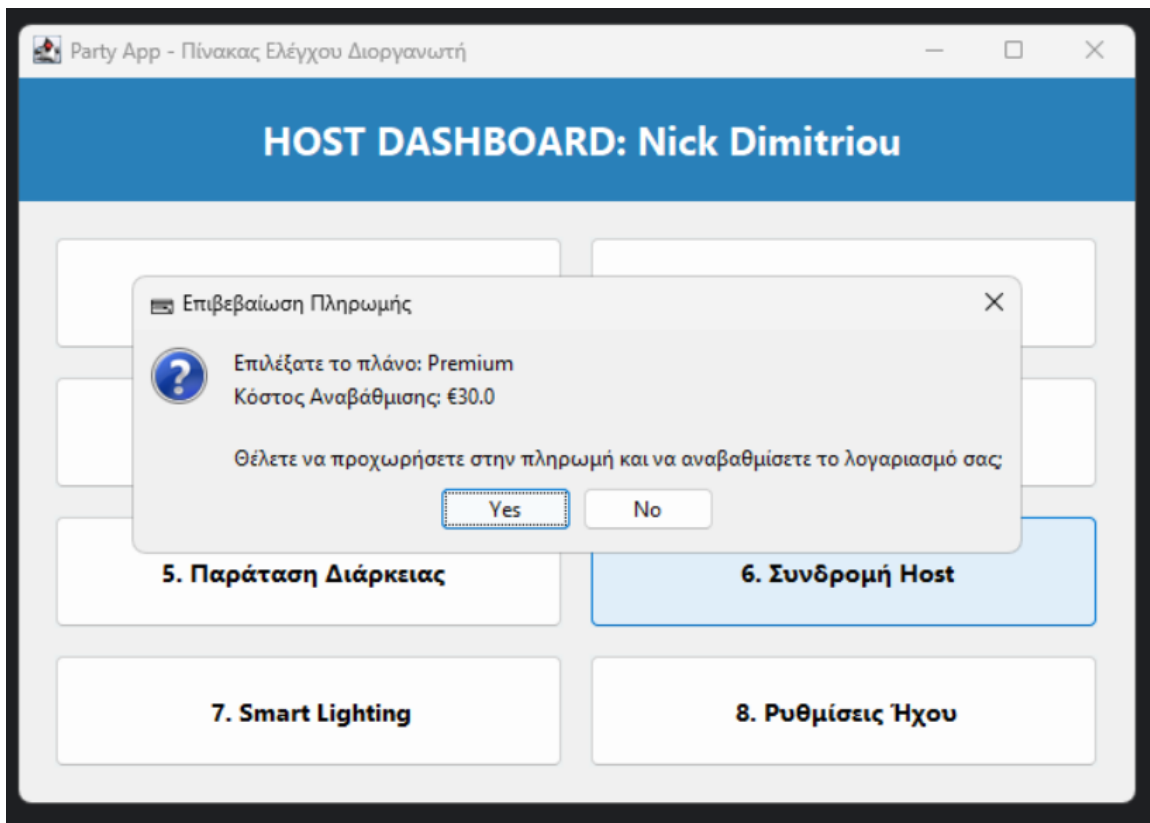
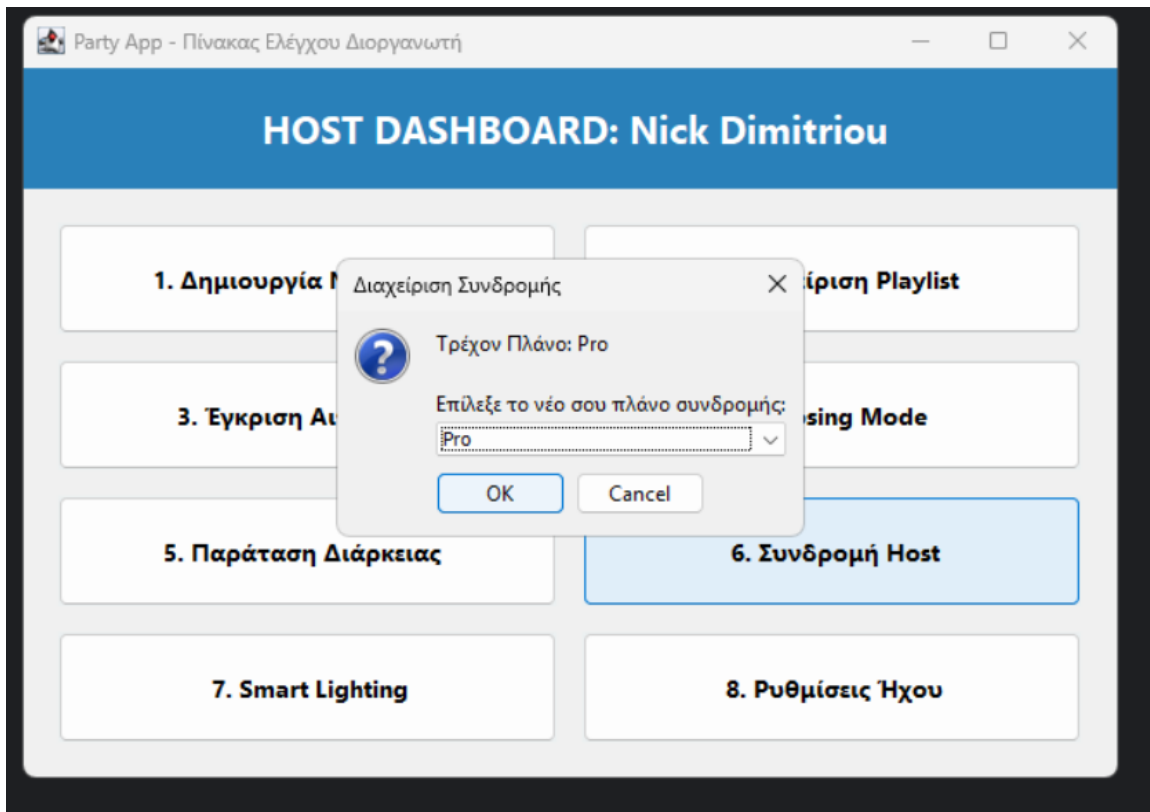


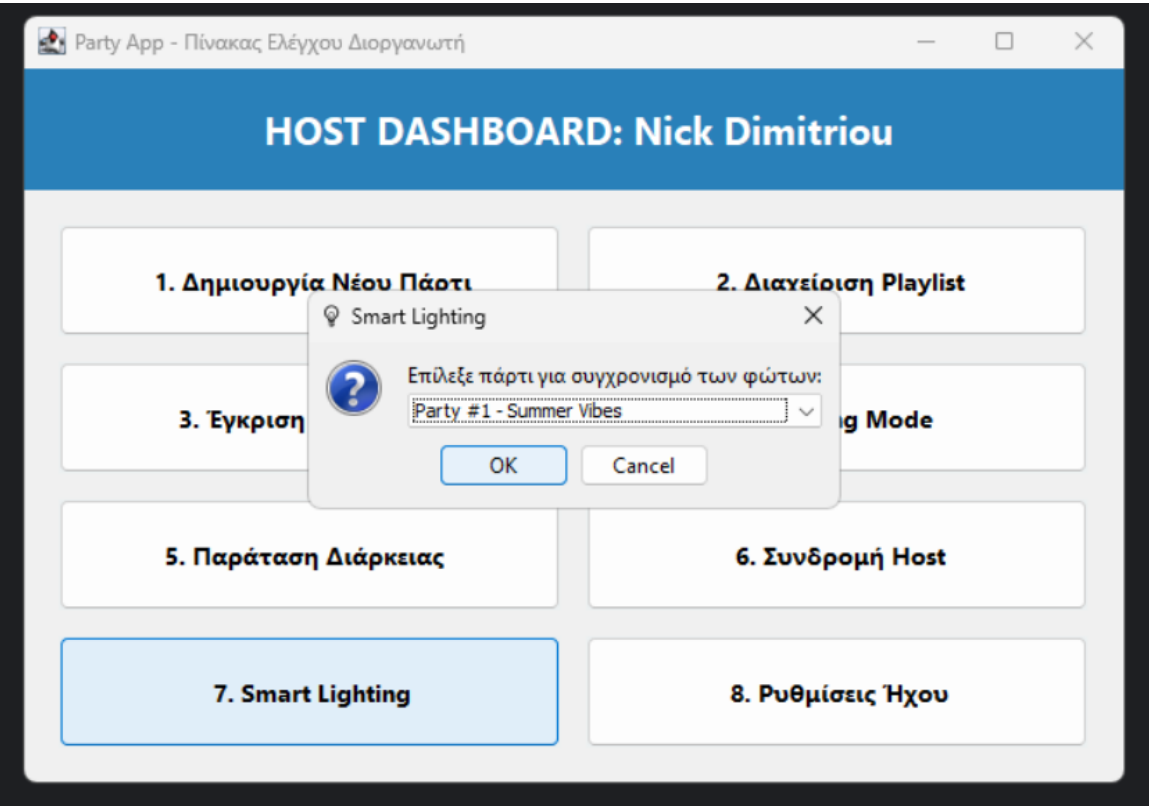
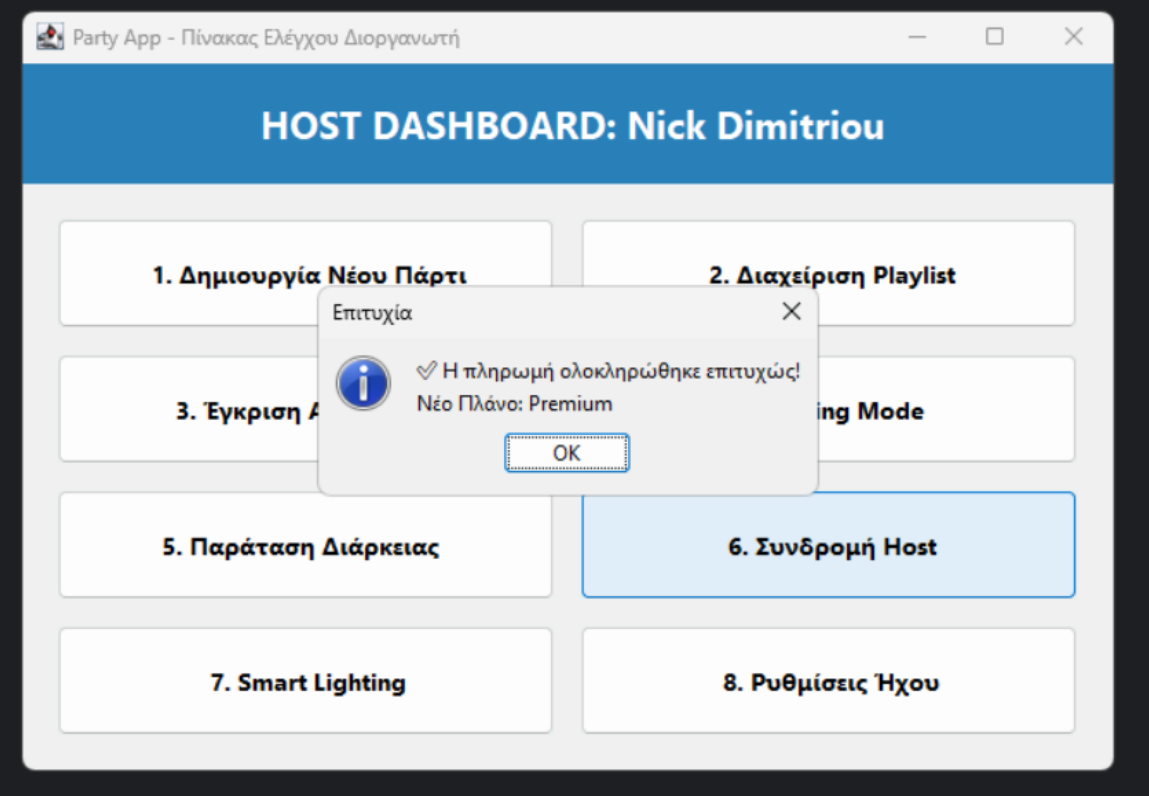


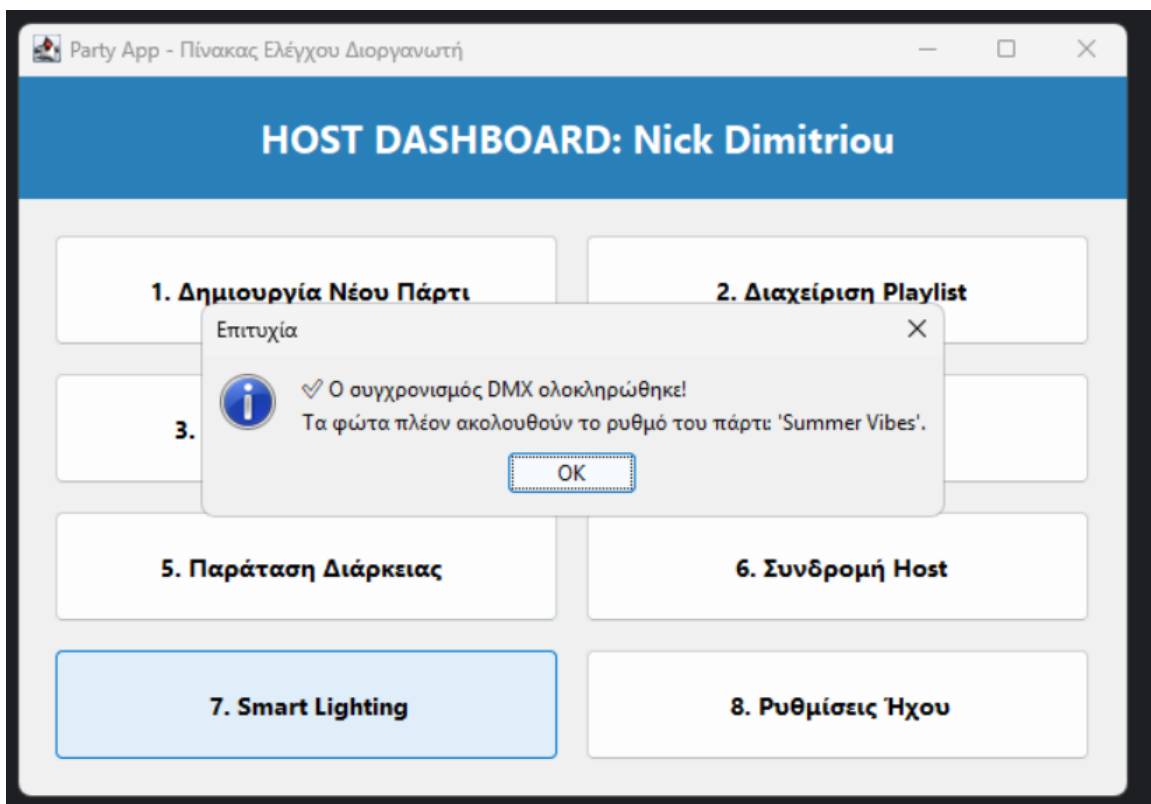
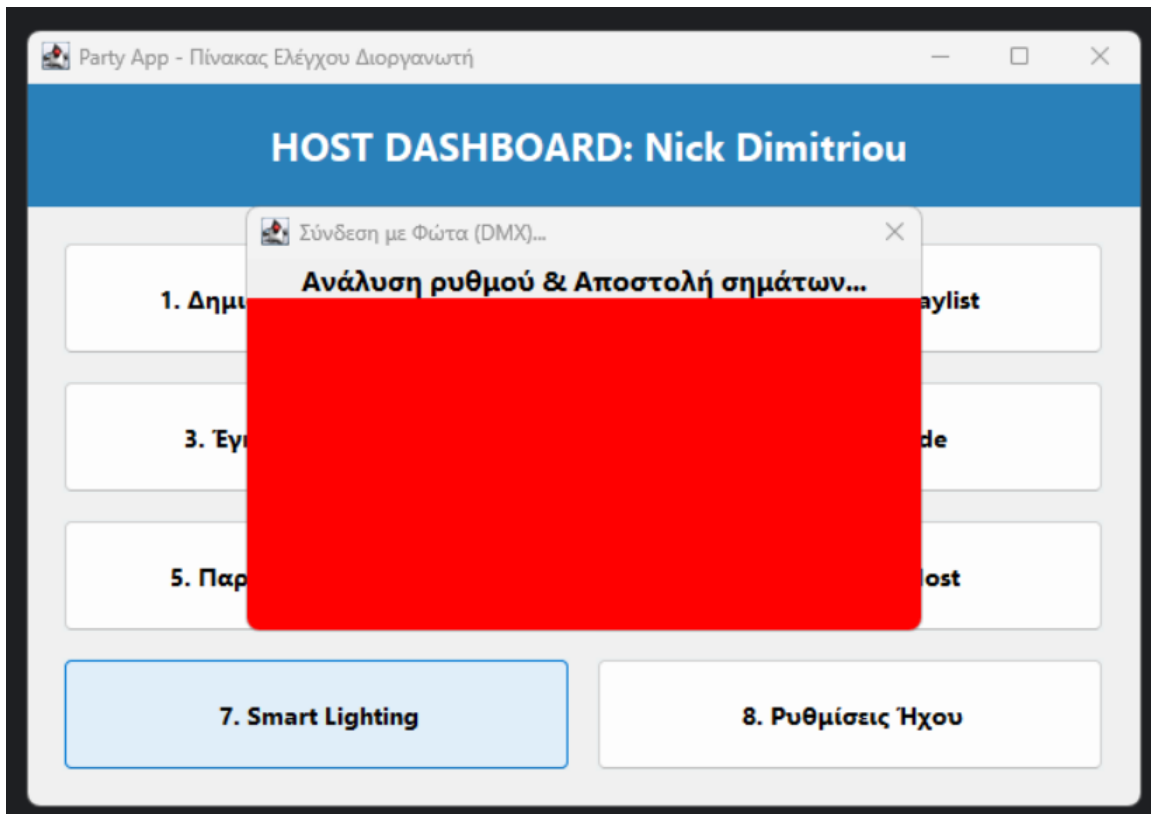


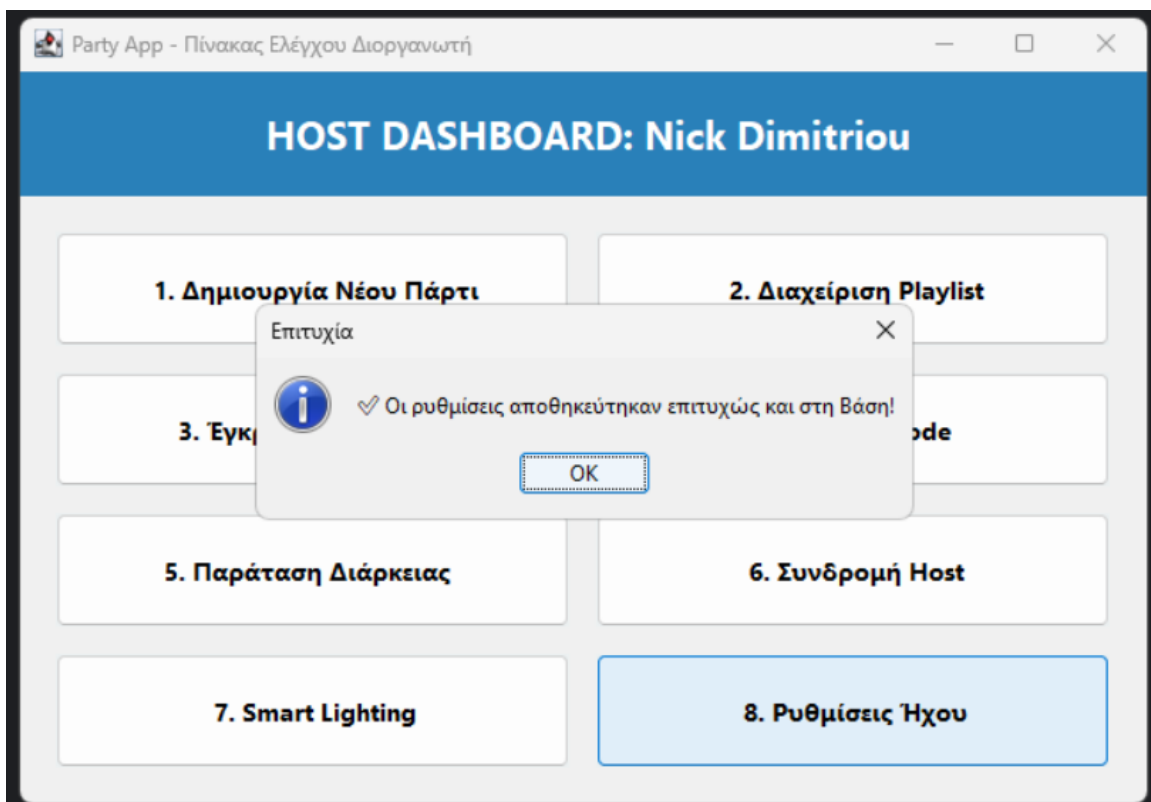
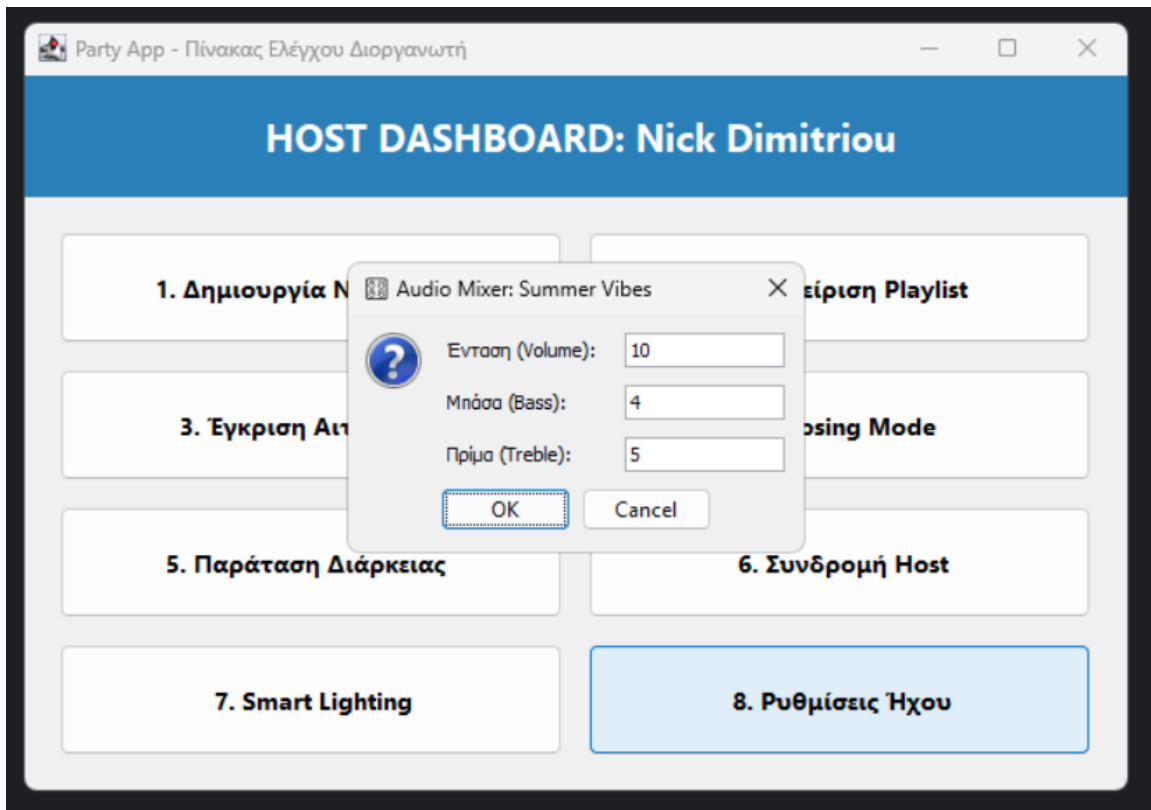














				subscriptionId	planType	status	monthlyCost	hostId	
☐		Επεξεργασία	 Αντιγραφή	 Διαγραφή	1	Premium	active	15.00	1
☐		Επεξεργασία	 Αντιγραφή	 Διαγραφή	2	Basic	active	0.00	2

				requestId	status	timestamp	queuePosition	guestId	songId	partyId
☐		Επεξεργασία	 Αντιγραφή  Διαγραφή	1	approved	2026-05-24 19:58:48	-1	3	8	1
☐		Επεξεργασία	 Αντιγραφή  Διαγραφή	2	pending	2026-05-24 20:02:55	-1	3	1	1
☐		Επεξεργασία	 Αντιγραφή  Διαγραφή	3	approved	2026-05-29 22:14:51	-1	3	8	1
☐		Επεξεργασία	 Αντιγραφή  Διαγραφή	4	pending	2026-05-29 22:16:37	-1	3	8	1
☐		Επεξεργασία	 Αντιγραφή  Διαγραφή	5	pending	2026-05-29 22:28:45	-1	3	7	1



- partyappdb
 - Νέο
 - audiosettings
 - environment
 - lightingsystem
 - party
 - partyguest
 - partyhost
 - payment
 - playlist
 - playlist_songs
 - qrcode
 - song
 - songrequest
 - subscription
 - user

☐ Εμφάνιση όλων | Αριθμός εγγραφών: 25 | Φιλτράρισμα εγγραφών: Αναζήτηση σε αυτόν τον πίνα

Extra options

		audiold	partyld	volume	bass	treble		
☐	Επεξεργασία	 Αντιγραφή	 Διαγραφή	1	1	22	4	5

☐ Επιλογή όλων | Με τους επιλεγμένους: Επεξεργασία Αντιγραφή Διαγραφή Εξαγωγή

☐ Εμφάνιση όλων | Αριθμός εγγραφών: 25 | Φιλτράρισμα εγγραφών: Αναζήτηση σε αυτόν τον πίνα

Λειτουργίες αποτελεσμάτων ερωτήματος

Εκτύπωση Αντιγραφή στο πρόχειρο Εξαγωγή Εμφάνιση διαγράμματος Δημιουργία προβολ